

// ZMAX //

zmax — Reference

v0.4.50

THE REFERENCE

every key · every command · every setting



MENKE
TECHNOLOGIES

Copyright and Attribution

zmax — the modal editor with an embedded language runtime

Copyright © 2026 MenkeTechnologies. All rights reserved.

No part of this book may be reproduced, distributed, or transmitted in any form or by any means without the prior written permission of the publisher, except for brief quotations in reviews and for the source excerpts identified below, which carry their own upstream licenses.

Published by MenkeTechnologies.

The trademarks and product names referenced in this book are the property of their respective owners. This book is an independent work: it is not affiliated with, authorized by, sponsored by, or otherwise endorsed by the Helix project, the Kakoune or Neovim projects, the Rust Foundation, or any other project named in these pages.

The software this book describes

zmax is distributed under the **Mozilla Public License 2.0**, and its source is at github.com/MenkeTechnologies/zmax.

The upstream this book is a fork of

zmax is a fork of **Helix**. The repository's `ATTRIBUTION.md` records the fork point exactly: upstream github.com/helix-editor/helix, forked at commit `6b7c7c6e0784d7c34295c24064deb36418c0b4b5` (2026-06-24), upstream version 25.7.1, upstream license Mozilla Public License 2.0.

MPL-2.0 is a file-level copyleft license: every source file derived from Helix remains under MPL-2.0, and modifications to those files keep their source available under the same terms. The unmodified `LICENSE` file in the zmax repository is the original Helix MPL-2.0 text and applies to all Helix-derived source files. New files added by zmax that contain no Helix-derived code may carry their own license header; absent such a header, a file is treated as MPL-2.0.

Helix itself is inspired by Kakoune and Neovim, and its selection-first editing model is heavily based on Kakoune.

The runtimes this book's software embeds

zmax vendors the MenkeTechnologies language runtimes it scripts with — `awkrs`, `elisprs`, `strykelang`, `vimlrs`, and `zshrs` — under `vendor/`. Each is released under the MIT License, Copyright © 2026 MenkeTechnologies, and each has its own repository under github.com/MenkeTechnologies.



MT PUBLISHING



MT OPENSOURCE

Contents

Copyright and Attribution	1
About This Reference	3
Keymap	4
Configuration and Settings	17
Typable Commands	38
Static Commands and Keybindings	84
Language Support	193

About This Reference

This is the zmax reference: every keybinding, every configuration setting, and every command the editor ships, generated from the source. It is the companion to the narrative book — where the book explains *why*, this enumerates *what*.

The sections are: the **keymap** (organized by mode and prefix), the **configuration schema** ([\[editor\]](#) and its tables), the full list of **typable : commands**, the full list of **static commands with their keybindings across all four keymap presets** (spacemacs, vim, helix, emacs) by mode, and the **language-support** matrix.

Keymap

Keymap

- Normal mode
 - Movement
 - Changes
 - * Shell
 - Selection manipulation
 - Search
 - Minor modes
 - * View mode
 - * Goto mode
 - * Match mode
 - * Window mode
 - * Space mode
 - Popup
 - Completion Menu
 - Signature-help Popup
 - * Unimpaired
- Insert mode
- Select / extend mode
- Picker
- Prompt

 Mappings marked (LSP) require an active language server for the file.

 Mappings marked (TS) require a tree-sitter grammar for the file type.

 Some terminals' default key mappings conflict with zmax's. If any of the mappings described on this page do not work as expected, check your terminal's mappings to ensure they do not conflict. See the [wiki](#) for known conflicts.

zmax ships four keymap presets, chosen with the `keymap` setting: `spacemacs` (the default), `vim`, `helix`, and `emacs`. The keys on this page are the **modal** presets (`spacemacs`, `vim`, `helix`); where they differ, the table cells note which preset a key belongs to. The `emacs` preset is modeless — you are always in insert mode, there is no normal mode, and commands are reached through Emacs chords (`Ctrl-x`, `Ctrl-c`, `Meta-`) — so the mode structure below does not describe it. For the exact keys each preset binds to every command, see the generated static-command table in the reference.

Normal mode

Normal mode is the launch mode for the modal keymap presets — `spacemacs` (the default), `vim`, and `helix` — and you return to it from other modes by pressing the `Escape` key. The `emacs` preset is modeless: it launches in insert mode and has no normal mode, so the mappings in this section do not apply to it.

Movement

NOTE: Unlike Vim, `f`, `F`, `t` and `T` are not confined to the current line.

Hereafter, `<n>` represents an integer by typing a sequence of digits.

Key	Description	Command
<code>h</code> , <code>Left</code>	Move left	<code>move_char_left</code>
<code>j</code> , <code>Down</code>	Move down	<code>move_visual_line_down</code>

Key	Description	Command
k, Up	Move up	move_visual_line_up
l, Right	Move right	move_char_right
w	Move next word start	move_next_word_start
b	Move previous word start	move_prev_word_start
e	Move next word end	move_next_word_end
W	Move next WORD start	move_next_long_word_start
B	Move previous WORD start	move_prev_long_word_start
E	Move next WORD end	move_next_long_word_end
t	Find till next char	find_till_char
f	Find next char	find_next_char
T	Find till previous char	till_prev_char
F	Find previous char	find_prev_char
<n>G, <n>gg	Go to line number <n>	goto_line
Alt-. .	Repeat last motion (f, t, m, [or])	repeat_last_motion
Home	Move to the start of the line	goto_line_start
End	Move to the end of the line	goto_line_end
Ctrl-b, PageUp	Move page up	page_up
Ctrl-f, PageDown	Move page down	page_down
Ctrl-u	Move cursor and page half page up	page_cursor_half_up
Ctrl-d	Move cursor and page half page down	page_cursor_half_down
Ctrl-i	Jump forward on the jumplist	jump_forward
Ctrl-o	Jump backward on the jumplist	jump_backward
Ctrl-s	Save the current selection to the jumplist	save_selection

Changes

Key	Description	Command
r	Replace with a character	replace
R	Replace with yanked text	replace_with_yanked
~	Switch case of the selected text	switch_case
`	Set the selected text to lower case	switch_to_lowercase
Alt-`	Set the selected text to upper case	switch_to_uppercase
i	Insert before selection	insert_mode
a	Insert after selection (append)	append_mode
I	Insert at the start of the line	insert_at_line_start

Key	Description	Command
A	Insert at the end of the line	insert_at_line_end
o	Open new line below selection	open_below
O	Open new line above selection	open_above
.	Repeat last insert	N/A
u	Undo change	undo
U	Redo change	redo
Alt-u	Move backward in history	earlier
Alt-U	Move forward in history	later
y	Yank selection	yank
p	Paste after selection	paste_after
P	Paste before selection	paste_before
" <reg>	Select a register to yank to or paste from	select_register
>	Indent selection	indent
<	Unindent selection	unindent
=	Format selection (LSP)	format_selections
d	Delete selection	delete_selection
Alt-d	Delete selection, without yanking	delete_selection_noyank
c	Change selection (delete and enter insert mode)	change_selection
Alt-c	Change selection (delete and enter insert mode, without yanking)	change_selection_noyank
[ctrl-a	Increment object (number) under cursor	increment
[ctrl-x	Decrement object (number) under cursor	decrement
Q	Start/stop macro recording to the selected register (experimental)	record_macro
q	Play back a recorded macro from the selected register (experimental)	replay_macro
[ctrl-z	Suspend Zmax and return to the shell (resume with fg)	suspend

☁ > and < leave the selection intact, so >>>> shifts the same block five levels without reselecting it. In Visual mode a count is levels as well (5> does the same in one keystroke); as an operator with a motion the count is lines, matching vim (2>> shifts two lines by one level). Vim drops out of Visual mode after >, which is what **vnoremap > >gv** exists to undo — zmax ships that behavior instead.

Shell

Key	Description	Command
	Pipe each selection through shell command, replacing with output	shell_pipe
Alt-	Pipe each selection into shell command, ignoring output	shell_pipe_to
!	Run shell command, inserting output before each selection	shell_insert_output

Key	Description	Command
<code>Alt-!</code>	Run shell command, appending output after each selection	<code>shell_append_output</code>
<code>\$</code>	Pipe each selection into shell command, keep selections where command returned 0	<code>shell_keep_pipe</code>

Selection manipulation

Key	Description	Command
<code>s</code>	Select all regex matches inside selections	<code>select_regex</code>
<code>S</code>	Split selection into sub selections on regex matches	<code>split_selection</code>
<code>Alt-s</code>	Split selection on newlines	<code>split_selection_on_newline</code>
<code>Alt-minus</code>	Merge selections	<code>merge_selections</code>
<code>Alt-_</code>	Merge consecutive selections	<code>merge_consecutive_selections</code>
<code>&</code>	Align selection in columns	<code>align_selections</code>
<code>-</code>	Trim whitespace from the selection	<code>trim_selections</code>
<code>;</code>	Collapse selection onto a single cursor	<code>collapse_selection</code>
<code>Alt-;</code>	Flip selection cursor and anchor	<code>flip_selections</code>
<code>Alt-:</code>	Ensures the selection is in forward direction	<code>ensure_selections_forward</code>
<code>,</code>	Keep only the primary selection	<code>keep_primary_selection</code>
<code>Alt-,</code>	Remove the primary selection	<code>remove_primary_selection</code>
<code>C</code>	Copy selection onto the next line (Add cursor below)	<code>copy_selection_on_next_line</code>
<code>Alt-C</code>	Copy selection onto the previous line (Add cursor above)	<code>copy_selection_on_prev_line</code>
<code>(</code>	Rotate main selection backward	<code>rotate_selections_backward</code>
<code>)</code>	Rotate main selection forward	<code>rotate_selections_forward</code>
<code>Alt-(</code>	Rotate selection contents backward	<code>rotate_selection_contents_backward</code>
<code>Alt-)</code>	Rotate selection contents forward	<code>rotate_selection_contents_forward</code>
<code>%</code>	Select entire file	<code>select_all</code>
<code>x</code>	Select current line, if already selected, extend to next line	<code>extend_line_below</code>
<code>X</code>	Extend selection to line bounds (line-wise selection)	<code>extend_to_line_bounds</code>
<code>Alt-x</code>	Shrink selection to line bounds (line-wise selection)	<code>shrink_to_line_bounds</code>
<code>J</code>	Join lines inside selection	<code>join_selections</code>
<code>Alt-J</code>	Join lines inside selection and select the inserted space	<code>join_selections_space</code>
<code>K</code>	Keep selections matching the regex	<code>keep_selections</code>
<code>Alt-K</code>	Remove selections matching the regex	<code>remove_selections</code>
<code>Ctrl-c</code>	Comment/uncomment the selections	<code>toggle_comments</code>

Key	Description	Command
Alt-o, Alt-up	Expand selection to parent syntax node (TS)	<code>expand_selection</code>
Alt-i, Alt-down	Shrink syntax tree object selection (TS)	<code>shrink_selection</code>
Alt-p, Alt-left	Select previous sibling node in syntax tree (TS)	<code>select_prev_sibling</code>
Alt-n, Alt-right	Select next sibling node in syntax tree (TS)	<code>select_next_sibling</code>
Alt-a	Select all sibling nodes in syntax tree (TS)	<code>select_all_siblings</code>
Alt-I, Alt-Shift-down	Select all children nodes in syntax tree (TS)	<code>select_all_children</code>
Alt-e	Move to end of parent node in syntax tree (TS)	<code>move_parent_node_end</code>
Alt-b	Move to start of parent node in syntax tree (TS)	<code>move_parent_node_start</code>

Search

Search commands all operate on the `/` register by default. To use a different register, use "`<char>`".

Key	Description	Command
<code>/</code>	Search for regex pattern	<code>search</code>
<code>?</code>	Search for previous pattern	<code>rsearch</code>
<code>n</code>	Select next search match	<code>search_next</code>
<code>N</code>	Select previous search match	<code>search_prev</code>
<code>*</code>	Use current selection as the search pattern, automatically wrapping with <code>\b</code> on word boundaries	<code>search_selection_detect_word_boundaries</code>
Alt-*	Use current selection as the search pattern	<code>search_selection</code>

Minor modes

These sub-modes are accessible from normal mode and typically switch back to normal mode after a command.

Key	Description	Command
v	Enter select (extend) mode	<code>select_mode</code>
g	Enter <code>goto mode</code>	N/A
m	Enter <code>match mode</code>	N/A
:	Enter command mode	<code>command_mode</code>
z	Enter <code>view mode</code>	N/A
Z	Enter sticky <code>view mode</code>	N/A
Ctrl-w	Enter <code>window mode</code>	N/A
Space	Enter <code>space mode</code>	N/A

These modes (except command mode) can be configured by [remapping keys](#).

View mode

Accessed by typing **z** in **normal mode**.

View mode is intended for scrolling and manipulating the view without changing the selection. The “sticky” variant of this mode (accessed by typing **Z** in normal mode) is persistent and can be exited using the escape key. This is useful when you’re simply looking over text and not actively editing it.

Key	Description	Command
z, c	Vertically center the line	<code>align_view_center</code>
t	Align the line to the top of the screen	<code>align_view_top</code>
b	Align the line to the bottom of the screen	<code>align_view_bottom</code>
m	Align the line to the middle of the screen (horizontally)	<code>align_view_middle</code>
j, down	Scroll the view downwards	<code>scroll_down</code>
k, up	Scroll the view upwards	<code>scroll_up</code>
[ctrl-f, PageDown]	Move page down	<code>page_down</code>
[ctrl-b, PageUp]	Move page up	<code>page_up</code>
[ctrl-u]	Move cursor and page half page up	<code>page_cursor_half_up</code>
[ctrl-d]	Move cursor and page half page down	<code>page_cursor_half_down</code>

Goto mode

Accessed by typing **g** in **normal mode**.

Jumps to various locations.

Key	Description	Command
<n>g	Go to line number <n>	<code>goto_file_start</code>
g	Go to the start of the file	<code>goto_file_start</code>
<n> 	Go to column number <n>	<code>goto_column</code>
 	Go to the start of line	<code>goto_column</code>
e	Go to the end of the file	<code>goto_last_line</code>
f	Go to files/URLs in selections	<code>goto_file</code>
h	Go to the start of the line	<code>goto_line_start</code>
l	Go to the end of the line	<code>goto_line_end</code>
s	Go to first non-whitespace character of the line	<code>goto_first_nonwhitespace</code>
t	Go to the top of the screen	<code>goto_window_top</code>
c	Go to the middle of the screen	<code>goto_window_center</code>
b	Go to the bottom of the screen	<code>goto_window_bottom</code>
d	Go to definition (LSP)	<code>goto_definition</code>
D	Go to declaration (LSP)	<code>goto_declaration</code>
y	Go to type definition (LSP)	<code>goto_type_definition</code>

Key	Description	Command
<code>r</code>	Go to references (LSP)	<code>goto_reference</code>
<code>i</code>	Go to implementation (LSP)	<code>goto_implementation</code>
<code>a</code>	Go to the last accessed/alternate file	<code>goto_last_accessed_file</code>
<code>m</code>	Go to the last modified/alternate file	<code>goto_last_modified_file</code>
<code>n</code>	Go to next buffer	<code>goto_next_buffer</code>
<code>p</code>	Go to previous buffer	<code>goto_previous_buffer</code>
<code>.</code>	Go to last modification in current file	<code>goto_last_modification</code>
<code>j</code>	Move down textual (instead of visual) line	<code>move_line_down</code>
<code>k</code>	Move up textual (instead of visual) line	<code>move_line_up</code>
<code>w</code>	Show labels at each word and select the word that belongs to the entered labels	<code>goto_word</code>

Match mode

Accessed by typing `m` in **normal mode**.

Please refer to the relevant sections for detailed explanations about **surround** and **textobjects**.

Key	Description	Command
<code>m</code>	Goto matching bracket (TS)	<code>match_brackets</code>
<code>s <char></code>	Surround current selection with <code><char></code>	<code>surround_add</code>
<code>r <from><to></code>	Replace surround character <code><from></code> with <code><to></code>	<code>surround_replace</code>
<code>d <char></code>	Delete surround character <code><char></code>	<code>surround_delete</code>
<code>a <object></code>	Select around textobject	<code>select_textobject_around</code>
<code>i <object></code>	Select inside textobject	<code>select_textobject_inner</code>

TODO: Mappings for selecting syntax nodes (a superset of `[]`).

Window mode

Accessed by typing `Ctrl-w` in **normal mode**.

This layer is similar to Vim keybindings as Kakoune does not support windows.

Key	Description	Command
<code>w, Ctrl-w</code>	Switch to next window	<code>rotate_view</code>
<code>v, Ctrl-v</code>	Vertical right split	<code>vsplit</code>
<code>s, Ctrl-s</code>	Horizontal bottom split	<code>hsplit</code>
<code>t, Ctrl-t</code>	Transpose the two adjacent splits	<code>transpose_view</code>
<code>f</code>	Go to files/URLs in selections in horizontal splits	<code>goto_file_hsplit</code>
<code>F</code>	Go to files/URLs in selections in vertical splits	<code>goto_file_vsplit</code>
<code>h, Ctrl-h, Left</code>	Move to left split	<code>jump_view_left</code>

Key	Description	Command
j, [ctrl-j, Down	Move to split below	jump_view_down
k, [ctrl-k, Up	Move to split above	jump_view_up
l, [ctrl-l, Right	Move to right split	jump_view_right
q, [ctrl-q	Close current window	wclose
o, [ctrl-o	Only keep the current window, closing all the others	wonly
H	Swap window to the left	swap_view_left
J	Swap window downwards	swap_view_down
K	Swap window upwards	swap_view_up
L	Swap window to the right	swap_view_right
ns, n[ctrl-s	New horizontal split with a scratch buffer	hsplit_new
nv, n[ctrl-v	New vertical split with a scratch buffer	vsplit_new

Space mode

Accessed by typing [Space](#) in [normal mode](#).

This layer is a kludge of mappings, mostly pickers.

Key	Description	Command
f	Open file picker at LSP workspace root	file_picker
F	Open file picker at current working directory	file_picker_in_current_directory
e	Open file explorer at workspace root	file_explorer
.	Open file explorer at current buffer's directory	file_explorer_in_current_buffer_directory
b	Open buffer picker	buffer_picker
j	Open jumplist picker	jumplist_picker
g	Open changed file picker	changed_file_picker
G	Debug (experimental)	N/A
k	Show documentation for item under cursor in a popup (LSP)	hover
s	Open document symbol picker (LSP or TS)	lsp_or_syntax_symbol_picker
S	Open workspace symbol picker (LSP or TS)	lsp_or_syntax_workspace_symbol_picker
d	Open document diagnostics picker (LSP)	diagnostics_picker
D	Open workspace diagnostics picker (LSP)	workspace_diagnostics_picker
r	Rename symbol (LSP)	rename_symbol
a	Apply code action (LSP)	code_action

Key	Description	Command
h	Select symbol references (LSP)	<code>select_references_to_symbol_under_cursor</code>
,	Open last fuzzy picker	<code>last_picker</code>
w	Enter window mode	N/A
c	Comment/uncomment selections	<code>toggle_comments</code>
C	Block comment/uncomment selections	<code>toggle_block_comments</code>
Alt-c	Line comment/uncomment selections	<code>toggle_line_comments</code>
p	Paste system clipboard after selections	<code>paste_clipboard_after</code>
P	Paste system clipboard before selections	<code>paste_clipboard_before</code>
y	Yank selections to clipboard	<code>yank_to_clipboard</code>
Y	Yank main selection to clipboard	<code>yank_main_selection_to_clipboard</code>
R	Replace selections by clipboard contents	<code>replace_selections_with_clipboard</code>
/	Global search in workspace folder	<code>global_search</code>
?	Open command palette	<code>command_palette</code>

 Global search displays results in a fuzzy picker, use **Space + '** to bring it back up after opening a file.

Popup Displays documentation for item under cursor. Remapping currently not supported.

Key	Description
Ctrl-u	Scroll up
Ctrl-d	Scroll down

Completion Menu Displays documentation for the selected completion item. Remapping currently not supported.

Key	Description
Shift-Tab, Ctrl-p, Up	Previous entry
Tab, Ctrl-n, Down	Next entry
Enter	Close menu and accept completion
Ctrl-c	Close menu and reject completion

Any other keypresses result in the completion being accepted.

Signature-help Popup Displays the signature of the selected completion item. Remapping currently not supported.

Key	Description
Alt-p	Previous signature
Alt-n	Next signature

Unimpaired

These mappings are in the style of **vim-unimpaired**.

Key	Description	Command
<code>]d</code>	Go to next diagnostic (LSP)	<code>goto_next_diag</code>
<code>[d</code>	Go to previous diagnostic (LSP)	<code>goto_prev_diag</code>
<code>]D</code>	Go to last diagnostic in document (LSP)	<code>goto_last_diag</code>
<code>[D</code>	Go to first diagnostic in document (LSP)	<code>goto_first_diag</code>
<code>]f</code>	Go to next function (TS)	<code>goto_next_function</code>
<code>[f</code>	Go to previous function (TS)	<code>goto_prev_function</code>
<code>]t</code>	Go to next type definition (TS)	<code>goto_next_class</code>
<code>[t</code>	Go to previous type definition (TS)	<code>goto_prev_class</code>
<code>]a</code>	Go to next argument/parameter (TS)	<code>goto_next_parameter</code>
<code>[a</code>	Go to previous argument/parameter (TS)	<code>goto_prev_parameter</code>
<code>]c</code>	Go to next comment (TS)	<code>goto_next_comment</code>
<code>[c</code>	Go to previous comment (TS)	<code>goto_prev_comment</code>
<code>]e</code>	Go to next entry (TS)	<code>goto_next_entry</code>
<code>[e</code>	Go to previous entry (TS)	<code>goto_prev_entry</code>
<code>]T</code>	Go to next test (TS)	<code>goto_next_test</code>
<code>[T</code>	Go to previous test (TS)	<code>goto_prev_test</code>
<code>]p</code>	Go to next paragraph	<code>goto_next_paragraph</code>
<code>[p</code>	Go to previous paragraph	<code>goto_prev_paragraph</code>
<code>]g</code>	Go to next change	<code>goto_next_change</code>
<code>[g</code>	Go to previous change	<code>goto_prev_change</code>
<code>]G</code>	Go to last change	<code>goto_last_change</code>
<code>[G</code>	Go to first change	<code>goto_first_change</code>
<code>]n</code>	Go to next merge-conflict marker	<code>goto_next_conflict</code>
<code>[n</code>	Go to previous merge-conflict marker	<code>goto_prev_conflict</code>
<code>[x</code>	Go to next (X)HTML element	<code>goto_next_xml_element</code>
<code>]x</code>	Go to previous (X)HTML element	<code>goto_prev_xml_element</code>
<code>]Space</code>	Add newline below	<code>add_newline_below</code>
<code>[Space</code>	Add newline above	<code>add_newline_above</code>

Wildfire

In normal mode, expand or shrink the selection by syntax-aware text objects.

Key	Description	Command
<code><ret></code>	Select / grow to the closest enclosing text object	<code>wildfire</code>
<code><backspace></code>	Shrink to the previously selected text object	<code>wildfire_shrink</code>

Git conflicts and merge

These live under the space-**g** git menu (see [Space mode](#)) and the unpaired **]n**/**[n** motions above.

Key	Description	Command
SPC g m , SPC g c r	Open the 3-pane merge-conflict resolver (:merge)	resolve_conflicts
SPC g c O	Resolve all conflicts by keeping our side	conflict_take_all_ours
SPC g c T	Resolve all conflicts by keeping their side	conflict_take_all_theirs
SPC g =	Side-by-side diff of the buffer vs. git HEAD (:diff)	git_diff

Insert mode

Accessed by typing **i** in [normal mode](#).

Insert mode bindings are minimal in the modal presets (**spacemacs**, **vim**, **helix**), which are built around spending most of your time in normal mode. In those presets, changes to the text are grouped for undo when you escape from insert mode back to normal mode. The **emacs** preset is modeless — you are always inserting — so it has no such escape and commits undo checkpoints explicitly with **Ctrl-s**.

💡 If you use a modal preset, new users are encouraged to learn the modal editing paradigm to get the smoothest experience. If you prefer a modeless editor, select the **emacs** preset with the **keymap** setting.

Key	Description	Command
Escape	Switch to normal mode	normal_mode
Ctrl-s	Commit undo checkpoint	commit_undo_checkpoint
Ctrl-x	Autocomplete	completion
Ctrl-r	Insert a register content	insert_register
Ctrl-w , Alt-Backspace	Delete previous word	delete_word_backward
Alt-d , Alt-Delete	Delete next word	delete_word_forward
Ctrl-u	Delete to start of line	kill_to_line_start
Ctrl-k	Delete to end of line	kill_to_line_end
Ctrl-h , Backspace , Shift-Backspace	Delete previous char	delete_char_backward
Ctrl-d , Delete	Delete next char	delete_char_forward
Ctrl-j , Enter	Insert new line	insert_newline
Tab	Advance a snippet tab stop, else expand a snippet trigger or emmet abbreviation, else smart tab	emmet_expand
Shift-Tab	Insert tab	insert_tab

💡 **Tab** is overloaded in insert mode: if a snippet is active it jumps to the next tab stop; otherwise it tries to expand the word before the cursor as a **snippet** trigger or an emmet abbreviation; failing both, it behaves as a **smart tab**.

These keys are not recommended, but are included for new users less familiar with modal editors.

Key	Description	Command
Up	Move to previous line	move_line_up

Key	Description	Command
Down	Move to next line	<code>move_line_down</code>
Left	Backward a char	<code>move_char_left</code>
Right	Forward a char	<code>move_char_right</code>
PageUp	Move one page up	<code>page_up</code>
PageDown	Move one page down	<code>page_down</code>
Home	Move to line start	<code>goto_line_start</code>
End	Move to line end	<code>goto_line_end_newline</code>

As you become more comfortable with modal editing, you may want to disable some insert mode bindings. You can do this by editing your `config.toml` file.

```
[keys.insert]
up = "no_op"
down = "no_op"
left = "no_op"
right = "no_op"
pageup = "no_op"
pagedown = "no_op"
home = "no_op"
end = "no_op"
```

Select / extend mode

Accessed by typing `v` in **normal mode**.

Select mode echoes Normal mode, but changes any movements to extend selections rather than replace them. Goto motions are also changed to extend, so that `vg|`, for example, extends the selection to the end of the line.

Search is also affected. By default, `n` and `N` will remove the current selection and select the next instance of the search term. Toggling this mode before pressing `n` or `N` makes it possible to keep the current selection. Toggling it on and off during your iterative searching allows you to selectively add search terms to your selections.

Picker

Keys to use within picker. Remapping currently not supported. See the documentation page on [pickers](#) for more info. [Prompt](#) keybinds also work in pickers, except where they conflict with picker keybinds.

Key	Description
<code>Shift-Tab</code> , <code>Up</code> , <code>Ctrl-p</code>	Previous entry
<code>Tab</code> , <code>Down</code> , <code>Ctrl-n</code>	Next entry
<code>PageUp</code> , <code>Ctrl-u</code>	Page up
<code>PageDown</code> , <code>Ctrl-d</code>	Page down
<code>Home</code>	Go to first entry
<code>End</code>	Go to last entry
<code>Enter</code>	Open selected
<code>Alt-Enter</code>	Open selected in the background without closing the picker
<code>Ctrl-s</code>	Open horizontally
<code>Ctrl-v</code>	Open vertically

Key	Description
<code>Ctrl-t</code>	Toggle preview
<code>Escape, Ctrl-c</code>	Close picker

Prompt

Keys to use within prompt, Remapping currently not supported.

Key	Description
<code>Escape, Ctrl-c</code>	Close prompt
<code>Alt-b, Ctrl-Left</code>	Backward a word
<code>Ctrl-b, Left</code>	Backward a char
<code>Alt-f, Ctrl-Right</code>	Forward a word
<code>Ctrl-f, Right</code>	Forward a char
<code>Ctrl-e, End</code>	Move prompt end
<code>Ctrl-a, Home</code>	Move prompt start
<code>Ctrl-w, Alt-Backspace, Ctrl-Backspace</code>	Delete previous word
<code>Alt-d, Alt-Delete, Ctrl-Delete</code>	Delete next word
<code>Ctrl-u</code>	Delete to start of line
<code>Ctrl-k</code>	Delete to end of line
<code>Backspace, Ctrl-h, Shift-Backspace</code>	Delete previous char
<code>Delete, Ctrl-d</code>	Delete next char
<code>Ctrl-s</code>	Insert a word under doc cursor, may be changed to Ctrl-r Ctrl-w later
<code>Ctrl-p, Up</code>	Select previous history
<code>Ctrl-n, Down</code>	Select next history
<code>Ctrl-r</code>	Insert the content of the register selected by following input char
<code>Tab</code>	Select next completion item
<code>BackTab</code>	Select previous completion item
<code>Enter</code>	Open selected

Configuration and Settings

Editor

- `[editor]` Section
- `[editor.clipboard-provider]` Section
- `[editor.statusline]` Section
- `[editor.lsp]` Section
- `[editor.cursor-shape]` Section
- `[editor.file-picker]` Section
- `[editor.file-explorer]` Section
- `[editor.buffer-picker]` Section
- `[editor.auto-pairs]` Section
- `[editor.auto-save]` Section
- `[editor.search]` Section
- `[editor.whitespace]` Section
- `[editor.indent-guides]` Section
- `[editor.gutters]` Section
 - `[editor.gutters.line-numbers]` Section
 - `[editor.gutters.diagnostics]` Section
 - `[editor.gutters.diff]` Section
 - `[editor.gutters.spacer]` Section
 - `[editor.gutters.code-action-hint]` Section
- `[editor.soft-wrap]` Section
- `[editor.smart-tab]` Section
- `[editor.inline-diagnostics]` Section
- `[editor.word-completion]` Section
- `[editor.workspace-trust]` Section

`[editor]` Section

Key	Description	Default
<code>scrolloff</code>	Number of lines of padding around the edge of the screen when scrolling. <code>0</code> (default) matches the <code>vim/spacemacs</code> presets — H/L reach the literal top/bottom row; set to e.g. <code>5</code> for the scroll margins the <code>helix</code> preset uses	<code>0</code>
<code>mouse</code>	Enable mouse mode	<code>true</code>
<code>mouse-yank-register</code>	Which register to use for mouse yanks.	<code>*</code>

Key	Description	Default
<code>middle-click-paste</code>	Middle click paste support	<code>true</code>
<code>default-yank-register</code>	Default register used for yank/paste	<code>''</code>
<code>scroll-lines</code>	Number of lines to scroll per scroll wheel step	<code>3</code>
<code>shell</code>	Shell to use when running external commands	Unix: <code>["sh", "-c"]</code> Windows: <code>["cmd", "/C"]</code>
<code>line-number</code>	Line number display: <code>absolute</code> simply shows each line's number, while <code>relative</code> shows the distance from the current line. When unfocused or in insert mode, <code>relative</code> will still show absolute line numbers	<code>"absolute"</code>
<code>cursorline</code>	Highlight all lines with a cursor	<code>false</code>
<code>cursorcolumn</code>	Highlight all columns with a cursor	<code>false</code>
<code>highlight-word-under-cursor</code>	Highlight every whole-word occurrence of the word under the primary cursor within the visible viewport (styled with the <code>ui.highlight.word</code> theme scope)	<code>true</code>
<code>continue-comments</code>	if zmax should automatically add a line comment token if you create a new line inside a comment.	<code>true</code>

Key	Description	Default
<code>gutters</code>	Gutters to display: Available are <code>diagnostics</code> and <code>diff</code> and <code>line-numbers</code> and <code>spacer</code> and <code>code-action-hint</code> , note that <code>diagnostics</code> also includes other features like breakpoints, 1-width padding will be inserted if <code>gutters</code> is non-empty	<code>["diagnostics", "spacer", "line-numbers", "spacer", "diff"]</code>
<code>auto-completion</code>	Enable automatic pop up of auto-completion	<code>true</code>
<code>path-completion</code>	Enable filepath completion. Show files and directories if an existing path at the cursor was recognized, either absolute or relative to the current opened document or current working directory (if the buffer is not yet saved). Defaults to true.	<code>true</code>
<code>auto-format</code>	Enable automatic formatting on save ¹	<code>true</code>
<code>idle-timeout</code>	Time in milliseconds since last keypress before idle timers trigger.	<code>250</code>
<code>completion-timeout</code>	Time in milliseconds after typing a word character before completions are shown, set to 5 for instant.	<code>250</code>
<code>preview-completion-insert</code>	Whether to apply completion item instantly when selected	<code>true</code>
<code>completion-trigger-len</code>	The min-length of word under cursor to trigger autocompletion	<code>2</code>

¹In most cases, you also need to enable the `auto-format` setting under `languages.toml`. You can find the reasoning [here](#).

Key	Description	Default
<code>completion-replace</code>	Whether to make completions always replace the entire word and not just the part before the cursor	<code>false</code>
<code>auto-info</code>	Whether to display info boxes	<code>true</code>
<code>true-color</code>	Whether to override automatic detection of terminal truecolor support in the event of a false negative	<code>false</code>
<code>transparent-background</code>	Don't paint the <code>ui.background</code> fill (editor surface, gutter/sign column, and the IDE file-tree sidebar), letting the terminal's own (e.g. translucent/blurred) background show through. Themed chrome — statusline, breadcrumbs, panel status bars — keeps its own colors	<code>false</code>
<code>undercurl</code>	Whether to override automatic detection of terminal undercurl support in the event of a false negative	<code>false</code>
<code>rulers</code>	List of column positions at which to display the rulers. Can be overridden by language specific <code>rulers</code> in <code>languages.toml</code> file	<code>[]</code>

Key	Description	Default
<code>bufferline</code>	Renders a line at the top of the editor displaying open buffers. Can be <code>always</code> , <code>never</code> or <code>multiple</code> (only shown if more than one buffer is in use)	<code>"never"</code>
<code>color-modes</code>	Whether to color the mode indicator with different colors depending on the mode itself	<code>false</code>
<code>text-width</code>	Maximum line length. Used for the <code>:reflow</code> command and soft-wrapping if <code>soft-wrap.wrap-at-text-width</code> is set	<code>80</code>
<code>workspace-lsp-roots</code>	Directories relative to the workspace root that are treated as LSP roots. Should only be set in <code>.zmax/config.toml</code>	<code>[]</code>
<code>default-line-ending</code>	The line ending to use for new documents. Can be <code>native</code> , <code>lf</code> , <code>crlf</code> , <code>ff</code> , <code>cr</code> or <code>nel</code> . <code>native</code> uses the platform's native line ending (<code>crlf</code> on Windows, otherwise <code>lf</code>).	<code>"native"</code>
<code>insert-final-newline</code>	Whether to automatically insert a trailing line-ending on write if missing	<code>true</code>

Key	Description	Default
<code>atomic-save</code>	Whether to use atomic operations to write documents to disk. This prevents data loss if the editor is interrupted while writing the file, but it stages the old contents in a sibling <code><file>XXXXXX.bck</code> file while the write runs — which confuses file watchers and leaves that file behind if <code>zmax</code> dies mid-write. Off by default: writes go straight to the file.	<code>false</code>
<code>backup</code>	Vim <code>backup</code> : keep a copy of a file's previous contents before overwriting it on write.	<code>false</code>
<code>backup-ext</code>	Vim <code>backupext</code> : suffix appended to the backup's name.	<code>"~"</code>
<code>backup-dir</code>	Vim <code>backupdir</code> : comma-separated directories hosting the backup copy; the first non-empty entry wins, <code>~/</code> is expanded, and the file's whole path is flattened into the name (each separator becomes <code>%</code>) so copies from different projects can't collide. Set it to <code>" "</code> to put the backup beside the file, vim-style.	<code>"~/ .zmax/backup"</code>
<code>backup-skip</code>	Vim <code>backupskip</code> : comma-separated globs (<code>*/?</code>); a matching file is written without a backup.	<code>" "</code>

Key	Description	Default
<code>swapfile</code>	Vim <code>swapfile</code> : keep a recovery swap file of unsaved changes and warn when one already exists on open.	<code>false</code>
<code>swap- directory</code>	Vim <code>directory</code> : where swap files live, with the same ~ expansion and path flattening as <code>backup-dir</code> . Set it to "" to keep them beside the edited file as dotfiles.	<code>"~/ .zmax/swap"</code>
<code>undofile</code>	Vim <code>undofile</code> : persist undo history to disk on write and reload it on open.	<code>false</code>
<code>undo-dir</code>	Vim <code>undodir</code> : where undo histories live (named by a hash of the file's canonical path).	<code>"~/ .zmax/undo"</code>
<code>patchmode</code>	Vim <code>patchmode</code> : when non-empty (e.g. <code>".orig"</code>), the <i>first</i> write of a file keeps its original contents as <code><file><patchmode></code> beside the file , so a patch can be made against the untouched version.	<code>""</code>
<code>trim-final- newlines</code>	Whether to automatically remove line-endings after the final one on write	<code>false</code>
<code>trim- trailing- whitespace</code>	Whether to automatically remove whitespace preceding line endings on write	<code>false</code>
<code>popup-border</code>	Draw border around <code>popup</code> , <code>menu</code> , <code>all</code> , or <code>none</code>	<code>"none"</code>

Key	Description	Default
<code>indent-heuristic</code>	How the indentation for a newly inserted line is computed: <code>simple</code> just copies the indentation level from the previous line, <code>tree-sitter</code> computes the indentation based on the syntax tree and <code>hybrid</code> combines both approaches. If the chosen heuristic is not available, a different one will be used as a fallback (the fallback order being <code>hybrid</code> -> <code>tree-sitter</code> -> <code>simple</code>).	"hybrid"
<code>jump-label-alphabet</code>	The characters that are used to generate two character jump labels. Characters at the start of the alphabet are used first.	"abcdefghijklmnopqrstuvwxyz"
<code>end-of-line-diagnostics</code>	Minimum severity of diagnostics to render at the end of the line. Set to <code>disable</code> to disable entirely. Refer to the setting about <code>inline-diagnostics</code> for more details	"hint"
<code>clipboard-provider</code>	Which API to use for clipboard interaction. One of <code>pasteboard</code> (MacOS), <code>wayland</code> , <code>x-clip</code> , <code>x-sel</code> , <code>win32-yank</code> , <code>termux</code> , <code>tmux</code> , <code>windows</code> , <code>termcode</code> , <code>none</code> , or a custom command set.	Platform and environment specific.
<code>editor-config</code>	Whether to read settings from <code>EditorConfig</code> files	true

Key	Description	Default
<code>rainbow-brackets</code>	Whether to render rainbow colors for matching brackets. Requires tree-sitter <code>rainbows.scm</code> queries for the language.	<code>false</code>
<code>kitty-keyboard-protocol</code>	Whether to enable Kitty Keyboard Protocol. Can be <code>enabled</code> , <code>disabled</code> or <code>auto</code>	<code>"auto"</code>
<code>startup</code>	What to open when launched with no file arguments. <code>startify</code> shows the start screen, <code>recent</code> opens the most-recently-used file, <code>session</code> restores the previous session's tabs and cursor, <code>file</code> opens <code>startup-file</code> . Each falls back to <code>startify</code> when its target is unavailable.	<code>"startify"</code>
<code>startup-file</code>	File to open on launch when <code>startup</code> is <code>"file"</code> . Ignored otherwise.	<code>" "</code>

Key	Description	Default
<code>sync-zwire-theme</code>	Bidirectionally sync the colorscheme with the <code>zwire</code> terminal host. When on, <code>zmax</code> reads <code>~/.zwire/global.toml</code> (<code>[theme]</code> <code>scheme + [theme.ui] light</code>) and applies the matching <code>zgui-<scheme> / zgui-<scheme>-light</code> theme at startup, on <code>:config-reload</code>, and live via a filesystem watcher on <code>global.toml</code> (applied immediately on change, no keypress needed) — overriding the configured/persisted theme. Conversely, committing a <code>zgui-*</code> theme in <code>zmax (:theme, the picker, :theme-toggle)</code> writes the scheme back into <code>global.toml</code>, which <code>zwire</code> fans out to the browser/HUD. Non-<code>zgui-*</code> themes are never pushed to <code>zwire</code>.	<code>false</code>

`[editor.clipboard-provider]` Section

`Zmax` can be configured either to use a builtin clipboard configuration or to use a provided command.

For instance, setting it to use OSC 52 termcodes, the configuration would be:

```
[editor]
clipboard-provider = "termcode"
```

Alternatively, `Zmax` can be configured to use arbitrary commands for clipboard integration:

```
[editor.clipboard-provider.custom]
yank = { command = "cat", args = ["test.txt"] }
paste = { command = "tee", args = ["test.txt"] }
primary-yank = { command = "cat", args = ["test-primary.txt"] } # optional
primary-paste = { command = "tee", args = ["test-primary.txt"] } # optional
```

For custom commands the contents of the yank/paste is communicated over stdin/stdout.

[editor.statusline] Section

Allows configuring the statusline at the bottom of the editor.

The configuration distinguishes between three areas of the status line:

```
[ ... .. LEFT ... .. | ... .. .. CENTER ... .. .. | ... .. RIGHT ... .. ]
```

Statusline elements can be defined as follows:

```
[editor.statusline]
left = ["mode", "spinner"]
center = ["file-name"]
right = ["diagnostics", "selections", "position", "file-encoding", "file-line-ending", "file-type"]
separator = "|"
mode.normal = "NORMAL"
mode.insert = "INSERT"
mode.select = "SELECT"
diagnostics = ["warning", "error"]
workspace-diagnostics = ["warning", "error"]
```

The [editor.statusline] key takes the following sub-keys:

Key	Description	Default
left	A list of elements aligned to the left of the statusline	["mode", "spinner", "file-name", "read-only-indicator", "file-modification-indicator"]
center	A list of elements aligned to the middle of the statusline	[]
right	A list of elements aligned to the right of the statusline	["diagnostics", "selections", "register", "position", "file-encoding"]
separator	The character used to separate elements in the statusline	" "
mode.normal	The text shown in the <code>mode</code> element for normal mode	"NOR"
mode.insert	The text shown in the <code>mode</code> element for insert mode	"INS"
mode.select	The text shown in the <code>mode</code> element for select mode	"SEL"
diagnostics	A list of severities which are displayed for the current buffer	["warning", "error"]
workspace-diagnostics	A list of severities which are displayed for the workspace	["warning", "error"]

The following statusline elements can be configured:

Key	Description
mode	The current editor mode (<code>mode.normal</code> / <code>mode.insert</code> / <code>mode.select</code>)
spinner	A progress spinner indicating LSP activity
file-name	The path/name of the opened file

Key	Description
<code>file-absolute-path</code>	The absolute path/name of the opened file
<code>file-base-name</code>	The basename of the opened file
<code>current-working-directory</code>	The current working directory
<code>file-modification-indicator</code>	The indicator to show whether the file is modified (a <code>[+]</code> appears when there are unsaved changes)
<code>file-encoding</code>	The encoding of the opened file if it differs from UTF-8
<code>file-line-ending</code>	The file line endings (CRLF or LF)
<code>file-indent-style</code>	The file indentation style
<code>read-only-indicator</code>	An indicator that shows <code>[readonly]</code> when a file cannot be written
<code>total-line-numbers</code>	The total line numbers of the opened file
<code>file-type</code>	The type of the opened file
<code>diagnostics</code>	The number of warnings and/or errors
<code>workspace-diagnostics</code>	The number of warnings and/or errors on workspace
<code>selections</code>	The primary selection index out of the number of active selections
<code>primary-selection-length</code>	The number of characters currently in primary selection
<code>position</code>	The cursor position
<code>position-percentage</code>	The cursor position as a percentage of the total number of lines
<code>separator</code>	The string defined in <code>editor.statusline.separator</code> (defaults to " ")
<code>spacer</code>	Inserts a space between elements (multiple/contiguous spacers may be specified)
<code>version-control</code>	The current branch name or detached commit hash of the opened workspace
<code>register</code>	The current selected register
<code>code-action-hint</code>	Indicator for when code actions are available

[`editor.lsp`] Section

Key	Description	Default
<code>enable</code>	Enables LSP integration. Setting to false will completely disable language servers regardless of language settings.	<code>true</code>
<code>display-messages</code>	Display LSP <code>window/showMessage</code> messages below statusline ²	<code>true</code>
<code>display-progress-messages</code>	Display LSP progress messages below statusline ³	<code>false</code>
<code>auto-signature-help</code>	Enable automatic popup of signature help (parameter hints)	<code>true</code>

²By default, a progress spinner is shown in the statusline beside the file path.

³By default, a progress spinner is shown in the statusline beside the file path.

Key	Description	Default
<code>auto-document-highlight</code>	Automatically highlight symbol references at the cursor	<code>false</code>
<code>display-inlay-hints</code>	Display inlay hints ⁴	<code>false</code>
<code>inlay-hints-length-limit</code>	Maximum displayed length (non-zero number) of inlay hints	Unset by default
<code>display-color-swatches</code>	Show color swatches next to colors	<code>true</code>
<code>display-signature-help-docs</code>	Display docs under signature help popup	<code>true</code>
<code>snippets</code>	Enables snippet completions. Requires a server restart (<code>:lsp-restart</code>) to take effect after <code>:config-reload/:set</code> . Include declaration in the goto references popup.	<code>true</code>
<code>goto-reference-include-declaration</code>		<code>true</code>

[`editor.cursor-shape`] Section

Defines the shape of cursor in each mode. Valid values for these options are `block`, `bar`, `underline`, or `hidden`.

🕒 Due to limitations of the terminal environment, only the primary cursor can change shape.

Key	Description	Default
<code>normal</code>	Cursor shape in <code>normal mode</code>	<code>"block"</code>
<code>insert</code>	Cursor shape in <code>insert mode</code>	<code>"block"</code>
<code>select</code>	Cursor shape in <code>select mode</code>	<code>"block"</code>

[`editor.file-picker`] Section

Set options for file picker and global search. Ignoring a file means it is not visible in the Zmax file picker and global search.

All git related options are only enabled in a git repository.

Key	Description	Default
<code>hidden</code>	Enables ignoring hidden files	<code>true</code>
<code>follow-symlinks</code>	Follow symlinks instead of ignoring them	<code>true</code>
<code>deduplicate-links</code>	Ignore symlinks that point at files already shown in the picker	<code>true</code>
<code>parents</code>	Enables reading ignore files from parent directories	<code>true</code>

⁴You may also have to activate them in the language server config for them to appear, not just in Zmax. Inlay hints in Zmax are still being improved on and may be a little bit laggy/janky under some circumstances. Please report any bugs you see so we can fix them!

Key	Description	Default
<code>ignore</code>	Enables reading <code>.ignore</code> files	<code>true</code>
<code>git-ignore</code>	Enables reading <code>.gitignore</code> files	<code>true</code>
<code>git-global</code>	Enables reading global <code>.gitignore</code> , whose path is specified in git's config: <code>core.excludesfile</code> option	<code>true</code>
<code>git-exclude</code>	Enables reading <code>.git/info/exclude</code> files	<code>true</code>
<code>max-depth</code>	Set with an integer value for maximum depth to recurse	Unset by default

Ignore files can be placed locally as `.ignore` or put in your home directory as `~/.ignore`. They support the usual ignore and negative ignore (unignore) rules used in `.gitignore` files.

Additionally, you can use Zmax-specific ignore files by creating a local `.zmax/ignore` file in the current workspace or a global `ignore` file located in your Zmax config directory: - Linux and Mac: `~/.zmax/ignore` - Windows: `%AppData%\zmax\ignore`

Example:

```
# unignore in file picker and global search
!.github/
!.gitignore
!.gitattributes
```

[`editor.file-explorer`] Section

In addition to the options for the file picker and global search, a similar set of options is presented to configure the file explorer separately. However, unlike the file picker, the defaults are set to avoid ignoring most files.

Note that the ignore files consulted by the file explorer when `ignore` is set to true are the same ones used by the file picker, including the aforementioned Zmax-specific ignore files.

Key	Description	Default
<code>hidden</code>	Enables ignoring hidden files	<code>false</code>
<code>follow-symlinks</code>	Follow symlinks instead of ignoring them	<code>false</code>
<code>parents</code>	Enables reading ignore files from parent directories	<code>false</code>
<code>ignore</code>	Enables reading <code>.ignore</code> files	<code>false</code>
<code>git-ignore</code>	Enables reading <code>.gitignore</code> files	<code>false</code>

Key	Description	Default
<code>git-global</code>	Enables reading global <code>.gitignore</code> , whose path is specified in git's config: <code>core.excludesfile</code> option	<code>false</code>
<code>git-exclude</code>	Enables reading <code>.git/info/exclude</code> files	<code>false</code>
<code>flatten-dirs</code>	Enables flattening single child directories	<code>true</code>

[`editor.buffer-picker`] Section

Set options for buffer picker.

Key	Description	Default
<code>start-position</code>	Controls behavior for which buffer is initially selected	<code>current</code>

Example

```
[editor.buffer-picker]
start-position = "previous"
```

[`editor.auto-pairs`] Section

Enables automatic insertion of pairs to parentheses, brackets, etc. Can be a simple boolean value, or a specific mapping of pairs of single characters.

To disable auto-pairs altogether, set `auto-pairs` to `false`:

```
[editor]
auto-pairs = false # defaults to `true`
```

The default pairs are `(){}""'</code>`, but these can be customized by setting `auto-pairs` to a TOML table:

```
[editor.auto-pairs]
'(' = ')'
'{' = '}'
'[' = ']'
'"' = '"'
"'" = "'"
'<' = '>'
```

Additionally, this setting can be used in a language config. Unless the editor setting is `false`, this will override the editor config in documents with this language.

Example `languages.toml` that adds `<>` and removes `' '`

```
[[language]]
name = "rust"

[language.auto-pairs]
'(' = ')'
'{' = '}'
'[' = ']'
'"' = '"'
'`' = '`'
'<' = '>'
```

[editor.auto-save] Section

Control auto save behavior.

Key	Description	Default
<code>focus-lost</code>	Enable automatic saving on the focus moving away from zmax. Requires <code>focus event support</code> from your terminal	<code>false</code>
<code>after-delay.enable</code>	Enable automatic saving after <code>auto-save.after-delay.timeout</code> milliseconds have passed since last edit.	<code>false</code>
<code>after-delay.timeout</code>	Time in milliseconds since last edit before auto save timer triggers.	<code>3000</code>

[editor.search] Section

Search specific options.

Key	Description	Default
<code>smart-case</code>	Enable smart case regex searching (case-insensitive unless pattern contains upper case characters)	<code>true</code>
<code>wrap-around</code>	Whether the search should wrap after depleting the matches	<code>true</code>

[editor.whitespace] Section

Options for rendering whitespace with visible characters. Use `:set whitespace.render all` to temporarily enable visible whitespace.

Key	Description	Default
<code>render</code>	Whether to render whitespace. May either be <code>all</code> or <code>none</code> , or a table with sub-keys <code>space</code> , <code>nbspace</code> , <code>nnbspace</code> , <code>tab</code> , and <code>newline</code>	<code>"none"</code>
<code>characters</code>	Literal characters to use when rendering whitespace. Sub-keys may be any of <code>tab</code> , <code>space</code> , <code>nbspace</code> , <code>nnbspace</code> , <code>newline</code> or <code>tabpad</code>	See example below

Example

```
[editor.whitespace]
render = "all"
# or control each character
[editor.whitespace.render]
space = "all"
tab = "all"
nbspace = "none"
nnbspace = "none"
newline = "none"

[editor.whitespace.characters]
space = "."
nbspace = "␣"
nnbspace = "␣"
tab = "→"
newline = "↵"
tabpad = "." # Tabs will look like "→..." (depending on tab width)
```

[editor.indent-guides] Section

Options for rendering vertical indent guides.

Key	Description	Default
<code>render</code>	Whether to render indent guides	<code>false</code>
<code>character</code>	Literal character to use for rendering the indent guide	<code>" "</code>
<code>skip-levels</code>	Number of indent levels to skip	<code>0</code>

Example:

```
[editor.indent-guides]
render = true
character = "|" # Some characters that work well: "|", "⋈", "⋊"
skip-levels = 1
```

[editor.gutters] Section

For simplicity, `editor.gutters` accepts an array of gutter types, which will use default settings for all gutter components.

```
[editor]
gutters = ["diff", "diagnostics", "line-numbers", "spacer"]
```

To customize the behavior of gutters, the `[editor.gutters]` section must be used. This section contains top level settings, as well as settings for specific gutter components as subsections.

Key	Description	Default
<code>layout</code>	A vector of gutters to display	<code>["blame", "diagnostics", "marks", "signs", "spacer", "line-numbers", "spacer", "diff"]</code>

Available gutter types: `diagnostics`, `line-numbers`, `spacer`, `diff`, `code-action-hint`, `marks`, `signs`, and `blame`. The `blame` gutter is a JetBrains-style “Annotate” column (author + relative time per line); it takes no space until toggled on with `SPC g B (toggle_blame_annotate)`. The `signs` gutter shows vim signs placed with `:sign place`; it takes no space until a sign is placed in the file (vim’s `signcolumn=auto`).

Example:

```
[editor.gutters]
layout = ["diff", "diagnostics", "line-numbers", "spacer"]
```

[editor.gutters.line-numbers] Section

Options for the line number gutter

Key	Description	Default
<code>min-width</code>	The minimum number of characters to use	<code>3</code>

Example:

```
[editor.gutters.line-numbers]
min-width = 1
```

[editor.gutters.diagnostics] Section

Currently unused

[editor.gutters.diff] Section

The `diff` gutter option displays colored bars indicating whether a `git` diff represents that a line was added, removed or changed. These colors are controlled by the theme attributes `diff.plus`, `diff.minus` and `diff.delta`.

Other diff providers will eventually be supported by a future plugin system.

There are currently no options for this section.

[editor.gutters.spacer] Section

Currently unused

[editor.gutters.code-action-hint] Section

The `code-action-hint` gutter option displays an indicator for whether a code action is available at current selection.

There are currently no options for this section.

[editor.soft-wrap] Section

Options for soft wrapping lines that exceed the view width:

Key	Description	Default
<code>enable</code>	Whether soft wrapping is enabled.	<code>false</code>
<code>max-wrap</code>	Maximum free space left at the end of the line.	<code>20</code>

Key	Description	Default
<code>max-indent-retain</code>	Maximum indentation to carry over when soft wrapping a line.	<code>40</code>
<code>wrap-indicator</code>	Text inserted before soft wrapped lines, highlighted with <code>ui.virtual.wrap</code>	<code>"↵ "</code>
<code>wrap-at-text-width</code>	Soft wrap at <code>text-width</code> instead of using the full viewport size.	<code>false</code>

Example:

```
[editor.soft-wrap]
enable = true
max-wrap = 25 # increase value to reduce forced mid-word wrapping
max-indent-retain = 0
wrap-indicator = "" # set wrap-indicator to "" to hide it
```

[editor.smart-tab] Section

Options for navigating and editing using tab key.

Key	Description	Default
<code>enable</code>	If set to true, then when the cursor is in a position with non-whitespace to its left, instead of inserting a tab, it will run <code>move_parent_node_end</code> . If there is only whitespace to the left, then it inserts a tab as normal. With the default bindings, to explicitly insert a tab character, press Shift-tab.	<code>true</code>
<code>supersede-menu</code>	Normally, when a menu is on screen, such as when auto complete is triggered, the tab key is bound to cycling through the items. This means when menus are on screen, one cannot use the tab key to trigger the <code>smart-tab</code> command. If this option is set to true, the <code>smart-tab</code> command always takes precedence, which means one cannot use the tab key to cycle through menu items. One of the other bindings must be used instead, such as arrow keys or <code>C-n/C-p</code> .	<code>false</code>

Due to lack of support for S-tab in some terminals, the default keybindings don't fully embrace smart-tab editing experience. If you enjoy smart-tab navigation and a terminal that supports the [Enhanced Keyboard protocol](#), consider setting extra keybindings:

```
[keys.normal]
tab = "move_parent_node_end"
S-tab = "move_parent_node_start"

[keys.insert]
S-tab = "move_parent_node_start"

[keys.select]
tab = "extend_parent_node_end"
S-tab = "extend_parent_node_start"
```

[editor.inline-diagnostics] Section

Options for rendering diagnostics inside the text like shown below

```
fn main() {
  let foo = bar;
    └─ no such value in this scope
}
```

Key	Description	Default
<code>cursor-line</code>	The minimum severity that a diagnostic must have to be shown inline on the line that contains the primary cursor. Set to <code>disable</code> to not show any diagnostics inline. This option does not have any effect when in insert-mode and will only take effect 350ms after moving the cursor to a different line.	"warning"
<code>other-lines</code>	The minimum severity that a diagnostic must have to be shown inline on a line that does not contain the cursor-line. Set to <code>disable</code> to not show any diagnostics inline.	"disable"
<code>prefix-len</code>	How many horizontal bars — are rendered before the diagnostic text.	1
<code>max-wrap</code>	Equivalent of the <code>editor.soft-wrap.max-wrap</code> option for diagnostics.	20
<code>max-diagnostics</code>	Maximum number of diagnostics to render inline for a given line	10

The allowed values for `cursor-line` and `other-lines` are: `error`, `warning`, `info`, `hint`.

The (first) diagnostic with the highest severity that is not shown inline is rendered at the end of the line (as long as its severity is higher than the `end-of-line-diagnostics` config option):

```
fn main() {
  let baz = 1;
  let foo = bar; a local variable with a similar name exists: baz
    └─ no such value in this scope
}
```

[`editor.word-completion`] Section

Options for controlling completion of words from open buffers.

Key	Description	Default
<code>enable</code>	Whether word completion is enabled	<code>true</code>
<code>trigger-length</code>	Number of word characters to type before triggering completion	7

Example:

```
[editor.word-completion]
enable = true
# Set the trigger length lower so that words are completed more often
trigger-length = 4
```

[`editor.workspace-trust`] Section

Controls implicit workspace trust. See the [workspace trust](#) chapter for the full feature.

Key	Description	Default
<code>level</code>	The default level of trust for every workspace.	<code>"servers"</code>
<code>prompt</code>	Whether to show a modal when opening a file in an untrusted workspace.	<code>true</code>
<code>trusted</code>	Glob patterns whose matching workspaces are trusted without a grant.	<code>[]</code>

Example:

```
[editor.workspace-trust]
# Even if `false`, the statusline `[△]` indicator is still shown.
prompt = false

# "none":    prompt for every workspace.
# "servers": trust LSP and DAP launches but still gate local config and git;
#            .zmax/config.toml, .zmax/languages.toml, etc. need :workspace-trust.
# "insecure": trust everything (discouraged).
level = "servers"

# Discouraged: skips .zmax/ change detection and trusts anything that lands
# under a matching path. `~` and environment variables are expanded.
trusted = ["~/src/github.com/me/*"]
```

Typable Commands

Commands run from the `:` command line, with their aliases.

Name	Description
<code>:terminal</code> , <code>:term</code>	Open an integrated terminal (PTY shell) running \$SHELL.
<code>:ide</code> , <code>:workbench</code>	Enter IDE mode (file-tree sidebar + panels, like <code>--ide</code> / F2).
<code>:diff</code> , <code>:gdiff</code>	Open a read-only side-by-side diff of the buffer vs. its git HEAD version.
<code>:diff-buffer-with-file</code>	Show a unified diff of the buffer's contents vs. its file on disk (emacs diff-buffer-with-file).
<code>:reveal</code> , <code>:open-repo</code>	Open this repository's homepage (GitHub/GitLab/Bitbucket/...) in the browser.
<code>:compare-ref</code> , <code>:compare-branch</code>	Diff the buffer against its version at a git ref (JetBrains Compare with Branch).
<code>:merge</code> , <code>:resolve</code>	Resolve the buffer's git merge conflicts in a 3-pane (ours/result/theirs) view.
<code>:magit</code> , <code>:git</code> , <code>:gst</code>	Open the Magit-style git status (stage/unstage/discard/commit changes by section).
<code>:hex</code> , <code>:hexview</code> , <code>:hexedit</code>	Open a read-only xxd-style hex viewer of a file's raw bytes (optional path; defaults to the buffer's file).
<code>:snippets</code> , <code>:snip</code>	Open the user snippet library editor (create/edit/delete reusable snippets).
<code>:org-cycle</code> , <code>:org-fold</code>	Toggle a fold over the current org heading's subtree (TAB-style outline cycling).
<code>:org-todo</code>	Cycle the current org heading's TODO keyword: none -> TODO -> DONE -> none.
<code>:org-promote</code>	Promote the current org heading one level (remove a leading star).
<code>:org-demote</code>	Demote the current org heading one level (add a leading star).
<code>:org-move-subtree-down</code> , <code>:org-metadown</code>	Move the org subtree at point down past its next sibling (emacs org-move-subtree-down).
<code>:org-move-subtree-up</code> , <code>:org-metaup</code>	Move the org subtree at point up past its previous sibling (emacs org-move-subtree-up).
<code>:org-schedule</code>	Add/update a SCHEDULED: timestamp on the heading at point: <code>:org-schedule 2026-07-15</code> (emacs org-schedule).
<code>:org-deadline</code>	Add/update a DEADLINE: timestamp on the heading at point: <code>:org-deadline 2026-07-15</code> (emacs org-deadline).
<code>:org-next-heading</code>	Move the cursor to the next org heading line.
<code>:org-prev-heading</code>	Move the cursor to the previous org heading line.
<code>:org-fold-all</code>	Fold every org heading subtree in the buffer.
<code>:org-unfold-all</code>	Unfold every fold in the buffer.
<code>:org-agenda</code> , <code>:agenda</code>	Open the org agenda: TODO/DONE items across open .org buffers and *.org files under the working directory, grouped by scheduled/deadline date.
<code>:org-remove-file</code>	Take the current file off the agenda file list (emacs org-remove-file).
<code>:org-agenda-file-to-front</code>	Move/add the current file to the top of the agenda file list; <code>end</code> adds it to the end instead (emacs org-agenda-file-to-front).

Name	Description
<code>:org-priority</code>	Cycle the current org heading's priority cookie: none -> [#A] -> [#B] -> [#C] -> none.
<code>:org-capture, :capture</code>	Prompt for a line of text and append it as a '* TODO ' entry to an inbox org file (default /inbox.org, or an explicit path argument).
<code>:exit, :x, :xit</code>	Write changes to disk if the buffer is modified and then quit. Accepts an optional path (:exit some/path.txt).
<code>:exit!, :x!, :xit!</code>	Force write changes to disk, creating necessary subdirectories, if the buffer is modified and then quit. Accepts an optional path (:exit! some/path.txt).
<code>:quit, :q</code>	Close the current view.
<code>:detach</code>	Detach the TUI and return to the shell, leaving the editor stopped in the background (neovim :detach).
<code>:help, :h</code>	Open the inline Help browser (searchable: commands, keybindings, topics).
<code>:wc, :words, :count</code>	Show document line/word/char counts (and selection stats).
<code>:blame</code>	Show git blame for the current line in the status bar.
<code>:reopen, :reopen-closed</code>	Reopen the most recently closed file.
<code>:zen</code>	Toggle the IDE workbench (Zen / focus mode).
<code>:emmet, :zencode</code>	Expand the emmet/zen HTML abbreviation before the cursor.
<code>:quit!, :q!</code>	Force close the current view, ignoring unsaved changes.
<code>:open, :o, :edit, :e, :ex</code>	Open a file from disk into the current view (vim :edit).
<code>:visual, :vi, :vis</code>	Leave Ex mode (gQ) and go back to Normal; outside Ex mode, open a file like :edit (vim :visual).
<code>:args, :ar, :argglobal, :argg, :arglocal, :argl</code>	Show the argument list, or set it to the given files and edit the first (vim :args).
<code>:argadd, :arga</code>	Add files to the argument list after the current entry (vim :argadd).
<code>:argedit, :arge</code>	Add a file to the argument list and edit it (vim :argedit).
<code>:argdelete, :argd</code>	Delete argument-list entries matching the given glob patterns (vim :argdelete).
<code>:argdedupe</code>	Remove duplicate entries from the argument list (vim :argdedupe).
<code>:next, :argnext</code>	Edit the next file in the argument list (vim :next).
<code>:previous, :Next, :prev, :argprev</code>	Edit the previous file in the argument list (vim :previous / :Next).
<code>:wnext, :wn</code>	Write the current buffer, then edit the next file in the argument list (vim :wnext).
<code>:wprevious, :wp, :wNext, :wN</code>	Write the current buffer, then edit the previous file in the argument list (vim :wprevious / :wNext).
<code>:find, :fin</code>	Find a file in the 'path' (buffer dir, cwd, then a recursive cwd walk) and edit it (vim :find).
<code>:sfind, :sf</code>	Split the window and edit a file found in the 'path' (vim :sfind).
<code>:first, :rewind, :rew</code>	Edit the first file in the argument list (vim :first / :rewind).
<code>:view, :vie, :find-file-read-only</code>	Edit a file read-only (vim :view / emacs find-file-read-only).
<code>:sview, :svie, :find-file-read-only-other-window</code>	Split and edit a file read-only (vim :sview / emacs find-file-read-only-other-window).
<code>:autocmd, :au</code>	Register an autocommand: :autocmd {events} {pattern} {command} (:autocmd ! clears the open group, or all).

Name	Description
<code>:augroup</code> , <code>:aug</code> , <code>:augr</code> , <code>:augro</code> , <code>:augrou</code>	Open the autocmd group the following <code>:autocmds</code> join (<code>:augroup</code> <code>END</code> closes it, <code>:augroup! {name}</code> deletes it) (vim <code>:augroup</code>).
<code>:last</code> , <code>:la</code>	Edit the last file in the argument list (vim <code>:last</code>).
<code>:argument</code> , <code>:argu</code>	Edit the Nth file in the argument list (vim <code>:argument</code>).
<code>:snext</code> , <code>:sn</code>	Split the window and edit the next argument (vim <code>:snext</code>).
<code>:sprevious</code> , <code>:sprev</code> , <code>:sNext</code> , <code>:sN</code>	Split the window and edit the previous argument (vim <code>:sprevious</code> / <code>:sNext</code>).
<code>:srewind</code> , <code>:sre</code> , <code>:sfirst</code> , <code>:sfir</code>	Split the window and edit the first argument (vim <code>:srewind</code> / <code>:sfirst</code>).
<code>:slast</code> , <code>:sla</code>	Split the window and edit the last argument (vim <code>:slast</code>).
<code>:sargument</code> , <code>:sa</code>	Split the window and edit the Nth argument (vim <code>:sargument</code>).
<code>:argdo</code>	Run an Ex command on each file in the argument list (vim <code>:argdo</code>).
<code>:all</code> , <code>:sall</code>	Open a window for each file in the argument list (vim <code>:all</code> / <code>:sall</code>).
<code>:compile</code>	Run a shell command and collect its errors into the compilation list (emacs <code>compile</code> / M-x <code>compile</code>).
<code>:projectile-test-project</code> , <code>:test-project</code>	Run the project's test command in the project root (projectile <code>projectile-test-project</code> , spacemacs SPC p T).
<code>:projectile-invalidate-cache</code>	Invalidate the project file cache so the next find-file re-scans (projectile <code>invalidate-cache</code> , spacemacs SPC p I).
<code>:dotnet-build</code>	Build a project or solution, collecting its errors (dotnet.el <code>dotnet-build</code> , spacemacs SPC m p b).
<code>:dotnet-clean</code>	Clean the .NET build output (dotnet.el <code>dotnet-clean</code> , spacemacs SPC m p c).
<code>:dotnet-publish</code>	Publish a .NET project for deployment (dotnet.el <code>dotnet-publish</code> , spacemacs SPC m p p).
<code>:dotnet-restore</code>	Restore the .NET project's dependencies (dotnet.el <code>dotnet-restore</code> , spacemacs SPC m p r s).
<code>:dotnet-run</code>	Build and run the project in a directory, remembering it (dotnet.el <code>dotnet-run</code> , spacemacs SPC m p r r).
<code>:dotnet-run-with-args</code>	Build and run the project with the given arguments (dotnet.el <code>dotnet-run-with-args</code> , spacemacs SPC m p r a).
<code>:dotnet-test</code>	Run a test project's unit tests, remembering it (dotnet.el <code>dotnet-test</code> , spacemacs SPC m p t).
<code>:dotnet-add-package</code>	Add a package reference to the project (dotnet.el <code>dotnet-add-package</code> , spacemacs SPC m p a p).
<code>:dotnet-add-reference</code>	Add a project reference: <code>:dotnet-add-reference {reference} {project}</code> (dotnet.el <code>dotnet-add-reference</code> , spacemacs SPC m p a r).
<code>:dotnet-new</code>	Create a project: <code>:dotnet-new {path} [template] [language]</code> (dotnet.el <code>dotnet-new</code> , spacemacs SPC m p n).
<code>:dotnet-sln-add</code>	Add a project to a solution: <code>:dotnet-sln-add {solution} {project}</code> (dotnet.el <code>dotnet-sln-add</code> , spacemacs SPC m p s a).
<code>:dotnet-sln-remove</code>	Remove a project from a solution: <code>:dotnet-sln-remove {solution} {project}</code> (dotnet.el <code>dotnet-sln-remove</code> , spacemacs SPC m p s r).
<code>:dotnet-sln-list</code>	List the projects a solution holds (dotnet.el <code>dotnet-sln-list</code> , spacemacs SPC m p s l).
<code>:dotnet-sln-new</code>	Create a solution at a path (dotnet.el <code>dotnet-sln-new</code> , spacemacs SPC m p s n).
<code>:dotnet-goto-sln</code>	Open the enclosing .sln file (dotnet.el <code>dotnet-goto-sln</code>).
<code>:dotnet-goto-csproj</code>	Open the enclosing .csproj file (dotnet.el <code>dotnet-goto-csproj</code>).

Name	Description
<code>:dotnet-goto-fsproj</code>	Open the enclosing .fsproj file (dotnet.el dotnet-goto-fsproj).
<code>:recompile</code>	Re-run the last compile command (emacs recompile).
<code>:make</code>	Run <code>make [args]</code> , collect errors into the quickfix list, jump to the first (vim :make).
<code>:lmake, :lmak</code>	Run <code>make [args]</code> and collect its errors into the location list, jumping to the first (vim :lmake).
<code>:tag, :ta</code>	Jump to the ctags definition of {name} from the tags file, pushing the tag stack (vim :tag).
<code>:tselect, :ts</code>	List every matching tag in a picker; select one to jump (vim :tselect).
<code>:tjump, :tj</code>	Jump to the tag if unique, else show the tag picker (vim :tjump).
<code>:stag, :sta</code>	Open the tag's definition in a new horizontal split and jump there (vim :stag).
<code>:ptag, :pt</code>	Show the tag's definition in the preview window, leaving the cursor where it is (vim :ptag).
<code>:tnext, :tn</code>	Jump to the next matching tag (vim :tnext).
<code>:tprevious, :tp, :tNext, :tN</code>	Jump to the previous matching tag (vim :tprevious).
<code>:tfirst, :trewind, :tr</code>	Jump to the first matching tag (vim :tfirst).
<code>:tlast</code>	Jump to the last matching tag (vim :tlast).
<code>:pop, :po</code>	Pop the tag stack, returning to where the last :tag jumped from (vim :pop).
<code>:tags</code>	Show the tag stack depth and the current matching tag (vim :tags).
<code>:messages, :mes</code>	Show the message log — every status/error shown this session (vim :messages).
<code>:Man, :man</code>	Open a man page in the run console (neovim :Man).
<code>:Man-next-manpage, :man-next-manpage</code>	Show the next page of the topic :Man looked up, wrapping at the end (emacs Man-next-manpage).
<code>:Man-previous-manpage, :man-previous-manpage</code>	Show the previous page of the topic :Man looked up, wrapping at the start (emacs Man-previous-manpage).
<code>:redir</code>	Capture message output to a register/file: <code>:redir @a</code>
<code>:arduino-compile, :averify, :arduino-verify</code>	Compile the sketch with arduino-cli for the selected board; errors go to the compilation list (Arduino IDE Verify).
<code>:arduino-upload, :aupload</code>	Compile and flash the sketch to the connected board (arduino-cli compile -upload), live in a terminal panel.
<code>:arduino-monitor, :amonitor, :serial-monitor</code>	Open the serial monitor for the selected port/ baud (arduino-cli monitor).
<code>:arduino-compile-verbose, :arduino-verify-verbose</code>	Verbose compile (<code>arduino-cli compile -v</code>); diagnostics into <i>compilation</i> .
<code>:arduino-compile-clean, :arduino-rebuild</code>	Compile without cached build artifacts (<code>arduino-cli compile --clean</code>).
<code>:arduino-compile-jobs, :arduino-compile-j</code>	Compile with N parallel jobs (<code>arduino-cli compile -j <n></code>).
<code>:arduino-compiledb, :arduino-compilation-database</code>	Generate compile_commands.json for the C/C++ LSP (<code>arduino-cli compile --only-compilation-database</code>).
<code>:arduino-compile-warnings, :arduino-warnings</code>	Compile at a warning level (<code>arduino-cli compile --warnings <none default more all></code>).
<code>:arduino-compile-profile, :arduino-build-profile</code>	Compile using a sketch build profile (<code>arduino-cli compile --profile <name></code>).

Name	Description
<code>:arduino-compile-debug-opt,</code> <code>:arduino-compile-for-debug</code>	Compile with debug-friendly optimization (<code>arduino-cli compile --optimize-for-debug</code>).
<code>:arduino-upload-verify,</code> <code>:arduino-upload-verified</code>	Build + flash, then verify the flashed program (<code>arduino-cli compile --upload --verify</code>).
<code>:arduino-upload-programmer,</code> <code>:arduino-upload-via-programmer</code>	Build + flash through a programmer (<code>arduino-cli compile --upload --programmer <id></code>).
<code>:arduino-upload-dir, :arduino-upload-input-dir</code>	Flash a pre-built binary folder without recompiling (<code>arduino-cli upload --input-dir <dir></code>).
<code>:arduino-upload-file,</code> <code>:arduino-upload-input-file</code>	Flash a specific pre-built binary without recompiling (<code>arduino-cli upload --input-file <file></code>).
<code>:arduino-monitor-raw, :arduino-monitor-noraw</code>	Serial monitor without output transformations (<code>arduino-cli monitor --raw</code>).
<code>:arduino-monitor-timestamp, :arduino-monitor-ts</code>	Serial monitor prefixing each line with a timestamp (<code>arduino-cli monitor --timestamp</code>).
<code>:arduino-board-programmers,</code> <code>:arduino-list-programmers</code>	List programmers the selected board supports (<code>arduino-cli board details --list-programmers</code>).
<code>:arduino-board-list-watch,</code> <code>:arduino-boards-watch</code>	Watch for boards connecting/disconnecting (<code>arduino-cli board list --watch</code>), live in a panel.
<code>:arduino-lib-list-updatable,</code> <code>:arduino-libs-updatable</code>	Installed libraries with a newer version available (<code>arduino-cli lib list --updatable</code>).
<code>:arduino-lib-install-git,</code> <code>:arduino-lib-install-url</code>	Install a library from a git repository (<code>arduino-cli lib install --git-url <url></code>).
<code>:arduino-lib-install-zip,</code> <code>:arduino-lib-install-archive</code>	Install a library from a local .zip (<code>arduino-cli lib install --zip-path <path></code>).
<code>:arduino-compile-quiet, :arduino-compile-silent</code>	Quiet compile, errors only (<code>arduino-cli compile -q</code>).
<code>:arduino-compile-properties,</code> <code>:arduino-build-properties</code>	Dump resolved build properties without building (<code>arduino-cli compile --show-properties</code>).
<code>:arduino-compile-preprocess,</code> <code>:arduino-preprocess</code>	Output the preprocessed sketch (<code>arduino-cli compile --preprocess</code>).
<code>:arduino-compile-dump-profile,</code> <code>:arduino-dump-profile</code>	Print a reproducible build profile for the sketch (<code>arduino-cli compile --dump-profile</code>).
<code>:arduino-compile-board-options,</code> <code>:arduino-board-options</code>	Compile with custom board menu options (<code>arduino-cli compile --board-options <opts></code>).
<code>:arduino-compile-build-property,</code> <code>:arduino-build-property</code>	Compile overriding a build property (<code>arduino-cli compile --build-property <key=value></code>).
<code>:arduino-compile-output-dir,</code> <code>:arduino-compile-out-dir</code>	Compile and save build artifacts to a directory (<code>arduino-cli compile --output-dir <dir></code>).
<code>:arduino-upload-verbose, :arduino-flash-verbose</code>	Verbose build + flash (<code>arduino-cli compile --upload -v</code>).
<code>:arduino-debug-info, :arduino-debug-config</code>	Print the debugger configuration without starting a session (<code>arduino-cli debug --info</code>).
<code>:arduino-debug-programmer,</code> <code>:arduino-debug-via-programmer</code>	Launch the debugger through a programmer (<code>arduino-cli debug --programmer <id></code>).
<code>:arduino-monitor-quiet, :arduino-monitor-silent</code>	Serial monitor suppressing non-error diagnostics (<code>arduino-cli monitor --quiet</code>).
<code>:arduino-monitor-describe,</code> <code>:arduino-monitor-port-info</code>	Describe the port's supported monitor settings (<code>arduino-cli monitor --describe</code>).

Name	Description
<code>:arduino-core-list-updatable,</code> <code>:arduino-cores-updatable</code>	Installed platforms with a newer version available (<code>arduino-cli core list --updatable</code>).
<code>:arduino-core-list-all,</code> <code>:arduino-cores-all</code>	Every installed platform incl. release channels (<code>arduino-cli core list --all</code>).
<code>:arduino-lib-list-all,</code> <code>:arduino-libs-all</code>	Installed libraries across all locations incl. built-in (<code>arduino-cli lib list --all</code>).
<code>:arduino-update-outdated,</code> <code>:arduino-update-show-outdated</code>	Refresh indexes then list upgradable cores/libraries (<code>arduino-cli update --show-outdated</code>).
<code>:arduino-boards-hidden,</code> <code>:arduino-board-listall-hidden</code>	Every known board including platform-hidden variants (<code>arduino-cli board listall --show-hidden</code>).
<code>:arduino-lib-search-names,</code> <code>:arduino-lib-search-name</code>	A names-only library search (<code>arduino-cli lib search <query> --names</code>).
<code>:arduino-sketch-archive-full,</code> <code>:arduino-archive-full</code>	Archive the sketch with its build output (<code>arduino-cli sketch archive --include-build-dir</code>).
<code>:arduino-lib-install-no-deps,</code> <code>:arduino-lib-install-nodeps</code>	Install a library without its declared dependencies (<code>arduino-cli lib install <name> --no-deps</code>).
<code>:arduino-board-details-full,</code> <code>:arduino-board-info-full</code>	The complete board detail dump for the selected FQBN (<code>arduino-cli board details --full</code>).
<code>:arduino-boards,</code> <code>:arduino-board</code>	Pick the target board (FQBN) from installed platforms (Arduino IDE board selector).
<code>:arduino-ports,</code> <code>:arduino-port</code>	Pick the serial port from connected devices (<code>arduino-cli board list</code>).
<code>:arduino-lib-search,</code> <code>:arduino-lib</code>	Search the Arduino library index and install the pick (Arduino IDE Library Manager).
<code>:arduino-core-install,</code> <code>:arduino-core</code>	Install a board-support core, e.g. <code>arduino:avr</code> (Arduino IDE Boards Manager).
<code>:pio-build,</code> <code>:pio-run,</code> <code>:platformio-build</code>	Build the PlatformIO project (<code>pio run</code>); errors go to the compilation list.
<code>:pio-upload,</code> <code>:platformio-upload</code>	Build and upload the PlatformIO project (<code>pio run -t upload</code>), live in a terminal panel.
<code>:pio-exec,</code> <code>:platformio-exec,</code> <code>:pio-run-exec</code>	Build and run the native program (<code>pio run -t exec</code>); args pass through as <code>--program-arg</code> .
<code>:pio-upload-monitor,</code> <code>:platformio-upload-monitor,</code> <code>:pio-flash-monitor</code>	Build, flash, then open the serial monitor in one shot (<code>pio run -t upload -t monitor</code>); optional port overrides <code>--monitor-port</code> .
<code>:pio-monitor,</code> <code>:platformio-monitor</code>	Open the PlatformIO serial monitor (<code>pio device monitor</code>).
<code>:pio-devices,</code> <code>:pio-device-list</code>	Pick a serial port from <code>pio device list</code> and set it for this project.
<code>:pio-device-logical,</code> <code>:platformio-device-logical</code>	List logical (disk) devices (<code>pio device list --logical</code>).
<code>:pio-device-mdns,</code> <code>:platformio-device-mdns</code>	List multicast-DNS / network (OTA) devices (<code>pio device list --mdns</code>).
<code>:pio-device-serial,</code> <code>:platformio-device-serial</code>	List serial ports only (<code>pio device list --serial</code>).
<code>:pio-pkg-list-libraries,</code> <code>:platformio-pkg-list-libraries,</code> <code>:pio-libs-only</code>	List installed libraries only (<code>pio pkg list --only-libraries</code>).
<code>:pio-pkg-list-platforms,</code> <code>:platformio-pkg-list-platforms</code>	List installed platforms only (<code>pio pkg list --only-platforms</code>).
<code>:pio-pkg-list-tools,</code> <code>:platformio-pkg-list-tools</code>	List installed tool packages only (<code>pio pkg list --only-tools</code>).
<code>:pio-test-json,</code> <code>:platformio-test-json</code>	Unit-test results as JSON (<code>pio test --json-output</code>), shown in a scratch buffer.

Name	Description
<code>:pio-project-config-json,</code> <code>:platformio-project-config-json</code>	Computed project configuration as JSON (<code>pio project config --json-output</code>).
<code>:pio-project-metadata-json,</code> <code>:platformio-project-metadata-json</code>	IDE/LSP metadata as JSON (<code>pio project metadata --json-output</code>).
<code>:pio-system-info-json,</code> <code>:platformio-system-info-json</code>	System-wide PlatformIO information as JSON (<code>pio system info --json-output</code>).
<code>:pio-pkg-search-page,</code> <code>:platformio-pkg-search-page</code>	A specific page of registry search results (<code>pio pkg search <query> --page <n></code>).
<code>:pio-pkg-unpublish-undo,</code> <code>:platformio-pkg-unpublish-undo</code>	Restore a previously unpublished package (<code>pio pkg unpublish <pkg> --undo</code>).
<code>:pio-init-no-deps, :platformio-init-no-deps</code>	Scaffold a project without installing dependencies (<code>pio project init --board <board> --no-install-dependencies</code>).
<code>:pio-home, :platformio-home</code>	Launch the PlatformIO Home GUI (<code>pio home</code>), live in a terminal panel; extra args tune the server.
<code>:pio-init, :platformio-init</code>	Scaffold a PlatformIO project for a board (<code>pio project init --board <id></code>).
<code>:embedded-baud, :serial-baud</code>	Set the serial monitor baud rate for this project (e.g. 9600, 115200).
<code>:arduino-plotter, :serial-plotter</code>	Live-graph the numbers streaming from the serial port (Arduino IDE Serial Plotter).
<code>:pio-plotter, :platformio-plotter</code>	Live-graph the PlatformIO serial monitor output (serial plotter).
<code>:serial-term, :serial-terminal</code>	Open a terminal emulator on a serial port (emacs <code>serial-term</code>): <code><port> [speed] [line]</code> .
<code>:arduino-new-sketch, :arduino-sketch-new</code>	Scaffold a new sketch (<code>arduino-cli sketch new</code>) and open its .ino (Arduino IDE New Sketch).
<code>:arduino-compile-export, :aexport,</code> <code>:arduino-export</code>	Compile and export the built binaries to the sketch folder (Arduino IDE Export Compiled Binary).
<code>:arduino-burn-bootloader, :arduino-bootloader</code>	Flash the bootloader to the selected board (Arduino IDE Burn Bootloader), live in a terminal panel.
<code>:arduino-board-info, :arduino-board-details</code>	Print the selected board's specs and menu options (Arduino IDE Get Board Info).
<code>:arduino-core-search, :arduino-boards-manager</code>	Search the Boards Manager index for a core (Arduino IDE Boards Manager).
<code>:arduino-core-list, :arduino-cores</code>	List installed board-support cores (Boards Manager, installed tab).
<code>:arduino-core-uninstall</code>	Uninstall a board-support core, e.g. <code>arduino:avr</code> .
<code>:arduino-core-update-index,</code> <code>:arduino-update-index</code>	Refresh the Boards Manager package index (<code>arduino-cli core update-index</code>).
<code>:arduino-core-upgrade</code>	Upgrade all installed board-support cores (<code>arduino-cli core upgrade</code>), live in a terminal panel.
<code>:arduino-lib-list, :arduino-libs</code>	List installed Arduino libraries (Library Manager, installed tab).
<code>:arduino-lib-uninstall</code>	Uninstall an Arduino library by name (<code>arduino-cli lib uninstall</code>).
<code>:arduino-lib-upgrade</code>	Upgrade all installed Arduino libraries (<code>arduino-cli lib upgrade</code>), live in a terminal panel.
<code>:arduino-lib-examples</code>	List a library's example sketches (<code>arduino-cli lib examples</code>).
<code>:arduino-sketch-archive, :arduino-archive</code>	Zip the whole sketch (Arduino IDE Sketch → Archive Sketch).
<code>:pio-clean, :platformio-clean</code>	Remove PlatformIO build artifacts (<code>pio run -t clean</code>); output goes to the compilation list.

Name	Description
<code>:pio-test, :platformio-test</code>	Run the PlatformIO project's unit tests (<code>pio test</code>); failures go to the compilation list.
<code>:pio-check, :platformio-check</code>	PlatformIO static code analysis (<code>pio check</code>); findings go to the compilation list.
<code>:pio-boards, :platformio-boards</code>	The PlatformIO Board Explorer (<code>pio boards [query]</code>); output in the Run console.
<code>:pio-boards-json, :platformio-boards-json</code>	The Board Explorer as JSON (<code>pio boards --json-output [query]</code>), shown in a scratch buffer.
<code>:pio-lib-install, :pio-pkg-install, :platformio-lib-install</code>	Add a library to the PlatformIO project (<code>pio pkg install -l <name></code>), live in a terminal panel.
<code>:pio-lib-list, :pio-pkg-list, :pio-libs</code>	List installed PlatformIO project packages (<code>pio pkg list</code>).
<code>:pio-lib-uninstall, :pio-pkg-uninstall</code>	Remove a PlatformIO project library (<code>pio pkg uninstall -l <name></code>), live in a terminal panel.
<code>:pio-lib-update, :pio-pkg-update</code>	Update installed PlatformIO project packages (<code>pio pkg update</code>), live in a terminal panel.
<code>:arduino-update</code>	Refresh the core and library indexes together (arduino-cli update).
<code>:arduino-upgrade</code>	Upgrade all installed cores and libraries (arduino-cli upgrade), live in a terminal panel.
<code>:arduino-outdated</code>	List cores and libraries with newer versions available (arduino-cli outdated).
<code>:arduino-lib-deps</code>	Show dependency status for a library (arduino-cli lib deps).
<code>:arduino-config, :arduino-config-dump</code>	Print the active arduino-cli configuration (arduino-cli config dump).
<code>:arduino-debug</code>	Launch the arduino-cli debugger for the selected board, live in a terminal panel.
<code>:pio-lib-search, :pio-pkg-search, :platformio-lib-search</code>	Search the PlatformIO registry for a library (<code>pio pkg search <query></code>).
<code>:pio-lib-outdated, :pio-pkg-outdated</code>	List installed project packages with newer versions available (<code>pio pkg outdated</code>).
<code>:pio-lib-show, :pio-pkg-show</code>	Show PlatformIO registry details for a package (<code>pio pkg show <pkg></code>).
<code>:pio-debug, :platformio-debug</code>	Launch the PlatformIO Unified Debugger for the project, live in a terminal panel.
<code>:pio-upgrade, :platformio-upgrade</code>	Upgrade PlatformIO Core itself (<code>pio upgrade</code>), live in a terminal panel.
<code>:pio-size, :platformio-size</code>	Show the program size report (<code>pio run -t size</code>).
<code>:pio-compiledb, :platformio-compiledb</code>	Generate compile_commands.json for the C/C++ LSP (<code>pio run -t compiledb</code>).
<code>:pio-buildfs, :platformio-buildfs</code>	Build the SPIFFS/LittleFS filesystem image (<code>pio run -t buildfs</code>).
<code>:pio-uploadfs, :platformio-uploadfs</code>	Flash the filesystem image to the board (<code>pio run -t uploadfs</code>), live in a terminal panel.
<code>:pio-uploadeep, :platformio-uploadeep</code>	Flash the EEPROM (<code>pio run -t uploadeep</code> , AVR boards), live in a terminal panel.
<code>:pio-bootloader, :platformio-bootloader</code>	Burn the bootloader (<code>pio run -t bootloader</code> , AVR boards), live in a terminal panel.
<code>:pio-fuses, :platformio-fuses</code>	Set the microcontroller fuses (<code>pio run -t fuses</code> , AVR boards), live in a terminal panel.
<code>:pio-nobuild, :platformio-nobuild</code>	Flash the existing firmware without rebuilding (<code>pio run -t nobuild</code>), live in a terminal panel.
<code>:pio-envdump, :platformio-envdump</code>	Dump the resolved build environment (<code>pio run -t envdump</code>).
<code>:pio-cleanall, :platformio-cleanall</code>	Remove all build artifacts including dependencies (<code>pio run -t cleanall</code>).

Name	Description
<code>:pio-project-config, :platformio-project-config</code>	Show the computed PlatformIO project configuration (<code>pio project config</code>).
<code>:pio-project-metadata, :platformio-project-metadata</code>	Dump the IDE/LSP metadata for the project (<code>pio project metadata</code>).
<code>:pio-pkg-exec, :platformio-pkg-exec</code>	Run a tool from an installed package (<code>pio pkg exec -- <argv></code>), live in a terminal panel.
<code>:pio-pkg-exec-pkg, :platformio-pkg-exec-pkg</code>	Run a tool from a specific installed package (<code>pio pkg exec -p <pkg> -- <argv></code>), live in a terminal panel.
<code>:pio-platform-install, :pio-core-install, :platformio-platform-install</code>	Install a development platform globally (<code>pio pkg install -g -p <spec></code>), live in a terminal panel.
<code>:pio-pkg-pack, :platformio-pkg-pack</code>	Build a tarball of the current package (<code>pio pkg pack [-o <path>]</code>), live in a terminal panel.
<code>:pio-pkg-publish, :platformio-pkg-publish</code>	Publish the current package to the PlatformIO registry (<code>pio pkg publish</code>); extra args forward <code>--owner/--type/--private/--no-notify</code> .
<code>:pio-project-config-lint, :platformio-project-config-lint, :pio-lint</code>	Validate platformio.ini without building (<code>pio project config --lint</code>).
<code>:pio-pkg-list-global, :platformio-pkg-list-global, :pio-pkg-list-g</code>	Globally installed packages (<code>pio pkg list -g</code>).
<code>:pio-pkg-update-global, :platformio-pkg-update-global, :pio-pkg-update-g</code>	Update globally installed packages (<code>pio pkg update -g</code>), live in a terminal panel.
<code>:pio-pkg-install-skip-deps, :platformio-pkg-install-skip-deps</code>	Install a package without its dependencies (<code>pio pkg install --skip-dependencies <spec></code>).
<code>:pio-test-junit, :platformio-test-junit</code>	Run unit tests and write a JUnit XML report (<code>pio test --junit-output-path <path></code>).
<code>:pio-test-json-path, :platformio-test-json-path</code>	Run unit tests and write a JSON report (<code>pio test --json-output-path <path></code>).
<code>:pio-test-port, :platformio-test-port</code>	Run unit tests over a specific serial port (<code>pio test --test-port <port></code>).
<code>:pio-test-upload-port, :platformio-test-upload-port</code>	Flash the test firmware to a specific serial port (<code>pio test --upload-port <port></code>).
<code>:pio-test-monitor-dtr, :platformio-test-monitor-dtr</code>	Set the DTR line state for the post-test monitor (<code>pio test --monitor-dtr <0 1></code>).
<code>:pio-test-monitor-rts, :platformio-test-monitor-rts</code>	Set the RTS line state for the post-test monitor (<code>pio test --monitor-rts <0 1></code>).
<code>:pio-project-metadata-path, :platformio-project-metadata-path</code>	Write the IDE/LSP metadata JSON to a file (<code>pio project metadata --json-output-path <path></code>).
<code>:pio-check-silent, :platformio-check-silent</code>	Quiet static analysis, warnings/errors only (<code>pio check -s</code>).
<code>:pio-upgrade-deps-only, :platformio-upgrade-deps-only</code>	Upgrade only PlatformIO Core's dependencies (<code>pio upgrade --only-dependencies</code>).
<code>:pio-init-env-prefix, :platformio-init-env-prefix</code>	Scaffold a project prefixing generated environment names (<code>pio project init --env-prefix <prefix></code>).
<code>:pio-pkg-unpublish, :platformio-pkg-unpublish</code>	Remove a previously published package (<code>pio pkg unpublish <pkg></code>), live in a terminal panel.
<code>:pio-system-info, :platformio-system-info</code>	Show system-wide PlatformIO information (<code>pio system info</code>).
<code>:pio-system-prune, :platformio-system-prune</code>	Remove unused PlatformIO caches/packages (<code>pio system prune -f</code>), live in a terminal panel.

Name	Description
<code>:pio-prune-cache, :platformio-prune-cache</code>	Prune only cached PlatformIO data (<code>pio system prune -f --cache</code>).
<code>:pio-prune-core, :platformio-prune-core</code>	Prune only unnecessary core packages (<code>pio system prune -f --core-packages</code>).
<code>:pio-prune-platform, :platformio-prune-platform</code>	Prune only unnecessary development-platform packages (<code>pio system prune -f --platform-packages</code>).
<code>:pio-prune-dry-run, :platformio-prune-dry-run</code>	Show what <code>pio system prune</code> would remove without deleting (<code>pio system prune --dry-run</code>).
<code>:pio-settings-get, :platformio-settings-get</code>	Print PlatformIO Core settings, all or one key (<code>pio settings get [name]</code>).
<code>:pio-settings-set, :platformio-settings-set</code>	Change a PlatformIO Core setting (<code>pio settings set <name> <value></code>).
<code>:pio-remote-agent-list, :pio-remote-agents, :platformio-remote-agent-list</code>	List active PlatformIO Remote agents (<code>pio remote agent list</code>).
<code>:pio-remote-agent-start, :platformio-remote-agent-start</code>	Start a PlatformIO Remote agent (<code>pio remote agent start [--name <n>] [--share <email>] [--working-dir <dir>]</code>).
<code>:pio-remote-devices, :pio-remote-device-list, :platformio-remote-devices</code>	List serial devices attached to remote agents (<code>pio remote device list</code>).
<code>:pio-remote-run, :platformio-remote-run</code>	Build/upload the project via a remote agent (<code>pio remote run</code>), live in a terminal panel.
<code>:pio-remote-test, :platformio-remote-test</code>	Run unit tests via a remote agent (<code>pio remote test</code>), live in a terminal panel.
<code>:pio-remote-update, :platformio-remote-update</code>	Update platforms/packages/libraries on remote agents (<code>pio remote update [--dry-run]</code>).
<code>:pio-account-login, :platformio-account-login</code>	Sign in to a PlatformIO account (<code>pio account login</code>), live in a terminal panel.
<code>:pio-account-logout, :platformio-account-logout</code>	Sign out of the PlatformIO account (<code>pio account logout</code>).
<code>:pio-account-show, :platformio-account-show</code>	Show the current PlatformIO account information (<code>pio account show</code>); extra args forward <code>--offline/--json-output</code> .
<code>:pio-account-token, :platformio-account-token</code>	Print the account auth token (<code>pio account token [--regenerate] [--json-output] [-p <password>]</code>).
<code>:pio-env, :pio-environment, :platformio-env</code>	Select the PlatformIO build environment (<code>[env:...]</code>); no arg picks from <code>platformio.ini</code> , <code>-</code> clears.
<code>:pio-list-targets, :platformio-list-targets</code>	Enumerate the project's build targets (<code>pio run --list-targets</code>).
<code>:pio-list-tests, :platformio-list-tests</code>	List the project's test suites (<code>pio test --list-tests</code>).
<code>:pio-test-filter, :platformio-test-filter</code>	Run only the tests matching a pattern (<code>pio test -f <pattern></code>).
<code>:pio-check-severity, :platformio-check-severity</code>	Static analysis filtered by minimum defect severity (<code>pio check --severity <low medium high></code>).
<code>:pio-tool-install, :platformio-tool-install</code>	Install a tool package globally (<code>pio pkg install -g -t <spec></code>), live in a terminal panel.
<code>:pio-boards-installed, :platformio-boards-installed</code>	List boards from installed platforms only (<code>pio boards --installed</code>).
<code>:pio-monitor-filter, :platformio-monitor-filter</code>	Add a serial monitor filter for this project (e.g. <code>time</code> , <code>log2file</code> , <code>hexlify</code> , <code>send_on_enter</code>).
<code>:pio-monitor-filters-clear, :platformio-monitor-filters-clear</code>	Remove all configured serial monitor filters.
<code>:pio-monitor-eol, :platformio-monitor-eol</code>	Set the serial monitor end-of-line mode (<code>CR</code> , <code>LF</code> , or <code>CRLF</code>).
<code>:pio-monitor-parity, :platformio-monitor-parity</code>	Set the serial monitor parity (<code>N</code> , <code>E</code> , <code>O</code> , <code>S</code> , or <code>M</code>).
<code>:pio-monitor-rts, :platformio-monitor-rts</code>	Set the initial RTS line state for the serial monitor (<code>0</code> or <code>1</code>).

Name	Description
<code>:pio-monitor-dtr, :platformio-monitor-dtr</code>	Set the initial DTR line state for the serial monitor (0 or 1).
<code>:pio-monitor-echo, :platformio-monitor-echo</code>	Toggle local echo in the serial monitor (--echo).
<code>:pio-monitor-raw, :platformio-monitor-raw</code>	Toggle raw serial monitor mode, disabling output transforms (--raw).
<code>:pio-monitor-encoding, :platformio-monitor-encoding</code>	Set the serial monitor encoding (e.g. UTF-8 , Latin-1 , hexlify); empty resets.
<code>:pio-monitor-flow, :platformio-monitor-flow</code>	Select serial monitor flow control (none , rtscts , or xonxoff).
<code>:pio-monitor-reconnect, :platformio-monitor-reconnect</code>	Toggle serial monitor auto-reconnect (on or off ; off = --no-reconnect).
<code>:pio-monitor-quiet, :platformio-monitor-quiet</code>	Toggle suppression of non-error serial monitor diagnostics (--quiet).
<code>:pio-monitor-exit-char, :platformio-monitor-exit-char</code>	Set the serial monitor exit-char ASCII code (--exit-char ; default 3 = Ctrl+C); empty resets.
<code>:pio-monitor-menu-char, :platformio-monitor-menu-char</code>	Set the serial monitor menu-char ASCII code (--menu-char ; default 20 = Ctrl+T); empty resets.
<code>:pio-pkg-show-type, :platformio-pkg-show-type</code>	Registry details scoped to a package type (pio pkg show --type <library platform tool> <pkg>).
<code>:pio-pkg-exec-call, :platformio-pkg-exec-call</code>	Run a package tool via the call form (pio pkg exec -c <argv...>), live in a terminal panel.
<code>:pio-build-verbose, :platformio-build-verbose</code>	Verbose PlatformIO build (pio run -v), routed through *compilation* .
<code>:pio-build-silent, :platformio-build-silent</code>	Quiet PlatformIO build showing warnings/errors only (pio run -s).
<code>:pio-run-jobs, :platformio-run-jobs</code>	Build with N parallel jobs (pio run -j <n>).
<code>:pio-build-no-auto-clean, :platformio-build-no-auto-clean</code>	Build without the pre-build clean (pio run --disable-auto-clean).
<code>:pio-target, :platformio-target</code>	Run an arbitrary PlatformIO build target (pio run -t <name>), live in a terminal panel.
<code>:pio-upload-to, :platformio-upload-to</code>	Build + flash to a specific port (pio run -t upload --upload-port <port>).
<code>:pio-test-verbose, :platformio-test-verbose</code>	Verbose PlatformIO unit tests (pio test -v).
<code>:pio-test-ignore, :platformio-test-ignore</code>	Run tests except those matching a pattern (pio test -i <pattern>).
<code>:pio-test-without-building, :platformio-test-without-building</code>	Test the last build without rebuilding (pio test --without-building).
<code>:pio-test-without-uploading, :platformio-test-without-uploading</code>	Run tests without flashing first (pio test --without-uploading).
<code>:pio-test-without-testing, :platformio-test-without-testing</code>	Build + upload but skip running the tests (pio test --without-testing).
<code>:pio-test-no-reset, :platformio-test-no-reset</code>	Do not reset the board between tests (pio test --no-reset).
<code>:pio-check-verbose, :platformio-check-verbose</code>	Verbose static code analysis (pio check -v).
<code>:pio-check-json, :platformio-check-json</code>	Static analysis as JSON (pio check --json-output), shown in a scratch buffer.
<code>:pio-check-flags, :platformio-check-flags</code>	Static analysis with extra tool flags (pio check --flags <flags>).
<code>:pio-check-fail-on, :platformio-check-fail-on</code>	Fail on defects at or above a severity (pio check --fail-on-defect <low medium high>).

Name	Description
<code>:pio-check-skip-packages,</code> <code>:platformio-check-skip-packages</code>	Analyse only project sources, skipping libraries (<code>pio check --skip-packages</code>).
<code>:pio-check-src-filters,</code> <code>:platformio-check-src-filters</code>	Restrict analysis to matching sources (<code>pio check --src-filters <pattern></code>).
<code>:pio-debug-verbose,</code> <code>:platformio-debug-verbose</code>	Verbose PlatformIO debugger session (<code>pio debug -v</code>).
<code>:pio-debug-interface,</code> <code>:platformio-debug-interface</code>	Debug with a specific interface (<code>pio debug --interface <name></code>).
<code>:pio-debug-load-mode,</code> <code>:platformio-debug-load-mode</code>	Control firmware reloading on debug start (<code>pio debug --load-mode <always modified manual></code>).
<code>:pio-build-conf,</code> <code>:platformio-build-conf,</code> <code>:pio-build-project-conf</code>	Build using an alternate platformio.ini (<code>pio run -c <path></code>).
<code>:pio-test-conf,</code> <code>:platformio-test-conf</code>	Run unit tests using an alternate platformio.ini (<code>pio test -c <path></code>).
<code>:pio-check-conf,</code> <code>:platformio-check-conf</code>	Static analysis using an alternate platformio.ini (<code>pio check -c <path></code>).
<code>:pio-debug-conf,</code> <code>:platformio-debug-conf</code>	Debug using an alternate platformio.ini (<code>pio debug -c <path></code>), live in a terminal panel.
<code>:pio-init-ide,</code> <code>:platformio-init-ide</code>	Generate IDE integration files (<code>pio project init --ide <ide></code>).
<code>:pio-init-sample,</code> <code>:platformio-init-sample</code>	Scaffold a project with example code (<code>pio project init --board <board> --sample-code</code>).
<code>:pio-init-option,</code> <code>:platformio-init-option</code>	Set a <code>platformio.ini</code> option while initialising (<code>pio project init -O <name=value></code>).
<code>:pio-pkg-install-force,</code> <code>:platformio-pkg-install-force</code>	Reinstall a package even if present (<code>pio pkg install -f <spec></code>).
<code>:pio-pkg-install-global,</code> <code>:platformio-pkg-install-global</code>	Install a package globally (<code>pio pkg install -g <spec></code>).
<code>:pio-lib-install-nosave,</code> <code>:platformio-lib-install-nosave</code>	Install a library without writing it to <code>platformio.ini</code> (<code>pio pkg install -l <name> --no-save</code>).
<code>:pio-pkg-search-sort,</code> <code>:platformio-pkg-search-sort</code>	Registry search with a sort order (<code>pio pkg search <query> --sort <relevance popularity trending added updated></code>).
<code>:pio-upgrade-dev,</code> <code>:platformio-upgrade-dev</code>	Upgrade PlatformIO Core to the development branch (<code>pio upgrade --dev</code>).
<code>:pio-remote-run-force,</code> <code>:platformio-remote-run-force</code>	Force the build to run on the remote agent (<code>pio remote run -r</code>).
<code>:pio-remote-agent-start-named,</code> <code>:platformio-remote-agent-start-named</code>	Start a named Remote agent (<code>pio remote agent start --name <name></code>).
<code>:pio-remote-monitor,</code> <code>:platformio-remote-monitor</code>	Serial monitor over a Remote agent (<code>pio remote device monitor</code>); extra args forward <code>-p/-b/-f/-eol/-sock</code> etc.
<code>:pio-settings-reset,</code> <code>:platformio-settings-reset</code>	Restore PlatformIO Core settings to their defaults (<code>pio settings reset</code>).
<code>:pio-system-completion,</code> <code>:platformio-system-completion</code>	Emit a shell completion script (<code>pio system completion <bash zsh fish powershell></code>).
<code>:pio-ci,</code> <code>:platformio-ci</code>	Build a standalone source tree in an isolated project (<code>pio ci <src> -b <board></code>), live in a terminal panel.
<code>:pio-account-register,</code> <code>:platformio-account-register</code>	Create a new PlatformIO account (<code>pio account register</code>), live in a terminal panel.
<code>:pio-account-password,</code> <code>:platformio-account-password</code>	Change the PlatformIO account password (<code>pio account password</code>), live in a terminal panel.

Name	Description
<code>:pio-account-update, :platformio-account-update</code>	Update PlatformIO profile information (<code>pio account update</code>), live in a terminal panel.
<code>:pio-account-forgot, :platformio-account-forgot</code>	Begin PlatformIO account password recovery (<code>pio account forgot</code>), live in a terminal panel.
<code>:pio-account-destroy, :platformio-account-destroy</code>	Permanently destroy the PlatformIO account (<code>pio account destroy</code>), live in a terminal panel.
<code>:pio-org-list, :platformio-org-list</code>	List PlatformIO organizations and their members (<code>pio org list</code>).
<code>:pio-org-create, :platformio-org-create</code>	Create a new PlatformIO organization (<code>pio org create <orgname></code>), live in a terminal panel.
<code>:pio-org-add, :platformio-org-add</code>	Add an owner to a PlatformIO organization (<code>pio org add <orgname> <username></code>), live in a terminal panel.
<code>:pio-org-remove, :platformio-org-remove</code>	Remove an owner from a PlatformIO organization (<code>pio org remove <orgname> <username></code>), live in a terminal panel.
<code>:pio-org-update, :platformio-org-update</code>	Update a PlatformIO organization (<code>pio org update <orgname></code>), live in a terminal panel.
<code>:pio-org-destroy, :platformio-org-destroy</code>	Destroy a PlatformIO organization (<code>pio org destroy <orgname></code>), live in a terminal panel.
<code>:pio-team-list, :platformio-team-list</code>	List teams in a PlatformIO organization (<code>pio team list <orgname></code>).
<code>:pio-team-create, :platformio-team-create</code>	Create a team in a PlatformIO organization (<code>pio team create <orgname:team></code>), live in a terminal panel.
<code>:pio-team-add, :platformio-team-add</code>	Add a member to a PlatformIO team (<code>pio team add <orgname:team> <username></code>), live in a terminal panel.
<code>:pio-team-remove, :platformio-team-remove</code>	Remove a member from a PlatformIO team (<code>pio team remove <orgname:team> <username></code>), live in a terminal panel.
<code>:pio-team-update, :platformio-team-update</code>	Update a PlatformIO team (<code>pio team update <orgname:team></code>), live in a terminal panel.
<code>:pio-team-destroy, :platformio-team-destroy</code>	Destroy a PlatformIO team (<code>pio team destroy <orgname:team></code>), live in a terminal panel.
<code>:pio-access-list, :platformio-access-list</code>	List published resources and their access levels (<code>pio access list</code>).
<code>:pio-access-grant, :platformio-access-grant</code>	Grant access to a resource (<code>pio access grant <level> <resource> <team></code>), live in a terminal panel.
<code>:pio-access-revoke, :platformio-access-revoke</code>	Revoke access to a resource (<code>pio access revoke <resource> <team></code>), live in a terminal panel.
<code>:pio-access-public, :platformio-access-public</code>	Make a published resource public (<code>pio access public <resource></code>), live in a terminal panel.
<code>:pio-access-private, :platformio-access-private</code>	Make a published resource private (<code>pio access private <resource></code>), live in a terminal panel.
<code>:pio, :platformio</code>	Run an arbitrary <code>pio</code> command in a terminal panel (any subcommand/flag, including <code>pio home</code>).
<code>:arduino-cli, :accli, :arduino</code>	Run an arbitrary <code>arduino-cli</code> command in a terminal panel (any subcommand/flag).
<code>:arduino-core-download, :arduino-download-core</code>	Fetch a core without installing it (<code>arduino-cli core download <package></code>), live in a terminal panel.
<code>:arduino-lib-download, :arduino-download-lib</code>	Fetch a library without installing it (<code>arduino-cli lib download <name></code>), live in a terminal panel.
<code>:arduino-lib-update-index, :arduino-lib-index</code>	Refresh the arduino-cli library index (<code>arduino-cli lib update-index</code>).
<code>:arduino-board-search, :arduino-boards-search</code>	Search the Boards Manager for a board (<code>arduino-cli board search [query]</code>).
<code>:arduino-cache-clean, :arduino-clean-cache</code>	Delete the Boards/Library Manager download cache (<code>arduino-cli cache clean</code>).
<code>:arduino-completion, :arduino-cli-completion</code>	Emit a shell completion script (<code>arduino-cli completion <bash zsh fish powershell></code>).
<code>:arduino-config-get, :arduino-cfg-get</code>	Read one arduino-cli configuration key (<code>arduino-cli config get <key></code>).

Name	Description
<code>:arduino-config-set, :arduino-cfg-set</code>	Set an arduino-cli configuration value (<code>arduino-cli config set <key> <value...></code>).
<code>:arduino-config-add, :arduino-cfg-add</code>	Append value(s) to an arduino-cli list setting (<code>arduino-cli config add <key> <value...></code>).
<code>:arduino-config-remove, :arduino-cfg-remove</code>	Remove value(s) from an arduino-cli list setting (<code>arduino-cli config remove <key> <value...></code>).
<code>:arduino-config-delete, :arduino-cfg-delete</code>	Delete an arduino-cli settings key and its sub-keys (<code>arduino-cli config delete <key></code>).
<code>:arduino-config-init, :arduino-cfg-init</code>	Write the current configuration to a config file (<code>arduino-cli config init</code>).
<code>:arduino-board-attach, :arduino-attach</code>	Attach a sketch to a board (<code>arduino-cli board attach -p <port> -b <fqbn> [sketch]</code>), live in a terminal panel.
<code>:arduino-profile-create, :arduino-profile-new</code>	Create/update a build profile in the sketch project file (<code>arduino-cli profile create</code>), live in a terminal panel.
<code>:arduino-profile-set-default, :arduino-profile-default</code>	Set the default build profile (<code>arduino-cli profile set-default <name></code>).
<code>:arduino-profile-lib-add, :arduino-profile-add-lib</code>	Add a library to the sketch build profile (<code>arduino-cli profile lib add <lib></code>).
<code>:arduino-profile-lib-remove, :arduino-profile-remove-lib</code>	Remove a library from the sketch build profile (<code>arduino-cli profile lib remove <lib></code>).
<code>:arduino-lib-install, :arduino-lib-add</code>	Install a library by name (<code>arduino-cli lib install <name></code>), live in a terminal panel.
<code>:arduino-daemon, :acli-daemon</code>	Run arduino-cli as a gRPC daemon (<code>arduino-cli daemon</code>), live in a terminal panel; extra args tune the server.
<code>:arduino-version, :acli-version</code>	Show the arduino-cli version (<code>arduino-cli version</code> ; add <code>--format json</code>).
<code>:next-error, :cnext-error</code>	Visit the next compilation error's location (emacs <code>next-error</code> / <code>M-g n</code>).
<code>:previous-error, :cprevious-error</code>	Visit the previous compilation error's location (emacs <code>previous-error</code> / <code>M-g p</code>).
<code>:first-error</code>	Visit the first compilation error's location (emacs <code>first-error</code>).
<code>:abbreviate, :ab</code>	List or define an abbreviation for Insert and Command-line mode (vim <code>:abbreviate</code>).
<code>:list-abbrevs</code>	Show all defined abbreviations in a buffer (emacs <code>list-abbrevs</code>).
<code>:define-global-abbrev</code>	Define a global abbreviation: <code>:define-global-abbrev NAME EXPANSION</code> (emacs <code>define-global-abbrev</code>).
<code>:define-mode-abbrev</code>	Define a major-mode-local abbreviation: <code>:define-mode-abbrev NAME EXPANSION</code> (emacs <code>define-mode-abbrev</code>).
<code>:abbrev-mode</code>	Toggle abbrev-mode (auto-expand abbrevs on a typed word separator); <code>:abbrev-mode on</code>
<code>:kill-all-abbrevs</code>	Remove all defined abbreviations (emacs <code>kill-all-abbrevs</code>).
<code>:iabbrev, :ia</code>	List or define an Insert-mode abbreviation (vim <code>:iabbrev</code>).
<code>:cabbrev, :ca</code>	List or define a Command-line-mode abbreviation (vim <code>:cabbrev</code>).
<code>:noreabbrev, :norea</code>	List or define a non-recursive Insert+Command-line abbreviation (vim <code>:noreabbrev</code>).
<code>:inoreabbrev, :inorea</code>	List or define a non-recursive Insert-mode abbreviation (vim <code>:inoreabbrev</code>).
<code>:cnoreabbrev, :cnorea</code>	List or define a non-recursive Command-line-mode abbreviation (vim <code>:cnoreabbrev</code>).
<code>:unabbreviate, :una</code>	Remove an abbreviation for both modes (vim <code>:unabbreviate</code>).

Name	Description
<code>:iunabbreviate, :iuna</code>	Remove an Insert-mode abbreviation (vim <code>:iunabbreviate</code>).
<code>:cunabbreviate, :cuna</code>	Remove a Command-line-mode abbreviation (vim <code>:cunabbreviate</code>).
<code>:abclear, :abc</code>	Remove all abbreviations for both modes (vim <code>:abclear</code>).
<code>:iabclear, :iabc</code>	Remove all Insert-mode abbreviations (vim <code>:iabclear</code>).
<code>:cabclear, :cabc</code>	Remove all Command-line-mode abbreviations (vim <code>:cabclear</code>).
<code>:bfirst, :brewind, :brew</code>	Go to the first buffer in the buffer list (vim <code>:bfirst</code> / <code>:brewind</code>).
<code>:blast, :bl</code>	Go to the last buffer in the buffer list (vim <code>:blast</code>).
<code>:bmodified, :bm</code>	Go to the next modified buffer (vim <code>:bmodified</code>).
<code>:ball, :sball, :unhide, :unh, :sunhide, :sun</code>	Open a window for each buffer in the buffer list (vim <code>:ball</code> ; <code>:unhide</code> / <code>:sunhide</code> — every zmax buffer is loaded).
<code>:badd</code>	Add a file to the buffer list without editing it (vim <code>:badd</code>).
<code>:balt</code>	Add a file to the buffer list and set it as the alternate file (vim <code>:balt</code>).
<code>:bufdo</code>	Run an Ex command in each listed buffer (vim <code>:bufdo</code>).
<code>:spellwrong, :spellw</code>	Mark words as misspelled (vim <code>:spellwrong</code>).
<code>:spellwrong!, :spellw!</code>	Mark words misspelled in the internal word list (vim <code>:spellwrong!</code>).
<code>:spellrare, :spellra</code>	Flag words as rare (vim <code>:spellrare</code>).
<code>:spellrare!, :spellra!</code>	Flag words as rare in the internal word list (vim <code>:spellrare!</code>).
<code>:spellundo, :spellu</code>	Remove words from the good/bad spell lists (vim <code>:spellundo</code>).
<code>:spellundo!, :spellu!</code>	Remove words from the internal word list only (vim <code>:spellundo!</code>).
<code>:spelldump, :spelled</code>	Open a buffer listing the user's known-good words (vim <code>:spelldump</code>).
<code>:spellinfo, :spelli</code>	Show the spell wordlist location and sizes (vim <code>:spellinfo</code>).
<code>:buffer-close, :bc, :bclose, :bd, :bdelete, :bun, :bunload, :bw, :bwipe, :bwipeout</code>	Close the current buffer.
<code>:buffer-close!, :bc!, :bclose!</code>	Close the current buffer forcefully, ignoring unsaved changes.
<code>:buffer-close-others, :bco, :bcloseother</code>	Close all buffers but the currently focused one.
<code>:buffer-close-others!, :bco!, :bcloseother!</code>	Force close all buffers but the currently focused one.
<code>:buffer-close-all, :bca, :bcloseall</code>	Close all buffers without quitting.
<code>:buffer-close-all!, :bca!, :bcloseall!</code>	Force close all buffers ignoring unsaved changes without quitting.
<code>:buffer-next, :bn, :bnext</code>	Goto next buffer.
<code>:buffer-previous, :bp, :bprev, :bN, :bNext</code>	Goto previous buffer.
<code>:sbnext, :sbn</code>	Split window and go to the next buffer (vim <code>:sbnext</code>).
<code>:sbprevious, :sbp, :sbNext, :sbN</code>	Split window and go to the previous buffer (vim <code>:sbprevious</code> / <code>:sbNext</code>).
<code>:sbfirst, :sbf, :sbrewind, :sbr</code>	Split window and go to the first buffer (vim <code>:sbfirst</code> / <code>:sbrewind</code>).
<code>:sblast, :sbl</code>	Split window and go to the last buffer (vim <code>:sblast</code>).

Name	Description
<code>:sbmodified, :sbm</code>	Split window and go to the next modified buffer (vim <code>:sbmodified</code>).
<code>:sbuffer, :sb</code>	Split window and go to the buffer whose path contains {name} (vim <code>:sbuffer / :sb</code>).
<code>:pedit, :ped</code>	Edit {file} in a preview (horizontal split) window (vim <code>:pedit</code>).
<code>:pbuffer, :pb</code>	Show the buffer whose path contains {name} in a preview (split) window (vim <code>:pbuffer</code>).
<code>:nohlsearch, :noh, :nohl</code>	Clear the persistent search highlight (vim <code>:nohlsearch</code>).
<code>:clearjumps</code>	Clear the current view's jump list (vim <code>:clearjumps</code>).
<code>:project-replace, :replace-in-files</code>	Regex-replace across all matching workspace files (JetBrains Replace in Files).
<code>:buffers, :ls, :files</code>	List open buffers in the buffer picker (vim <code>:buffers/:ls/:files</code>).
<code>:jumps</code>	List the jump list in a picker (vim <code>:jumps</code>).
<code>:oldfiles</code>	Pick from recently edited files (vim <code>:oldfiles</code>).
<code>:marks</code>	List the buffer's marks in a picker (vim <code>:marks</code>).
<code>:changes</code>	List the buffer's changelist in a picker (vim <code>:changes</code>).
<code>:history</code>	Pick from the command-line history (vim <code>:history</code>).
<code>:delmarks, :delm</code>	Delete the listed named marks (vim <code>:delmarks abc</code>).
<code>:delmarks!, :delm!</code>	Delete all letter marks (vim <code>:delmarks!</code>).
<code>:write, :w, :sav, :saveas</code>	Write changes to disk. Accepts an optional path (<code>:write some/path.txt</code>)
<code>:write!, :w!</code>	Force write changes to disk creating necessary subdirectories. Accepts an optional path (<code>:write! some/path.txt</code>)
<code>:write-buffer-close, :wbc</code>	Write changes to disk and closes the buffer. Accepts an optional path (<code>:write-buffer-close some/path.txt</code>)
<code>:write-buffer-close!, :wbc!</code>	Force write changes to disk creating necessary subdirectories and closes the buffer. Accepts an optional path (<code>:write-buffer-close! some/path.txt</code>)
<code>:new, :n, :enew</code>	Create a new scratch buffer.
<code>:Scratch, :scratch</code>	Open a new scratch buffer, optionally with a language (SPC b S).
<code>:RevealInFinder, :reveal-in-finder</code>	Reveal the current file in the OS file manager (JetBrains Reveal in Finder).
<code>:compose-mail, :mail, :compose</code>	Open a message-mode mail draft (emacs <code>compose-mail</code> , C-x m). <code>:compose-mail [to] [subject...]</code>
<code>:compose-mail-other-window</code>	Open a mail draft in a split (emacs <code>compose-mail-other-window</code>).
<code>:message-send</code>	Assemble and queue the current mail draft (emacs <code>message-send</code> , C-c C-s).
<code>:message-send-and-exit</code>	Queue the draft and kill its buffer (emacs <code>message-send-and-exit</code> , C-c C-c).
<code>:message-kill-buffer</code>	Kill the mail compose buffer (emacs <code>message-kill-buffer</code> , C-c C-k).
<code>:message-goto-to</code>	Move point to the To: header (emacs <code>message-goto-to</code> , C-c C-f C-t).
<code>:message-goto-subject</code>	Move point to the Subject: header (emacs <code>message-goto-subject</code> , C-c C-f C-s).
<code>:message-goto-cc</code>	Move point to the Cc: header (emacs <code>message-goto-cc</code> , C-c C-f C-c).

Name	Description
<code>:message-goto-bcc</code>	Move point to the Bcc: header (emacs message-goto-bcc, C-c C-f C-b).
<code>:message-goto-body, :mail-text</code>	Move point to the message body (emacs message-goto-body / mail-text, C-c C-b).
<code>:message-insert-signature</code>	Insert the signature block at point (emacs message-insert-signature, C-c C-w).
<code>:mml-attach-file, :mail-add-attachment</code>	Attach a file as a MIME part (emacs mml-attach-file, C-c C-a). :mml-attach-file
<code>:format, :fmt</code>	Format the file using an external formatter or language server.
<code>:indent-style</code>	Set the indentation style for editing. ('t' for tabs or 1-16 for number of spaces.)
<code>:line-ending</code>	Set the document's default line ending. Options: crlf, lf.
<code>:earlier, :ear</code>	Jump back to an earlier point in edit history. Accepts a number of steps or a time span.
<code>:later, :lat</code>	Jump to a later point in edit history. Accepts a number of steps or a time span.
<code>:undotree, :undo-tree, :UndotreeToggle</code>	Open the branching undo-history browser (vim undotree).
<code>:undolist, :undol</code>	List the undo states as a text popup (vim :undolist).
<code>:injections, :injection-rules</code>	List the active language-injection rules (defaults + injections.toml).
<code>:injection-info, :what-injection</code>	Report the effective (possibly injected) language at the cursor.
<code>:edit-fragment, :edit-injected-fragment</code>	Edit the injected-language fragment at point in its own buffer (JetBrains Edit Fragment).
<code>:apply-fragment, :apply-injected-fragment</code>	Write the fragment buffer back into its host string.
<code>:inject-language, :inject-lang</code>	Inject a language into the string at point via a /* language=... */ hint (JetBrains inject-here).
<code>:write-quit, :wq</code>	Write changes to disk and close the current view. Accepts an optional path (:wq some/path.txt)
<code>:write-quit!, :wq!</code>	Write changes to disk and close the current view forcefully. Accepts an optional path (:wq! some/path.txt)
<code>:write-all, :wa</code>	Write changes from all buffers to disk.
<code>:write-all!, :wa!</code>	Forcefully write changes from all buffers to disk creating necessary subdirectories.
<code>:write-quit-all, :wqa, :xa</code>	Write changes from all buffers to disk and close all views.
<code>:write-quit-all!, :wqa!, :xa!</code>	Forcefully write changes from all buffers to disk, creating necessary subdirectories, and close all views (ignoring unsaved changes).
<code>:quit-all, :qa, :qall, :quita, :quitall</code>	Close all views.
<code>:quit-all!, :qa!</code>	Force close all views ignoring unsaved changes.
<code>:cquit, :cq</code>	Quit with exit code (default 1). Accepts an optional integer exit code (:cq 2).
<code>:cquit!, :cq!</code>	Force quit with exit code (default 1) ignoring unsaved changes. Accepts an optional integer exit code (:cq! 2).
<code>:theme, :colorscheme, :colo</code>	Change the editor theme (show current theme if no name specified).
<code>:describe-theme</code>	Show a theme's faces and their fg/bg colors in a scratch buffer (emacs describe-theme).
<code>:disable-theme</code>	Turn off the active theme, reverting to the default (emacs disable-theme).
<code>:hunk-reset, :reset-hunk, :hunk-undo</code>	Undo the git hunk under the cursor, restoring it from HEAD (gitsigns reset_hunk).
<code>:hunk-next, :next-hunk</code>	Move the cursor to the next git hunk.

Name	Description
<code>:hunk-prev</code> , <code>:prev-hunk</code>	Move the cursor to the previous git hunk.
<code>:conflict-ours</code> , <code>:diffget-ours</code> , <code>:conflict-keep-ours</code>	Resolve the merge conflict at the cursor by keeping OUR side (HEAD).
<code>:conflict-theirs</code> , <code>:diffget-theirs</code> , <code>:conflict-keep-theirs</code>	Resolve the merge conflict at the cursor by keeping THEIR side (incoming).
<code>:conflict-both</code> , <code>:conflict-keep-both</code>	Resolve the merge conflict at the cursor by keeping BOTH sides.
<code>:conflict-next</code>	Jump to the next merge-conflict marker.
<code>:conflict-prev</code>	Jump to the previous merge-conflict marker.
<code>:theme-toggle</code> , <code>:toggle-theme</code> , <code>:light-dark</code>	Toggle between a dark and light theme (<code>:theme-toggle [dark] [light]</code>).
<code>:theme-next</code>	Switch to the next theme (alphabetical).
<code>:theme-prev</code>	Switch to the previous theme (alphabetical).
<code>:run</code> , <code>:r!</code>	Run a command in the IDE Run tool window (defaults to <code>cargo run</code>).
<code>:search-project</code> , <code>:rg</code>	Search the project (ripgrep) and show jumpable results in the Run console.
<code>:grep</code> , <code>:gr</code>	Run 'grep', put the matches in the quickfix list and jump to the first (vim :grep).
<code>:grep!</code>	Like :grep, but do not jump to the first match.
<code>:grepadd</code> , <code>:grepa</code>	Like :grep, but append the matches to the quickfix list (vim :grepadd).
<code>:lgrep</code> , <code>:lgr</code>	Like :grep, but fill the location list (vim :lgrep).
<code>:lgrepadd</code> , <code>:lgrepa</code>	Like :lgrep, but append to the location list (vim :lgrepadd).
<code>:vimgrep</code> , <code>:vimg</code>	Search files for <code>/[pattern]/</code> , fill the quickfix list and jump to the first match (vim :vimgrep).
<code>:vimgrep!</code>	Like :vimgrep, but do not jump to the first match.
<code>:vimgrepadd</code> , <code>:vimgrepa</code>	Like :vimgrep, but append the matches to the quickfix list (vim :vimgrepadd).
<code>:lvimgrep</code> , <code>:lv</code> , <code>:lvim</code>	Like :vimgrep, but fill the location list (vim :lvimgrep).
<code>:lvimgrepadd</code> , <code>:lvimgrepa</code>	Like :lvimgrep, but append to the location list (vim :lvimgrepadd).
<code>:helpgrep</code> , <code>:helpg</code>	Search the help: open the inline Help browser filtered to {pattern} (vim :helpgrep).
<code>:lhelpgrep</code> , <code>:lh</code>	Location-list variant of :helpgrep — open the Help browser filtered to {pattern} (vim :lhelpgrep).
<code>:copen</code> , <code>:cwindow</code> , <code>:cw</code>	Open the quickfix list window.
<code>:cbottom</code> , <code>:cbo</code> , <code>:cbot</code> , <code>:cbott</code> , <code>:cbotto</code>	Scroll the quickfix window to its last entry (vim :cbottom).
<code>:cclose</code> , <code>:ccl</code>	Close the quickfix list window.
<code>:cnext</code> , <code>:cn</code>	Jump to the next quickfix entry.
<code>:cprevious</code> , <code>:cprev</code> , <code>:cp</code> , <code>:cN</code>	Jump to the previous quickfix entry.
<code>:cfirst</code> , <code>:crewind</code> , <code>:cr</code>	Jump to the first quickfix entry.
<code>:clast</code> , <code>:cla</code>	Jump to the last quickfix entry.
<code>:cc</code>	Jump to the [count]th quickfix entry (or the current one).
<code>:cnfile</code> , <code>:cnf</code>	Jump to the first quickfix entry in the next file.

Name	Description
<code>:cpfile, :cpf, :cNf, :cNfile</code>	Jump to the last quickfix entry in the previous file.
<code>:cabove, :cabo</code>	Jump to the [count]th quickfix entry above the cursor line.
<code>:cbelow, :cbe1</code>	Jump to the [count]th quickfix entry below the cursor line.
<code>:cbefore, :cbe</code>	Jump to the [count]th quickfix entry before the cursor position.
<code>:cafter, :caf</code>	Jump to the [count]th quickfix entry after the cursor position.
<code>:cbuffer, :cb</code>	Read the current buffer as error lines into the quickfix list.
<code>:cgetbuffer</code>	Read the current buffer into the quickfix list without jumping.
<code>:caddbuffer</code>	Append the current buffer's error lines to the quickfix list.
<code>:cexpr, :cex</code>	Parse the argument text into the quickfix list and jump to the first entry.
<code>:cgetexpr</code>	Parse the argument text into the quickfix list without jumping.
<code>:caddexpr</code>	Append the argument text's entries to the quickfix list.
<code>:cfile, :cf</code>	Read a file of error lines into the quickfix list and jump to the first entry.
<code>:cgetfile</code>	Read a file of error lines into the quickfix list without jumping.
<code>:lopen, :lwindow, :lw</code>	Open the location list window for the current window.
<code>:lbottom, :lbo, :lbot, :lbott, :lbotto</code>	Scroll the location list window to its last entry (vim :lbottom).
<code>:lclose, :lcl</code>	Close the location list window.
<code>:lnext, :lne, :ln</code>	Jump to the next location list entry.
<code>:lprevious, :lprev, :lp, :lN</code>	Jump to the previous location list entry.
<code>:lfirst, :lrewind, :lr</code>	Jump to the first location list entry.
<code>:llast, :lla</code>	Jump to the last location list entry.
<code>:ll</code>	Jump to the [count]th location list entry (or the current one).
<code>:labove, :lab</code>	Jump to the [count]th location list entry above the cursor line.
<code>:lbelow, :lbel</code>	Jump to the [count]th location list entry below the cursor line.
<code>:lbefore, :lbe</code>	Jump to the [count]th location list entry before the cursor position.
<code>:lafter, :laf</code>	Jump to the [count]th location list entry after the cursor position.
<code>:lbuffer, :lb</code>	Read the current buffer as error lines into the location list.
<code>:lgetbuffer</code>	Read the current buffer into the location list without jumping.
<code>:lexpr, :lex</code>	Parse the argument text into the location list and jump to the first entry.
<code>:lgetexpr</code>	Parse the argument text into the location list without jumping.
<code>:lfile, :lf</code>	Read a file of error lines into the location list and jump to the first entry.
<code>:lgetfile</code>	Read a file of error lines into the location list without jumping.
<code>:tabnew, :tabe, :tabedit</code>	Open a new tabpage (optionally editing a file).
<code>:tabnext, :tabn</code>	Go to the next tabpage (or tab [count]).
<code>:tabprevious, :tabp, :tabNext, :tabN</code>	Go to the previous tabpage.

Name	Description
<code>:tabclose, :tabc</code>	Close the current tabpage.
<code>:tab-rename</code>	Name the current tab, or clear it when given no name (emacs <code>tab-rename</code>).
<code>:tab-switch</code>	Switch to a tab by name or 1-based number (emacs <code>tab-switch</code>).
<code>:tab-undo</code>	Reopen the most recently closed tab (emacs <code>tab-undo</code>).
<code>:tab-bar-history-back</code>	Return to the previously visited tab (emacs <code>tab-bar-history-back</code>).
<code>:tab-bar-history-forward</code>	Re-visit a tab left via history-back (emacs <code>tab-bar-history-forward</code>).
<code>:tabonly, :tabo</code>	Close all tabpages except the current one.
<code>:tabfirst, :tabrewind, :tabr</code>	Go to the first tabpage.
<code>:tablast</code>	Go to the last tabpage.
<code>:tabmove, :tabm</code>	Move the current tabpage to position [N] (default: last).
<code>:tabdo</code>	Run an ex-command in every tabpage.
<code>:windo</code>	Run an ex-command in every window of the current tabpage.
<code>:wincmd</code>	Run a window (CTRL-W) command by key, e.g. <code>:wincmd h</code> focuses left.
<code>:source, :so</code>	Source a Vimscript file through the embedded vimlrs interpreter.
<code>:runtime, :ru</code>	Source a file from the runtimepath (zmax config dir) via vimlrs (vim <code>:runtime</code>).
<code>:diffthis, :diff</code>	Show the current buffer's changes as a side-by-side diff vs git HEAD (vim <code>:diffthis</code>).
<code>:diffupdate, :diffu</code>	Recompute and redisplay the buffer's diff vs git HEAD (vim <code>:diffupdate</code>).
<code>:diffoff, :diffo</code>	Turn off diff mode: remove the side-by-side diff overlay (vim <code>:diffoff</code>).
<code>:diffsplit, :diffs, :diffsp, :diffspl, :diffspli</code>	Show the differences between this buffer and {file}, side by side (vim <code>:diffsplit</code>).
<code>:diffpatch, :diffp, :diffpa, :diffpat, :diffpatc</code>	Patch a copy of this buffer with {patchfile} (patch(1)) and show the differences (vim <code>:diffpatch</code>).
<code>:match</code>	Highlight {pattern} in match group 1, or clear it with <code>:match none</code> (vim <code>:match</code>).
<code>:2match</code>	Highlight {pattern} in match group 2, or clear it with <code>:2match none</code> (vim <code>:2match</code>).
<code>:3match</code>	Highlight {pattern} in match group 3, or clear it with <code>:3match none</code> (vim <code>:3match</code>).
<code>:helptags, :helpt</code>	Write a tags file for the vim-format *.txt help files in {dir} (vim <code>:helptags</code>).
<code>:sign, :sig</code>	Define/place/unplace/list/jump gutter signs (vim <code>:sign</code>); e.g. <code>:sign define warn text=» texthl=WarningMsg</code> then <code>:sign place 1 line=10 name=warn</code> .
<code>:undojoin, :undoj</code>	Join the next change with the previous undo block, so one undo reverts both (vim <code>:undojoin</code>).
<code>:image-mode, :image-display, :image-toggle-display</code>	Display the current image file in the terminal (emacs <code>image-mode</code> / <code>image-toggle-display</code>).
<code>:image-rotate</code>	Rotate the current image 90 degrees and redisplay (emacs <code>image-rotate</code>).
<code>:image-flip-horizontally</code>	Flip the current image left-to-right and redisplay (emacs <code>image-flip-horizontally</code>).
<code>:image-flip-vertically</code>	Flip the current image top-to-bottom and redisplay (emacs <code>image-flip-vertically</code>).

Name	Description
<code>:add-file-local-variable</code>	Set a file-local variable in the Local Variables block (emacs <code>add-file-local-variable</code>).
<code>:add-file-local-variable-prop-line</code>	Set a file-local variable in the first-line <code>-*-</code> prop line (emacs <code>add-file-local-variable-prop-line</code>).
<code>:delete-file-local-variable</code>	Remove a file-local variable from the Local Variables block (emacs <code>delete-file-local-variable</code>).
<code>:delete-file-local-variable-prop-line</code>	Remove a file-local variable from the <code>-*-</code> prop line (emacs <code>delete-file-local-variable-prop-line</code>).
<code>:image-increase-size</code>	Scale the current image up by 25% and redisplay (emacs <code>image-increase-size</code>).
<code>:image-decrease-size</code>	Scale the current image down by 20% and redisplay (emacs <code>image-decrease-size</code>).
<code>:image-transform-set-percent</code>	Set the current image's scale to N percent and redisplay (emacs <code>image-transform-set-percent</code>).
<code>:image-transform-set-scale</code>	Set the current image's scale from a multiplier, e.g. 1.5 (emacs <code>image-transform-set-scale</code>).
<code>:image-transform-fit-to-window</code>	Fit the current image to the window (emacs <code>image-transform-fit-to-window</code>).
<code>:image-transform-reset-to-original</code> , <code>:image-transform-reset-to-initial</code>	Drop all rotation/flip/scale transforms and redisplay (emacs <code>image-transform-reset-to-original</code>).
<code>:image-reset-speed</code>	Reset the current animation's speed to the file's own frame delays, keeping its direction (emacs <code>image-reset-speed</code>).
<code>:image-reverse-speed</code>	Play the current animation backwards at the same rate (emacs <code>image-reverse-speed</code>).
<code>:image-next-file</code>	Open the next image file in the directory (emacs <code>image-next-file</code>).
<code>:image-previous-file</code>	Open the previous image file in the directory (emacs <code>image-previous-file</code>).
<code>:image-goto-frame</code>	Show frame N (from 1) of a multi-frame image (emacs <code>image-goto-frame</code>).
<code>:image-next-frame</code>	Switch to the Nth frame after the current one, default 1 (emacs <code>image-next-frame</code>).
<code>:image-previous-frame</code>	Switch to the Nth frame before the current one, default 1 (emacs <code>image-previous-frame</code>).
<code>:image-increase-speed</code>	Double the current animated image's playback speed (emacs <code>image-increase-speed</code>).
<code>:image-decrease-speed</code>	Halve the current animated image's playback speed (emacs <code>image-decrease-speed</code>).
<code>:image-mode-copy-file-name-as-kill</code>	Copy the image file's path to the clipboard register (emacs <code>image-mode-copy-file-name-as-kill</code>).
<code>:doc-view-mode</code> , <code>:doc-view-minor-mode</code> , <code>:doc-view-toggle-display</code>	Render the current document's page in the terminal (emacs <code>doc-view-mode</code>).
<code>:doc-view-next-page</code> , <code>:doc-view-scroll-up-or-next-page</code>	Show the next page of the document (emacs <code>doc-view-next-page</code>).
<code>:doc-view-previous-page</code> , <code>:doc-view-scroll-down-or-previous-page</code>	Show the previous page of the document (emacs <code>doc-view-previous-page</code>).
<code>:doc-view-first-page</code>	Show the first page of the document (emacs <code>doc-view-first-page</code>).
<code>:doc-view-last-page</code>	Show the last page of the document (emacs <code>doc-view-last-page</code>).
<code>:doc-view-goto-page</code>	Show page N of the document (emacs <code>doc-view-goto-page</code>).
<code>:doc-view-enlarge</code>	Increase the document render resolution and redisplay (emacs <code>doc-view-enlarge</code>).
<code>:doc-view-shrink</code>	Decrease the document render resolution and redisplay (emacs <code>doc-view-shrink</code>).

Name	Description
<code>:doc-view-open-text</code>	Extract the document's text into a scratch buffer (emacs <code>doc-view-open-text</code>).
<code>:doc-view-search</code> , <code>:doc-view-search-backward</code>	Search the document's text for a pattern (emacs <code>doc-view-search</code>).
<code>:doc-view-clear-cache</code> , <code>:doc-view-kill-proc</code> , <code>:doc-view-kill-proc-and-buffer</code>	Forget the doc-view render state (emacs <code>doc-view-clear-cache</code>).
<code>:doc-view-set-slice</code>	Crop the displayed page to X Y WIDTH HEIGHT pixels (emacs <code>doc-view-set-slice</code>).
<code>:doc-view-reset-slice</code>	Drop the doc-view crop slice and show the full page (emacs <code>doc-view-reset-slice</code>).
<code>:doc-view-show-tooltip</code>	Show the current page info (Page N of M) on the status line (emacs <code>doc-view-show-tooltip</code>).
<code>:append</code> , <code>:a</code>	Insert typed lines after the current line; end input with a line containing only '.' (vim <code>:append</code>).
<code>:insert</code> , <code>:i</code>	Insert typed lines before the current line; end input with a line containing only '.' (vim <code>:insert</code>).
<code>:change</code> , <code>:c</code>	Replace the current line with typed lines; end input with a line containing only '.' (vim <code>:change</code>).
<code>:doautocmd</code> , <code>:do</code> , <code>:doa</code>	Fire the autocommands registered for {event} on the current buffer (vim <code>:doautocmd</code>).
<code>:doautoall</code> , <code>:doautoa</code>	Fire the autocommands for {event} on every loaded buffer (vim <code>:doautoall</code>).
<code>:drop</code> , <code>:dr</code>	Jump to a buffer already editing {file}, else edit it (vim <code>:drop</code>).
<code>:lua</code>	Run a Lua snippet through the system lua interpreter and echo its output (vim <code>:lua</code>).
<code>:luafile</code> , <code>:luafile</code>	Run a Lua script file through the system lua interpreter (vim <code>:luafile</code>).
<code>:perl</code> , <code>:pe</code>	Evaluate a Perl snippet via the embedded stryke interpreter (a Perl superset; vim <code>:perl</code> , in-process, no system perl).
<code>:perlfile</code> , <code>:perlf</code>	Run a Perl script file through the system perl interpreter (vim <code>:perlfile</code>).
<code>:python</code> , <code>:py</code>	Evaluate a Python snippet via the embedded pythonrs interpreter (vim <code>:python</code> ; in-process, no system python).
<code>:pyfile</code> , <code>:pyf</code>	Run a Python script file through the system python interpreter (vim <code>:pyfile</code>).
<code>:py3</code> , <code>:python3</code>	Run a Python 3 snippet through the system python3 interpreter and echo its output (vim <code>:py3</code> / <code>:python3</code>).
<code>:py3file</code> , <code>:py3f</code> , <code>:python3file</code>	Run a Python 3 script file through the system python3 interpreter (vim <code>:py3file</code>).
<code>:ruby</code> , <code>:rub</code> , <code>:rb</code>	Evaluate a Ruby snippet via the embedded rubylang interpreter (vim <code>:ruby</code> ; in-process, no system ruby).
<code>:rubyfile</code> , <code>:rubyf</code>	Run a Ruby script file through the system ruby interpreter (vim <code>:rubyfile</code>).
<code>:tabs</code>	List the tabpages and switch to the selected one.
<code>:cdo</code>	Run an ex-command on each quickfix entry.
<code>:cfdo</code>	Run an ex-command on the first quickfix entry of each file.
<code>:ldo</code>	Run an ex-command on each location list entry.
<code>:lfdo</code>	Run an ex-command on the first location entry of each file.
<code>:clist</code> , <code>:cl</code>	Show the quickfix list.
<code>:llist</code> , <code>:lli</code>	Show the location list.
<code>:colder</code> , <code>:col</code>	Go to an older quickfix list.
<code>:cnewer</code> , <code>:cnew</code>	Go to a newer quickfix list.

Name	Description
<code>:chistory, :chi</code>	Show the quickfix list history position.
<code>:lolder, :lol</code>	Go to an older location list (vim :lolder).
<code>:lnewer, :lnew</code>	Go to a newer location list (vim :lnewer).
<code>:lhistory, :lhi</code>	Show the location list history position (vim :lhistory).
<code>:lnfile, :lnf</code>	Jump to the first location-list entry in the next file (vim :lnfile).
<code>:lNfile, :lNf</code>	Jump to the last location-list entry in the previous file (vim :lNfile).
<code>:caddfile</code>	Append a file of error lines to the quickfix list.
<code>:laddbuffer</code>	Append the current buffer's error lines to the location list.
<code>:laddexpr</code>	Append the argument text's entries to the location list.
<code>:laddfile</code>	Append a file of error lines to the location list.
<code>:lpfile, :lpf</code>	Jump to the last location entry in the previous file.
<code>:shell-quote, :sh-quote</code>	Wrap the selection in safe shell single-quotes.
<code>:wrap-tag</code>	Wrap each selection in
<code>:csv-column, :csv-col</code>	Replace the selected CSV/TSV with just its Nth column (1-based).
<code>:code-fence, :fence</code>	Wrap the selection in a fenced Markdown code block with optional language.
<code>:md-table, :table-fmt</code>	Align the selected Markdown pipe table (pad columns, rebuild separator row).
<code>:json-query, :json-get</code>	Replace the selected JSON with the value at a dot-path (e.g. users.0.name).
<code>:json-flatten, :json-paths</code>	Flatten the selected JSON into greppable <code>path = value</code> lines.
<code>:json-to-csv, :json-csv</code>	Convert the selected JSON array of objects to CSV (sorted header union).
<code>:json-unflatten, :json-unpaths</code>	Rebuild nested JSON from <code>path = value</code> lines (inverse of :json-flatten).
<code>:toml-to-json, :toml-json</code>	Convert the selected TOML to pretty-printed JSON.
<code>:json-to-toml, :json-toml</code>	Convert the selected JSON to pretty-printed TOML.
<code>:json-sort, :json-sort-array</code>	Sort the selected JSON array (optionally by an object field: :json-sort name).
<code>:json-pick, :json-select</code>	Keep only the named keys in the selected JSON object/array (e.g. :json-pick name age).
<code>:json-omit, :json-drop</code>	Drop the named keys from the selected JSON object/array (e.g. :json-omit password).
<code>:json-unique, :json-uniq</code>	Remove duplicate elements from the selected JSON array (optionally by a field).
<code>:json-group-by, :json-group</code>	Group the selected JSON array of objects by a field (e.g. :json-group-by city).
<code>:extract, :matches</code>	Replace the selection with every regex match, one per line (group 1 if present).
<code>:filter, :keep-lines, :delete-non-matching-lines</code>	Keep only the selected lines matching a regex (Emacs keep-lines; in-buffer grep).
<code>:reject, :remove-lines, :flush-lines, :delete-matching-lines</code>	Drop the selected lines matching a regex (Emacs flush-lines; in-buffer grep -v).
<code>:count-matches, :count-regex, :how-many</code>	Report how many regex matches (and matching lines) are in the selection (Emacs how-many).

Name	Description
<code>:copy-matching-lines</code> , <code>:copy-lines-matching</code>	Copy lines matching a regex (region, or point to end) to the kill ring (Emacs <code>copy-matching-lines</code>).
<code>:kill-matching-lines</code> , <code>:kill-lines-matching</code>	Delete lines matching a regex (region, or point to end) and save them to the kill ring (Emacs <code>kill-matching-lines</code>).
<code>:uniq-count</code> , <code>:frequency</code>	Collapse the selected lines to <code>count line</code> , sorted by frequency (<code>uniq -c</code>).
<code>:stats</code> , <code>:describe</code>	Show count/sum/mean/min/max of the numbers in the selection (non-destructive).
<code>:seq</code> , <code>:sequence</code>	Insert an integer sequence, one per line: <code>:seq [step]</code> .
<code>:field</code> , <code>:cut</code>	Keep only the Nth whitespace field of each selected line (<code>awk '{print \$N}'</code>).
<code>:running-total</code> , <code>:cumsum</code>	Replace each numeric line with the cumulative total so far.
<code>:diff-lines</code> , <code>:deltas</code>	Replace each numeric line with its delta from the previous (inverse of <code>running-total</code>).
<code>:sum-column</code> , <code>:sumcol</code>	Sum the Nth whitespace field across the selected lines (non-destructive).
<code>:shuffle</code> , <code>:shuf</code>	Randomly reorder the selected lines (Fisher-Yates).
<code>:sample</code> , <code>:random-lines</code>	Keep N random lines from the selection, preserving order (<code>:sample 10</code>).
<code>:jsonl-to-json</code> , <code>:jsonl-json</code>	Convert the selected JSONL/NDJSON (one value per line) to a JSON array.
<code>:json-to-jsonl</code> , <code>:json-jsonl</code>	Convert the selected JSON array to JSONL (one compact value per line).
<code>:head</code> , <code>:first-lines</code>	Keep only the first N lines of the selection (<code>:head 10</code>).
<code>:tail</code> , <code>:last-lines</code>	Keep only the last N lines of the selection (<code>:tail 10</code>).
<code>:rev</code> , <code>:reverse-each-line</code>	Reverse the characters of each selected line independently (Unix <code>rev</code>).
<code>:json-table</code> , <code>:json-tbl</code>	Render the selected JSON array of objects as an aligned plain-text table.
<code>:hexdump</code> , <code>:xxd</code>	Render the selection as an xxd-style hex dump (offset, hex bytes, ASCII).
<code>:dedup</code> , <code>:unique-lines</code> , <code>:delete-duplicate-lines</code>	Remove all duplicate lines globally, keeping first occurrence and order. Emacs <code>delete-duplicate-lines</code> .
<code>:caesar</code> , <code>:shift-letters</code>	Caesar-shift the selection's letters by N (e.g. <code>:caesar 13</code> = ROT13; N may be negative).
<code>:base32-encode</code> , <code>:base32</code>	Base32-encode the selection (RFC 4648).
<code>:base32-decode</code> , <code>:unbase32</code>	Base32-decode the selection (RFC 4648).
<code>:crc32</code> , <code>:checksum</code>	Show the CRC32 (IEEE) checksum of the selection in hex and decimal (non-destructive).
<code>:rot47</code> , <code>:rot-47</code>	Apply ROT47 to the selection (rotates all printable ASCII; self-inverse).
<code>:morse-encode</code> , <code>:morse</code>	Encode the selection (A-Z, 0-9) to Morse code (words separated by /).
<code>:morse-decode</code> , <code>:unmorse</code>	Decode Morse code in the selection back to text.
<code>:human-bytes</code> , <code>:humanize-size</code>	Convert each numeric line (a byte count) to a human-readable size like 1.5 KiB.
<code>:ordinal</code> , <code>:ordinalize</code>	Convert each numeric line to its ordinal (1 → 1st, 22 → 22nd).
<code>:to-snake</code> , <code>:snake-case</code>	Convert the selected identifier to <code>snake_case</code> .
<code>:to-kebab</code> , <code>:kebab-case</code>	Convert the selected identifier to kebab-case.
<code>:to-camel</code> , <code>:camel-case</code>	Convert the selected identifier to camelCase.

Name	Description
<code>:to-pascal, :pascal-case</code>	Convert the selected identifier to PascalCase.
<code>:to-constant, :screaming-snake, :upper-snake</code>	Convert the selected identifier to CONSTANT_CASE.
<code>:to-binary, :text-to-binary</code>	Convert the selection to space-separated 8-bit binary.
<code>:from-binary, :binary-to-text</code>	Convert space-separated binary in the selection back to text.
<code>:natural-sort, :sort-natural</code>	Sort the selected lines in natural order (file2 before file10).
<code>:pad-right, :ljust</code>	Left-justify each selected line, padding with spaces to a minimum width.
<code>:pad-left, :rjust</code>	Right-justify each selected line, padding with spaces to a minimum width.
<code>:json-keys, :json-fields</code>	List the keys of the selected JSON object (or union across an array of objects).
<code>:json-type, :json-describe</code>	Show the JSON type and size of the selection in the status line (non-destructive).
<code>:after, :cut-after</code>	Keep the text after the first on each selected line.
<code>:before, :cut-before</code>	Keep the text before the first on each selected line.
<code>:swapcase, :invert-case</code>	Invert the case of each character in the selection (Hello → hELLO).
<code>:strip-invisible, :strip-zero-width</code>	Remove zero-width / invisible Unicode characters from the selection.
<code>:lines-to-json, :lines-to-array</code>	Wrap the selected lines into a JSON array of strings.
<code>:json-to-lines, :array-to-lines</code>	Unwrap a JSON array in the selection into one line per element.
<code>:checkbox-list, :task-list</code>	Turn the selected lines into a Markdown task list (- [] item).
<code>:unwrap-paragraphs, :unhardwrap</code>	Join hard-wrapped lines within each paragraph into single lines.
<code>:sql-in, :sql-in-list</code>	Build a SQL IN-list ('a', 'b', 'c') from the selected lines.
<code>:dec-to-hex, :to-hex-num</code>	Convert each decimal number line to hexadecimal.
<code>:hex-to-dec, :from-hex-num</code>	Convert each hexadecimal number line to decimal.
<code>:unicode-escape, :u-escape</code>	Escape non-ASCII characters in the selection as \u{XXXX}.
<code>:unicode-unescape, :u-unescape</code>	Decode \u{XXXX} and \uXXXX escapes in the selection back to characters.
<code>:sort-by-length, :sortlen</code>	Sort the selected lines by length (shortest first).
<code>:count-unique, :distinct-count</code>	Report the number of distinct vs total selected lines (non-destructive).
<code>:rotate-lines, :rotate</code>	Cyclically rotate the selected lines by N (negative rotates the other way).
<code>:unquote-lines, :strip-quotes-lines</code>	Remove surrounding quotes from each selected line independently.
<code>:quote-lines, :quote-each</code>	Wrap each selected line in double quotes (escaping " and ").
<code>:repeat, :repeat-text</code>	Repeat the selected text N times (:repeat 3).
<code>:capitalize-lines, :capitalize</code>	Uppercase the first letter of each selected line.
<code>:remove-blank-lines, :remove-empty</code>	Remove all blank (whitespace-only) lines from the selection.
<code>:trim-lines, :trim</code>	Trim leading and trailing whitespace from each selected line.
<code>:kv-to-json, :env-to-json</code>	Convert key=value / key: value lines in the selection to a JSON object.
<code>:json-to-kv, :json-to-env</code>	Convert the selected JSON object to key=value lines.

Name	Description
<code>:json-pluck, :json-values-of</code>	Extract one field's value from each object in a JSON array, one per line.
<code>:to-html-list, :html-list</code>	Convert the selected lines into an HTML
<code>:from-html-list, :html-list-to-lines</code>	Extract
<code>:csv-to-html-table, :csv-to-html</code>	Convert the selected CSV/TSV (first row = headers) to an HTML
<code>:slugify-lines, :slug-lines</code>	Slugify each selected line independently (URL-friendly).
<code>:lines-to-csv-row, :join-csv</code>	Join the selected lines into one CSV row (RFC-4180 quoting).
<code>:csv-row-to-lines, :split-csv</code>	Split a CSV row in the selection into one field per line (quote-aware).
<code>:deslugify, :unslugify</code>	Turn a slug back into a Title Cased phrase (hyphens/underscores to spaces).
<code>:csv-to-tsv, :csv-tsv</code>	Convert the selected CSV to tab-separated values (quote-aware).
<code>:tsv-to-csv, :tsv-csv</code>	Convert the selected TSV to CSV (RFC-4180 quoting).
<code>:strip-line-numbers, :unnumber</code>	Remove a leading line number (and separator) from each selected line.
<code>:markdown-link, :md-link</code>	Wrap the selected text as a Markdown link text .
<code>:extract-urls, :urls</code>	Replace the selection with the http(s) URLs found in it, one per line.
<code>:extract-emails, :emails</code>	Replace the selection with the email addresses found in it, one per line.
<code>:extract-ips, :ips</code>	Replace the selection with the IPv4 addresses found in it, one per line.
<code>:extract-quoted, :quoted-strings</code>	Replace the selection with the contents of double-quoted strings, one per line.
<code>:extract-between, :between</code>	Extract substrings between and delimiters, one per line.
<code>:wrap-with, :surround-with</code>	Wrap the selection with the given text on both sides (:wrap-with **).
<code>:extract-numbers, :numbers</code>	Replace the selection with the numbers found in it, one per line.
<code>:json-validate, :json-check</code>	Report whether the selection is valid JSON (with error location) — non-destructive.
<code>:csv-validate, :csv-check</code>	Check all CSV rows have the same field count (non-destructive).
<code>:ordered-list, :numbered-list</code>	Turn the selected lines into a Markdown ordered list (1. 2. 3.).
<code>:strip-list-markers, :unlist</code>	Strip leading bullet/number/checkbox list markers from each selected line.
<code>:sort-words, :sort-fields</code>	Sort the whitespace-separated words within each selected line.
<code>:unique-words, :dedup-words</code>	Remove duplicate words within each selected line (first occurrence kept).
<code>:sum-fields, :row-sum</code>	Replace each line with the sum of its numeric fields (row total).
<code>:avg-fields, :row-avg</code>	Replace each line with the mean of its numeric fields.
<code>:max-fields, :row-max</code>	Replace each line with the maximum of its numeric fields.
<code>:min-fields, :row-min</code>	Replace each line with the minimum of its numeric fields.
<code>:range-fields, :row-range</code>	Replace each line with the range (max - min) of its numeric fields.
<code>:to-env-export, :export-vars</code>	Prefix each KEY=value line with <code>export</code> (turn a .env into shell exports).

Name	Description
<code>:strip-export, :unexport</code>	Remove a leading <code>export</code> from each selected line.
<code>:dos2unix, :crlf-to-lf</code>	Convert CRLF/CR line endings in the selection to LF.
<code>:unix2dos, :lf-to-crlf</code>	Convert LF line endings in the selection to CRLF.
<code>:percent-of-total, :percentages</code>	Replace each numeric line with its percentage of the column total.
<code>:running-max, :cummax</code>	Replace each numeric line with the running maximum so far.
<code>:running-min, :cummin</code>	Replace each numeric line with the running minimum so far.
<code>:to-fixed, :round-to</code>	Format each numeric line to N decimal places (<code>:to-fixed 2</code>).
<code>:clamp, :clip</code>	Clamp each numeric line to the [min, max] range (<code>:clamp 0 100</code>).
<code>:scale, :multiply-by</code>	Multiply each numeric line by a factor (<code>:scale 1.5</code>).
<code>:offset, :add-to-each</code>	Add N to each numeric line (<code>:offset 10</code> ; negative subtracts).
<code>:abs, :absolute-value</code>	Replace each numeric line with its absolute value.
<code>:linkify, :auto-link</code>	Wrap bare URLs in the selection with Markdown link syntax <code>url</code> .
<code>:strip-markdown-links, :unlink</code>	Replace <code>text</code> Markdown links with just their text.
<code>:strip-emphasis, :strip-md-emphasis</code>	Remove Markdown bold/italic/code emphasis markers from the selection.
<code>:strip-html-comments, :strip-comments-html</code>	Remove HTML/Markdown comments from the selection.
<code>:remove-trailing-commas, :fix-trailing-commas</code>	Remove trailing commas before <code>}</code> or <code>]</code> (JSON5/JS to strict JSON).
<code>:add-trailing-commas, :trailing-commas</code>	Add trailing commas before <code>}</code> or <code>]</code> (cleaner JS/JSON5 diffs).
<code>:smart-quotes, :curly-quotes</code>	Convert straight quotes to typographic curly quotes (context-aware).
<code>:typographic-dashes, :em-dash</code>	Convert <code>—</code> to em dash, <code>–</code> to en dash, <code>...</code> to ellipsis.
<code>:de-typography, :ascii-punctuation</code>	Normalize curly quotes/dashes/ellipsis back to ASCII punctuation.
<code>:to-ascii, :transliterate</code>	Transliterate accented Latin characters to ASCII (café → cafe).
<code>:nato, :phonetic</code>	Spell the selection in the NATO phonetic alphabet (A → Alfa).
<code>:transpose-grid, :transpose-ws</code>	Transpose a whitespace-separated grid (rows become columns).
<code>:repeat-lines, :duplicate-each</code>	Repeat each selected line N times (<code>:repeat-lines 3</code>).
<code>:rename-word, :rename-local</code>	Rename every whole-word occurrence of the symbol under the cursor in this buffer.
<code>:grep-word, :gw, :find-references</code>	Search the project for the whole word under the cursor (jumpable in Run).
<code>:todos, :project-todos, :fixme</code>	Scan the whole project for TODO/FIXME/HACK/XXX/BUG/NOTE markers (jumpable in Run).
<code>:registers, :reg, :display</code>	Show the contents of all registers.
<code>:yank-join</code>	Yank joined selections. A separator can be provided as first argument. Default value is newline.
<code>:clipboard-yank</code>	Yank main selection into system clipboard.
<code>:clipboard-yank-join</code>	Yank joined selections into system clipboard. A separator can be provided as first argument. Default value is newline.
<code>:primary-clipboard-yank</code>	Yank main selection into system primary clipboard.

Name	Description
<code>:primary-clipboard-yank-join</code>	Yank joined selections into system primary clipboard. A separator can be provided as first argument. Default value is newline.
<code>:clipboard-paste-after</code>	Paste system clipboard after selections.
<code>:clipboard-paste-before</code>	Paste system clipboard before selections.
<code>:clipboard-paste-replace</code>	Replace selections with content of system clipboard.
<code>:primary-clipboard-paste-after</code>	Paste primary clipboard after selections.
<code>:primary-clipboard-paste-before</code>	Paste primary clipboard before selections.
<code>:primary-clipboard-paste-replace</code>	Replace selections with content of system primary clipboard.
<code>:show-clipboard-provider</code>	Show clipboard provider name in status bar.
<code>:change-current-directory, :cd, :chdir, :lcd, :lchdir, :tcd, :tchdir</code>	Change the current working directory.
<code>:show-directory-stack</code>	Show the directory stack as a delimited string.
<code>:push-directory, :pushd</code>	Save and then change the current directory.
<code>:pop-directory, :popd</code>	Remove the top entry from the directory stack, and cd to the new top directory..
<code>:show-directory, :pwd</code>	Show the current working directory.
<code>:encoding</code>	Set encoding. Based on https://encoding.spec.whatwg.org .
<code>:character-info, :char, :ascii</code>	Get info about the character under the primary cursor.
<code>:reload, :rl</code>	Discard changes and reload from the source file.
<code>:reload-all, :rla</code>	Discard changes and reload all documents from the source files.
<code>:checktime, :checkt</code>	Reload loaded buffers that changed on disk (vim :checktime).
<code>:filetype, :filet</code>	Report the buffer's detected language / accept on
<code>:scriptnames, :scr</code>	List the sourced config files (vim :scriptnames).
<code>:wshada, :wsh, :wsha, :wshad</code>	Write the registers and every buffer's marks to the shada state file (nvim :wshada; zmax's own line format, not nvim's msgpack).
<code>:rshada, :rsh, :rsha, :rshad</code>	Read the registers and marks back from the shada state file (nvim :rshada).
<code>:syncbind, :sync, :syncb, :syncbi, :syncbin</code>	Scroll the 'scrollbind' (follow-mode) windows back into step with this one (vim :syncbind).
<code>:uptime, :upt, :uptim</code>	Show how long this editor process has been running (nvim :uptime).
<code>:mksession, :mks</code>	Write a session file (cwd + buffers) that :source restores (vim :mksession).
<code>:mkvimrc, :mkv</code>	Write the current runtime mappings to a vimrc file (vim :mkvimrc; mappings only).
<code>:mkexrc, :mk</code>	Write the current runtime mappings to an exrc file (vim :mkexrc; mappings only).
<code>:mkview, :mkvie</code>	Write the current window's view (cursor position) to a file (vim :mkview).
<code>:loadview, :lo</code>	Restore the current window's view by sourcing its :mkview file (vim :loadview).
<code>:swapname, :sw</code>	Show the current buffer's swap file name (vim :swapname).
<code>:preserve, :pre</code>	Flush the buffer to its swap file now (vim :preserve).

Name	Description
<code>:recover, :rec</code>	Replace the buffer with the contents of its swap file (vim <code>:recover</code>).
<code>:wundo, :wun</code>	Write the buffer's undo history to {file} (vim <code>:wundo</code>).
<code>:rundo, :rund</code>	Read the buffer's undo history from {file} (vim <code>:rundo</code>).
<code>:git-stage, :stage, :git-add</code>	Stage the current buffer's file (git add).
<code>:vc-root-version-diff, :vc-root-diff</code>	Unified diff of the whole working tree vs a revision, default HEAD (emacs <code>vc-root-version-diff</code>).
<code>:vc-revision-other-window</code>	Show a past revision of the current file, default HEAD (emacs <code>vc-revision-other-window</code>).
<code>:git-unstage, :unstage</code>	Unstage the current buffer's file (git reset HEAD).
<code>:stash, :git-stash</code>	git stash the working-tree changes (then reload open buffers).
<code>:stash-pop, :git-stash-pop</code>	git stash pop the most recent stash (then reload open buffers).
<code>:update, :u</code>	Write changes only if the file has been modified.
<code>:lsp-workspace-command</code>	Open workspace command picker
<code>:lsp-restart</code>	Restarts the given language servers, or all language servers that are used by the current file if no arguments are supplied
<code>:lsp-health, :lsp-status</code>	Show a health report of language servers: which are ready, initializing, or not running, plus each server's supported features
<code>:set, :se</code>	Set options with vim syntax (<code>:set nu, :set nowrap, :set tw=80</code>); no args lists all options.
<code>:setlocal, :setl</code>	Set options for the current buffer only, leaving the global default alone (vim <code>:setlocal</code>).
<code>:setglobal, :setg</code>	Set the global default without changing this buffer's value or behavior (vim <code>:setglobal</code>).
<code>:map</code>	Map {lhs} to {rhs} in normal+select modes (Vim <code>:map</code>).
<code>:noremap</code>	Non-recursive <code>:map</code> in normal+select modes.
<code>:nmap</code>	Map {lhs} to {rhs} in normal mode (Vim <code>:nmap</code>).
<code>:nnoremap</code>	Non-recursive normal-mode map (Vim <code>:nnoremap</code>).
<code>:imap</code>	Map {lhs} to {rhs} in insert mode (Vim <code>:imap</code>).
<code>:inoremap</code>	Non-recursive insert-mode map (Vim <code>:inoremap</code>).
<code>:vmap</code>	Map {lhs} to {rhs} in select/visual mode (Vim <code>:vmap</code>).
<code>:vnoremap</code>	Non-recursive select/visual-mode map (Vim <code>:vnoremap</code>).
<code>:xmap</code>	Map {lhs} to {rhs} in visual mode (Vim <code>:xmap</code>).
<code>:xnoremap</code>	Non-recursive visual-mode map (Vim <code>:xnoremap</code>).
<code>:smap</code>	Map {lhs} to {rhs} in select mode (Vim <code>:smap</code>).
<code>:snoremap</code>	Non-recursive select-mode map (Vim <code>:snoremap</code>).
<code>:omap</code>	Map {lhs} to {rhs} in operator-pending mode (Vim <code>:omap</code>).
<code>:onoremap</code>	Non-recursive operator-pending map (Vim <code>:onoremap</code>).
<code>:unmap</code>	Remove a runtime {lhs} mapping (normal+select).
<code>:nunmap</code>	Remove a runtime normal-mode {lhs} mapping.
<code>:iunmap</code>	Remove a runtime insert-mode {lhs} mapping.

Name	Description
<code>:vunmap</code>	Remove a runtime select/visual-mode {lhs} mapping.
<code>:xunmap</code>	Remove a runtime visual-mode {lhs} mapping.
<code>:mapclear</code>	Clear runtime normal+select-mode mappings.
<code>:nmapclear</code>	Clear runtime normal-mode mappings.
<code>:imapclear</code>	Clear runtime insert-mode mappings.
<code>:vmapclear</code>	Clear runtime select/visual-mode mappings.
<code>:xmapclear</code>	Clear runtime visual-mode mappings (Vim :xmapclear).
<code>:smapclear</code>	Clear runtime select-mode mappings (Vim :smapclear).
<code>:omapclear</code>	Clear runtime operator-pending mappings (Vim :omapclear).
<code>:sunmap</code>	Remove a runtime select-mode {lhs} mapping (Vim :sunmap).
<code>:ounmap</code>	Remove a runtime operator-pending {lhs} mapping (Vim :ounmap).
<code>:cmap, :cm, :cma, :cnoremap, :cno, :cnor, :cnore</code>	Map {lhs} to {rhs} in Command-line mode (Vim :cmap/:cnoremap).
<code>:cunmap, :cu, :cun, :cunm</code>	Remove a Command-line-mode {lhs} mapping (Vim :cunmap).
<code>:cmapclear, :cmapc, :cmapcl</code>	Clear all Command-line-mode mappings (Vim :cmapclear).
<code>:lmap, :lm, :lma, :lnoremap, :lno, :lnor</code>	Map {lhs} to {rhs} in Lang-Arg mode — the 'keymap' table (Vim :lmap).
<code>:lunmap, :lu, :lun, :lunm</code>	Remove a Lang-Arg {lhs} mapping (Vim :lunmap).
<code>:lmapclear, :lmapc, :lmapcl</code>	Clear all Lang-Arg mappings (Vim :lmapclear).
<code>:describe-input-method, :describe-current-input-method</code>	Show the active (or named) keymap's documentation and key translations (emacs describe-input-method).
<code>:list-input-methods</code>	List every input method installed in the 'runtimepath' (emacs list-input-methods).
<code>:activate-transient-input-method, :transient-input-method</code>	Enable an input method for the next character only, then disable it (emacs C-x).
<code>:quail-show-key</code>	Show which key sequence of the active input method inputs the character after the cursor (emacs quail-show-key).
<code>:toggle-debug-on-error</code>	Toggle whether a failing command opens a backtrace (emacs toggle-debug-on-error).
<code>:test-buffer</code>	Run the tests of the current buffer's file (spacemacs SPC m t b).
<code>:tmap, :tma, :tnoremap, :tno, :tnor, :tnore</code>	Map {lhs} to {rhs} in Terminal mode (Vim :tmap/:tnoremap).
<code>:tunmap, :tunma, :tunm</code>	Remove a Terminal-mode {lhs} mapping (Vim :tunmap).
<code>:tmapclear, :tmapc, :tmapcl</code>	Clear all Terminal-mode mappings (Vim :tmapclear).
<code>:loadkeymap, :loadk, :loadke</code>	Load the following keymaps until EOF — only in a sourced file (Vim :loadkeymap).
<code>:gui, :gu, :gvim, :gv</code>	Start the GUI (Vim :gui/:gvim) — zmax is a TUI, so this is E25.
<code>:winsize, :wi, :win</code>	Get or set the window size (Vim :winsize) — the editor area, in cells.
<code>:winpos, :winp</code>	Get or set the window position on the screen (Vim :winpos) — E188 in a terminal.
<code>:fclose, :fc, :fcl</code>	Close the topmost floating window — picker, popup or panel (Vim :fclose).

Name	Description
<code>:restart</code>	Restart the editor, re-executing it with the same arguments (Vim <code>:restart</code>).
<code>:diffput</code> , <code>:diffpu</code> , <code>:dp</code>	Write the hunk under the selection into the diff base (Vim <code>:diffput</code>).
<code>:filter</code> , <code>:filt</code>	Run a command and show only its output lines matching a pattern (Vim <code>:filter</code>).
<code>:spellrepall</code> , <code>:spellr</code> , <code>:spellre</code>	Replace every bad word the last z= replaced, with the same word (Vim <code>:spellrepall</code>).
<code>:menu</code> , <code>:me</code> , <code>:men</code>	Add a menu item {path} {rhs} for normal/visual/select/op-pending (Vim <code>:menu</code>); bare <code>:menu</code> lists them.
<code>:noremenu</code> , <code>:noreme</code> , <code>:noem</code>	Non-remappable <code>:menu</code> (Vim <code>:noremenu</code>).
<code>:unmenu</code> , <code>:unme</code>	Remove the menu item {path} (<code>:unmenu *</code> removes all) (Vim <code>:unmenu</code>).
<code>:amenu</code> , <code>:am</code> , <code>:ame</code>	Add a menu item for all modes (Vim <code>:amenu</code>).
<code>:anoremenu</code> , <code>:an</code> , <code>:anoreme</code>	Non-remappable <code>:amenu</code> (Vim <code>:anoremenu</code>).
<code>:aunmenu</code> , <code>:aun</code>	Remove an all-mode menu item (Vim <code>:aunmenu</code>).
<code>:nmenu</code> , <code>:nme</code>	Add a normal-mode menu item (Vim <code>:nmenu</code>).
<code>:nnoremenu</code> , <code>:nnoreme</code>	Non-remappable <code>:nmenu</code> (Vim <code>:nnoremenu</code>).
<code>:nunmenu</code> , <code>:nunme</code>	Remove a normal-mode menu item (Vim <code>:nunmenu</code>).
<code>:vmenu</code> , <code>:vme</code>	Add a visual+select-mode menu item (Vim <code>:vmenu</code>).
<code>:vnoremenu</code> , <code>:vnoreme</code>	Non-remappable <code>:vmenu</code> (Vim <code>:vnoremenu</code>).
<code>:vunmenu</code> , <code>:vunme</code>	Remove a visual+select-mode menu item (Vim <code>:vunmenu</code>).
<code>:xmenu</code> , <code>:xme</code>	Add a visual-mode menu item (Vim <code>:xmenu</code>).
<code>:xnoremenu</code> , <code>:xnoreme</code>	Non-remappable <code>:xmenu</code> (Vim <code>:xnoremenu</code>).
<code>:xunmenu</code> , <code>:xunme</code>	Remove a visual-mode menu item (Vim <code>:xunmenu</code>).
<code>:smenu</code> , <code>:sme</code>	Add a select-mode menu item (Vim <code>:smenu</code>).
<code>:snoremenu</code> , <code>:snoreme</code>	Non-remappable <code>:smenu</code> (Vim <code>:snoremenu</code>).
<code>:sunmenu</code> , <code>:sunme</code>	Remove a select-mode menu item (Vim <code>:sunmenu</code>).
<code>:omenu</code> , <code>:ome</code>	Add an operator-pending menu item (Vim <code>:omenu</code>).
<code>:onoremenu</code> , <code>:onoreme</code>	Non-remappable <code>:omenu</code> (Vim <code>:onoremenu</code>).
<code>:ounmenu</code> , <code>:ounme</code>	Remove an operator-pending menu item (Vim <code>:ounmenu</code>).
<code>:imenu</code> , <code>:ime</code>	Add an insert-mode menu item (Vim <code>:imenu</code>).
<code>:inoremenu</code> , <code>:inoreme</code>	Non-remappable <code>:imenu</code> (Vim <code>:inoremenu</code>).
<code>:iunmenu</code> , <code>:iunme</code>	Remove an insert-mode menu item (Vim <code>:iunmenu</code>).
<code>:cmenu</code> , <code>:cme</code>	Add a command-line-mode menu item (Vim <code>:cmenu</code>).
<code>:cnoremenu</code> , <code>:cnoreme</code>	Non-remappable <code>:cmenu</code> (Vim <code>:cnoremenu</code>).
<code>:cunmenu</code> , <code>:cunme</code>	Remove a command-line-mode menu item (Vim <code>:cunmenu</code>).
<code>:tmenu</code> , <code>:tln</code>	Add a terminal-mode menu item (Vim <code>:tmenu</code>).
<code>:tlnoremenu</code> , <code>:tlno</code>	Non-remappable <code>:tmenu</code> (Vim <code>:tlnoremenu</code>).

Name	Description
<code>:tlunmenu, :tlu</code>	Remove a terminal-mode menu item (Vim :tlunmenu).
<code>:tmenu, :tm</code>	Set a menu item's tooltip (Vim :tmenu).
<code>:tunmenu, :tu</code>	Remove a menu item's tooltip (Vim :tunmenu).
<code>:emenu, :em</code>	Execute the menu item {path} (Vim :emenu).
<code>:popup, :popu</code>	Pop up the menu {name} as a picker and run what is chosen (Vim :popup).
<code>:menutranslate, :menut</code>	Translate a menu path component (:menutranslate clear drops the table) (Vim :menutranslate).
<code>:echohl, :echoh</code>	Use {group} for the following :echo commands; :echohl None stops (vim :echohl).
<code>:mkspell, :mksp</code>	Compile word lists into the spell dictionary {outname} that :set spellang={outname} then uses (vim :mkspell).
<code>:packdel, :packd</code>	Remove an installed package from the 'packpath' (nvim :packdel).
<code>:packupdate, :packu</code>	Update the installed packages (git pull --ff-only in each) (nvim :packupdate).
<code>:package-menu-filter-upgradable, :packupgradable</code>	List only the installed packages that have updates available (emacs package-menu-filter-upgradable).
<code>:normal, :noim, :normal!, :norm!</code>	Execute {commands} as normal-mode keystrokes (vim :normal[!]).
<code>:mark, :k</code>	Set mark {x} at the cursor position (vim :mark / :k).
<code>:buffer, :buf, :b</code>	Switch to the open buffer whose path contains {name} (vim :buffer / :b).
<code>:resize, :res</code>	Adjust the current window height (vim :resize [+/-]{N}).
<code>:let</code>	Set a vimscript variable via the embedded interpreter (:let x = 42).
<code>:Files</code>	Fuzzy-find files with fzf and open the selection (fzf.vim :Files).
<code>:GFiles, :GitFiles</code>	Fuzzy-find git-tracked files with fzf and open the pick (fzf.vim :GFiles).
<code>:Rg, :Ag, :RG</code>	Ripgrep the tree with fzf; open the pick at its line (fzf.vim :Rg/:Ag).
<code>:Todo, :TODO, :Todos</code>	TODO tool window: ripgrep TODO/FIXME/HACK/XXX across the tree, jump to the pick.
<code>:Locate</code>	locate(1) files with fzf and open the pick (fzf.vim :Locate).
<code>:BLines</code>	Fuzzy-search the current buffer's lines with fzf, jump to the pick (fzf.vim :BLines).
<code>:Lines</code>	Fuzzy-search lines across all open buffers with fzf, open the pick (fzf.vim :Lines).
<code>:History</code>	Fuzzy-pick a recently opened file with fzf and open it (fzf.vim :History).
<code>:History:</code>	Fuzzy-pick a past : command with fzf and run it (fzf.vim :History:).
<code>:History/</code>	Fuzzy-pick a past search with fzf and re-run it (fzf.vim :History/).
<code>:Filetypes</code>	Fuzzy-pick a language with fzf and set the buffer's filetype (fzf.vim :Filetypes).
<code>:Commits</code>	Fuzzy-pick a repo commit with fzf and show it (fzf.vim :Commits).
<code>:BCommits</code>	Fuzzy-pick a commit touching the current file with fzf and show it (fzf.vim :BCommits).
<code>:fzf-git-show</code>	(internal) show a <sha> subject fzf pick via git show.
<code>:Jumps</code>	Fuzzy-pick a jumplist entry with fzf and open it (fzf.vim :Jumps).

Name	Description
<code>:RecentLocations, :recent-locations</code>	Recent Locations (JetBrains): jump ring newest-first, deduped, with context.
<code>:LocalHistory, :local-history</code>	Local History (JetBrains): pick a saved snapshot of this file and open it.
<code>:local-history-open</code>	(internal) open a <code>:LocalHistory</code> snapshot pick.
<code>:Windows</code>	Fuzzy-pick an open window with fzf and focus it (fzf.vim <code>:Windows</code>).
<code>:Marks</code>	Fuzzy-pick a mark in the current buffer with fzf and jump to it (fzf.vim <code>:Marks</code>).
<code>:Tags</code>	Fuzzy-pick a ctags tag across the tree with fzf and jump to it (fzf.vim <code>:Tags</code>).
<code>:BTags</code>	Fuzzy-pick a ctags tag in the current file with fzf and jump to it (fzf.vim <code>:BTags</code>).
<code>:Snippets</code>	Fuzzy-pick a snippet with fzf and insert its body (fzf.vim <code>:Snippets</code>).
<code>:Maps</code>	Fuzzy-browse the current keymaps with fzf (fzf.vim <code>:Maps</code>).
<code>:Helptags</code>	Fuzzy-pick a help tag with fzf and open the Help browser at it (fzf.vim <code>:Helptags</code>).
<code>:fzf-window</code>	(internal) focus the Nth window of a <code>:Windows</code> fzf pick.
<code>:fzf-snippet</code>	(internal) insert the body of a <code>:Snippets</code> fzf pick.
<code>:fzf-map</code>	(internal) echo a <code>:Maps</code> fzf pick.
<code>:fzf-helptag</code>	(internal) open the Help browser at a <code>:Helptags</code> fzf pick.
<code>:fzf-goto</code>	(internal) open a <code>file:line:col:text</code> fzf pick at the line.
<code>:fzf-line</code>	(internal) jump to the line of an <code>N: text</code> fzf pick.
<code>:Colors</code>	Fuzzy-pick a colorscheme with fzf (fzf.vim <code>:Colors</code>).
<code>:Buffers</code>	Fuzzy-pick an open buffer with fzf and switch to it (fzf.vim <code>:Buffers</code>).
<code>:Commands</code>	Fuzzy-pick a <code>:</code> command with fzf and run it (fzf.vim <code>:Commands</code>).
<code>:fold, :fo</code>	Create a fold over the selected/current lines (vim <code>:fold</code>).
<code>:foldopen, :foldo</code>	Open the fold under the cursor (vim <code>:foldopen</code>).
<code>:foldclose, :foldc</code>	Close the fold under the cursor (vim <code>:foldclose</code>).
<code>:&, :&&, :s-repeat</code>	Repeat the last <code>:substitute</code> on the current line (vim <code>:& / :&&</code>).
<code>:sleep, :sl</code>	Do nothing for {count} seconds (vim <code>:sleep</code>).
<code>:echomsg, :echom</code>	Echo an expression and save it in the message history (vim <code>:echomsg</code>).
<code>:eval, :ev</code>	Evaluate an expression and discard the result (vim <code>:eval</code>).
<code>:call, :cal</code>	Call a function (vim <code>:call</code>).
<code>:defer, :defe</code>	Call a function when the current function is done; arguments are evaluated now (vim <code>:defer</code>).
<code>:execute, :exe</code>	Execute the string result of an expression as an Ex command (vim <code>:execute</code>).
<code>:const, :cons</code>	Create a variable as a constant (vim <code>:const</code>).
<code>:unlet, :unl</code>	Delete a variable (vim <code>:unlet</code>).
<code>:lockvar, :lockv</code>	Lock variables against <code>:let / :unlet</code> (vim <code>:lockvar [depth] {name}</code>).

Name	Description
<code>:unlockvar, :unlo</code>	Unlock variables locked with <code>:lockvar</code> (vim <code>:unlockvar [depth] {name}</code>).
<code>:ptnext, :ptn</code>	Show the next matching tag in a preview split (vim <code>:ptnext</code>).
<code>:ptprevious, :ptp, :ptNext, :ptN</code>	Show the previous matching tag in a preview split (vim <code>:ptprevious / :ptNext</code>).
<code>:ptfirst, :ptrewind, :ptr</code>	Show the first matching tag in a preview split (vim <code>:ptfirst / :ptrewind</code>).
<code>:ptlast, :ptl</code>	Show the last matching tag in a preview split (vim <code>:ptlast</code>).
<code>:pclose, :pc</code>	Close the preview window (vim <code>:pclose</code>).
<code>:startgreplace, :startg</code>	Start Virtual Replace mode (vim <code>:startgreplace</code>).
<code>:print, :p</code>	Display the selected lines (or current line) in a scratch buffer (vim <code>:print</code>).
<code>:number, :nu, :#</code>	Like <code>:print</code> , with line numbers (vim <code>:number / :#</code>).
<code>:list, :l</code>	Like <code>:print</code> , marking each line end with <code>\$</code> (vim <code>:list</code>).
<code>:print-line-number, :=</code>	Echo the last line number of the buffer (vim <code>:=</code>).
<code>:version, :ver</code>	Show the zmax version and compiled feature summary (vim <code>:version</code>).
<code>:intro, :int</code>	Show the introductory message (vim <code>:intro</code>).
<code>:redrawstatus</code>	Redraw the status line (vim <code>:redrawstatus</code> ; approximated by a full redraw).
<code>:redrawtabline</code>	Redraw the tab line (vim <code>:redrawtabline</code> ; approximated by a full redraw).
<code>:silent, :sil</code>	Run {cmd} silently (vim <code>:silent[!]</code>); message suppression is best-effort.
<code>:unsilent</code>	Run {cmd} with messages shown (vim <code>:unsilent</code>).
<code>:verbose, :verb</code>	Run {cmd} verbosely, optional leading count (vim <code>:verbose</code>).
<code>:noautocmd, :noa</code>	Run {cmd} without triggering autocommands (vim <code>:noautocmd</code>).
<code>:sandbox, :san</code>	Run {cmd} in the sandbox (vim <code>:sandbox</code> ; best-effort).
<code>:confirm, :conf</code>	Run {cmd} confirming risky actions (vim <code>:confirm</code> ; best-effort).
<code>:browse, :bro</code>	Run {cmd}, picking its file with the file picker when none is given (vim <code>:browse</code>).
<code>:hide, :hid</code>	Run {cmd} keeping the current buffer hidden (vim <code>:hide</code>).
<code>:vertical, :vert</code>	Run {cmd} with vertical split placement (vim <code>:vertical</code> ; best-effort).
<code>:horizontal, :hor</code>	Run {cmd} with horizontal split placement (vim <code>:horizontal</code>).
<code>:tab</code>	Run {cmd} opening its window in a new tab (vim <code>:tab</code> ; best-effort placement).
<code>:lsp-stop</code>	Stops the given language servers, or all language servers that are used by the current file if no arguments are supplied
<code>:tree-sitter-scopes</code>	Display tree sitter scopes, primarily for theming and development.
<code>:tree-sitter-highlight-name, :Inspect</code>	Display the tree-sitter highlight capture under the cursor (neovim <code>:Inspect</code>).
<code>:tree-sitter-layers</code>	Display language names of tree-sitter injection layers under the cursor.
<code>:debug-start, :dbg</code>	Start a debug session from a given template with given parameters.

Name	Description
<code>:debug-remote</code> , <code>:dbg-tcp</code>	Connect to a debug adapter by TCP address and start a debugging session from a given template with given parameters.
<code>:debug-eval</code>	Evaluate expression in current debug context.
<code>:close</code> , <code>:clo</code>	Close the current window (vim <code>:close</code>). Refuses to close the last window.
<code>:only</code> , <code>:on</code>	Close all windows except the current one (vim <code>:only</code>).
<code>:vsplit</code> , <code>:vs</code>	Open the file in a vertical split.
<code>:vsplit-new</code> , <code>:vnew</code>	Open a scratch buffer in a vertical split.
<code>:hsplit</code> , <code>:hs</code> , <code>:sp</code> , <code>:split</code>	Open the file in a horizontal split.
<code>:hsplit-new</code> , <code>:hnew</code>	Open a scratch buffer in a horizontal split.
<code>:tutor</code>	Open the tutorial.
<code>:goto</code> , <code>:g</code>	Goto line number.
<code>:set-language</code> , <code>:lang</code> , <code>:setf</code> , <code>:setfiletype</code>	Set the language of current buffer (show current language if no value specified).
<code>:lpr-buffer</code> , <code>:print-buffer</code>	Print the whole buffer via the external lpr spooler (emacs <code>lpr-buffer</code>).
<code>:lpr-region</code> , <code>:print-region</code>	Print the selected region via the external lpr spooler (emacs <code>lpr-region</code>).
<code>:dictionary-search</code> , <code>:dictionary</code>	Look up a word (or the word at point) with the external dict client (emacs <code>dictionary-search</code>).
<code>:denote</code> , <code>:denote-create-note</code>	Create a timestamped IDENTIFIER–title__keywords.org note in ~/Documents/notes and open it (<code>denote</code>).
<code>:denote-link</code> , <code>:denote-insert-link</code>	Insert an org <code>[[denote:ID][title]]</code> link to a note; with no argument, list the notes (<code>denote-link</code>).
<code>:calendar-hebrew-list-yahrzeits</code> , <code>:list-yahrzeit-dates</code>	List a Hebrew death date's yahrzeit Gregorian dates over N years (emacs <code>calendar-hebrew-list-yahrzeits</code>).
<code>:completion-preview-mode</code>	Show the top completion candidate inline as you type, in this buffer (emacs <code>completion-preview-mode</code>).
<code>:global-completion-preview-mode</code>	Show the top completion candidate inline as you type, everywhere (emacs <code>global-completion-preview-mode</code>).
<code>:paragraph-indent-minor-mode</code>	Treat an indented line as the start of a paragraph (emacs <code>paragraph-indent-minor-mode</code>).
<code>:paragraph-indent-text-mode</code>	Plain text with indentation starting paragraphs (emacs <code>paragraph-indent-text-mode</code>).
<code>:shell-dirtrack-mode</code>	Track the shell's directory from the <code>cd/pushd/popd</code> you type (emacs <code>shell-dirtrack-mode</code>).
<code>:dirtrack-mode</code>	Track the shell's directory by reading it out of the prompt (emacs <code>dirtrack-mode</code>).
<code>:dirs</code>	Ask the shell where it actually is and resynchronise (emacs <code>dirs</code>).
<code>:bug-reference-url-format</code>	Set the tracker URL template bug references open, %s being the number (emacs <code>bug-reference-url-format</code>).
<code>:icomplete-mode</code>	Show matching candidates on the minibuffer line while typing (emacs <code>icomplete-mode</code>).
<code>:icomplete-vertical-mode</code>	Show icomplete candidates one per line (emacs <code>icomplete-vertical-mode</code>).
<code>:fido-mode</code> , <code>:fido-vertical-mode</code>	Icomplete with ido flavouring: Enter takes the top candidate (emacs <code>fido-mode</code>).
<code>:minibuffer-electric-default-mode</code>	Show the prompt's (default X) segment only until it is typed over (emacs <code>minibuffer-electric-default-mode</code>).
<code>:file-name-shadow-mode</code>	Dim the part of a typed file name that its later components override (emacs <code>file-name-shadow-mode</code>).

Name	Description
<code>:eldoc-mode</code> , <code>:global-eldoc-mode</code> , <code>:turn-on-eldoc-mode</code>	Toggle automatic signature/parameter hints at point (emacs eldoc-mode).
<code>:normal-mode</code>	Re-detect the buffer's major mode from its file (emacs normal-mode).
<code>:text-mode</code> , <code>:fundamental-mode</code>	Switch the buffer to plain text with no code syntax (emacs text-mode/fundamental-mode).
<code>:set-option</code>	Set a config option at runtime. For example to disable smart case search, use <code>:set-option search.smart-case false</code> .
<code>:toggle-option</code> , <code>:toggle</code>	Toggle a config option at runtime. For example to toggle smart case search, use <code>:toggle search.smart-case</code> .
<code>:get-option</code> , <code>:get</code>	Get the current value of a config option.
<code>:move-line-down</code>	Move the current line down by one (drag down).
<code>:move-line-up</code>	Move the current line up by one (drag up).
<code>:cycle-case</code>	Cycle the case style of the symbol under the cursor.
<code>:change-case</code>	Change the symbol under the cursor to camel
<code>:left</code> , <code>:le</code>	Left-align line(s), setting leading indent to {n} (default 0) — vim :left.
<code>:right</code> , <code>:ri</code>	Right-align line(s) to width {n} (default 80) — vim :right.
<code>:center</code> , <code>:ce</code>	Center line(s) within width {n} (default 80) — vim :center.
<code>:undo</code>	Undo the last change (vim :undo).
<code>:redo</code> , <code>:red</code>	Redo the last undone change (vim :redo).
<code>:retab</code>	Replace tabs with spaces (tab-width per buffer) — vim :retab.
<code>:join</code> , <code>:j</code>	Join the current line(s) with the next, separated by a space (vim :j).
<code>:join!</code> , <code>:j!</code>	Join the current line(s) with the next, no separating space (vim :j!).
<code>:put</code> , <code>:pu</code>	Put (paste) a register's contents as new line(s) below the cursor (vim :put).
<code>:put!</code> , <code>:pu!</code>	Put (paste) a register's contents as new line(s) above the cursor (vim :put!).
<code>:iput</code> , <code>:ip</code>	Put a register's contents below the cursor, indenting to the current line (vim :iput).
<code>:execute-register</code> , <code>:@</code>	Execute a register's contents as Ex command line(s) (vim :@{reg}).
<code>:ijump</code> , <code>:ij</code>	Jump to the first line containing an identifier (vim :ijump).
<code>:djump</code> , <code>:dj</code>	Jump to the first #define of a macro (vim :djump).
<code>:isplit</code> , <code>:isp</code>	Split the window and jump to the first line containing an identifier (vim :isplit).
<code>:dsplit</code> , <code>:dsp</code>	Split the window and jump to the first #define of a macro (vim :dsplit).
<code>:ilist</code> , <code>:il</code>	List every line containing an identifier in a scratch buffer (vim :ilist).
<code>:digraphs</code> , <code>:dig</code>	List the digraph table in a scratch buffer (vim :digraphs).
<code>:z</code>	Print a window of lines from the cursor into a scratch buffer (vim :z).
<code>:checkpath</code> , <code>:checkp</code>	List the files #included by this buffer that are not found in 'path' (:checkpath! lists all).
<code>:checkpath!</code>	List every file #included by this buffer, with where it resolved to in 'path'.

Name	Description
<code>:vertical</code> , <code>:vert</code> , <code>:verti</code> , <code>:vertic</code> , <code>:vertica</code>	Run {cmd}; a window it opens is split vertically (vim :vertical).
<code>:horizontal</code> , <code>:hor</code> , <code>:hori</code> , <code>:horiz</code> , <code>:horizo</code> , <code>:horizon</code> , <code>:horizont</code> , <code>:horizonta</code>	Run {cmd}; a window it opens is split horizontally (vim :horizontal).
<code>:tab</code>	Run {cmd}; where it would open a window, open a new tab page instead (vim :tab).
<code>:silent</code> , <code>:sil</code> , <code>:sile</code> , <code>:silen</code>	Run {cmd} without its status message (vim :silent).
<code>:silent!</code> , <code>:sil!</code> , <code>:sile!</code> , <code>:silen!</code>	Run {cmd} without its status message, and swallow its error (vim :silent!).
<code>:unsilent</code> , <code>:uns</code> , <code>:unsi</code> , <code>:unsil</code> , <code>:unsile</code> , <code>:unsilen</code>	Run {cmd} with its messages shown, cancelling an enclosing :silent (vim :unsilent).
<code>:noautocmd</code> , <code>:noa</code> , <code>:noau</code> , <code>:noaut</code> , <code>:noauto</code> , <code>:noautoc</code> , <code>:noautocm</code>	Run {cmd} without firing any autocommand (vim :noautocmd).
<code>:confirm</code> , <code>:conf</code> , <code>:confi</code> , <code>:confir</code>	Run {cmd} with 'confirm' on: it asks instead of failing on unsaved changes (vim :confirm).
<code>:sandbox</code> , <code>:sandb</code> , <code>:sandbo</code>	Run {cmd} in the sandbox: shelling out and writing files are refused (vim :sandbox).
<code>:abovewleft</code> , <code>:abo</code> , <code>:abov</code> , <code>:above</code> , <code>:abovew</code> , <code>:abovewl</code> , <code>:abovewlef</code>	Run {cmd}; a window it opens goes above (horizontal) or left (vertical) of this one (vim :abovewleft).
<code>:leftabove</code> , <code>:lefta</code> , <code>:leftab</code> , <code>:leftabo</code> , <code>:leftabov</code>	Run {cmd}; a window it opens goes left (vertical) or above (horizontal) this one (vim :leftabove).
<code>:belowright</code> , <code>:bel</code> , <code>:belo</code> , <code>:below</code> , <code>:belowr</code> , <code>:belowri</code> , <code>:belowrig</code> , <code>:belowrigh</code>	Run {cmd}; a window it opens goes below (horizontal) or right (vertical) of this one (vim :belowright).
<code>:rightbelow</code> , <code>:rightb</code> , <code>:rightbe</code> , <code>:rightbel</code> , <code>:rightbelo</code>	Run {cmd}; a window it opens goes right (vertical) or below (horizontal) this one (vim :rightbelow).
<code>:topleft</code> , <code>:to</code> , <code>:top</code> , <code>:topl</code> , <code>:tople</code> , <code>:topleft</code>	Run {cmd}; a window it opens is pushed to the top (horizontal) or far left (vertical) (vim :topleft).
<code>:botright</code> , <code>:bo</code> , <code>:bot</code> , <code>:botr</code> , <code>:botri</code> , <code>:botrig</code> , <code>:botrigh</code>	Run {cmd}; a window it opens is pushed to the bottom (horizontal) or far right (vertical) (vim :botright).
<code>:keepalt</code> , <code>:keepa</code> , <code>:keepal</code>	Run {cmd} without changing the alternate file (vim :keepalt).
<code>:keepjumps</code> , <code>:keepj</code> , <code>:keepju</code> , <code>:keepjum</code> , <code>:keepjump</code>	Run {cmd} without changing the jumplist (vim :keepjumps).
<code>:keepmarks</code> , <code>:kee</code> , <code>:keep</code> , <code>:keepm</code> , <code>:keepma</code> , <code>:keepmar</code> , <code>:keepmark</code>	Run {cmd} without moving any mark (vim :keepmarks).
<code>:lockmarks</code> , <code>:loc</code> , <code>:lock</code> , <code>:lockm</code> , <code>:lockma</code> , <code>:lockmar</code> , <code>:lockmark</code>	Run {cmd} without moving any mark, including '[' and ']' (vim :lockmarks).
<code>:keeppatterns</code> , <code>:keep</code> , <code>:keepa</code> , <code>:keeppat</code> , <code>:keeppatt</code> , <code>:keeppatte</code> , <code>:keeppatter</code> , <code>:keeppattern</code>	Run {cmd} without changing the last search pattern (vim :keeppatterns).
<code>:noswapfile</code> , <code>:nos</code> , <code>:nosw</code> , <code>:noswa</code> , <code>:noswap</code> , <code>:noswapf</code> , <code>:noswapfi</code> , <code>:noswapfil</code>	Run {cmd} without touching the recovery swap file (vim :noswapfile).
<code>:packadd</code> , <code>:pa</code>	Load the package {name} from 'packpath': add it to 'runtimepath' and source its plugin/*.vim (vim :packadd).
<code>:packloadall</code> , <code>:packl</code>	Load every package under pack/*/start/ on the 'packpath' (vim :packloadall).
<code>:language</code> , <code>:lan</code>	Set the locale (\$LANG/\$LC_*) for the editor and every process it starts; bare form reports it (vim :language).
<code>:log</code>	Open zmax's log file read-only.
<code>:lsp</code>	Language-server control: <code>:lsp</code> <code>info restart stop command</code> .
<code>:trust</code>	Trust the current workspace (language servers + local config). <code>++deny</code> never prompts, <code>++remove</code> revokes (nvim :trust).

Name	Description
<code>:exusage, :exu</code>	List the available Ex commands in a scratch buffer (vim <code>:exusage</code>).
<code>:viusage, :viu</code>	List the Normal-mode commands in a scratch buffer (vim <code>:viusage</code>).
<code>:dlist, :dli</code>	List every <code>#define</code> line of a macro in a scratch buffer (vim <code>:dlist</code>).
<code>:isearch, :is</code>	Echo the first line containing an identifier (vim <code>:isearch</code>).
<code>:dsearch, :ds</code>	Echo the first <code>#define</code> line of a macro (vim <code>:dsearch</code>).
<code>:delete-lines, :d, :del, :delete, :kill-whole-line</code>	Delete the current line(s) into the unnamed register (vim <code>:d</code> / emacs <code>kill-whole-line</code>).
<code>:dl</code>	Delete the current line(s) and echo the last deleted line in <code>:list</code> format (vim <code>:dl</code>).
<code>:yank-lines, :y, :ya, :yank</code>	Yank the current line(s) into the unnamed register (vim <code>:y</code>).
<code>:indent-lines</code>	Indent the current line(s) by one shiftwidth (vim <code>:></code>).
<code>:dedent-lines</code>	Dedent the current line(s) by one shiftwidth (vim <code>:<</code>).
<code>:move-lines, :m</code>	Move the current line to after line {address}: <code>:m{addr}</code> (e.g. <code>:m0</code> , <code>:m\$</code> , <code>:m.+2</code>).
<code>:copy-lines, :t, :co, :copy</code>	Copy the current line to after line {address}: <code>:t{addr}</code> (e.g. <code>:t0</code> , <code>:t\$</code>).
<code>:global</code>	Run a command on matching lines: <code>:g/pattern/d</code> (delete). Also <code>:g!/pat/d</code> .
<code>:vglobal</code>	Run a command on non-matching lines: <code>:v/pattern/d</code> (delete).
<code>:substitute, :s</code>	Substitute: <code>:s/pattern/replacement/[flags]</code> . Also <code>:%s/.../.../g</code> for the whole file.
<code>:smagic</code>	Substitute forcing 'magic': <code>:smagic/pattern/replacement/[flags]</code> (vim <code>:smagic</code>).
<code>:snomagic</code>	Substitute forcing 'nomagic' (pattern literal): <code>:snomagic/pattern/replacement/[flags]</code> (vim <code>:snomagic</code>).
<code>:replace-word, :rw, :subword</code>	Global replace of the word under the cursor across the file: <code>:replace-word bar → :%s/\bfoo\b/bar/g</code> . Add <code>i</code> as a 2nd arg for case-insensitive.
<code>:Subvert, :S</code>	vim-abolish case-preserving substitute: <code>:S/foo/bar/g</code> rewrites <code>foo/Foo/FOO → bar/Bar/BAR</code> .
<code>:Thesaurus, :thesaurus</code>	Look up synonyms for the word under the cursor (or <code>:Thesaurus word</code>) and replace it.
<code>:split-line</code>	Split the current line at the cursor, keeping the cursor in place.
<code>:just-one-space</code>	Collapse spaces and tabs around the cursor to a single space.
<code>:delete-horizontal-space</code>	Delete all spaces and tabs around the cursor (emacs <code>M-</code>).
<code>:fixup-whitespace</code>	Collapse whitespace around the cursor to one space, or none by context (emacs <code>fixup-whitespace</code>).
<code>:cycle-spacing</code>	Cycle the whitespace around the cursor: one space, then none, then restore.
<code>:tabify</code>	Convert runs of spaces in the region to tabs at tab stops (emacs <code>tabify</code>).
<code>:untabify</code>	Expand tabs in the region to spaces at the buffer tab width (emacs <code>untabify</code>).
<code>:delete-blank-lines</code>	Collapse consecutive blank lines down to a single blank line.
<code>:uniquify-lines, :uniq</code>	Delete duplicate lines, keeping the first occurrence.
<code>:reverse, :reverse-lines, :reverse-region, :tac</code>	Reverse the order of the selected lines (or the whole buffer). Emacs <code>reverse-region</code> .
<code>:uuid, :guid</code>	Insert a random UUID v4 at each cursor (replaces any selection).

Name	Description
<code>:goto-offset, :goto-char</code>	Move the cursor to an absolute character offset.
<code>:goto-byte, :go, :gob</code>	Move the cursor to a 1-based byte offset (Vim <code>:goto</code>).
<code>:pad-numbers, :zero-pad</code>	Zero-pad every integer in the selection to digits.
<code>:increment-numbers, :incr-numbers</code>	Add N (default 1; negative to decrement) to every integer in the selection.
<code>:bases, :base-info</code>	Show the selected integer in decimal, hex, octal, and binary.
<code>:lorem, :lipsum</code>	Insert N words (default 30) of lorem-ipsum placeholder text.
<code>:date</code>	Insert the current UTC date (YYYY-MM-DD) at each cursor.
<code>:datetime, :now</code>	Insert the current UTC date and time (YYYY-MM-DD HH:MM:SS) at each cursor.
<code>:timestamp, :epoch</code>	Insert the current Unix epoch (seconds) at each cursor.
<code>:sum, :total</code>	Sum the numbers in the selection; reports sum/avg/min/max/count in the status line.
<code>:calc, :eval-math</code>	Evaluate an arithmetic expression (+ - * / % ^), or each selection in place.
<code>:join-with, :joinw</code>	Join the selected lines into one with a separator (default “,”).
<code>:split-on, :splito</code>	Split the selected line(s) on a separator (default “,”) into one item per line.
<code>:squeeze-blank-lines, :squeeze</code>	Collapse consecutive blank lines in the selection to one (cat -s).
<code>:dedup-adjacent, :uniq-adjacent</code>	Collapse consecutive duplicate lines in the selection (Unix uniq).
<code>:number-lines, :nl</code>	Prepend line numbers to the selected lines (optional start, default 1).
<code>:string-rectangle, :string-replace-rectangle</code>	Replace the selected rectangle's column span with a string on every line (emacs C-x r t).
<code>:string-insert-rectangle</code>	Insert a string at the rectangle's left column on every selected line, shifting text right (emacs string-insert-rectangle).
<code>:align, :tabularize</code>	Align the selected lines on a delimiter (default =) so it shares a column. A <code>/regex/</code> argument aligns on every match into columns (Tabular-style).
<code>:encrypt, :age-encrypt</code>	Encrypt the selection (or whole buffer) with an age passphrase; prompts for the passphrase, replaces in place with ASCII-armored ciphertext.
<code>:decrypt, :age-decrypt</code>	Decrypt the selected age ciphertext (or whole buffer) with a passphrase; prompts for the passphrase, replaces in place with the plaintext.
<code>:sort-by-field, :sortf</code>	Sort the selected lines by their Nth whitespace field (default 1).
<code>:sort-numeric-fields, :sortnf</code>	Sort the selected lines by the numeric value of their Nth whitespace field (default 1).
<code>:sort-columns, :sortc</code>	Sort the selected lines alphabetically by the character-column range [beg, end).
<code>:sort-paragraphs, :sortp</code>	Sort the paragraphs (blank-line separated blocks) of the selection alphabetically.
<code>:fill-individual-paragraphs</code>	Fill each paragraph of the selection separately, splitting on indentation changes and using each paragraph's indentation as its fill prefix (emacs fill-individual-paragraphs).
<code>:fill-nonuniform-paragraphs</code>	Fill each paragraph of the selection separately, splitting only on blank lines and using the smallest indentation of each paragraph as its fill prefix (emacs fill-nonuniform-paragraphs).
<code>:sort-pages</code>	Sort the ^L-delimited pages in the selection (or buffer) alphabetically (emacs sort-pages).

Name	Description
<code>:sort-lines, :sortl</code>	Sort the selected lines (or the whole buffer) — vim-style line sort.
<code>:highlight-regexp, :hi-lock</code>	Add a persistent highlight for the regexp (emacs highlight-regexp).
<code>:highlight-phrase</code>	Highlight the phrase, matching across whitespace/line breaks (emacs highlight-phrase).
<code>:highlight-lines-matching-regexp</code>	Highlight whole lines matching the regexp (emacs highlight-lines-matching-regexp).
<code>:unhighlight-regexp</code>	Remove the highlight for the regexp, or all highlights if none given (emacs unhighlight-regexp).
<code>:highlight-symbol-at-point, :hi-lock-face-symbol-at-point</code>	Highlight every whole-word occurrence of the symbol under the cursor (emacs highlight-symbol-at-point).
<code>:hi-lock-write-interactive-patterns</code>	Insert the active Hi-Lock patterns at point as commented Hi-lock: lines (emacs hi-lock-write-interactive-patterns).
<code>:hi-lock-find-patterns</code>	Activate Hi-Lock highlights from commented Hi-lock: lines at the top of the buffer (emacs hi-lock-find-patterns).
<code>:outline-hide-by-heading-regexp</code>	Fold the subtree of every heading whose line matches the regexp (emacs outline-hide-by-heading-regexp).
<code>:outline-show-by-heading-regexp</code>	Reveal the subtree of every heading whose line matches the regexp (emacs outline-show-by-heading-regexp).
<code>:set-fill-column</code>	Set the fill width to N, or the current cursor column if omitted (emacs set-fill-column).
<code>:customize, :customize-browse</code>	Open Preferences on the Settings tab (emacs customize / customize-browse).
<code>:customize-variable, :customize-option</code>	Open Settings pre-filtered to a variable name (emacs customize-variable).
<code>:customize-group</code>	Open Settings pre-filtered to a group name (emacs customize-group).
<code>:bs-customize</code>	Open Settings for the buffer-selection (bs) group (emacs bs-customize).
<code>:text-scale-pinch</code>	Adjust the text-scale amount by a pinch scale factor; no visible effect on a terminal (emacs text-scale-pinch).
<code>:customize-apropos</code>	Open Settings pre-filtered to a regexp/substring (emacs customize-apropos).
<code>:customize-unsaved, :customize-changed, :customize-saved</code>	Open Settings showing only options changed from their default (emacs customize-unsaved).
<code>:customize-face, :customize-themes, :customize-create-theme</code>	Open the Color Scheme (theme/face editor) tab (emacs customize-face / customize-themes).
<code>:set-face-foreground</code>	Set a theme face's foreground color: <code>:set-face-foreground</code> (emacs set-face-foreground).
<code>:set-face-background</code>	Set a theme face's background color: <code>:set-face-background</code> (emacs set-face-background).
<code>:set-fringe-style</code>	Set the gutter (fringe) style: none clears it, default restores it (emacs set-fringe-style).
<code>:set-right-margin</code>	Re-fill the region to N columns narrower than text-width (emacs set-right-margin).
<code>:write-region</code>	Write the region (or whole buffer) to a file, overwriting it (emacs write-region).
<code>:append-to-file</code>	Append the region (or whole buffer) to the end of a file (emacs append-to-file).
<code>:set-justification-left</code>	Flush the region's lines to the left margin (emacs set-justification-left).
<code>:set-justification-right</code>	Right-justify the region's lines to the fill width (emacs set-justification-right).
<code>:set-justification-center</code>	Centre the region's lines within the fill width (emacs set-justification-center).

Name	Description
<code>:set-justification-full</code>	Justify the region's lines to both margins (emacs <code>set-justification-full</code>).
<code>:set-justification-none</code>	Turn justification off for the region (emacs <code>set-justification-none</code>).
<code>:set-left-margin</code>	Set the region's left margin to a column of spaces (emacs <code>set-left-margin</code>).
<code>:increase-left-margin</code>	Indent the region by standard-indent columns (emacs <code>increase-left-margin</code>).
<code>:decrease-left-margin</code>	Outdent the region by standard-indent columns (emacs <code>decrease-left-margin</code>).
<code>:append-to-buffer</code>	Insert the region at the end of another open buffer (emacs <code>append-to-buffer</code>).
<code>:prepend-to-buffer</code>	Insert the region at the start of another open buffer (emacs <code>prepend-to-buffer</code>).
<code>:copy-to-buffer</code>	Replace another open buffer's contents with the region (emacs <code>copy-to-buffer</code>).
<code>:rename-buffer</code>	Give the current buffer an explicit display name (emacs <code>rename-buffer</code>).
<code>:rename-uniquely</code>	Rename the current buffer to a unique name with a numeric suffix (emacs <code>rename-uniquely</code>).
<code>:desktop-save</code>	Save file-visiting buffers and point to a desktop file (emacs <code>desktop-save</code>).
<code>:desktop-read</code>	Reopen the buffers recorded in a desktop file (emacs <code>desktop-read</code>).
<code>:desktop-change-dir</code>	Switch to the desktop saved in another directory (emacs <code>desktop-change-dir</code>).
<code>:desktop-revert</code>	Re-read the last loaded desktop (emacs <code>desktop-revert</code>).
<code>:desktop-clear</code>	Close all buffers and erase the desktop file (emacs <code>desktop-clear</code>).
<code>:expand-region-abbrevs</code>	Expand every abbrev found in the region (emacs <code>expand-region-abbrevs</code>).
<code>:getenv</code>	Report the value of an environment variable (emacs <code>getenv</code>).
<code>:setenv</code>	Set (or, with no value, unset) an environment variable (emacs <code>setenv</code>).
<code>:apropos-command</code>	List commands whose name matches a regexp (emacs <code>apropos-command</code>).
<code>:apropos</code>	List commands (by name or doc) and config variables matching a regexp (emacs <code>apropos</code>).
<code>:apropos-documentation</code>	List commands whose documentation matches a regexp (emacs <code>apropos-documentation</code>).
<code>:apropos-variable</code>	List config variables whose name matches a regexp (emacs <code>apropos-variable</code>).
<code>:apropos-user-option</code>	List user-customizable options whose name matches a regexp (emacs <code>apropos-user-option</code>).
<code>:apropos-value</code>	List config variables whose value matches a regexp (emacs <code>apropos-value</code>).
<code>:goto-line-relative</code>	Go to a line counting from the narrowed region start (emacs <code>goto-line-relative</code>).
<code>:emacs-version</code>	Report the running editor version (emacs <code>emacs-version</code>).
<code>:add-name-to-file</code>	Hard-link the current buffer's file to a new name (emacs <code>add-name-to-file</code>).
<code>:browse-url</code>	Open a URL in the OS default browser (emacs <code>browse-url</code>).
<code>:set-visited-file-name</code>	Change the file the current buffer is visiting (emacs <code>set-visited-file-name</code>).
<code>:make-symbolic-link</code>	Create a symbolic link (emacs <code>make-symbolic-link</code>): , or for the current file.

Name	Description
<code>:set-file-modes</code>	Set the mode bits of a file, chmod-style, MODE as octal (emacs set-file-modes).
<code>:copy-directory</code>	Recursively copy a directory to a destination (emacs copy-directory).
<code>:insert-file-literally</code>	Insert the raw contents of a file at point (emacs insert-file-literally).
<code>:list-directory</code>	Show a listing of a directory in a scratch buffer (emacs list-directory).
<code>:write-abbrev-file</code>	Write every abbrev to the named file (emacs write-abbrev-file).
<code>:read-abbrev-file</code>	Read abbrevs from the named file, merging them into the table (emacs read-abbrev-file).
<code>:bookmark-write</code>	Write every bookmark to the named file (emacs bookmark-write).
<code>:bookmark-load</code>	Load bookmarks from the named file, merging them into the store (emacs bookmark-load).
<code>:multi-occur-in-matching-buffers, :multi-occur</code>	List matches for SEARCH-REGEXP across all buffers whose name matches BUFFER-REGEXP (emacs multi-occur-in-matching-buffers).
<code>:transpose-regions</code>	Swap two char ranges: START1 END1 START2 END2 (emacs transpose-regions).
<code>:transpose-words</code>	Transpose the word before the cursor with the word after it.
<code>:transpose-chars</code>	Transpose the two characters around the cursor.
<code>:duplicate-line, :dup</code>	Duplicate the current line below.
<code>:delete-trailing-whitespace, :dtw</code>	Delete trailing whitespace from every line in the buffer.
<code>:sort</code>	Sort ranges in selection.
<code>:insert-char</code>	Insert the character a code point names, e.g. 263A or #x2318, optionally N times (emacs insert-char).
<code>:rectangle-number-lines</code>	Number the lines of the selected rectangle, optionally from N with a format (emacs rectangle-number-lines).
<code>:auto-compression-mode</code>	Whether a .gz/.bz2/.xz file opens as its contents or its raw bytes (emacs auto-compression-mode).
<code>:center-region, :center-line, :center-paragraph</code>	Centre the selected lines within the text width (emacs center-region/center-line/center-paragraph).
<code>:comment-box</code>	Draw a box of comment characters around the selection (emacs comment-box).
<code>:reflow</code>	Hard-wrap the current selection of lines to a given width.
<code>:tree-sitter-subtree, :ts-subtree, :InspectTree</code>	Display the smallest tree-sitter subtree that spans the primary selection (neovim :InspectTree).
<code>:config-reload</code>	Refresh user config.
<code>:keymap</code>	Switch the active keymap preset: spacemacs, vim, helix, or emacs.
<code>:config-open</code>	Open the user config.toml file.
<code>:config-open-workspace</code>	Open the workspace config.toml file.
<code>:init-open, :init-el-open</code>	Open the Emacs Lisp init file (<config-dir>/init.el) sourced at startup.
<code>:log-open</code>	Open the zmax log file.
<code>:insert-output</code>	Run shell command, inserting output before each selection.
<code>:append-output</code>	Run shell command, appending output after each selection.
<code>:pipe, : </code>	Pipe each selection to the shell command.

Name	Description
<code>:pipe-to</code>	Pipe each selection to the shell command, ignoring output.
<code>:xpipe, :xp, :\ ></code>	Filter each selection through a chain of embedded-language stages (awk, ruby, php, python, ...) in-process, with no subprocess.
<code>:xpipe-to</code>	Run an embedded-language pipeline on each selection, ignoring the output.
<code>:xpipe-insert</code>	Run an embedded-language pipeline with no input, inserting its output before each selection.
<code>:xpipe-append</code>	Run an embedded-language pipeline with no input, appending its output after each selection.
<code>:run-shell-command, :sh, :!</code>	Run a shell command
<code>:elisp, :eval-expression, :el</code>	Evaluate an Emacs Lisp expression against the editor, asking for it when none is given (embedded elisprs).
<code>:vim, :viml, :vimscript</code>	Evaluate a Vimscript (VimL) expression via the embedded vimlrs interpreter.
<code>:awk, :awk-filter</code>	Filter the selection (or whole buffer) through an awk program (embedded awkrs).
<code>:zsh, :zshell</code>	Run a command in the embedded zsh shell (state persists); output shown in a popup.
<code>:stryke, :st</code>	Evaluate stryke (strykelang) source via the embedded interpreter (state persists).
<code>:php</code>	Evaluate PHP source via the embedded phplang interpreter.
<code>:node, :js, :javascript</code>	Evaluate JavaScript source via the embedded node-js interpreter.
<code>:arb, :arb-filter</code>	Filter the selection (or whole buffer) through an arb spec's <code>out</code> pipeline (embedded arblang).
<code>:tcl, :tclsh</code>	Evaluate Tcl source via the embedded tclrs interpreter (state persists).
<code>:rlang, :rscript</code>	Evaluate R source via the embedded rlang interpreter (<code>:r</code> is <code>:read</code>).
<code>:zwire-host, :zh</code>	Send a raw JSON request to the zwire-host daemon; show the reply (e.g. <code>{"cmd":"hostinfo"}</code>).
<code>:zwire-sysinfo, :zsys</code>	Show live system stats (cpu/mem/load) from the shared zwire-host daemon.
<code>:zwire-hostinfo</code>	Show machine facts (os/arch/cpus/hostname) from the shared zwire-host daemon.
<code>:zwire-exec, :zx</code>	Run a command through the zwire-host daemon and insert its output at the cursor.
<code>:zwire-crawl, :zwc</code>	Recursively crawl the filesystem via zwire-host and insert matching paths at the cursor.
<code>:zwire-job, :zj</code>	Ship a long-running command to the zwire-host daemon; get notified on the status line when it finishes.
<code>:zwire-jobs, :jobs, :zjs</code>	List background zwire-host jobs still running, plus recent completions.
<code>:zwire-job-output, :zjo</code>	Insert a finished background job's output at the cursor (most recent, or a given id).
<code>:zmax-native, :plugin</code>	Manage native (compiled Rust) plugins (<code>zmax-native</code> SDK). Package manager: <code>:zmax-native add owner/repo, get, sync, remove, registry, info, update, gc, clean</code> . Raw host: <code>load <path>, unload <name>, list</code> . Alias: <code>:plugin</code> .
<code>:repl</code>	Open the embedded-language REPL (elisp/viml/stryke/awk/zsh/node/ruby/php/python/arb/tcl/r); optional starting language.
<code>:reset-diff-change, :diffget, :diffg</code>	Reset the diff change at the cursor position.
<code>:clear-register</code>	Clear given register. If no argument is provided, clear all registers.

Name	Description
<code>:set-register</code>	Set contents of the given register.
<code>:redraw</code>	Clear and re-render the whole UI
<code>:profile, :prof</code>	Profile functions and scripts (start
<code>:profdel, :profd</code>	Stop profiling a function or script (halts the profiler, keeping samples)
<code>:scriptencoding, :scripte</code>	Declare the encoding used in a sourced script (zmax reads scripts as UTF-8)
<code>:org-export, :org-export-dispatch</code>	Export the current Org buffer to Markdown (scratch buffer, or to a path arg) — Emacs C-c C-e
<code>:editorconfig, :editorconfig-mode</code>	Apply the nearest .editorconfig settings to the current buffer (Emacs editorconfig-mode)
<code>:eww, :browse-web</code>	Fetch a URL and render the HTML to text in a buffer (Emacs eww, in-editor web browser)
<code>:eww-open-file</code>	Render a local HTML file in the eww buffer (Emacs eww-open-file)
<code>:eww-search-words, :eww-search</code>	Web-search the given words (or word under cursor) and render results (Emacs eww-search-words)
<code>:translate, :google-translate</code>	Translate the word under cursor (or given text) between the configured languages
<code>:translate-set-languages</code>	Set source and target languages for :translate (e.g. en fr, or auto en)
<code>:translate-reverse</code>	Reverse the source and target translate languages
<code>:irc-connect, :erc, :irc</code>	Connect to an IRC server and register: :irc-connect <host[:port]> (Emacs erc/rcirc)
<code>:irc-join</code>	Join an IRC channel on the active session: :irc-join #channel
<code>:irc-say, :irc-msg</code>	Send a message on the active IRC session: :irc-say
<code>:irc-view, :irc-buffer</code>	Show the current IRC session transcript in a buffer (Emacs erc buffer)
<code>:irc-quit</code>	Disconnect the active IRC session: :irc-quit [message]
<code>:ietf-docs-open, :ietf</code>	Open an IETF document (rfc2119, draft-..., std66) named by the argument or the word under the cursor; ! re-downloads (spacemacs ietf layer).
<code>:slack-start</code>	Authenticate a Slack token and start a session: :slack-start [token] (else \$SLACK_TOKEN).
<code>:slack-select-rooms, :slack-channel-select</code>	List the Slack team's rooms, or select one: :slack-select-rooms [#channel]
<code>:slack-buffer</code>	Show the selected Slack room's recent history: :slack-buffer [count].
<code>:slack-message</code>	Post a message to the selected Slack room: :slack-message .
<code>:slack-quit</code>	Close the Slack session (emacs-slack slack-ws-close).
<code>:sudo-edit</code>	Open a file with elevated privileges, reading it through sudo when it is root-only: :sudo-edit [file] (spacemacs SPC f E).
<code>:sudo-write</code>	Write the current buffer to a root-owned file through sudo tee: :sudo-write [file].
<code>:regenerate-tags, :projectile-regenerate-tags</code>	Rebuild the project's TAGS file with <code>ctags -Re</code> and visit it (projectile-regenerate-tags, SPC p G).
<code>:syntime</code>	Profile syntax highlighting: :syntime on
<code>:move, :mv</code>	Move the current buffer and its corresponding file to a different path
<code>:move!, :mv!</code>	Move the current buffer and its corresponding file to a different path creating necessary subdirectories
<code>:delete-file, :remove-file</code>	Delete the current buffer's file from disk and close the buffer (vim-eunuch :Delete).

Name	Description
<code>:chmod-x</code> , <code>:chmodx</code> , <code>:make-executable</code>	Make the current file executable (<code>chmod a+x</code>). Unix only.
<code>:mkdir</code>	Create a directory and any missing parents; with no arg, the current file's parent.
<code>:yank-diagnostic</code>	Yank diagnostic(s) under primary cursor to register, or clipboard by default
<code>:read</code> , <code>:r</code>	Load a file into buffer, or the output of a shell command with <code>:r !cmd</code>
<code>:insert-file</code>	Insert a file's contents at the cursor, asking for the file when none is given (emacs <code>insert-file</code>).
<code>:insert-buffer</code>	Insert another open buffer's contents at the cursor, asking for the buffer when none is given (emacs <code>insert-buffer</code>).
<code>:echo</code>	Prints the given arguments to the statusline.
<code>:echoerr</code> , <code>:echoe</code>	Prints the given arguments to the statusline as an error (vim <code>:echoerr</code>).
<code>:noop</code>	Does nothing.
<code>:workspace-trust</code>	Allow language servers and local config for the current workspace.
<code>:workspace-untrust</code>	Revoke the current workspace's trust grant or exclusion.
<code>:workspace-exclude</code>	Mark the current workspace as never-prompt. Never prompts for trust again.
<code>:stjump</code> , <code>:stj</code>	Jump to a tag in a new split, offering a picker when several tags match (vim <code>:stjump</code>).
<code>:ptjump</code> , <code>:ptj</code>	Like <code>:tjump</code> , showing the tag in the preview window and leaving the cursor where it is (vim <code>:ptjump</code>).
<code>:stselect</code> , <code>:sts</code>	List matching tags in a picker; the chosen one opens in a new split (vim <code>:stselect</code>).
<code>:ptselect</code> , <code>:pts</code>	Like <code>:tselect</code> , showing the chosen tag in the preview window — a split here (vim <code>:ptselect</code>).
<code>:ppop</code> , <code>:pp</code>	Pop the tag stack into the preview window — a split here (vim <code>:ppop</code>).
<code>:ltag</code> , <code>:lt</code>	Jump to a tag and put every matching tag in the location list (vim <code>:ltag</code>).
<code>:psearch</code> , <code>:ps</code>	Show the first line matching an identifier in the preview window, keeping focus (vim <code>:psearch</code>).
<code>:perl-do</code> , <code>:perl-do</code>	Run a Perl snippet on every line (<code>\$_</code> is the line, and replaces it) (vim <code>:perl-do</code>).
<code>:ruby-do</code> , <code>:ruby-do</code>	Run a Ruby snippet on every line (<code>\$_</code> is the line, and replaces it) (vim <code>:ruby-do</code>).
<code>:luado</code> , <code>:luad</code>	Run a Lua chunk on every line (args line, linenr; the return value replaces the line) (vim <code>:luado</code>).
<code>:pydo</code> , <code>:pyd</code>	Run a Python function body on every line (args line, linenr; the return value replaces the line) (vim <code>:pydo</code>).
<code>:py3do</code> , <code>:py3d</code> , <code>:pyxdo</code> , <code>:pyxd</code>	Run a Python 3 function body on every line (args line, linenr) (vim <code>:py3do</code> / <code>:pyxdo</code>).
<code>:pyx</code> , <code>:pythonx</code>	Run a Python snippet through python3 and echo its output (vim <code>:pyx</code> / <code>:pythonx</code>).
<code>:pyxfile</code> , <code>:pyxf</code>	Run a Python script file through python3 (vim <code>:pyxfile</code>).
<code>:command</code> , <code>:com</code>	Define a user command (<code>:command LI :lopen</code>), or list the ones defined (vim <code>:command</code>).
<code>:command!</code> , <code>:com!</code>	Define a user command, replacing an existing definition of that name (vim <code>:command!</code>).
<code>:delcommand</code> , <code>:delc</code>	Delete a user-defined command (vim <code>:delcommand</code>).
<code>:comclear</code> , <code>:comc</code>	Delete every user-defined command (vim <code>:comclear</code>).
<code>:folddoopen</code> , <code>:foldd</code>	Run a command on every line that is not inside a closed fold (vim <code>:folddoopen</code>).

Name	Description
<code>:folddoclosed</code> , <code>:folddoc</code>	Run a command on every line inside a closed fold (vim <code>:folddoclosed</code>).
<code>:breakadd</code> , <code>:breaka</code>	Set a debugger breakpoint: <code>:breakadd</code> here
<code>:breakdel</code> , <code>:breakd</code>	Delete a debugger breakpoint by <code>:breaklist</code> number, by position, or all with <code>*</code> (vim <code>:breakdel</code>).
<code>:breaklist</code> , <code>:breakl</code>	List the debugger breakpoints, numbered for <code>:breakdel</code> (vim <code>:breaklist</code>).
<code>:debug</code> , <code>:deb</code> , <code>:debu</code>	Run an Ex command under the script debugger prompt (vim <code>:debug</code>).
<code>:debuggreedy</code> , <code>:debugg</code> , <code>:debuggr</code> , <code>:debuggre</code> , <code>:debuggree</code> , <code>:debuggreed</code>	Read debug mode commands from the normal input stream instead of the prompt (vim <code>:debuggreedy</code>).
<code>:0debuggreedy</code> , <code>:0debugg</code>	Undo <code>:debuggreedy</code> — debug mode commands are typed at the prompt again (vim <code>:0debuggreedy</code>).
<code>:highlight</code> , <code>:hi</code>	List, show or set the theme's highlight groups (<code>:hi</code> Comment <code>guifg=#5c6370</code> <code>gui=italic</code>) (vim <code>:highlight</code>).
<code>:syntax</code> , <code>:sy</code> , <code>:syn</code>	Turn the buffer's syntax highlighting on/off, or report the language (vim <code>:syntax</code>).
<code>:compiler</code> , <code>:comp</code>	Select the compiler for <code>:make</code> by setting <code>makeprg</code> (<code>:compiler</code> cargo) (vim <code>:compiler</code>).
<code>:checkhealth</code> , <code>:che</code>	Run the health checks (clipboard, language servers, grammars) and show the report (nvim <code>:checkhealth</code>).
<code>:helpclose</code> , <code>:helpc</code>	Close the help window (vim <code>:helpclose</code>).
<code>:options</code> , <code>:opt</code>	Open the options window — Preferences on the Settings tab (vim <code>:options</code>).
<code>:mode</code> , <code>:mod</code>	Redraw the screen (vim <code>:mode</code>).
<code>:startreplace</code> , <code>:starttr</code>	Start Replace mode (vim <code>:startreplace</code>).
<code>:tabfind</code> , <code>:tabf</code>	Find a file in the 'path' and edit it in a new tab page (vim <code>:tabfind</code>).
<code>:spellgood!</code> , <code>:spe!</code>	Add words to the known-good internal word list (vim <code>:spellgood!</code>).
<code>:spellgood</code> , <code>:spe</code>	Add words to the known-good spell list (vim <code>:spellgood</code>).
<code>:~</code>	Repeat the last <code>:substitute</code> (vim <code>:~</code>).

Static Commands and Keybindings

Every static (bindable) command and the keys bound to it in each keymap preset (spacemacs, vim, helix, emacs), by mode. Presets that share a binding are listed together.

Name	Description	Keybinds
<code>no_op</code>	Do nothing	
<code>move_char_left</code>	Move left	spacemacs — normal: <code>h</code> , insert: <code><[-b></code> , <code><left></code> vim — normal: <code>h</code> , <code><[-h></code> , insert: <code><left></code> helix — normal: <code>h</code> , <code><left></code> , insert: <code><left></code> emacs, cua — normal: <code><[-b></code> , <code><left></code> , insert: <code><[-b></code> , <code><left></code>
<code>move_char_right</code>	Move right	spacemacs — normal: <code>l</code> , insert: <code><[-f></code> , <code><right></code> vim — normal: <code>l</code> , <code><space></code> , insert: <code><right></code> helix — normal: <code>l</code> , <code><right></code> , insert: <code><right></code> emacs, cua — normal: <code><[-f></code> , <code><right></code> , insert: <code><[-f></code> , <code><right></code>
<code>move_line_up</code>	Move up	spacemacs, vim — normal: <code>k</code> , <code><[-p></code> , insert: <code><up></code> helix — normal: <code>gk</code>
<code>move_line_down</code>	Move down	spacemacs, vim — normal: <code>j</code> , <code><[-j></code> , <code><[-n></code> , insert: <code><down></code> helix — normal: <code>gj</code>
<code>shift_line_up</code>	Move the line/selection up (JetBrains Move Statement Up)	spacemacs, vim — normal: <code><A-up></code>
<code>shift_line_down</code>	Move the line/selection down (JetBrains Move Statement Down)	spacemacs, vim — normal: <code><A-down></code>
<code>drag_line_down</code>	Drag the current line down (SPC x . j)	spacemacs — normal: <code><space>x . J</code> , <code><space>x . j</code> , <code><space>x . <down></code> , select: <code><space>x . J</code> , <code><space>x . j</code> , <code><space>x . <down></code>
<code>drag_line_up</code>	Drag the current line up (SPC x . k)	spacemacs — normal: <code><space>x . K</code> , <code><space>x . k</code> , <code><space>x . <up></code> , select: <code><space>x . K</code> , <code><space>x . k</code> , <code><space>x . <up></code>
<code>toggle_test_file</code>	Toggle between implementation and test file (SPC p a)	spacemacs — normal: <code><space>pa</code> , select: <code><space>pa</code>
<code>fold_comments</code>	Fold multi-line comment blocks (SPC c h)	spacemacs — normal: <code><space>ch</code> , select: <code><space>ch</code>
<code>move_visual_line_up</code>	Move up	spacemacs, vim — normal: <code>gk</code> , <code>g<up></code> , insert: <code><[-g>k</code> , <code><[-g><up></code> , <code><[-g><[-k></code> helix — normal: <code>k</code> , <code><up></code> , insert: <code><up></code> emacs, cua — normal: <code><up></code> , <code><[-p></code> , insert: <code><up></code> , <code><[-p></code>
<code>move_visual_line_down</code>	Move down	spacemacs, vim — normal: <code>gj</code> , <code>g<down></code> , insert: <code><[-g>j</code> , <code><[-g><[-j></code> , <code><[-g><down></code> helix — normal: <code>j</code> , <code><down></code> , insert: <code><down></code> emacs, cua — normal: <code><[-n></code> , <code><down></code> , insert: <code><[-n></code> , <code><down></code>

Name	Description	Keybinds
<code>extend_char_left</code>	Extend left	helix — select: <code>h</code> , <code><left></code> emacs — select: <code><C-b></code> , <code><left></code> cua — select: <code><C-b></code> , <code><left></code> , <code><S-left></code>
<code>extend_char_right</code>	Extend right	vim — select: <code><space></code> helix — select: <code>l</code> , <code><right></code> emacs — select: <code><C-f></code> , <code><right></code> cua — select: <code><C-f></code> , <code><right></code> , <code><S-right></code>
<code>extend_line_up</code>	Extend up	helix — select: <code>gk</code>
<code>extend_line_down</code>	Extend down	helix — select: <code>gj</code>
<code>extend_visual_line_up</code>	Extend up	helix — select: <code>k</code> , <code><up></code> emacs — select: <code><up></code> , <code><C-p></code> cua — select: <code><up></code> , <code><C-p></code> , <code><S-up></code>
<code>extend_visual_line_down</code>	Extend down	helix — select: <code>j</code> , <code><down></code> emacs — select: <code><C-n></code> , <code><down></code> cua — select: <code><C-n></code> , <code><down></code> , <code><S-down></code>
<code>copy_selection_on_next_line</code>	Copy selection on next line	helix — normal: <code>C</code> , select: <code>C</code>
<code>copy_selection_on_prev_line</code>	Copy selection on previous line	helix — normal: <code><A-C></code> , select: <code><A-C></code>
<code>column_selection</code>	Turn the selection into a rectangular column block (IntelliJ column selection)	
<code>visual_block_mode</code>	Enter/leave vim visual-block selection (CTRL-V)	spacemacs , vim — normal: <code><C-v></code> , <code>g<C-h></code> , select: <code><C-v></code>
<code>block_reproject</code>	Rebuild the visual-block rectangle from its anchor (internal motion helper)	
<code>visual_line_mode</code>	Enter/leave vim visual-line selection (V)	spacemacs , vim — normal: <code>V</code> , select: <code>V</code>
<code>line_reproject</code>	Rebuild the visual-line whole-line span from its anchor (internal motion helper)	
<code>block_dollar</code>	Visual-block: extend each row to its own line end (CTRL-V \$)	spacemacs , vim — select: <code>\$</code>
<code>block_swap_corners</code>	Visual-block: move cursor to the opposite corner (o)	spacemacs , vim — select: <code>o</code>
<code>block_swap_columns</code>	Visual-block: move cursor to the other column edge (O)	spacemacs , vim — select: <code>O</code>
<code>block_insert</code>	Visual-block: insert at the left column on every row (I)	spacemacs , vim — select: <code>I</code>
<code>block_append</code>	Visual-block: append at the right column, padding short rows (A)	spacemacs , vim — select: <code>A</code>
<code>move_next_word_start</code>	Move to start of next word	spacemacs — normal: <code><A-f></code> , <code><A-right></code> , insert: <code><A-f></code> helix — normal: <code>w</code>
<code>move_prev_word_start</code>	Move to start of previous word	spacemacs — normal: <code><A-b></code> , <code><A-left></code> , <code><C-left></code> , insert: <code><A-b></code> , <code><C-left></code> , <code><S-left></code> vim — normal: <code><C-left></code> , insert: <code><C-left></code> , <code><S-left></code> helix — normal: <code>bemacs</code> , <code>cua</code> — normal: <code><A-b></code> , insert: <code><A-b></code>
<code>move_next_word_end</code>	Move to end of next word	helix — normal: <code>eemacs</code> , <code>cua</code> — normal: <code><A-f></code> , insert: <code><A-f></code>
<code>move_prev_word_end</code>	Move to end of previous word	
<code>move_next_long_word_start</code>	Move to start of next long word	helix — normal: <code>W</code>
<code>move_prev_long_word_start</code>	Move to start of previous long word	helix — normal: <code>B</code>
<code>move_next_long_word_end</code>	Move to end of next long word	helix — normal: <code>E</code>
<code>move_prev_long_word_end</code>	Move to end of previous long word	

Name	Description	Keybinds
<code>move_next_sub_word_start</code>	Move to start of next sub word	
<code>move_prev_sub_word_start</code>	Move to start of previous sub word	
<code>move_next_sub_word_end</code>	Move to end of next sub word	
<code>move_prev_sub_word_end</code>	Move to end of previous sub word	
<code>vim_move_next_word_start</code>	Move to start of next word (vim caret)	
<code>vim_move_prev_word_start</code>	Move to start of previous word (vim caret)	
<code>vim_move_next_word_end</code>	Move to end of next word (vim caret)	
<code>vim_move_prev_word_end</code>	Move to end of previous word (vim caret)	spacemacs, vim — normal: ge
<code>vim_move_next_long_word_start</code>	Move to start of next long word (vim caret)	spacemacs, vim — normal: W
<code>vim_move_prev_long_word_start</code>	Move to start of previous long word (vim caret)	spacemacs, vim — normal: B
<code>vim_move_next_long_word_end</code>	Move to end of next long word (vim caret)	spacemacs, vim — normal: E
<code>vim_move_prev_long_word_end</code>	Move to end of previous long word (vim caret)	spacemacs, vim — normal: gE
<code>move_parent_node_end</code>	Move to end of the parent node	spacemacs — normal: <space>k\$, select: <space>k\$helix — normal: <A-e>
<code>move_parent_node_start</code>	Move to beginning of the parent node	spacemacs — normal: <space>k0 , select: <space>k0helix — normal: <A-b>
<code>extend_next_word_start</code>	Extend to start of next word	helix — select: w
<code>extend_prev_word_start</code>	Extend to start of previous word	helix — select: bemacs, cua — select: <A-b>
<code>extend_next_word_end</code>	Extend to end of next word	helix — select: emacs, cua — select: <A-f>
<code>extend_prev_word_end</code>	Extend to end of previous word	
<code>extend_next_long_word_start</code>	Extend to start of next long word	helix — select: W
<code>extend_prev_long_word_start</code>	Extend to start of previous long word	helix — select: B
<code>extend_next_long_word_end</code>	Extend to end of next long word	helix — select: E
<code>extend_prev_long_word_end</code>	Extend to end of prev long word	
<code>extend_next_sub_word_start</code>	Extend to start of next sub word	
<code>extend_prev_sub_word_start</code>	Extend to start of previous sub word	
<code>extend_next_sub_word_end</code>	Extend to end of next sub word	
<code>extend_prev_sub_word_end</code>	Extend to end of prev sub word	
<code>extend_parent_node_end</code>	Extend to end of the parent node	helix — select: <A-e>
<code>extend_parent_node_start</code>	Extend to beginning of the parent node	helix — select: <A-b>
<code>find_till_char</code>	Move till next occurrence of char	helix — normal: t
<code>find_next_char</code>	Move to next occurrence of char	helix — normal: f
<code>extend_till_char</code>	Extend till next occurrence of char	helix — select: t
<code>extend_next_char</code>	Extend to next occurrence of char	helix — select: f

Name	Description	Keybinds
<code>till_prev_char</code>	Move till previous occurrence of char	helix — normal: <code>T</code>
<code>find_prev_char</code>	Move to previous occurrence of char	helix — normal: <code>F</code>
<code>vim_find_next_char</code>	Move to next occurrence of char on this line (vim f)	spacemacs , vim — normal: <code>f</code>
<code>vim_find_prev_char</code>	Move to previous occurrence of char on this line (vim F)	spacemacs , vim — normal: <code>F</code>
<code>vim_find_till_char</code>	Move till next occurrence of char on this line (vim t)	spacemacs , vim — normal: <code>t</code>
<code>vim_till_prev_char</code>	Move till previous occurrence of char on this line (vim T)	spacemacs , vim — normal: <code>T</code>
<code>sneak_forward</code>	Sneak: jump forward to a two-character sequence	
<code>sneak_backward</code>	Sneak: jump backward to a two-character sequence	
<code>sneak_or_substitute_char</code>	Sneak forward, or substitute char when vim-sneak is off	spacemacs , vim — normal: <code>s</code>
<code>sneak_or_substitute_line</code>	Sneak backward, or substitute line when vim-sneak is off	spacemacs , vim — normal: <code>S</code>
<code>vim_change_line</code>	Change count lines, keeping the first line's indent (vim cc)	spacemacs , vim — normal: <code>cc</code>
<code>extend_till_prev_char</code>	Extend till previous occurrence of char	helix — select: <code>T</code>
<code>extend_prev_char</code>	Extend to previous occurrence of char	helix — select: <code>F</code>
<code>repeat_last_motion</code>	Repeat last motion	spacemacs — normal: <code><C-x>z</code> , select: <code><C-x>z</code> , insert: <code><C-x>z</code> helix — normal: <code><A-.></code> , select: <code><A-.></code>
<code>repeat_find_char</code>	Repeat last find in same direction (:) ;	spacemacs , vim — normal: <code>;</code> , select: <code>;</code>
<code>repeat_find_char_reverse</code>	Repeat last find in opposite direction (,) ,	spacemacs , vim — normal: <code>,</code> , select: <code>,</code>
<code>replace</code>	Replace with new char	spacemacs , vim — select: <code>r</code> helix — normal: <code>r</code> , select: <code>r</code>
<code>switch_case</code>	Switch (toggle) case	spacemacs , vim — select: <code>-</code> helix — normal: <code>~</code> , select: <code>~</code>
<code>switch_case_forward</code>	Toggle case and advance cursor (vim ~)	spacemacs , vim — normal: <code>~</code>
<code>switch_to_uppercase</code>	Switch to uppercase	spacemacs — normal: <code><space>xU</code> , <code><C-x><C-u></code> , select: <code><space>xU</code> , <code><C-x><C-u></code> , insert: <code><C-x><C-u></code> helix — normal: <code><A-`></code> , select: <code><A-`></code> emacs — insert: <code><C-x><C-u></code> cua — select: <code><C-X><C-u></code> , insert: <code><C-x><C-u></code>
<code>switch_to_lowercase</code>	Switch to lowercase	spacemacs — normal: <code><space>xu</code> , <code><C-x><C-l></code> , select: <code><space>xu</code> , <code><C-x><C-l></code> , insert: <code><C-x><C-l></code> helix — normal: <code>`</code> , select: <code>`</code> emacs — insert: <code><C-x><C-l></code> cua — select: <code><C-X><C-l></code> , insert: <code><C-x><C-l></code>
<code>upcase_word</code>	Upper-case the word after point (emacs upcase-word, M-u)	spacemacs — normal: <code><A-u></code> emacs , cua — normal: <code><A-u></code> , insert: <code><A-u></code>
<code>downcase_word</code>	Lower-case the word after point (emacs downcase-word, M-l)	spacemacs — normal: <code><A-l></code> emacs , cua — normal: <code><A-l></code> , insert: <code><A-l></code>
<code>capitalize_word</code>	Capitalize the word after point (emacs capitalize-word, M-c)	spacemacs — normal: <code><A-c></code> emacs , cua — normal: <code><A-c></code> , insert: <code><A-c></code>
<code>upcase_prev_word</code>	Upper-case the previous word (emacs M- M-u)	spacemacs , vim — normal: <code><A-minus>u</code> , <code><A-minus><A-u></code>
<code>downcase_prev_word</code>	Lower-case the previous word (emacs M- M-l)	spacemacs , vim — normal: <code><A-minus>l</code> , <code><A-minus><A-l></code>

Name	Description	Keybinds
<code>capitalize_prev_word</code>	Capitalize the previous word (emacs M-M-c)	spacemacs, vim — normal: <code><A-minus>c</code> , <code><A-minus><A-c></code>
<code>capitalize_region</code>	Title-case every word in the region (emacs <code>capitalize-region</code>)	
<code>upcase_initials_region</code>	Upper-case the first letter of each word in the region (emacs <code>upcase-initials-region</code>)	
<code>page_up</code>	Move page up	spacemacs — normal: <code><A-v></code> , <code><C-b></code> , <code><S-minus></code> , insert: <code><A-v></code> , <code><S-up></code> , <code><pageup></code> vim — normal: <code><C-b></code> , <code><S-minus></code> , insert: <code><S-up></code> , <code><pageup></code> helix — normal: <code><C-b></code> , <code>Z<C-b></code> , <code>z<C-b></code> , <code><pageup></code> , <code>Z<pageup></code> , <code>z<pageup></code> , select: <code><C-b></code> , <code>Z<C-b></code> , <code>z<C-b></code> , <code><pageup></code> , <code>Z<pageup></code> , <code>z<pageup></code> , insert: <code><pageup></code> emacs, cua — normal: <code><A-v></code> , <code><pageup></code> , insert: <code><A-v></code> , <code><pageup></code>
<code>page_down</code>	Move page down	spacemacs, vim — normal: <code><C-f></code> , <code><S-+></code> , <code><S-ret></code> , insert: <code><S-down></code> , <code><pagedown></code> helix — normal: <code><C-f></code> , <code>Z<C-f></code> , <code>z<C-f></code> , <code><pagedown></code> , <code>Z<pagedown></code> , <code>z<pagedown></code> , select: <code><C-f></code> , <code>Z<C-f></code> , <code>z<C-f></code> , <code><pagedown></code> , <code>Z<pagedown></code> , <code>z<pagedown></code> , insert: <code><pagedown></code> emacs — normal: <code><C-v></code> , <code><pagedown></code> , insert: <code><C-v></code> , <code><pagedown></code> cua — normal: <code><pagedown></code> , insert: <code><pagedown></code>
<code>half_page_up</code>	Move half page up	
<code>half_page_down</code>	Move half page down	
<code>page_cursor_up</code>	Move page and cursor up	
<code>page_cursor_down</code>	Move page and cursor down	
<code>page_cursor_half_up</code>	Move page and cursor half up	spacemacs, vim — normal: <code><C-u></code> helix — normal: <code><C-u></code> , <code>Z<C-u></code> , <code>z<C-u></code> , <code>Z<backspace></code> , <code>z<backspace></code> , select: <code><C-u></code> , <code>Z<C-u></code> , <code>z<C-u></code> , <code>Z<backspace></code> , <code>z<backspace></code>
<code>page_cursor_half_down</code>	Move page and cursor half down	spacemacs, vim — normal: <code><C-d></code> helix — normal: <code><C-d></code> , <code>Z<C-d></code> , <code>z<C-d></code> , <code>Z<space></code> , <code>z<space></code> , select: <code><C-d></code> , <code>Z<C-d></code> , <code>z<C-d></code> , <code>Z<space></code> , <code>z<space></code>
<code>extend_page_up</code>	Extend the selection a page up (vim)	cua — select: <code><S-pageup></code>
<code>extend_page_down</code>	Extend the selection a page down (vim)	cua — select: <code><S-pagedown></code>
<code>select_all</code>	Select whole document	spacemacs — normal: <code><C-x>h</code> , select: <code><C-x>h</code> , insert: <code><C-x>h</code> helix — normal: <code>%</code> , select: <code>%</code> emacs — normal: <code><C-x>h</code> , insert: <code><C-x>h</code> cua — normal: <code><C-x>h</code> , select: <code><C-X>h</code> , insert: <code><C-x>h</code>
<code>select_regex</code>	Select all regex matches inside selections	helix — normal: <code>s</code> , select: <code>s</code>

Name	Description	Keybinds
<code>select_all_instances</code>	Select all occurrences of the current selection in the buffer	
<code>split_selection</code>	Split selections on regex matches	helix — normal: <code>S</code> , select: <code>S</code>
<code>split_selection_on_newline</code>	Split selection on newlines	helix — normal: <code><A-s></code> , select: <code><A-s></code>
<code>merge_selections</code>	Merge selections	helix — normal: <code><A-minus></code> , select: <code><A-minus></code>
<code>merge_consecutive_selections</code>	Merge consecutive selections	helix — normal: <code><A-_-></code> , select: <code><A-_-></code>
<code>search</code>	Search for regex pattern	spacemacs — normal: <code>/</code> , <code><C-s></code> , <code><A-[C-s]></code> , select: <code>/vim</code> — normal: <code>/</code> , select: <code>/helix</code> — normal: <code>/</code> , <code>Z/</code> , <code>z/</code> , select: <code>/</code> , <code>Z/</code> , <code>z/emacs</code> , cua — normal: <code><C-s></code> , <code><A-[C-s]></code> , insert: <code><C-s></code> , <code><A-[C-s]></code>
<code>rsearch</code>	Reverse search for regex pattern	spacemacs — normal: <code>?</code> , <code><A-[C-r]></code> , select: <code>?vim</code> — normal: <code>?</code> , select: <code>?helix</code> — normal: <code>?</code> , <code>Z?</code> , <code>z?</code> , select: <code>?</code> , <code>Z?</code> , <code>z?emacs</code> , cua — normal: <code><C-r></code> , <code><A-[C-r]></code> , insert: <code><C-r></code> , <code><A-[C-r]></code>
<code>delete_to_search_forward</code>	Delete up to the next match (vim d/pat)	spacemacs, vim — normal: <code>d/</code>
<code>delete_to_search_backward</code>	Delete back to the previous match (vim d?pat)	spacemacs, vim — normal: <code>d?</code>
<code>change_to_search_forward</code>	Change up to the next match (vim c/pat)	spacemacs, vim — normal: <code>c/</code>
<code>change_to_search_backward</code>	Change back to the previous match (vim c?pat)	spacemacs, vim — normal: <code>c?</code>
<code>yank_to_search_forward</code>	Yank up to the next match (vim y/pat)	spacemacs, vim — normal: <code>y/</code>
<code>yank_to_search_backward</code>	Yank back to the previous match (vim y?pat)	spacemacs, vim — normal: <code>y?</code>
<code>search_next</code>	Select next search match	spacemacs — normal: <code><space>sh</code> , select: <code><space>shhelix</code> — normal: <code>n</code> , <code>Zn</code> , <code>zn</code> , select: <code>Zn</code> , <code>zn</code>
<code>search_prev</code>	Select previous search match	helix — normal: <code>N</code> , <code>ZN</code> , <code>zN</code> , select: <code>ZN</code> , <code>zN</code>
<code>extend_search_next</code>	Add next search match to selection	helix — select: <code>n</code>
<code>extend_search_prev</code>	Add previous search match to selection	helix — select: <code>N</code>
<code>select_gn_match</code>	vim gn: select the search match at/after the cursor (for cgn/dgn)	
<code>select_gn_match_prev</code>	vim gN: select the search match at/before the cursor	
<code>search_next_vim</code>	vim n: repeat last search in its direction	spacemacs, vim — normal: <code>n</code>
<code>search_prev_vim</code>	vim N: repeat last search in the opposite direction	spacemacs, vim — normal: <code>N</code>
<code>extend_search_next_vim</code>	vim n (visual): extend to the repeated match	spacemacs, vim — select: <code>n</code>
<code>extend_search_prev_vim</code>	vim N (visual): extend to the reverse match	spacemacs, vim — select: <code>N</code>
<code>add_selection_to_next_match</code>	Add the next occurrence of the selection as a new cursor	
<code>select_all_occurrences</code>	Select every occurrence of the selection as a cursor (JetBrains Select All Occurrences)	spacemacs — normal: <code><space>xo</code> , select: <code><space>xo</code>
<code>search_selection</code>	Use current selection as search pattern	helix — normal: <code><A-*></code> , select: <code><A-*></code>

Name	Description	Keybinds
<code>search_selection_detect_word_boundaries</code>	Use current selection as the search pattern, automatically wrapping with <code>\b</code> on word boundaries	helix — normal: <code>*</code> , select: <code>*</code>
<code>make_search_word_bounded</code>	Modify current search to make it word bounded	
<code>global_search</code>	Global search in workspace folder	spacemacs — normal: <code><space>/</code> , <code><space>po</code> , <code><space>ps</code> , <code><space>sP</code> , <code><space>sb</code> , <code><space>sd</code> , <code><space>sf</code> , <code><space>sp</code> , <code><space>ss</code> , <code><space>saa</code> , <code><space>sab</code> , <code><space>sad</code> , <code><space>saf</code> , <code><space>sap</code> , <code><space>sgb</code> , <code><space>sgd</code> , <code><space>sgf</code> , <code><space>sgg</code> , <code><space>sgp</code> , <code><space>skb</code> , <code><space>skd</code> , <code><space>skf</code> , <code><space>skp</code> , <code><space>srb</code> , <code><space>srD</code> , <code><space>srF</code> , <code><space>srp</code> , <code><space>srR</code> , select: <code><space>/</code> , <code><space>po</code> , <code><space>ps</code> , <code><space>sP</code> , <code><space>sb</code> , <code><space>sd</code> , <code><space>sf</code> , <code><space>sp</code> , <code><space>ss</code> , <code><space>saa</code> , <code><space>sab</code> , <code><space>sad</code> , <code><space>saf</code> , <code><space>sap</code> , <code><space>sgb</code> , <code><space>sgd</code> , <code><space>sgf</code> , <code><space>sgg</code> , <code><space>sgp</code> , <code><space>skb</code> , <code><space>skd</code> , <code><space>skf</code> , <code><space>skp</code> , <code><space>srb</code> , <code><space>srD</code> , <code><space>srF</code> , <code><space>srp</code> , <code><space>srR</code> helix — normal: <code><space>/</code> , select: <code><space>/</code>
<code>dumb_jump_go</code>	Jump to the definition of the symbol at point by grepping for it (dumb-jump-go)	
<code>global_search_symbol</code>	Global search seeded with the symbol under the cursor	spacemacs — normal: <code><space>*</code> , <code><space>sB</code> , <code><space>sD</code> , <code><space>sF</code> , <code><space>saA</code> , <code><space>saB</code> , <code><space>saD</code> , <code><space>saF</code> , <code><space>saP</code> , <code><space>sgB</code> , <code><space>sgF</code> , <code><space>sgG</code> , <code><space>skB</code> , <code><space>skD</code> , <code><space>skF</code> , <code><space>skP</code> , <code><space>srB</code> , <code><space>srD</code> , <code><space>srF</code> , <code><space>srP</code> , <code><space>srR</code> , select: <code><space>*</code> , <code><space>sB</code> , <code><space>sD</code> , <code><space>sF</code> , <code><space>saA</code> , <code><space>saB</code> , <code><space>saD</code> , <code><space>saF</code> , <code><space>saP</code> , <code><space>sgB</code> , <code><space>sgF</code> , <code><space>sgG</code> , <code><space>skB</code> , <code><space>skD</code> , <code><space>skF</code> , <code><space>skP</code> , <code><space>srB</code> , <code><space>srD</code> , <code><space>srF</code> , <code><space>srP</code> , <code><space>srR</code>
<code>clear_search_highlight</code>	Clear persistent search highlight (SPC s c)	spacemacs — normal: <code><space>sc</code> , select: <code><space>sc</code>

Name	Description	Keybinds
<code>regex_convert_form</code>	Convert the selected regex between PCRE and Emacs forms (SPC x r c)	spacemacs — normal: <code><space>xrc</code> , <code><space>xrep</code> , <code><space>xrpe</code> , select: <code><space>xrc</code> , <code><space>xrep</code> , <code><space>xrpe</code>
<code>regex_emacs_to_rx_replace</code>	Convert the selected Emacs regex to rx form (SPC x r e x)	spacemacs — normal: <code><space>xret</code> , <code><space>xrex</code> , select: <code><space>xret</code> , <code><space>xrex</code>
<code>regex_emacs_to_rx_explain</code>	Explain the selected Emacs regex as rx (SPC x r e /)	spacemacs — normal: <code><space>xre/</code> , select: <code><space>xre/</code>
<code>regex_pcre_to_rx_replace</code>	Convert the selected PCRE regex to rx form (SPC x r x)	spacemacs — normal: <code><space>xrt</code> , <code><space>xrx</code> , <code><space>xrpx</code> , select: <code><space>xrt</code> , <code><space>xrx</code> , <code><space>xrpx</code>
<code>regex_pcre_to_rx_explain</code>	Explain the selected PCRE regex as rx (SPC x r /)	spacemacs — normal: <code><space>xr/</code> , <code><space>xrp/</code> , select: <code><space>xr/</code> , <code><space>xrp/</code>
<code>justify_left</code>	Left-justify (fill) the region (SPC x j l)	spacemacs — normal: <code><space>xjl</code> , select: <code><space>xjl</code>
<code>justify_right</code>	Right-justify the region (SPC x j r)	spacemacs — normal: <code><space>xjr</code> , select: <code><space>xjr</code>
<code>justify_center</code>	Center-justify the region (SPC x j c)	spacemacs — normal: <code><space>xjc</code> , select: <code><space>xjc</code>
<code>justify_full</code>	Full-justify the region (SPC x j f)	spacemacs — normal: <code><space>xjf</code> , select: <code><space>xjf</code>
<code>justify_none</code>	Remove justification / left-fill (SPC x j n)	spacemacs — normal: <code><space>xjn</code> , select: <code><space>xjn</code>
<code>count_words_region</code>	Count occurrences per word in the selection (SPC x w c)	spacemacs — normal: <code><space>xwc</code> , select: <code><space>xwc</code>
<code>goto_next_close_paren</code>	Go forward to next closing paren (SPC k j)	spacemacs — normal: <code><space>kj</code> , select: <code><space>kj</code>
<code>goto_prev_open_paren</code>	Go backward to previous opening paren (SPC k k)	spacemacs — normal: <code><space>kk</code> , select: <code><space>kk</code>
<code>ediff_windows</code>	Diff the two front windows side by side (SPC D w w)	spacemacs — normal: <code><space>Dwl</code> , <code><space>Dww</code> , select: <code><space>Dwl</code> , <code><space>Dww</code>
<code>ediff_buffer</code>	Diff the current buffer against a picked buffer (SPC D b b)	spacemacs — normal: <code><space>Dbb</code> , select: <code><space>Dbb</code>
<code>ediff_dotfile_and_template</code>	Diff the config file against the shipped template (SPC f e D)	
<code>compare_with_clipboard</code>	Diff the current buffer against the clipboard (JetBrains Compare with Clipboard)	spacemacs — normal: <code><space>Dc</code> , select: <code><space>Dc</code>
<code>transpose_paragraph</code>	Swap the current paragraph with the previous one (SPC x t p)	spacemacs — normal: <code><space>xtp</code> , select: <code><space>xtp</code>
<code>transpose_line</code>	Swap the current line with the previous one (emacs transpose-lines, C-x C-t)	spacemacs — normal: <code><[-x><[-t></code> , select: <code><[-x><[-t></code> , insert: <code><[-x><[-t></code> emacs — insert: <code><[-x><[-t></code> cua — select: <code><[-x><[-t></code> , insert: <code><[-X><[-t></code>
<code>move_element_right</code>	Swap the syntax node under the cursor with its next sibling (JetBrains Move Element Right)	spacemacs — normal: <code><space>x<gt></code> , select: <code><space>x<gt></code>
<code>move_element_left</code>	Swap the syntax node under the cursor with its previous sibling (JetBrains Move Element Left)	spacemacs — normal: <code><space>x<lt></code> , select: <code><space>x<lt></code>
<code>convert_indent_to_spaces</code>	Convert leading indentation to spaces (JetBrains Convert Indents to Spaces)	
<code>convert_indent_to_tabs</code>	Convert leading indentation to tabs (JetBrains Convert Indents to Tabs)	

Name	Description	Keybinds
<code>transpose_sexp</code>	Swap the current s-expression with the previous one (SPC x t e)	spacemacs — normal: <code><A-[t></code> , <code><space>xte</code> , select: <code><space>xte</code>
<code>transpose_sentence</code>	Swap the current sentence with the previous one (SPC x t s)	spacemacs — normal: <code><space>xts</code> , select: <code><space>xts</code>
<code>make_3_windows</code>	Lay out three vertical windows (SPC w 3)	spacemacs — normal: <code><[w>3</code> , <code><space>w3</code> , <code><space>u<space>w3</code> , select: <code><space>w3</code> , <code><space>u<space>w3</code> vim — normal: <code><[w>3</code>
<code>make_4_windows</code>	Lay out a 2x2 window grid (SPC w 4)	spacemacs — normal: <code><[w>4</code> , <code><space>w4</code> , <code><space>u<space>w4</code> , select: <code><space>w4</code> , <code><space>u<space>w4</code> vim — normal: <code><[w>4</code>
<code>narrow_to_function</code>	Narrow the buffer to the enclosing function (SPC n f)	spacemacs — normal: <code><[x>nd</code> , <code><space>nf</code> , select: <code><[x>nd</code> , <code><space>nf</code> , insert: <code><[x>nd</code>
<code>align_at_equals</code>	Align region at = (SPC x a =)	spacemacs — normal: <code><space>xa=</code> , select: <code><space>xa=</code>
<code>align_at_comma</code>	Align region at , (SPC x a ,)	spacemacs — normal: <code><space>xa,</code> , select: <code><space>xa,</code>
<code>align_at_colon</code>	Align region at : (SPC x a :)	spacemacs — normal: <code><space>xa:</code> , select: <code><space>xa:</code>
<code>align_at_semicolon</code>	Align region at ; (SPC x a ;)	spacemacs — normal: <code><space>xa;</code> , select: <code><space>xa;</code>
<code>align_at_ampersand</code>	Align region at & (SPC x a &)	spacemacs — normal: <code><space>xaf</code> , select: <code><space>xaf</code>
<code>align_at_lparen</code>	Align region at ((SPC x a (	spacemacs — normal: <code><space>xa(</code> , select: <code><space>xa(</code>
<code>align_at_rparen</code>	Align region at) (SPC x a)	spacemacs — normal: <code><space>xa)</code> , select: <code><space>xa)</code>
<code>align_at_lbracket</code>	Align region at [(SPC x a [	spacemacs — normal: <code><space>xa[</code> , select: <code><space>xa[</code>
<code>align_at_rbracket</code>	Align region at] (SPC x a])	spacemacs — normal: <code><space>xa]</code> , select: <code><space>xa]</code>
<code>align_at_lbrace</code>	Align region at { (SPC x a {)	spacemacs — normal: <code><space>xa{</code> , select: <code><space>xa{</code>
<code>align_at_rbrace</code>	Align region at } (SPC x a })	spacemacs — normal: <code><space>xa}</code> , select: <code><space>xa}</code>
<code>align_at_dot</code>	Align region at . (SPC x a .)	spacemacs — normal: <code><space>xa.</code> , select: <code><space>xa.</code>
<code>align_at_arithmetic</code>	Align region at arithmetic operators (SPC x a m)	spacemacs — normal: <code><space>xam</code> , select: <code><space>xam</code>
<code>align_at_regex</code>	Align region at a user-specified regexp (SPC x a r)	spacemacs — normal: <code><space>xar</code> , select: <code><space>xar</code>
<code>align_current</code>	Auto-align the region into columns, per blank-line section (emacs align-current)	
<code>align_entire</code>	Auto-align the whole region into columns as one section (emacs align-entire)	
<code>align_at_bar</code>	Align every	into columns (SPC x a
<code>align_left_at_char</code>	Left-align region at a typed delimiter (SPC x a l)	spacemacs — normal: <code><space>xal</code> , select: <code><space>xal</code>
<code>align_right_at_char</code>	Right-align region at a typed delimiter (SPC x a L)	spacemacs — normal: <code><space>xal</code> , select: <code><space>xal</code>
<code>buffer_to_window_1</code>	Move current buffer to window 1 (SPC b . 1)	spacemacs — normal: <code><space>b.1</code> , select: <code><space>b.1</code>
<code>buffer_to_window_2</code>	Move current buffer to window 2 (SPC b . 2)	spacemacs — normal: <code><space>b.2</code> , select: <code><space>b.2</code>

Name	Description	Keybinds
<code>buffer_to_window_3</code>	Move current buffer to window 3 (SPC b . 3)	spacemacs — normal: <code><space>b . 3</code> , select: <code><space>b . 3</code>
<code>buffer_to_window_4</code>	Move current buffer to window 4 (SPC b . 4)	spacemacs — normal: <code><space>b . 4</code> , select: <code><space>b . 4</code>
<code>buffer_to_window_5</code>	Move current buffer to window 5 (SPC b . 5)	spacemacs — normal: <code><space>b . 5</code> , select: <code><space>b . 5</code>
<code>buffer_to_window_6</code>	Move current buffer to window 6 (SPC b . 6)	spacemacs — normal: <code><space>b . 6</code> , select: <code><space>b . 6</code>
<code>buffer_to_window_7</code>	Move current buffer to window 7 (SPC b . 7)	spacemacs — normal: <code><space>b . 7</code> , select: <code><space>b . 7</code>
<code>buffer_to_window_8</code>	Move current buffer to window 8 (SPC b . 8)	spacemacs — normal: <code><space>b . 8</code> , select: <code><space>b . 8</code>
<code>buffer_to_window_9</code>	Move current buffer to window 9 (SPC b . 9)	spacemacs — normal: <code><space>b . 9</code> , select: <code><space>b . 9</code>
<code>goto_window_1</code>	Go to window 1 (SPC 1)	spacemacs — normal: <code><C-w> . 1</code> , <code><C-w>[1, <C-w>{ 1, <space>1, <space>w . 1, <space>w[1, <space>w{ 1, <space>b . <C-1></code> , select: <code><space>1, <space>w . 1, <space>w[1, <space>w{ 1, <space>b . <C-1></code> vim — normal: <code><C-w> . 1, <C-w>[1, <C-w>{ 1</code>
<code>goto_window_2</code>	Go to window 2 (SPC 2)	spacemacs — normal: <code><C-w> . 2</code> , <code><C-w>[2, <C-w>{ 2, <space>2, <space>w . 2, <space>w[2, <space>w{ 2, <space>b . <C-2></code> , select: <code><space>2, <space>w . 2, <space>w[2, <space>w{ 2, <space>b . <C-2></code> vim — normal: <code><C-w> . 2, <C-w>[2, <C-w>{ 2</code>
<code>goto_window_3</code>	Go to window 3 (SPC 3)	spacemacs — normal: <code><C-w> . 3</code> , <code><C-w>[3, <C-w>{ 3, <space>3, <space>w . 3, <space>w[3, <space>w{ 3, <space>b . <C-3></code> , select: <code><space>3, <space>w . 3, <space>w[3, <space>w{ 3, <space>b . <C-3></code> vim — normal: <code><C-w> . 3, <C-w>[3, <C-w>{ 3</code>
<code>goto_window_4</code>	Go to window 4 (SPC 4)	spacemacs — normal: <code><C-w> . 4</code> , <code><C-w>[4, <C-w>{ 4, <space>4, <space>w . 4, <space>w[4, <space>w{ 4, <space>b . <C-4></code> , select: <code><space>4, <space>w . 4, <space>w[4, <space>w{ 4, <space>b . <C-4></code> vim — normal: <code><C-w> . 4, <C-w>[4, <C-w>{ 4</code>
<code>goto_window_5</code>	Go to window 5 (SPC 5)	spacemacs — normal: <code><C-w> . 5</code> , <code><C-w>[5, <C-w>{ 5, <space>5, <space>w . 5, <space>w[5, <space>w{ 5, <space>b . <C-5></code> , select: <code><space>5, <space>w . 5, <space>w[5, <space>w{ 5, <space>b . <C-5></code> vim — normal: <code><C-w> . 5, <C-w>[5, <C-w>{ 5</code>

Name	Description	Keybinds
<code>goto_window_6</code>	Go to window 6 (SPC 6)	spacemacs — normal: <code><C-w>.6,</code> <code><C-w>[6, <C-w>{6, <space>6,</code> <code><space>w.6, <space>w[6,</code> <code><space>w{6, <space>b.<C-6>,</code> select: <code><space>6, <space>w.6,</code> <code><space>w[6, <space>w{6,</code> <code><space>b.<C-6></code> vim — normal: <code><C-w>.6, <C-w>[6, <C-w>{6</code>
<code>goto_window_7</code>	Go to window 7 (SPC 7)	spacemacs — normal: <code><C-w>.7,</code> <code><C-w>[7, <C-w>{7, <space>7,</code> <code><space>w.7, <space>w[7,</code> <code><space>w{7, <space>b.<C-7>,</code> select: <code><space>7, <space>w.7,</code> <code><space>w[7, <space>w{7,</code> <code><space>b.<C-7></code> vim — normal: <code><C-w>.7, <C-w>[7, <C-w>{7</code>
<code>goto_window_8</code>	Go to window 8 (SPC 8)	spacemacs — normal: <code><C-w>.8,</code> <code><C-w>[8, <C-w>{8, <space>8,</code> <code><space>w.8, <space>w[8,</code> <code><space>w{8, <space>b.<C-8>,</code> select: <code><space>8, <space>w.8,</code> <code><space>w[8, <space>w{8,</code> <code><space>b.<C-8></code> vim — normal: <code><C-w>.8, <C-w>[8, <C-w>{8</code>
<code>goto_window_9</code>	Go to window 9 (SPC 9)	spacemacs — normal: <code><C-w>.9,</code> <code><C-w>[9, <C-w>{9, <space>9,</code> <code><space>w.9, <space>w[9,</code> <code><space>w{9, <space>b.<C-9>,</code> select: <code><space>9, <space>w.9,</code> <code><space>w[9, <space>w{9,</code> <code><space>b.<C-9></code> vim — normal: <code><C-w>.9, <C-w>[9, <C-w>{9</code>
<code>delete_window_and_buffer</code>	Close window and kill its buffer (SPC w . x)	spacemacs — normal: <code><C-w>.x,</code> <code><C-w>[x, <C-w>{x, <C-x>40,</code> <code><space>w.x, <space>w[x,</code> <code><space>w{x, <space>u<space>bD,</code> <code><space>u<space>wD, select:</code> <code><C-x>40, <space>w.x, <space>w[x,</code> <code><space>w{x, <space>u<space>bD,</code> <code><space>u<space>wD, insert:</code> <code><C-x>40</code> vim — normal: <code><C-w>.x,</code> <code><C-w>[x, <C-w>{x</code> emacs — insert: <code><C-x>40</code> cua — select: <code><C-X>40,</code> insert: <code><C-x>40</code>
<code>eval_elisp_region</code>	Evaluate the selection as elisp (SPC m e r)	spacemacs — normal: <code><space>mer,</code> select: <code><space>mer</code>
<code>eval_elisp_buffer</code>	Evaluate the buffer as elisp (SPC m e b)	spacemacs — normal: <code><space>meb,</code> select: <code><space>meb</code>
<code>eval_elisp_line</code>	Evaluate the current line as elisp (SPC m e e)	spacemacs — normal: <code><space>me\$,</code> <code><space>mee, <space>mel, select:</code> <code><space>me\$, <space>mee,</code> <code><space>mel</code>
<code>eval_elisp_defun</code>	Evaluate the enclosing form as elisp (SPC m e f)	spacemacs — normal: <code><space>mec,</code> <code><space>mef, select: <space>mec,</code> <code><space>mef</code>
<code>eval_print_last_sexp</code>	Evaluate the sexp before point and insert its value (emacs eval-print-last-sexp)	

Name	Description	Keybinds
<code>eval_last_sexp</code>	Evaluate the sexp before point and echo its value (emacs eval-last-sexp, C-x C-e)	spacemacs — normal: <code><[-x><[-e></code> , select: <code><[-x><[-e></code> , insert: <code><[-x><[-e></code>
<code>compare_windows</code>	Compare this window with the next, moving both points to the first difference (emacs compare-windows)	
<code>layout_create</code>	Create a new window-layout from the current windows (SPC I l)	spacemacs — normal: <code><space>ll</code> , <code><space>lww</code> , select: <code><space>ll</code> , <code><space>lww</code>
<code>layout_next</code>	Switch to the next layout (SPC I n)	spacemacs — normal: <code><space>ln</code> , <code><space>lw1</code> , <code><space>lwn</code> , <code><space>l<[-l></code> , select: <code><space>ln</code> , <code><space>lw1</code> , <code><space>lwn</code> , <code><space>l<[-l></code>
<code>layout_prev</code>	Switch to the previous layout (SPC I p)	spacemacs — normal: <code><space>lN</code> , <code><space>lp</code> , <code><space>lwN</code> , <code><space>lwh</code> , <code><space>lpw</code> , <code><space>l<[-h></code> , select: <code><space>lN</code> , <code><space>lp</code> , <code><space>lwN</code> , <code><space>lwh</code> , <code><space>lpw</code> , <code><space>l<[-h></code>
<code>layout_last</code>	Switch to the last-used layout (SPC I TAB)	spacemacs — normal: <code><space>l<tab></code> , <code><space>lw<tab></code> , select: <code><space>l<tab></code> , <code><space>lw<tab></code>
<code>layout_default</code>	Switch to the default (first) layout (SPC I h)	spacemacs — normal: <code><space>lh</code> , select: <code><space>lh</code>
<code>layout_delete</code>	Delete the current layout, keeping its buffers (SPC I d)	spacemacs — normal: <code><space>lD</code> , <code><space>lX</code> , <code><space>ld</code> , <code><space>lx</code> , <code><space>lwd</code> , select: <code><space>lD</code> , <code><space>lX</code> , <code><space>ld</code> , <code><space>lx</code> , <code><space>lwd</code>
<code>layout_save</code>	Save layouts to disk (SPC I s)	spacemacs — normal: <code><space>lS</code> , select: <code><space>lS</code>
<code>layout_rename</code>	Rename the current layout (SPC I R)	spacemacs — normal: <code><space>lR</code> , <code><space>lwR</code> , select: <code><space>lR</code> , <code><space>lwR</code>
<code>layout_load</code>	Load layouts from disk (SPC I L)	spacemacs — normal: <code><space>ll</code> , <code><space>lo</code> , select: <code><space>ll</code> , <code><space>lo</code>
<code>layout_goto_1</code>	Switch to layout 1 (SPC I 1)	spacemacs — normal: <code><space>l1</code> , <code><space>lw1</code> , <code><space>l<[-1></code> , <code><space>lw<[-1></code> , select: <code><space>l1</code> , <code><space>lw1</code> , <code><space>l<[-1></code> , <code><space>lw<[-1></code>
<code>layout_goto_2</code>	Switch to layout 2 (SPC I 2)	spacemacs — normal: <code><space>l2</code> , <code><space>lw2</code> , <code><space>l<[-2></code> , <code><space>lw<[-2></code> , select: <code><space>l2</code> , <code><space>lw2</code> , <code><space>l<[-2></code> , <code><space>lw<[-2></code>
<code>layout_goto_3</code>	Switch to layout 3 (SPC I 3)	spacemacs — normal: <code><space>l3</code> , <code><space>lw3</code> , <code><space>l<[-3></code> , <code><space>lw<[-3></code> , select: <code><space>l3</code> , <code><space>lw3</code> , <code><space>l<[-3></code> , <code><space>lw<[-3></code>
<code>layout_goto_4</code>	Switch to layout 4 (SPC I 4)	spacemacs — normal: <code><space>l4</code> , <code><space>lw4</code> , <code><space>l<[-4></code> , <code><space>lw<[-4></code> , select: <code><space>l4</code> , <code><space>lw4</code> , <code><space>l<[-4></code> , <code><space>lw<[-4></code>

Name	Description	Keybinds
<code>layout_goto_5</code>	Switch to layout 5 (SPC I 5)	spacemacs — normal: <code><space>l5</code> , <code><space>lw5</code> , <code><space>l<[-5></code> , <code><space>lw<[-5></code> , select: <code><space>l5</code> , <code><space>lw5</code> , <code><space>l<[-5></code> , <code><space>lw<[-5></code>
<code>layout_goto_6</code>	Switch to layout 6 (SPC I 6)	spacemacs — normal: <code><space>l6</code> , <code><space>lw6</code> , <code><space>l<[-6></code> , <code><space>lw<[-6></code> , select: <code><space>l6</code> , <code><space>lw6</code> , <code><space>l<[-6></code> , <code><space>lw<[-6></code>
<code>layout_goto_7</code>	Switch to layout 7 (SPC I 7)	spacemacs — normal: <code><space>l7</code> , <code><space>lw7</code> , <code><space>l<[-7></code> , <code><space>lw<[-7></code> , select: <code><space>l7</code> , <code><space>lw7</code> , <code><space>l<[-7></code> , <code><space>lw<[-7></code>
<code>layout_goto_8</code>	Switch to layout 8 (SPC I 8)	spacemacs — normal: <code><space>l8</code> , <code><space>lw8</code> , <code><space>l<[-8></code> , <code><space>lw<[-8></code> , select: <code><space>l8</code> , <code><space>lw8</code> , <code><space>l<[-8></code> , <code><space>lw<[-8></code>
<code>layout_goto_9</code>	Switch to layout 9 (SPC I 9)	spacemacs — normal: <code><space>l9</code> , <code><space>lw9</code> , <code><space>l<[-9></code> , <code><space>lw<[-9></code> , select: <code><space>l9</code> , <code><space>lw9</code> , <code><space>l<[-9></code> , <code><space>lw<[-9></code>
<code>toggle_modeline_position</code>	Toggle cursor position in the mode line (SPC t m p)	spacemacs — normal: <code><space>tmp</code> , select: <code><space>tmp</code>
<code>toggle_modeline_vcs</code>	Toggle version-control info in the mode line (SPC t m v)	spacemacs — normal: <code><space>tmv</code> , select: <code><space>tmv</code>
<code>toggle_modeline_major_mode</code>	Toggle the major mode (file type) in the mode line (SPC t m M)	
<code>toggle_system_monitor</code>	Toggle the system monitor in the minibuffer (symon-mode, SPC t m s)	
<code>toggle_centered_cursor</code>	Keep the cursor vertically centered (SPC t -)	spacemacs — normal: <code><space>wc[-</code> , <code><space>wcc</code> , <code><space>t<minus></code> , select: <code><space>wc[-</code> , <code><space>wcc</code> , <code><space>t<minus></code>
<code>toggle_hl_line</code>	Highlight the current line (emacs hl-line-mode / global-hl-line-mode)	
<code>toggle_electric_pair</code>	Auto-insert matching close delimiters (emacs electric-pair-mode)	
<code>toggle_auto_revert</code>	Reload buffers when their file changes on disk (emacs auto-revert-mode)	
<code>auto_revert_tail_mode</code>	Follow the tail of the file as it grows, like tail -f (emacs auto-revert-tail-mode)	
<code>set_fill_prefix</code>	Set the fill-prefix from line start to point (emacs set-fill-prefix)	spacemacs — normal: <code><[-x>.</code> , select: <code><[-x>.</code> , insert: <code><[-x>.</code>
<code>set_goal_column</code>	Make vertical motion stick to the current column (emacs set-goal-column)	spacemacs — normal: <code><[-x><[-n></code> , select: <code><[-x><[-n></code> , insert: <code><[-x><[-n></code>
<code>toggle_fill_column</code>	Toggle a fill-column ruler (SPC t f)	spacemacs — normal: <code><space>tf</code> , select: <code><space>tf</code>
<code>toggle_long_line_marker</code>	Toggle an 80th-column ruler (SPC t 8)	spacemacs — normal: <code><space>t8</code> , <code><space>t<[-8></code> , select: <code><space>t8</code> , <code><space>t<[-8></code>
<code>toggle_soft_wrap</code>	Toggle soft-wrap of long lines (IntelliJ View > Soft-Wrap)	spacemacs — normal: <code><[-x>xt</code> , select: <code><[-x>xt</code> , insert: <code><[-x>xt</code>
<code>toggle_whitespace_render</code>	Toggle rendering of whitespace characters (IntelliJ View > Show Whitespaces)	

Name	Description	Keybinds
<code>toggle_line_numbers</code>	Toggle the line-numbers gutter (IntelliJ View > Show Line Numbers)	
<code>toggle_indent_guides</code>	Toggle indentation guides (IntelliJ View > Show Indent Guides)	
<code>toggle_inlay_hints</code>	Toggle display of LSP inlay hints (IntelliJ View > Inlay Hints)	
<code>toggle_auto_highlight</code>	Toggle automatic symbol-under-cursor highlight (SPC t h a)	spacemacs — normal: <code><space>tha</code> , select: <code><space>tha</code>
<code>toggle_syntax_highlighting</code>	Toggle syntax highlighting for the current buffer (SPC t h s)	spacemacs — normal: <code><space>ths</code> , select: <code><space>ths</code>
<code>toggle_diagnostics</code>	Toggle diagnostics display / flycheck (SPC t s)	spacemacs — normal: <code><space>ts</code> , select: <code><space>ts</code>
<code>ediff_file</code>	Diff a prompted file against the current buffer (SPC D f f)	spacemacs — normal: <code><space>Dff</code> , select: <code><space>Dff</code>
<code>ediff_documentation</code>	Show the Ediff manual and the ediff sessions this build ships (ediff-documentation, SPC D h)	
<code>ediff_3_files</code>	3-way diff of three prompted files, read-only (SPC D f 3)	spacemacs — normal: <code><space>Df3</code> , select: <code><space>Df3</code>
<code>ediff_directories</code>	Compare two directories: list same-name files that differ or are unique, open ediff on one (emacs ediff-directories, SPC D d d)	spacemacs — normal: <code><space>Ddd</code> , select: <code><space>Ddd</code>
<code>ediff_directories3</code>	Compare three directories: list same-name files present in all three that differ, open a 3-way ediff (emacs ediff-directories3, SPC D d 3)	spacemacs — normal: <code><space>Dd3</code> , select: <code><space>Dd3</code>
<code>ediff_merge_directories_with_ancestor</code>	Merge the same-name files of two directories against a third holding their ancestors (emacs ediff-merge-directories-with-ancestor, SPC D m d 3)	
<code>ediff_directory_revisions</code>	Compare a directory's files against their revisions (emacs ediff-directory-revisions, SPC D d r)	
<code>ediff_merge_revisions_with_ancestor</code>	Merge two revisions of this file against their common ancestor (emacs ediff-merge-revisions-with-ancestor, SPC D m r 3)	
<code>ediff_merge_revisions</code>	Merge two revisions of this file, no ancestor (emacs ediff-merge-revisions, SPC D m r r)	
<code>ediff_regions</code>	Ediff two regions linewise: mark A, then diff B (SPC D r l)	spacemacs — normal: <code><space>Drl</code> , select: <code><space>Drl</code>
<code>ediff_regions_wordwise</code>	Ediff two regions wordwise: mark A, then diff B by word (SPC D r w)	spacemacs — normal: <code><space>Drw</code> , select: <code><space>Drw</code>
<code>ediff_merge_file</code>	Merge a picked file into the current buffer (editable, SPC D m f f)	spacemacs — normal: <code><space>Dmff</code> , select: <code><space>Dmff</code>
<code>ediff_3_buffers</code>	3-way diff of three open buffers, read-only (SPC D b 3)	spacemacs — normal: <code><space>Db3</code> , select: <code><space>Db3</code>
<code>kill_buffers_by_regex</code>	Kill all buffers whose name matches a regex (SPC b M)	spacemacs — normal: <code><space>b<C-D></code> , select: <code><space>b<C-D></code>
<code>narrow_to_page</code>	Narrow the buffer to the current page (SPC n p)	spacemacs — normal: <code><C-x>np</code> , <code><space>np</code> , select: <code><C-x>np</code> , <code><space>np</code> , insert: <code><C-x>np</code>
<code>copy_file</code>	Copy the current file to a prompted destination (SPC f c)	spacemacs — normal: <code><space>fc</code> , select: <code><space>fc</code>

Name	Description	Keybinds
<code>find_file_replace_buffer</code>	Open a file and replace the current buffer with it (SPC f A)	spacemacs — normal: <code><space>fA</code> , <code><[-x]<[-v></code> , select: <code><space>fA</code> , <code><[-x]<[-v></code> , insert: <code><[-x]<[-v>emacs</code> — insert: <code><[-x]<[-v>cuA</code> — select: <code><[-X]<[-v></code> , insert: <code><[-x]<[-v></code>
<code>open_file_literally</code>	Open a file with no syntax/language (fundamental mode, SPC f l)	spacemacs — normal: <code><space>fl</code> , select: <code><space>fl</code>
<code>locate_file</code>	Locate a file via system locate/mdfind and open it (SPC f L)	spacemacs — normal: <code><space>fl</code> , select: <code><space>fl</code>
<code>edit_project_config</code>	Edit the project-local .zmax/config.toml (SPC p e)	spacemacs — normal: <code><space>pe</code> , select: <code><space>pe</code>
<code>man_page_search</code>	Search man pages via apropos and view the selected page (SPC h m)	spacemacs — normal: <code><[-h>S</code> , <code><space>hm</code> , select: <code><[-h>S</code> , <code><space>hm</code> , insert: <code><[-h>S</code>
<code>info_search</code>	Search GNU info manuals (apropos) and view the selected node (SPC h i)	spacemacs — normal: <code><[-h>i</code> , <code><space>hi</code> , select: <code><[-h>i</code> , <code><space>hi</code> , insert: <code><[-h>i</code>
<code>info_goto_emacs_key_command_node</code>	Go to the Emacs manual node for the command a key runs (emacs Info-goto-emacs-key-command-node, C-h K)	
<code>info_lookup_file</code>	Look the file name at point up in the Info file index (emacs info-lookup-file)	
<code>dash_at_point</code>	Look the symbol at point up in the Dash/Zeal offline docsets (spacemacs dash layer, SPC d d)	
<code>dash_at_point_with_docset</code>	Look the symbol at point up in a chosen Dash/Zeal docset (spacemacs dash layer, SPC d D)	
<code>speed_reading</code>	Speed-read the buffer one word at a time, RSVP style (spacemacs speed-reading layer, spray-mode)	
<code>diagnostics_verify_setup</code>	Report the buffer's diagnostics/LSP setup (SPC e v)	spacemacs — normal: <code><space>ev</code> , select: <code><space>ev</code>
<code>clear_diagnostics</code>	Clear all diagnostics for the current buffer (SPC e c)	spacemacs — normal: <code><space>ec</code> , select: <code><space>ec</code>
<code>ai_chat</code>	Ask the AI provider about the selection/buffer (SPC a i)	spacemacs — normal: <code><space>ai</code> , select: <code><space>ai</code>
<code>ai_chat_panel</code>	Open the streaming AI chat drawer (SPC a p)	spacemacs — normal: <code><space>ap</code> , select: <code><space>ap</code>
<code>ai_model_picker</code>	Pick the AI model at runtime (SPC a m)	
<code>toggle_ai_privacy</code>	Toggle AI privacy mode (SPC a P)	spacemacs — normal: <code><space>aP</code> , select: <code><space>aP</code>
<code>ai_apply_block</code>	Apply the last AI code block to the selection (SPC a y)	spacemacs — normal: <code><space>ay</code> , select: <code><space>ay</code>
<code>ai_add_file_context</code>	Add a file as @context for the next AI chat (SPC a @)	spacemacs — normal: <code><space>a@</code> , select: <code><space>a@</code>
<code>ai_codebase_context</code>	Add codebase-search results as @context (SPC a b)	spacemacs — normal: <code><space>ab</code> , select: <code><space>ab</code>
<code>ai_symbol_context</code>	Add the symbol-under-cursor's definitions as @context (SPC a s)	spacemacs — normal: <code><space>as</code> , select: <code><space>as</code>
<code>ai_terminal_command</code>	Generate a shell command from natural language (SPC a k)	spacemacs — normal: <code><space>ak</code> , select: <code><space>ak</code>
<code>ai_inline_edit</code>	AI inline edit/generate on the selection (SPC a e)	spacemacs — normal: <code><space>ae</code> , select: <code><space>ae</code>
<code>ai_inline_edit_preview</code>	AI inline edit with a diff preview (SPC a E)	spacemacs — normal: <code><space>aE</code> , select: <code><space>aE</code>

Name	Description	Keybinds
<code>ai_accept_edit</code>	Accept the pending AI inline-edit preview (SPC a .)	spacemacs — normal: <code><space>a .</code> , select: <code><space>a .</code>
<code>ai_explain</code>	Explain the selected code with AI (SPC a x)	spacemacs — normal: <code><space>ax</code> , select: <code><space>ax</code>
<code>ai_generate_tests</code>	Generate tests for the selection with AI (SPC a u)	spacemacs — normal: <code><space>au</code> , select: <code><space>au</code>
<code>ai_commit_message</code>	Generate a git commit message with AI (SPC a c)	
<code>ai_agent</code>	Run the autonomous AI agent on a task (SPC a a)	spacemacs — normal: <code><space>aa</code> , select: <code><space>aa</code>
<code>ai_agent_review</code>	Toggle agent review (dry-run) mode — propose changes without applying (SPC a R)	spacemacs — normal: <code><space>aR</code> , select: <code><space>aR</code>
<code>ai_complete</code>	AI code completion at the cursor (SPC a TAB)	spacemacs — normal: <code><space>a<tab></code> , select: <code><space>a<tab></code>
<code>ai_docs_context</code>	@docs keyword-search over the docs directory for AI context (SPC a D)	spacemacs — normal: <code><space>aD</code> , select: <code><space>aD</code>
<code>ai_web_context</code>	@web live web-search results as AI context (SPC a w)	spacemacs — normal: <code><space>aw</code> , select: <code><space>aw</code>
<code>toggle_ai_autocomplete</code>	Toggle real-time AI ghost-text autocomplete (SPC a g)	spacemacs — normal: <code><space>ag</code> , select: <code><space>ag</code>
<code>ghost_text_accept</code>	Accept the AI ghost-text suggestion, else Tab (insert mode)	spacemacs, vim — insert: <code><tab></code>
<code>ghost_text_accept_word</code>	Accept the next word of the AI ghost-text suggestion (partial accept)	spacemacs, vim — insert: <code><A-right></code>
<code>ai_revert_agent</code>	Revert the workspace to the last agent checkpoint (SPC a z)	spacemacs — normal: <code><space>az</code> , select: <code><space>az</code>
<code>ai_fix</code>	AI-fix the diagnostic(s) on the current line (SPC a F)	spacemacs — normal: <code><space>aF</code> , select: <code><space>aF</code>
<code>describe_diagnostics_checker</code>	Describe the buffer's checkers/language servers (SPC e h)	spacemacs — normal: <code><space>eh</code> , select: <code><space>eh</code>
<code>describe_text_properties</code>	Describe the tree-sitter node stack at the cursor (SPC h d t)	spacemacs — normal: <code><space>hdt</code> , select: <code><space>hdt</code>
<code>copy_system_info</code>	Copy system info (version/OS/arch) to the clipboard (SPC h d s)	spacemacs — normal: <code><space>hds</code> , select: <code><space>hds</code>
<code>copy_last_keys</code>	Copy the most recently pressed keys to the clipboard (SPC h d l)	spacemacs — normal: <code><space>hdl</code> , select: <code><space>hdl</code>
<code>ace_window</code>	Jump to a window by its number, prompted (ace-window, SPC w . a)	spacemacs — normal: <code><C-w>.a</code> , <code><C-w>[a</code> , <code><C-w>{a</code> , <code><space>w.a</code> , <code><space>w[a</code> , <code><space>w{a</code> , select: <code><space>w.a</code> , <code><space>w[a</code> , <code><space>w{a</code> , vim — normal: <code><C-w>.a</code> , <code><C-w>[a</code> , <code><C-w>{a</code>
<code>browse_news</code>	Browse zmax release notes / NEWS (SPC h n)	spacemacs — normal: <code><C-h>n</code> , <code><space>hn</code> , <code><C-h><C-n></code> , select: <code><C-h>n</code> , <code><space>hn</code> , <code><C-h><C-n></code> , insert: <code><C-h>n</code> , <code><C-h><C-n></code>
<code>browse_faq</code>	Browse the zmax FAQ (SPC h f)	spacemacs — normal: <code><space>hf</code> , select: <code><space>hf</code>
<code>layer_search</code>	Search zmax capability areas / layers (SPC h l)	spacemacs — normal: <code><space>hl</code> , select: <code><space>hl</code>
<code>show_environment</code>	Show the editor's environment variables (SPC f e e)	spacemacs — normal: <code><space>fee</code> , select: <code><space>fee</code>
<code>reimport_shell_env</code>	Re-import the shell environment into the editor (SPC f e C-e)	spacemacs — normal: <code><space>feE</code> , <code><space>fe<C-e></code> , select: <code><space>feE</code> , <code><space>fe<C-e></code>
<code>image_converter_add_handler</code>	Register an external converter for an image suffix (emacs image-converter-add-handler)	

Name	Description	Keybinds
<code>image_converter_list_handlers</code>	List the registered image converters (emacs image-converter)	
<code>conda_env_list</code>	List the available conda environments (Spacemacs conda-env-list)	
<code>conda_env_activate</code>	Activate a conda environment for spawned processes (Spacemacs conda-env-activate)	
<code>conda_env_deactivate</code>	Deactivate the active conda environment (Spacemacs conda-env-deactivate)	
<code>quickurl</code>	Insert the URL of the quickurl entry named by the word at point (emacs quickurl)	
<code>quickurl_ask</code>	Prompt for a quickurl entry and insert its URL (emacs quickurl-ask)	
<code>quickurl_add_url</code>	Add a named URL to the quickurl list (emacs quickurl-add-url)	
<code>quickurl_list</code>	List the stored quickurl entries (emacs quickurl-list)	
<code>quickurl_browse_url</code>	Browse the quickurl entry named by the word at point (emacs quickurl-browse-url)	
<code>quickurl_browse_url_ask</code>	Prompt for a quickurl entry and browse it (emacs quickurl-browse-url-ask)	
<code>rebox_dwim</code>	Box the selected lines / rebuild the comment box at point (rebox2 rebox-dwim)	
<code>rebox_cycle</code>	Cycle the comment box at point to the next rebox style (rebox2 rebox-cycle)	
<code>goto_buffer_window</code>	Focus the window already showing a chosen buffer (SPC b w)	spacemacs — normal: <code><space>bw</code> , select: <code><space>bw</code>
<code>git_file_dispatch</code>	Magit-style file operations dispatch for the current file (SPC g f m)	spacemacs — normal: <code><space>gfm</code> , select: <code><space>gfm</code>
<code>describe_current_modes</code>	Describe the current editor/buffer modes (SPC h d m)	spacemacs — normal: <code><C-h>m</code> , <code><space>hdm</code> , select: <code><C-h>m</code> , <code><space>hdm</code> , insert: <code><C-h>m</code>
<code>describe_command</code>	Describe a command — its doc and key bindings (C-h f)	spacemacs — normal: <code><C-h>x</code> , select: <code><C-h>x</code> , insert: <code><C-h>x</code>
<code>where_is</code>	Show the keys a command is bound to (C-h w)	spacemacs — normal: <code><C-h>w</code> , select: <code><C-h>w</code> , insert: <code><C-h>w</code>
<code>describe_key</code>	Describe a key — pick a binding, show its command and doc (C-h k)	spacemacs — normal: <code><C-h>k</code> , select: <code><C-h>k</code> , insert: <code><C-h>k</code>
<code>describe_bindings</code>	List every key binding of the current mode (C-h b)	spacemacs — normal: <code><C-h>b</code> , <code><space>tkb</code> , select: <code><C-h>b</code> , <code><space>tkb</code> , insert: <code><C-h>b</code>
<code>describe_coding_system</code>	Describe the buffer's coding system / encoding (C-h C)	spacemacs — normal: <code><C-h>C</code> , select: <code><C-h>C</code> , insert: <code><C-h>C</code>
<code>describe_language_environment</code>	Describe the language environment / locale (C-h L)	spacemacs — normal: <code><C-h>L</code> , select: <code><C-h>L</code> , insert: <code><C-h>L</code>
<code>describe_syntax</code>	Describe the buffer's syntax / tree-sitter status (C-h s)	spacemacs — normal: <code><C-h>s</code> , select: <code><C-h>s</code> , insert: <code><C-h>s</code>
<code>describe_fontset</code>	Describe the fontset — a tty uses none, so this reports so (emacs describe-fontset)	
<code>view_lossage</code>	Show the recently pressed keys (C-h l)	spacemacs — normal: <code><C-h>l</code> , select: <code><C-h>l</code> , insert: <code><C-h>l</code>
<code>describe_char</code>	Describe the character after point — code, Unicode block, category (emacs describe-char, C-u C-x =)	

Name	Description	Keybinds
<code>emoji_describe</code>	Say what the emoji after point is called (emacs emoji-describe)	
<code>emoji_list</code>	Pick an emoji by name and insert it (emacs emoji-list)	
<code>view_hello_file</code>	Show a multi-script greeting sample (emacs view-hello-file, C-h h)	spacemacs — normal: <code><C-h>h</code> , select: <code><C-h>h</code> , insert: <code><C-h>h</code>
<code>view_echo_area_messages</code>	Show the last echo-area message (emacs view-echo-area-messages, C-h e)	spacemacs — normal: <code>g<lt></code> , <code><C-h>e</code> , select: <code><C-h>e</code> , insert: <code><C-h>evim</code> — normal: <code>g<lt></code>
<code>describe_copying</code>	Show zmax's copying license, the GPL (emacs describe-copying, C-h C-c)	spacemacs — normal: <code><C-h><C-c></code> , select: <code><C-h><C-c></code> , insert: <code><C-h><C-c></code>
<code>describe_distribution</code>	How to get zmax / GNU software (emacs describe-distribution, C-h C-d)	spacemacs — normal: <code><C-h><C-d></code> , <code><C-h><C-o></code> , select: <code><C-h><C-d></code> , <code><C-h><C-o></code> , insert: <code><C-h><C-d></code> , <code><C-h><C-o></code>
<code>describe_gnu_project</code>	Open the GNU project page (emacs describe-gnu-project, C-h g)	spacemacs — normal: <code><C-h>g</code> , select: <code><C-h>g</code> , insert: <code><C-h>g</code>
<code>describe_no_warranty</code>	Show the GPL no-warranty sections (emacs describe-no-warranty, C-h C-w)	spacemacs — normal: <code><C-h><C-w></code> , select: <code><C-h><C-w></code> , insert: <code><C-h><C-w></code>
<code>view_emacs_faq</code>	Open the GNU Emacs FAQ (emacs view-emacs-FAQ, C-h C-f)	spacemacs — normal: <code><C-h><C-f></code> , select: <code><C-h><C-f></code> , insert: <code><C-h><C-f></code>
<code>view_emacs_todo</code>	Open the Emacs TODO list (emacs view-emacs-todo)	spacemacs — normal: <code><C-h><C-t></code> , select: <code><C-h><C-t></code> , insert: <code><C-h><C-t></code>
<code>view_emacs_problems</code>	Open the Emacs known-problems file (emacs view-emacs-problems)	spacemacs — normal: <code><C-h><C-p></code> , select: <code><C-h><C-p></code> , insert: <code><C-h><C-p></code>
<code>view_emacs_debugging</code>	Open the Emacs debugging manual (emacs view-emacs-debugging)	
<code>view_order_manuals</code>	Open where to get the GNU manuals (emacs view-order-manuals)	spacemacs — normal: <code><C-h><C-m></code> , select: <code><C-h><C-m></code> , insert: <code><C-h><C-m></code>
<code>view_external_packages</code>	Open GNU ELPA / external packages (emacs view-external-packages)	spacemacs — normal: <code><C-h><C-e></code> , select: <code><C-h><C-e></code> , insert: <code><C-h><C-e></code>
<code>describe_keymap</code>	List every binding of the current mode's keymap (emacs describe-keymap)	spacemacs — normal: <code><space>hdK</code> , <code><space>tkM</code> , <code><space>tkm</code> , select: <code><space>hdK</code> , <code><space>tkM</code> , <code><space>tkm</code>
<code>describe_prefix_bindings</code>	List the sub-bindings of a prefix (emacs describe-prefix-bindings)	
<code>describe_categories</code>	List the character categories zmax recognises (emacs describe-categories)	
<code>list_character_sets</code>	List the Unicode blocks zmax knows (emacs list-character-sets)	
<code>list_charset_chars</code>	List the printable characters of each Unicode block (emacs list-charset-chars)	
<code>list_coding_systems</code>	List the coding systems / encodings zmax supports (emacs list-coding-systems)	
<code>describe_language_package</code>	Describe the language-support config for the buffer (SPC h d p)	spacemacs — normal: <code><space>hdp</code> , select: <code><space>hdp</code>

Name	Description	Keybinds
<code>package_search</code>	Search configured language packages and describe one (SPC h p)	spacemacs — normal: <code><[-h>P</code> , <code><[-h>p</code> , <code><space>hp</code> , <code><space>hdP</code> , select: <code><[-h>P</code> , <code><[-h>p</code> , <code><space>hp</code> , <code><space>hdP</code> , insert: <code><[-h>P</code> , <code><[-h>p</code>
<code>config_variable_search</code>	Search editor config variables, copy path on select (SPC h .)	spacemacs — normal: <code><space>h.</code> , select: <code><space>h.</code>
<code>apropos_local_value</code>	Search buffer-local variables by value (emacs apropos-local-value, SPC h v)	spacemacs — normal: <code><space>hv</code> , select: <code><space>hv</code>
<code>clone_indirect_buffer</code>	Clone the current buffer into a shared-document split (SPC b N i)	spacemacs — normal: <code><[-x>4c</code> , <code><space>bNI</code> , <code><space>bNi</code> , select: <code><[-x>4c</code> , <code><space>bNI</code> , <code><space>bNi</code> , insert: <code><[-x>4c</code>
<code>clone_indirect_from_buffer</code>	Open an existing buffer in a shared-document split (SPC b N C-i)	spacemacs — normal: <code><space>bN<[-i></code> , select: <code><space>bN<[-i></code>
<code>open_junk_file</code>	Open a fresh timestamped junk file (SPC f J)	spacemacs — normal: <code><space>fJ</code> , select: <code><space>fJ</code>
<code>open_hex</code>	Open the current file in the hex editor (SPC f h, hexl)	spacemacs — normal: <code><space>fh</code> , select: <code><space>fh</code>
<code>open_file_external</code>	Open the current file with the OS default program (SPC f o)	spacemacs — normal: <code><space>fo</code> , select: <code><space>fo</code>
<code>git_init</code>	Initialize a new git repository (SPC g i)	spacemacs — normal: <code><space>gi</code> , select: <code><space>gi</code>
<code>view_file_at_rev</code>	View the current file at a branch/commit (SPC g f f)	spacemacs — normal: <code><[-x>v~</code> , <code><space>gff</code> , select: <code><[-x>v~</code> , <code><space>gff</code> , insert: <code><[-x>v~</code>
<code>extend_line</code>	Select current line, if already selected, extend to another line based on the anchor	spacemacs — normal: <code><space>kV</code> , select: <code><space>kV</code>
<code>extend_line_below</code>	Select current line, if already selected, extend to next line	helix — normal: <code>x</code> , select: <code>x</code>
<code>extend_line_above</code>	Select current line, if already selected, extend to previous line	
<code>select_line_above</code>	Select current line, if already selected, extend or shrink line above based on the anchor	helix — normal: <code>X</code> , select: <code>X</code>
<code>select_line_below</code>	Select current line, if already selected, extend or shrink line below based on the anchor	
<code>extend_to_line_bounds</code>	Extend selection to line bounds	helix — normal: <code>X</code> , select: <code>X</code>
<code>extend_chars_right_vim</code>	Extend count graphemes right, line-bounded (dl/cl/yl)	
<code>extend_chars_left_vim</code>	Extend count graphemes left, line-bounded (dh/ch/yh)	helix — normal: <code>X</code> , select: <code>X</code>
<code>extend_line_below_linewise</code>	Extend whole lines down for a linewise operator (dj/cj/yj)	
<code>extend_line_above_linewise</code>	Extend whole lines up for a linewise operator (dk/ck/yk)	helix — normal: <code>X</code> , select: <code>X</code>
<code>extend_next_paragraph</code>	Extend to next paragraph for an operator (d/c/y)	
<code>extend_prev_paragraph</code>	Extend to previous paragraph for an operator (d{/c{/y/)	helix — normal: <code>X</code> , select: <code>X</code>
<code>select_paragraph_forward_vim</code>	vim }: paragraph operator target with linewise promotion	
<code>select_paragraph_backward_vim</code>	vim {: paragraph operator target with linewise promotion	helix — normal: <code>X</code> , select: <code>X</code>
<code>select_paragraph_forward_vim_linewise</code>	vim V): force-linewise paragraph operator target	

Name	Description	Keybinds
<code>select_paragraph_backward_vim_linewise</code>	vim V{; force-linewise paragraph operator target	
<code>shrink_to_line_bounds</code>	Shrink selection to line bounds	helix — normal: <code><A-x></code> , select: <code><A-x></code>
<code>delete_selection</code>	Delete selection	helix — normal: <code>d</code> , select: <code>d</code>
<code>delete_selection_linewise</code>	Delete selection (vim linewise, EOF-aware)	
<code>delete_selection_noyank</code>	Delete selection without yanking	helix — normal: <code><A-d></code> , select: <code><A-d></code>
<code>change_selection</code>	Change selection	helix — normal: <code>c</code> , select: <code>c</code>
<code>change_selection_noyank</code>	Change selection without yanking	helix — normal: <code><A-c></code> , select: <code><A-c></code>
<code>collapse_selection</code>	Collapse selection into single cursor	helix — normal: <code>;</code> , select: <code>;</code> emacs , cua — normal: <code><[-g></code> , <code><[-pause></code> , insert: <code><[-g></code> , <code><[-pause></code>
<code>flip_selections</code>	Flip selection cursor and anchor	spacemacs — normal: <code><[-x><[-x></code> , select: <code><[-x><[-x></code> , insert: <code><[-x><[-x></code> helix — normal: <code><A-;></code> , select: <code><A-;></code> emacs — normal: <code><[-x><[-x></code> , insert: <code><[-x><[-x></code> cua — normal: <code><[-x><[-x></code> , select: <code><[-X><[-x></code> , insert: <code><[-x><[-x></code> helix — normal: <code><A-:></code> , select: <code><A-:></code>
<code>ensure_selections_forward</code>	Ensure all selections face forward	
<code>insert_mode</code>	Insert before selection	spacemacs — normal: <code>i</code> , <code><ins></code> , <code><space>k</code> <code>i</code> , select: <code><space>k</code> vim — normal: <code>i</code> , <code><ins></code> helix — normal: <code>i</code> , select: <code>i</code> emacs , cua — normal: <code>a</code> , <code>i</code> spacemacs , vim — normal: <code>a</code> helix — normal: <code>a</code> , select: <code>a</code>
<code>append_mode</code>	Append after selection	spacemacs , vim — normal: <code>R</code>
<code>replace_mode</code>	Enter Replace mode (overtyping)	
<code>virtual_replace_mode</code>	Enter Virtual Replace mode, overtyping in screen space (vim gR)	spacemacs , vim — normal: <code>gR</code>
<code>command_mode</code>	Enter command mode	spacemacs — normal: <code><space>:</code> , <code><space>k</code> <code>:</code> , select: <code>:</code> , <code><space>:</code> , <code><space>k</code> vim — select: <code>:</code> helix — normal: <code>:</code> , select: <code>:</code>
<code>ex_mode</code>	Enter Ex mode: type : commands one after another until :visual (vim gQ)	spacemacs , vim — normal: <code>gQ</code>
<code>file_picker</code>	Open file picker	spacemacs — normal: <code><space>ff</code> , <code><space>pf</code> , <code><space>ph</code> , <code><space>pp</code> , <code><[-x><[-f></code> , <code><[-x><[-r></code> , select: <code><space>ff</code> , <code><space>pf</code> , <code><space>ph</code> , <code><space>pp</code> , <code><[-x><[-f></code> , <code><[-x><[-r></code> , insert: <code><[-x><[-f></code> , <code><[-x><[-r></code> helix — normal: <code><space>f</code> , select: <code><space>f</code> emacs — normal: <code><[-x><[-f></code> , insert: <code><[-x><[-f></code> cua — normal: <code><[-x><[-f></code> , select: <code><[-X><[-f></code> , insert: <code><[-x><[-f></code>
<code>bury_buffer</code>	Stop showing the current buffer without killing it (emacs bury-buffer)	spacemacs — normal: <code><space>b</code> . <code><[-d></code> , select: <code><space>b</code> . <code><[-d></code>
<code>toggle_column_indexing</code>	Toggle 0/1-based column indexing in the statusline (Spacemacs SPC t z)	spacemacs — normal: <code><space>tz</code> , select: <code><space>tz</code>
<code>next_file_in_dir</code>	Open the next file in the current file's directory (Spacemacs] f)	

Name	Description	Keybinds
<code>prev_file_in_dir</code>	Open the previous file in the current file's directory (Spacemacs [f)	
<code>file_picker_in_current_buffer_directory</code>	Open file picker at current buffer's directory	
<code>file_picker_in_current_directory</code>	Open file picker at current working directory	spacemacs — normal: <code><[-x><[-d></code> , select: <code><[-x><[-d></code> , insert: <code><[-x><[-d></code> helix — normal: <code><space>F</code> , select: <code><space>F</code>
<code>file_explorer</code>	Open file explorer in workspace root	spacemacs — normal: <code><space>ad</code> , <code><space>af</code> , <code><space>ft</code> , <code><space>pd</code> , <code><space>pt</code> , <code><space>atrd</code> , <code><space>atrr</code> , select: <code><space>ad</code> , <code><space>af</code> , <code><space>ft</code> , <code><space>pd</code> , <code><space>pt</code> , <code><space>atrd</code> , <code><space>atrr</code> helix — normal: <code><space>e</code> , select: <code><space>e</code>
<code>file_explorer_in_current_buffer_directory</code>	Open file explorer at current buffer's directory	spacemacs — normal: <code><space>fd</code> , <code><space>fj</code> , <code><space>jD</code> , <code><space>jd</code> , select: <code><space>fd</code> , <code><space>fj</code> , <code><space>jD</code> , <code><space>jd</code> helix — normal: <code><space>.</code> , select: <code><space>.</code>
<code>file_explorer_in_current_directory</code>	Open file explorer at current working directory	
<code>buffer_menu</code>	Open the Buffer Menu (emacs buffer-menu / C-x C-b)	spacemacs — normal: <code><space>bM</code> , select: <code><space>bM</code>
<code>list_buffers</code>	List open buffers in the Buffer Menu (emacs list-buffers)	spacemacs — normal: <code><[-x><[-b></code> , select: <code><[-x><[-b></code> , insert: <code><[-x><[-b></code> emacs — normal: <code><[-x><[-b></code> , insert: <code><[-x><[-b></code> cua — normal: <code><[-x><[-b></code> , select: <code><[-X><[-b></code> , insert: <code><[-x><[-b></code>
<code>calendar</code>	Open the Calendar month grid (emacs calendar)	
<code>diary</code>	Show today's diary entries (emacs diary)	
<code>diary_view_entries</code>	Show diary entries for the current date (emacs diary-view-entries)	
<code>diary_show_all_entries</code>	Open the diary file (emacs diary-show-all-entries)	
<code>diary_insert_entry</code>	Add a diary entry for today (emacs diary-insert-entry)	
<code>diary_insert_weekly_entry</code>	Add a weekly diary entry for today (emacs diary-insert-weekly-entry)	
<code>diary_mark_entries</code>	Mark calendar dates that have diary entries (emacs diary-mark-entries)	
<code>diary_list_entries</code>	List diary entries for the current date (emacs diary-list-entries)	
<code>diary_mail_entries</code>	Mail the next week's diary entries to yourself (emacs diary-mail-entries)	
<code>diary_fancy_display</code>	Show the day's diary entries in fancy format (emacs diary-fancy-display)	
<code>diary_simple_display</code>	Show the day's diary entries in simple format (emacs diary-simple-display)	
<code>diary_sort_entries</code>	Sort the day's diary entries by time (emacs diary-sort-entries)	
<code>diary_include_other_diary_files</code>	Include entries from #include'd diary files (emacs diary-include-other-diary-files)	

Name	Description	Keybinds
<code>diary_mark_included_diary_files</code>	Mark dates from #include'd diary files (emacs diary-mark-included-diary-files)	
<code>icalendar_export_file</code>	Export the diary file to ~/diary.ics (emacs icalendar-export-file)	
<code>icalendar_export_region</code>	Export the selected diary region to ~/diary.ics (emacs icalendar-export-region)	
<code>icalendar_import_file</code>	Import ~/diary.ics into the diary file (emacs icalendar-import-file)	
<code>icalendar_import_buffer</code>	Import the current buffer's iCalendar into the diary (emacs icalendar-import-buffer)	
<code>diary_print_entries</code>	Print the day's diary entries (emacs diary-print-entries)	
<code>diary_day_of_year</code>	Report today's day-of-year and days remaining (emacs diary-day-of-year)	
<code>diary_hebrew_date</code>	Today's Hebrew calendar date (emacs diary-hebrew-date)	
<code>diary_islamic_date</code>	Today's Islamic calendar date (emacs diary-islamic-date)	
<code>diary_french_date</code>	Today's French Revolutionary date (emacs diary-french-date)	
<code>diary_bahai_date</code>	Today's Baha'i calendar date (emacs diary-bahai-date)	
<code>diary_coptic_date</code>	Today's Coptic calendar date (emacs diary-coptic-date)	
<code>diary_ethiopic_date</code>	Today's Ethiopic calendar date (emacs diary-ethiopic-date)	
<code>diary_astro_day_number</code>	Today's astronomical (Julian) day number (emacs diary-astro-day-number)	
<code>diary_hebrew_omer</code>	Report today's Omer count, if any (emacs diary-hebrew-omer)	
<code>diary_hebrew_rosh_hodesh</code>	Report if today is Rosh Hodesh (emacs diary-hebrew-rosh-hodesh)	
<code>diary_hebrew_birthday</code>	Report any Hebrew birthday falling today (emacs diary-hebrew-birthday)	
<code>diary_hebrew_yahrzeit</code>	Report any yahrzeit falling today (emacs diary-hebrew-yahrzeit)	
<code>diary_insert_monthly_entry</code>	Add a monthly diary entry for today (emacs diary-insert-monthly-entry)	
<code>diary_insert_yearly_entry</code>	Add a yearly diary entry for today (emacs diary-insert-yearly-entry)	
<code>diary_insert_anniversary_entry</code>	Add a diary-anniversary entry for today (emacs diary-insert-anniversary-entry)	
<code>diary_insert_block_entry</code>	Add a diary-block entry for today (emacs diary-insert-block-entry)	
<code>diary_insert_cyclic_entry</code>	Add a diary-cyclic entry for today (emacs diary-insert-cyclic-entry)	
<code>diary_hebrew_insert_entry</code>	Add a Hebrew-date diary entry for today (emacs diary-hebrew-insert-entry)	
<code>diary_hebrew_insert_monthly_entry</code>	Add a monthly Hebrew diary entry (emacs diary-hebrew-insert-monthly-entry)	
<code>diary_hebrew_insert_yearly_entry</code>	Add a yearly Hebrew diary entry (emacs diary-hebrew-insert-yearly-entry)	

Name	Description	Keybinds
<code>diary_hebrew_insert_anniversary_entry</code>	Add a Hebrew anniversary diary entry (emacs diary-hebrew-insert-anniversary-entry)	
<code>diary_islamic_insert_entry</code>	Add an Islamic-date diary entry for today (emacs diary-islamic-insert-entry)	
<code>diary_islamic_insert_monthly_entry</code>	Add a monthly Islamic diary entry (emacs diary-islamic-insert-monthly-entry)	
<code>diary_islamic_insert_yearly_entry</code>	Add a yearly Islamic diary entry (emacs diary-islamic-insert-yearly-entry)	
<code>diary_islamic_insert_anniversary_entry</code>	Add an Islamic anniversary diary entry (emacs diary-islamic-insert-anniversary-entry)	
<code>diary_bahai_insert_entry</code>	Add a Baha'i-date diary entry for today (emacs diary-bahai-insert-entry)	
<code>diary_bahai_insert_monthly_entry</code>	Add a monthly Baha'i diary entry (emacs diary-bahai-insert-monthly-entry)	
<code>diary_bahai_insert_yearly_entry</code>	Add a yearly Baha'i diary entry (emacs diary-bahai-insert-yearly-entry)	
<code>diary_bahai_insert_anniversary_entry</code>	Add a Baha'i anniversary diary entry (emacs diary-bahai-insert-anniversary-entry)	
<code>diary_chinese_date</code>	Today's Chinese calendar date (emacs diary-chinese-date)	
<code>diary_chinese_insert_entry</code>	Add a Chinese-date diary entry for today (emacs diary-chinese-insert-entry)	
<code>diary_chinese_insert_monthly_entry</code>	Add a monthly Chinese diary entry (emacs diary-chinese-insert-monthly-entry)	
<code>diary_chinese_insert_yearly_entry</code>	Add a yearly Chinese diary entry (emacs diary-chinese-insert-yearly-entry)	
<code>diary_chinese_insert_anniversary_entry</code>	Add a Chinese anniversary diary entry (emacs diary-chinese-insert-anniversary-entry)	
<code>appt_add</code>	Add an appointment reminder (emacs appt-add)	
<code>appt_delete</code>	Delete appointment reminders (emacs appt-delete)	
<code>appt_activate</code>	Toggle appointment checking (emacs appt-activate)	
<code>calendar_print_other_dates</code>	Report today's date in all other calendars (emacs calendar-print-other-dates)	
<code>calendar_julian_print_date</code>	Today's Julian (Roman) calendar date (emacs calendar-julian-print-date)	
<code>calendar_iso_print_date</code>	Today's ISO 8601 week date (emacs calendar-iso-print-date)	
<code>calendar_hebrew_print_date</code>	Today's Hebrew calendar date (emacs calendar-hebrew-print-date)	
<code>calendar_islamic_print_date</code>	Today's Islamic calendar date (emacs calendar-islamic-print-date)	
<code>calendar_persian_print_date</code>	Today's Persian calendar date (emacs calendar-persian-print-date)	
<code>calendar_coptic_print_date</code>	Today's Coptic calendar date (emacs calendar-coptic-print-date)	

Name	Description	Keybinds
<code>calendar_ethiopic_print_date</code>	Today's Ethiopic calendar date (emacs calendar-ethiopic-print-date)	
<code>calendar_french_print_date</code>	Today's French Revolutionary date (emacs calendar-french-print-date)	
<code>calendar_bahai_print_date</code>	Today's Baha'i date, approx (emacs calendar-bahai-print-date)	
<code>calendar_chinese_print_date</code>	Today's Chinese calendar date (emacs calendar-chinese-print-date)	
<code>calendar_astro_print_day_</code> <code>number</code>	Astronomical (Julian) day number (emacs calendar-astro-print-day-number)	
<code>calendar_mayan_print_date</code>	Today's Mayan date (emacs calendar-mayan-print-date)	
<code>calendar_day_of_year</code>	Day-of-year of today (emacs calendar-day-of-year)	
<code>calendar_goto_day_of_year</code>	Echo the date for a day-of-year (emacs calendar-goto-day-of-year)	
<code>calendar_count_days_region</code>	Count days between two dates (emacs calendar-count-days-region)	
<code>calendar_list_holidays</code>	List this year's holidays (emacs calendar-list-holidays)	
<code>holidays</code>	List this year's holidays (emacs holidays)	
<code>holiday_list</code>	List this year's holidays (emacs holiday-list)	
<code>list_holidays</code>	List this year's holidays (emacs list-holidays)	
<code>calendar_lunar_phases</code>	This month's moon phases, approx (emacs calendar-lunar-phases)	
<code>calendar_sunrise_sunset</code>	Sunrise/sunset today, approx (emacs calendar-sunrise-sunset)	
<code>calendar_other_month</code>	Open the Calendar at another month (emacs calendar-other-month)	
<code>calendar_set_date_style</code>	Cycle american/european/iso date style (emacs calendar-set-date-style)	
<code>calendar_hebrew_goto_date</code>	Echo Gregorian for a Hebrew date (emacs calendar-hebrew-goto-date)	
<code>calendar_islamic_goto_date</code>	Echo Gregorian for an Islamic date (emacs calendar-islamic-goto-date)	
<code>calendar_julian_goto_date</code>	Echo Gregorian for a Julian date (emacs calendar-julian-goto-date)	
<code>calendar_iso_goto_week</code>	Echo Gregorian for an ISO week date (emacs calendar-iso-goto-week)	
<code>calendar_persian_goto_date</code>	Echo Gregorian for a Persian date (emacs calendar-persian-goto-date)	
<code>calendar_coptic_goto_date</code>	Echo Gregorian for a Coptic date (emacs calendar-coptic-goto-date)	
<code>calendar_ethiopic_goto_date</code>	Echo Gregorian for an Ethiopic date (emacs calendar-ethiopic-goto-date)	
<code>calendar_french_goto_date</code>	Echo Gregorian for a French Revolutionary date (emacs calendar-french-goto-date)	
<code>calendar_bahai_goto_date</code>	Echo Gregorian for a Baha'i date (emacs calendar-bahai-goto-date)	
<code>calendar_chinese_goto_date</code>	Echo Gregorian for a Chinese date (emacs calendar-chinese-goto-date)	
<code>calendar_astro_goto_day_number</code>	Echo Gregorian for an astro day number (emacs calendar-astro-goto-day-number)	

Name	Description	Keybinds
<code>calendar_mayan_goto_long_count</code>	Echo Gregorian for a Mayan long count (emacs calendar-mayan-goto-long-count)	
<code>calc_dispatch</code>	Open the RPN Calc stack calculator (emacs calc / C-x *)	spacemacs — normal: <code><space>ac</code> , select: <code><space>ac</code>
<code>occur</code>	List lines matching a regexp in an <i>Occur</i> overlay (emacs occur / M-s o)	spacemacs — normal: <code><A-s>o</code>
<code>isearch_forward_word</code>	Incremental whole-word search forward (emacs isearch-forward-word)	spacemacs — normal: <code><A-s>w</code> , <code><A-s><A-w></code>
<code>isearch_forward_symbol</code>	Incremental whole-symbol search forward (emacs isearch-forward-symbol)	spacemacs — normal: <code><A-s>_</code>
<code>isearch_forward_thing_at_point</code>	Search for the symbol/word at point (emacs isearch-forward-thing-at-point)	
<code>isearch_forward_symbol_at_point</code>	Search for the symbol at point (emacs isearch-forward-symbol-at-point)	spacemacs — normal: <code><A-s>.</code> , <code><A-s><A-.></code>
<code>isearch_toggle_regexp</code>	Toggle regexp matching for the current search (emacs isearch-toggle-regexp)	
<code>isearch_toggle_word</code>	Toggle whole-word matching for the current search (emacs isearch-toggle-word)	
<code>isearch_toggle_symbol</code>	Toggle whole-symbol matching for the current search (emacs isearch-toggle-symbol)	
<code>isearch_toggle_case_fold</code>	Toggle case-folding for the current search (emacs isearch-toggle-case-fold)	
<code>isearch_toggle_lax_whitespace</code>	Toggle lax-whitespace matching for the current search (emacs isearch-toggle-lax-whitespace)	
<code>isearch_toggle_char_fold</code>	Toggle character folding — resume matches résumé (emacs isearch-toggle-char-fold)	
<code>isearch_toggle_invisible</code>	Toggle matching inside folded (invisible) text (emacs isearch-toggle-invisible)	
<code>isearch_yank_char</code>	Extend the search with the next buffer char (emacs isearch-yank-char)	
<code>isearch_yank_word_or_char</code>	Extend the search with the next word or char (emacs isearch-yank-word-or-char)	
<code>isearch_yank_symbol_or_char</code>	Extend the search with the next symbol or char (emacs isearch-yank-symbol-or-char)	
<code>isearch_yank_word</code>	Extend the search with the next word (emacs isearch-yank-word)	
<code>isearch_yank_symbol</code>	Extend the search with the next symbol (emacs isearch-yank-symbol)	
<code>isearch_yank_line</code>	Extend the search to end of line (emacs isearch-yank-line)	
<code>isearch_yank_until_char</code>	Extend the search up to a given char (emacs isearch-yank-until-char)	
<code>isearch_yank_kill</code>	Extend the search with the kill-ring top (emacs isearch-yank-kill)	
<code>isearch_yank_pop</code>	Extend the search with a kill-ring entry (emacs isearch-yank-pop)	
<code>isearch_yank_x_selection</code>	Extend the search with the clipboard selection (emacs isearch-yank-x-selection)	

Name	Description	Keybinds
<code>isearch_del_char</code>	Shorten the search string by one char (emacs isearch-del-char)	
<code>isearch_delete_char</code>	Shorten the search string by one char (emacs isearch-delete-char)	
<code>isearch_edit_string</code>	Edit the search string in a prompt (emacs isearch-edit-string)	
<code>isearch_ring_advance</code>	Cycle to an older search-ring entry (emacs isearch-ring-advance)	
<code>isearch_ring_retreat</code>	Cycle to a newer search-ring entry (emacs isearch-ring-retreat)	
<code>isearch_exit</code>	End the current incremental search (emacs isearch-exit)	
<code>isearch_abort</code>	Abort the search, return to origin (emacs isearch-abort)	
<code>isearch_cancel</code>	Cancel the search, return to origin (emacs isearch-cancel)	
<code>isearch_quote_char</code>	Add a literal char to the search string (emacs isearch-quote-char)	
<code>isearch_complete</code>	Complete the search string from history (emacs isearch-complete)	
<code>isearch_char_by_name</code>	Add a char by digraph mnemonic to the search (emacs isearch-char-by-name)	
<code>isearch_emoji_by_name</code>	Add a char by digraph mnemonic to the search (emacs isearch-emoji-by-name)	
<code>isearch_occur</code>	Run occur with the current search pattern (emacs isearch-occur)	
<code>query_replace</code>	Replace matches one by one, asking (emacs query-replace M-%)	emacs, cua — insert: <code><A-%></code>
<code>query_replace_regexp</code>	Replace regexp matches one by one, asking (emacs query-replace-regexp C-M-%)	emacs, cua — insert: <code><A-[-%></code>
<code>tags_query_replace</code>	Query-replace a regexp across every file in the tags table (emacs tags-query-replace)	
<code>isearch_query_replace</code>	Query-replace the current search pattern (emacs isearch-query-replace)	
<code>isearch_query_replace_regexp</code>	Query-replace the current search regexp (emacs isearch-query-replace-regexp)	
<code>isearch_highlight_regexp</code>	Highlight matches of the current search (emacs isearch-highlight-regexp)	
<code>isearch_highlight_lines_matching_regexp</code>	List/highlight lines matching the search (emacs isearch-highlight-lines-matching-regexp)	
<code>rmail</code>	Open the Rmail mail reader on ~/RMAIL (emacs rmail)	spacemacs — normal: <code><space>ar</code> , select: <code><space>ar</code>
<code>rmail_continue</code>	Rmail: return to the message being composed (emacs rmail-continue)	
<code>rmail_resend</code>	Rmail: resend (bounce) the current message (emacs rmail-resend)	
<code>rmail_retry_failure</code>	Rmail: re-compose the message a bounce returned (emacs rmail-retry-failure)	
<code>rmail_mime</code>	Rmail: toggle the decoded MIME display (emacs rmail-mime)	
<code>rmail_mime_next_item</code>	Rmail: move to the next MIME entity (emacs rmail-mime-next-item)	
<code>rmail_mime_previous_item</code>	Rmail: move to the previous MIME entity (emacs rmail-mime-previous-item)	

Name	Description	Keybinds
<code>rmail_mime_toggle_hidden</code>	Rmail: show/collapse the MIME entity at point (emacs <code>rmail-mime-toggle-hidden</code>)	
<code>rmail_epa_decrypt</code>	Rmail: decrypt the message's OpenPGP armor with gpg (emacs <code>rmail-epa-decrypt</code>)	
<code>rmail_redecode_body</code>	Rmail: re-decode the message body with another coding system (emacs <code>rmail-redecode-body</code>)	
<code>undigestify_rmail_message</code>	Rmail: split the current digest into its messages (emacs <code>undigestify-rmail-message</code>)	
<code>unforward_rmail_message</code>	Rmail: extract the message a forward carries (emacs <code>unforward-rmail-message</code>)	
<code>dired</code>	Open the Dired directory editor (emacs <code>C-x d</code>)	spacemacs — normal: <code><[-x>d</code> , <code><space>pD</code> , select: <code><[-x>d</code> , <code><space>pD</code> , insert: <code><[-x>d</code> emacs — insert: <code><[-x>d</code> cua — select: <code><[-X>d</code> , insert: <code><[-x>d</code>
<code>dired_jump</code>	Open Dired on the current buffer's directory (emacs <code>C-x C-j</code>)	spacemacs — normal: <code><[-x><[-j></code> , select: <code><[-x><[-j></code> , insert: <code><[-x><[-j></code> emacs — insert: <code><[-x><[-j></code> cua — select: <code><[-X><[-j></code> , insert: <code><[-x><[-j></code>
<code>dired_other_window</code>	Open Dired (overlay; emacs <code>dired-other-window C-x 4 d</code>)	spacemacs — normal: <code><[-x>4d</code> , select: <code><[-x>4d</code> , insert: <code><[-x>4d</code>
<code>dired_jump_other_window</code>	Open Dired on the buffer's dir (overlay; emacs <code>dired-jump-other-window</code>)	spacemacs — normal: <code><[-x>4<[-j></code> , select: <code><[-x>4<[-j></code> , insert: <code><[-x>4<[-j></code>
<code>dired_at_point</code>	Open Dired on the file name at point (emacs <code>dired-at-point</code>)	
<code>tex_insert_braces</code>	TeX: insert a {} brace pair (emacs <code>tex-insert-braces</code>)	
<code>tex_insert_quote</code>	TeX: insert “ or ’ smart quotes (emacs <code>tex-insert-quote</code>)	
<code>tex_terminate_paragraph</code>	TeX: end the paragraph (emacs <code>tex-terminate-paragraph</code>)	
<code>latex_insert_block</code>	LaTeX: insert a \begin{...}\end{...} block (emacs <code>latex-insert-block</code>)	
<code>latex_close_block</code>	LaTeX: close the innermost open environment (emacs <code>latex-close-block</code>)	
<code>tex_validate</code>	TeX: check {}/\$/begin-end balance (emacs <code>tex-validate-region</code>)	
<code>tex_mode</code>	TeX: enter TeX editing mode (emacs <code>tex-mode</code>)	
<code>latex_mode</code>	LaTeX: enter LaTeX editing mode (emacs <code>latex-mode</code>)	
<code>latex_electric_env_pair_mode</code>	LaTeX: toggle electric \begin/\end pairing (emacs <code>latex-electric-env-pair-mode</code>)	
<code>tex_file</code>	TeX: run LaTeX on the current file (emacs <code>tex-file</code>)	
<code>tex_buffer</code>	TeX: compile the current buffer (emacs <code>tex-buffer</code>)	
<code>tex_region</code>	TeX: compile the current file (emacs <code>tex-region</code>)	
<code>tex_compile</code>	TeX: run LaTeX on the current file (emacs <code>tex-compile</code>)	

Name	Description	Keybinds
<code>tex_bibtex_file</code>	TeX: run BibTeX on the current file (emacs tex-bibtex-file)	
<code>tex_view</code>	TeX: open the compiled PDF (emacs tex-view)	
<code>tex_print</code>	TeX: print the compiled PDF via lpr (emacs tex-print)	
<code>tex_recenter_output_buffer</code>	TeX: recenter the TeX output (emacs tex-recenter-output-buffer)	
<code>tex_kill_job</code>	TeX: kill the running TeX job (emacs tex-kill-job)	
<code>sgml_tag</code>	SGML: wrap region/point in a .. (emacs sgml-tag)	
<code>sgml_close_tag</code>	SGML: close the innermost open element (emacs sgml-close-tag)	
<code>sgml_delete_tag</code>	SGML: delete the enclosing tag pair, keeping content (emacs sgml-delete-tag)	
<code>sgml_skip_tag_forward</code>	SGML: move past a balanced tag group (emacs sgml-skip-tag-forward)	
<code>sgml_skip_tag_backward</code>	SGML: move back over a balanced tag group (emacs sgml-skip-tag-backward)	
<code>sgml_name_char</code>	SGML: insert a &entity; for a character (emacs sgml-name-char)	
<code>sgml_tag_help</code>	SGML: describe an HTML element (emacs sgml-tag-help)	
<code>sgml_attributes</code>	SGML: insert attributes at point (emacs sgml-attributes)	
<code>sgml_name_8bit_mode</code>	SGML: toggle 8-bit entity name display (emacs sgml-name-8bit-mode)	
<code>sgml_validate</code>	SGML: validate the file with onsgmls/nsngmls (emacs sgml-validate)	
<code>sgml_mode</code>	SGML: enter SGML editing mode (emacs sgml-mode)	
<code>html_mode</code>	HTML: enter HTML editing mode (emacs html-mode)	
<code>htmlfontify_buffer</code>	HTML: export the buffer as highlighted HTML (emacs htmlfontify-buffer)	
<code>nroff_forward_text_line</code>	nroff: forward one text line, skip requests (emacs nroff-forward-text-line)	
<code>nroff_backward_text_line</code>	nroff: backward one text line, skip requests (emacs nroff-backward-text-line)	
<code>nroff_count_text_lines</code>	nroff: count text lines in region (emacs nroff-count-text-lines)	
<code>code_action</code>	Perform code action	spacemacs — normal: <code><space>la</code> , select: <code><space>lahelix</code> — normal: <code><space>a</code> , select: <code><space>a</code>
<code>extract_refactor</code>	Extract refactoring (method/variable/constant) via LSP (IntelliJ Extract)	
<code>extract_function</code>	Extract Method/Function via LSP (IntelliJ Extract Method)	
<code>extract_variable</code>	Introduce Variable via LSP (IntelliJ Introduce Variable)	
<code>extract_constant</code>	Extract Constant via LSP (IntelliJ Extract Constant)	
<code>extract_field</code>	Introduce Field via LSP (IntelliJ Introduce Field)	
<code>extract_parameter</code>	Introduce Parameter via LSP (IntelliJ Introduce Parameter)	

Name	Description	Keybinds
<code>inline_refactor</code>	Inline refactoring (variable/method) via LSP (IntelliJ Inline)	
<code>rewrite_refactor</code>	Rewrite refactoring (change signature etc.) via LSP	
<code>refactor_this</code>	Show all applicable refactorings (IntelliJ Refactor This)	
<code>organize_imports</code>	Organize/optimize imports via LSP source action (IntelliJ Ctrl-Alt-O)	spacemacs — normal: <code><space>l0</code> , select: <code><space>l0</code>
<code>implement_methods</code>	Implement missing interface/trait members via LSP (IntelliJ Ctrl-I)	spacemacs — normal: <code><space>li</code> , select: <code><space>li</code>
<code>override_methods</code>	Override inherited members via LSP (IntelliJ Ctrl-O)	spacemacs — normal: <code><space>lv</code> , select: <code><space>lv</code>
<code>generate_code</code>	Generate code (getters/constructors/impls) via LSP (SPC I g)	spacemacs — normal: <code><space>lg</code> , select: <code><space>lg</code>
<code>change_signature</code>	Change signature refactor via LSP	
<code>pull_members_up</code>	Pull members up refactor via LSP (IntelliJ)	
<code>push_members_down</code>	Push members down refactor via LSP (IntelliJ)	
<code>safe_delete</code>	Safe Delete: delete the symbol under the cursor only if unused, else show its usages (JetBrains Safe Delete)	
<code>buffer_picker</code>	Open buffer picker	spacemacs — normal: <code><C-x>b</code> , <code><space>bb</code> , <code><space>lb</code> , <code><space>lt</code> , <code><space>pb</code> , <code><space>b.b</code> , select: <code><C-x>b</code> , <code><space>bb</code> , <code><space>lb</code> , <code><space>lt</code> , <code><space>pb</code> , <code><space>b.b</code> , insert: <code><C-x>bhelix</code> — normal: <code><space>b</code> , select: <code><space>bemacs</code> — normal: <code><C-x>b</code> , insert: <code><C-x>bcua</code> — normal: <code><C-x>b</code> , select: <code><C-X>b</code> , insert: <code><C-x>b</code>
<code>jumplist_picker</code>	Open jumplist picker	spacemacs — normal: <code><space>jj</code> , select: <code><space>jjhelix</code> — normal: <code><space>j</code> , select: <code><space>j</code>
<code>register_picker</code>	Browse registers and paste the chosen one	spacemacs — normal: <code><space>re</code> , <code><space>rr</code> , <code><space>ry</code> , select: <code><space>re</code> , <code><space>rr</code> , <code><space>ry</code>
<code>marks_picker</code>	Fuzzy-pick a vim mark and jump to it (:Marks)	spacemacs — normal: <code><space>fb</code> , <code><space>rm</code> , select: <code><space>fb</code> , <code><space>rm</code>
<code>buffer_line_picker</code>	Fuzzy-search lines in the current buffer (:BLines)	spacemacs — normal: <code><space>sl</code> , select: <code><space>sl</code>
<code>command_history_picker</code>	Fuzzy-pick and run a past command line (:History)	spacemacs — normal: <code><space>r:</code> , <code><C-x><esc><esc></code> , select: <code><space>r:</code> , <code><C-x><esc><esc></code> , insert: <code><C-x><esc><esc></code>
<code>cmdline_window</code>	Edit the command-line history in a buffer; runs the line (vim q:)	
<code>cmdline_window_execute</code>	Run the command-line under the cursor and close the window (vim q:)	
<code>search_history_picker</code>	Fuzzy-pick and re-run a past search (:History/)	spacemacs — normal: <code><space>r/</code> , select: <code><space>r/</code>
<code>unicode_picker</code>	Fuzzy-pick a character/digraph and insert it (helm-unicode)	spacemacs — normal: <code><C-x>8e</code> , <code><space>iu</code> , <code><C-x>8<ret></code> , select: <code><C-x>8e</code> , <code><space>iu</code> , <code><C-x>8<ret></code> , insert: <code><C-x>8e</code> , <code><C-x>8<ret></code>

Name	Description	Keybinds
<code>git_file_log_picker</code>	Commit log for the current file (:BCommits)	spacemacs — normal: <code><space>gt</code> , <code><space>gfl</code> , select: <code><space>gt</code> , <code><space>gfl</code>
<code>git_repo_log_picker</code>	Commit log for the whole repo (:Commits)	spacemacs — normal: <code><space>gl</code> , select: <code><space>gl</code>
<code>theme_picker</code>	Open fuzzy theme picker with live preview	spacemacs — normal: <code><space>Tc</code> , select: <code><space>Tc</code>
<code>wrap_sexp</code>	Wrap the selection in parentheses	spacemacs — normal: <code><space>kw</code> , select: <code><space>kw</code>
<code>symbol_picker</code>	Open symbol picker	spacemacs — normal: <code>g0</code> , <code><space>ji</code> , <code><space>pg</code> , <code><space>sj</code> , select: <code><space>ji</code> , <code><space>pg</code> , <code><space>sjvim</code> — normal: <code>g0</code>
<code>syntax_symbol_picker</code>	Open symbol picker from syntax information	
<code>lsp_or_syntax_symbol_picker</code>	Open symbol picker from LSP or syntax information	helix — normal: <code><space>s</code> , select: <code><space>s</code>
<code>changed_file_picker</code>	Open changed file picker	spacemacs — normal: <code><space>bm</code> , select: <code><space>bmhelix</code> — normal: <code><space>g</code> , select: <code><space>g</code>
<code>frecent_file_picker</code>	Open recent files ranked by frequency (z algorithm)	spacemacs — normal: <code><space>fr</code> , select: <code><space>fr</code>
<code>reopen_last_closed</code>	Reopen the most recently closed file	spacemacs — normal: <code><space>bu</code> , <code><space>fu</code> , select: <code><space>bu</code> , <code><space>fu</code>
<code>harpoon_add</code>	Pin the current file to the harpoon list	spacemacs — normal: <code><space>Ha</code> , select: <code><space>Ha</code>
<code>harpoon_jump</code>	Jump to the harpoon mark in slot [count]	spacemacs — normal: <code><space>Hj</code> , select: <code><space>Hj</code>
<code>harpoon_1</code>	Jump to harpoon mark 1	spacemacs — normal: <code><space>H1</code> , select: <code><space>H1</code>
<code>harpoon_2</code>	Jump to harpoon mark 2	spacemacs — normal: <code><space>H2</code> , select: <code><space>H2</code>
<code>harpoon_3</code>	Jump to harpoon mark 3	spacemacs — normal: <code><space>H3</code> , select: <code><space>H3</code>
<code>harpoon_4</code>	Jump to harpoon mark 4	spacemacs — normal: <code><space>H4</code> , select: <code><space>H4</code>
<code>harpoon_next</code>	Open the next harpoon mark	spacemacs — normal: <code><space>Hn</code> , select: <code><space>Hn</code>
<code>harpoon_prev</code>	Open the previous harpoon mark	spacemacs — normal: <code><space>Hp</code> , select: <code><space>Hp</code>
<code>bookmark_toggle</code>	Toggle a line bookmark (JetBrains F11)	spacemacs — normal: <code><space>rt</code> , select: <code><space>rt</code>
<code>bookmark_next</code>	Jump to the next line bookmark (JetBrains)	spacemacs — normal: <code><space>rn</code> , select: <code><space>rn</code>
<code>bookmark_prev</code>	Jump to the previous line bookmark (JetBrains)	spacemacs — normal: <code><space>rN</code> , select: <code><space>rN</code>
<code>harpoon_menu</code>	Open the harpoon marks menu	spacemacs — normal: <code><space>Hh</code> , <code><space>Hl</code> , select: <code><space>Hh</code> , <code><space>Hl</code>
<code>harpoon_remove</code>	Unpin the current file from harpoon	spacemacs — normal: <code><space>Hd</code> , select: <code><space>Hd</code>
<code>select_references_to_symbol_under_cursor</code>	Select symbol references	spacemacs — normal: <code><space>se</code> , <code><space>sh</code> , select: <code><space>se</code> , <code><space>shhelix</code> — normal: <code><space>h</code> , select: <code><space>h</code>
<code>workspace_symbol_picker</code>	Open workspace symbol picker	spacemacs — normal: <code><A-C- .></code> , <code><C-c></code> , <code>J</code> , <code><space>jI</code> , <code><space>sS</code> , select: <code><C-c></code> , <code>J</code> , <code><space>jI</code> , <code><space>sS</code> , insert: <code><C-c></code> , <code>J</code>

Name	Description	Keybinds
<code>syntax_workspace_symbol_picker</code>	Open workspace symbol picker from syntax information	helix — normal: <code><space>S</code> , select: <code><space>S</code> spacemacs — normal: <code><space>e1</code> , <code><space>en1</code> , <code><space>epl</code> , select: <code><space>e1</code> , <code><space>en1</code> , <code><space>epl</code> helix — normal: <code><space>d</code> , select: <code><space>d</code> spacemacs — normal: <code><space>e1</code> , select: <code><space>e1</code> helix — normal: <code><space>D</code> , select: <code><space>D</code> spacemacs — normal: <code><space>'</code> , <code><space>rl</code> , <code><space>rs</code> , <code><space>sl</code> , select: <code><space>'</code> , <code><space>rl</code> , <code><space>rs</code> , <code><space>sl</code> helix — normal: <code><space>'</code> , select: <code><space>'</code> spacemacs, vim — normal: <code>I</code> , <code>gI</code> helix — normal: <code>I</code> , select: <code>I</code> spacemacs, vim — normal: <code>A</code> helix — normal: <code>A</code> , select: <code>A</code> spacemacs, vim — normal: <code>o</code> helix — normal: <code>o</code> , select: <code>o</code> spacemacs, vim — normal: <code>O</code> helix — normal: <code>O</code> , select: <code>O</code> spacemacs — normal: <code><[-c>;</code> , select: <code><[-c>;</code> , insert: <code><[-c>;</code>
<code>lsp_or_syntax_workspace_symbol_picker</code>	Open workspace symbol picker from LSP or syntax information	
<code>diagnostics_picker</code>	Open diagnostic picker	
<code>workspace_diagnostics_picker</code>	Open workspace diagnostic picker	spacemacs — normal: <code><space>e1</code> , select: <code><space>e1</code> helix — normal: <code><space>D</code> , select: <code><space>D</code> spacemacs — normal: <code><space>'</code> , <code><space>rl</code> , <code><space>rs</code> , <code><space>sl</code> , select: <code><space>'</code> , <code><space>rl</code> , <code><space>rs</code> , <code><space>sl</code> helix — normal: <code><space>'</code> , select: <code><space>'</code> spacemacs, vim — normal: <code>I</code> , <code>gI</code> helix — normal: <code>I</code> , select: <code>I</code> spacemacs, vim — normal: <code>A</code> helix — normal: <code>A</code> , select: <code>A</code> spacemacs, vim — normal: <code>o</code> helix — normal: <code>o</code> , select: <code>o</code> spacemacs, vim — normal: <code>O</code> helix — normal: <code>O</code> , select: <code>O</code> spacemacs — normal: <code><[-c>;</code> , select: <code><[-c>;</code> , insert: <code><[-c>;</code>
<code>last_picker</code>	Open last picker	
<code>insert_at_line_start</code>	Insert at start of line	
<code>insert_at_line_end</code>	Insert at end of line	spacemacs — normal: <code><[-c>;</code> , select: <code><[-c>;</code> , insert: <code><[-c>;</code> spacemacs — normal: <code><[-c>.</code> , select: <code><[-c>.</code> , insert: <code><[-c>.</code> spacemacs — normal: <code><[-\><[-g></code> , <code><[-\><[-n></code> vim — normal: <code><[-c></code> , <code><[-\><[-g></code> , <code><[-\><[-n></code> helix — normal: <code><esc></code> , select: <code>v</code> , insert: <code><esc></code> spacemacs — normal: <code>v</code> , <code>gh</code> , <code><[-space></code> , <code><space>kv</code> , <code><space>k<[-v></code> , select: <code><space>kv</code> , <code><space>k<[-v></code> vim — normal: <code>v</code> , <code>gh</code> helix — normal: <code>vemacs</code> , <code>cua</code> — normal: <code><[-space></code> helix — select: <code><esc></code>
<code>open_below</code>	Open new line below selection	
<code>open_above</code>	Open new line above selection	
<code>complete_current_statement</code>	Complete the current statement (close brackets, add terminator, open next line) (JetBrains)	spacemacs — normal: <code><[-c></code> , select: <code><[-c></code> , insert: <code><[-c></code> spacemacs — normal: <code><[-c>.</code> , select: <code><[-c>.</code> , insert: <code><[-c>.</code> spacemacs — normal: <code><[-\><[-g></code> , <code><[-\><[-n></code> vim — normal: <code><[-c></code> , <code><[-\><[-g></code> , <code><[-\><[-n></code> helix — normal: <code><esc></code> , select: <code>v</code> , insert: <code><esc></code> spacemacs — normal: <code>v</code> , <code>gh</code> , <code><[-space></code> , <code><space>kv</code> , <code><space>k<[-v></code> , select: <code><space>kv</code> , <code><space>k<[-v></code> vim — normal: <code>v</code> , <code>gh</code> helix — normal: <code>vemacs</code> , <code>cua</code> — normal: <code><[-space></code> helix — select: <code><esc></code>
<code>postfix_expand</code>	Postfix completion: expand <code>expr.kw</code> (if/for/while/match/let/return/not/...) (JetBrains)	
<code>normal_mode</code>	Enter normal mode	
<code>select_mode</code>	Enter selection extend mode	spacemacs — normal: <code><[-\><[-g></code> , <code><[-\><[-n></code> vim — normal: <code><[-c></code> , <code><[-\><[-g></code> , <code><[-\><[-n></code> helix — normal: <code><esc></code> , select: <code>v</code> , insert: <code><esc></code> spacemacs — normal: <code>v</code> , <code>gh</code> , <code><[-space></code> , <code><space>kv</code> , <code><space>k<[-v></code> , select: <code><space>kv</code> , <code><space>k<[-v></code> vim — normal: <code>v</code> , <code>gh</code> helix — normal: <code>vemacs</code> , <code>cua</code> — normal: <code><[-space></code> helix — select: <code><esc></code>
<code>exit_select_mode</code>	Exit selection mode	

Name	Description	Keybinds
<code>goto_definition</code>	Goto definition	spacemacs — normal: <code>g</code>], <code>gd</code> , <code><A-.></code> , <code><[-w>]</code> , <code>g<[-]></code> , <code><[-c></code> , <code>j</code> , <code><[-w>g]</code> , <code><[-x>5.</code> , <code><space>gd</code> , <code><space>jf</code> , <code><space>jv</code> , <code><space>w</code>], <code><[-w><[-]></code> , <code><space>m</code> gg, <code><space>w</code> g], <code><[-w>g<[-]></code> , <code><space>w<[-]></code> , <code><space>w</code> g<[-]>, select: <code><[-]></code> , <code><[-c></code> , <code>j</code> , <code><[-x>5.</code> , <code><space>gd</code> , <code><space>jf</code> , <code><space>jv</code> , <code><space>w</code>], <code><space>m</code> gg, <code><space>w</code> g], <code><space>w<[-]></code> , <code><space>w</code> g<[-]>, insert: <code><[-c></code> , <code>j</code> , <code><[-x>5.</code> vim — normal: <code>g</code>], <code>gd</code> , <code><[-w>]</code> , <code>g<[-]></code> , <code><[-w>g]</code> , <code><[-w><[-]></code> , <code><[-w>g<[-]></code> , select: <code><[-]></code> helix — normal: <code>gd</code> , select: gdemacs , cua — normal: <code><A-.></code> , insert: <code><A-.></code>
<code>tag_jump</code>	Jump to the definition of the symbol under the cursor, recording where from on the tag stack (vim CTRL-J)	spacemacs , vim — normal: <code><[-]></code>
<code>preview_tag</code>	Show the tag under the cursor in the preview window (vim CTRL-W)	spacemacs — normal: <code><[-w>]</code> , <code><space>w</code> }, select: <code><space>w</code> vim — normal: <code><[-w>]</code>
<code>preview_tjump</code>	Show the tag under the cursor in the preview window, listing ambiguous matches (vim CTRL-W g)	spacemacs — normal: <code><[-w>g]</code> , <code><space>w</code> g}, select: <code><space>w</code> g} vim — normal: <code><[-w>g]</code>
<code>tag_pop</code>	Jump back to the position the last tag jump started from (vim CTRL-T, :pop)	spacemacs , vim — normal: <code><[-t></code>
<code>peek_definition</code>	Peek the definition in a popup without navigating (JetBrains Quick Definition)	spacemacs — normal: <code><space>l</code> q, select: <code><space>l</code> q
<code>goto_declaration</code>	Goto declaration	spacemacs — normal: <code>gD</code> , <code><[-w>i</code> , <code><space>gD</code> , <code><space>w</code> i, <code><[-w><[-i></code> , <code><space>w<[-i></code> , select: <code><space>gD</code> , <code><space>w</code> i, <code><space>w<[-i></code> vim — normal: <code>gD</code> , <code><[-w>i</code> , <code><[-w><[-i></code> helix — normal: <code>gD</code> , select: <code>gD</code>
<code>add_newline_above</code>	Add newline above	helix — normal: <code>[<space></code> , select: <code>[<space></code>
<code>add_newline_below</code>	Add newline below	helix — normal: <code>]<space></code> , select: <code>]<space></code>
<code>goto_type_definition</code>	Goto type definition	spacemacs — normal: <code>gy</code> , <code><space>g</code> y, select: <code><space>g</code> y vim — normal: <code>gy</code> helix — normal: <code>gy</code> , select: <code>gy</code>
<code>goto_implementation</code>	Goto implementation	helix — normal: <code>gi</code> , select: <code>gi</code>
<code>goto_file_start</code>	Goto line number <code><n></code> else file start	spacemacs — normal: <code>gg</code> , <code><A-lt></code> , <code><[-home></code> , insert: <code><A-lt></code> , <code><[-home></code> vim — normal: <code>gg</code> , <code><[-home></code> , insert: <code><[-home></code> helix — normal: <code>ggemacs</code> , cua — normal: <code><A-lt></code> , select: <code><A-lt></code> , insert: <code><A-lt></code>
<code>goto_file_end</code>	Goto file end	spacemacs — insert: <code><A-gt></code> , <code><[-end></code> vim — insert: <code><[-end></code>
<code>extend_to_file_start</code>	Extend to line number <code><n></code> else file start	helix — select: <code>gg</code>
<code>extend_to_file_end</code>	Extend to file end	

Name	Description	Keybinds
<code>goto_file</code>	Goto files/URLs in selections	spacemacs — normal: <code>[f,]f, gf, gx, <C-w>gf, <C-w>gf, <space>fF, <space>jU, <space>ju, <space>pF, <space>wgf, <space>wgf, select: <space>fF, <space>jU, <space>ju, <space>pF, <space>wgf, <space>wgf</code> vim — normal: <code>[f,]f, gf, gx, <C-w>gf, <C-w>gf</code> helix — normal: <code>gf, select: gf</code>
<code>goto_file_hsplit</code>	Goto files in selections (hsplit)	spacemacs — normal: <code><C-w>F, <space>wF, <C-w><C-f>, <space>w<C-f>, select: <space>wF, <space>w<C-f></code> vim — normal: <code><C-w>F, <C-w><C-f></code> helix — normal: <code><C-w>f, <space>wF, select: <C-w>f, <space>wF</code>
<code>goto_file_vsplit</code>	Goto files in selections (vsplit)	helix — normal: <code><C-w>F, <space>wF, select: <C-w>F, <space>wF</code>
<code>goto_file_readonly</code>	Visit the file at point read-only (emacs ffap-read-only, C-x C-r)	
<code>goto_file_new_tab</code>	Visit the file at point in a new tab (emacs ffap-other-tab, C-x t C-f)	
<code>goto_file_other_frame</code>	Visit the file at point in a new frame (emacs ffap-other-frame, C-x 5 f)	
<code>goto_reference</code>	Goto references	spacemacs, vim — normal: <code>gr</code> helix — normal: <code>gr, select: gr</code> emacs, cua — normal: <code><A-?>, insert: <A-?></code>
<code>call_hierarchy_incoming_calls</code>	Call hierarchy: who calls the symbol (JetBrains Ctrl-Alt-H)	spacemacs — normal: <code><space>gh, select: <space>gh</code>
<code>call_hierarchy_outgoing_calls</code>	Call hierarchy: what the symbol calls	spacemacs — normal: <code><space>gH, select: <space>gH</code>
<code>type_hierarchy_supertypes</code>	Type hierarchy: supertypes of the symbol (JetBrains Ctrl-H)	spacemacs — normal: <code><space>gT, select: <space>gT</code>
<code>type_hierarchy_subtypes</code>	Type hierarchy: subtypes of the symbol	
<code>goto_window_top</code>	Goto window top	spacemacs, vim — normal: <code>H</code> helix — normal: <code>gt, select: gt</code>
<code>what_line</code>	Report the line number of point (emacs what-line)	
<code>what_page</code>	Report the page number and line within the page (emacs what-page)	
<code>count_lines_page</code>	Report lines on the current page, before + after point (emacs count-lines-page, C-x l)	spacemacs — normal: <code><C-x>l, select: <C-x>l, insert: <C-x>l</code>
<code>what_cursor_position</code>	Report the character at point, its code, position, percentage and column (emacs what-cursor-position, C-x =)	spacemacs — normal: <code><C-x>=, select: <C-x>=, insert: <C-x>=</code>
<code>move_to_window_line_top_bottom</code>	Move point to window centre/top/bottom, cycling (emacs move-to-window-line-top-bottom, M-r)	spacemacs — normal: <code><A-r></code> emacs, cua — normal: <code><A-r>, insert: <A-r></code>
<code>goto_window_center</code>	Goto window center	spacemacs, vim — normal: <code>M</code> helix — normal: <code>gc, select: gc</code>
<code>goto_window_bottom</code>	Goto window bottom	spacemacs, vim — normal: <code>L</code> helix — normal: <code>gb, select: gb</code>

Name	Description	Keybinds
<code>goto_last_accessed_file</code>	Goto last accessed file	spacemacs — normal: <code><[-^></code> , <code><[-w>^</code> , <code>g<tab></code> , <code><[-tab></code> , <code><space>w^</code> , <code><[-w><[-^></code> , <code><[-w>g<tab></code> , <code><space><tab></code> , <code><space>w<[-^></code> , <code><space>wg<tab></code> , select: <code><space>w^</code> , <code><space><tab></code> , <code><space>w<[-^></code> , <code><space>wg<tab></code> vim — normal: <code><[-^></code> , <code><[-w>^</code> , <code>g<tab></code> , <code><[-tab></code> , <code><[-w><[-^></code> , <code><[-w>g<tab></code> helix — normal: <code>ga</code> , select: <code>ga</code>
<code>goto_last_modified_file</code>	Goto last modified file	helix — normal: <code>gm</code> , select: <code>gm</code>
<code>goto_last_modification</code>	Goto last modification	spacemacs, vim — normal: <code>g.</code> helix — normal: <code>g.</code> , select: <code>g.</code>
<code>goto_older_change</code>	vim g,: jump to an older change-list position	spacemacs, vim — normal: <code>g;</code>
<code>goto_newer_change</code>	vim g,: jump to a newer change-list position	spacemacs, vim — normal: <code>g,</code>
<code>goto_line</code>	Goto line	spacemacs — normal: <code><A-g>g</code> , <code><A-g><A-g></code> helix — normal: <code>G</code> , select: <code>G</code> emacs, cua — normal: <code><A-g>g</code> , <code><A-g><A-g></code> , insert: <code><A-g>g</code> , <code><A-g><A-g></code>
<code>goto_last_line</code>	Goto last line	spacemacs — normal: <code>G</code> , <code><A-gt></code> , <code><[-end></code> vim — normal: <code>G</code> , <code><[-end></code> helix — normal: <code>geemacs</code> , <code>cua</code> — normal: <code><A-gt></code> , select: <code><A-gt></code> , insert: <code><A-gt></code> helix — select: <code>ge</code>
<code>extend_to_last_line</code>	Extend to last line	
<code>goto_first_diag</code>	Goto first diagnostic	spacemacs — normal: <code><space>ef</code> , <code><space>enf</code> , <code><space>epf</code> , select: <code><space>ef</code> , <code><space>enf</code> , <code><space>epf</code> helix — normal: <code>[D</code> , select: <code>[D</code>
<code>copy_diagnostic</code>	Copy the diagnostic message(s) on the current line	
<code>goto_last_diag</code>	Goto last diagnostic	spacemacs — normal: <code><space>e.</code> , select: <code><space>e.</code> helix — normal: <code>]D</code> , select: <code>]D</code>
<code>goto_next_diag</code>	Goto next diagnostic	spacemacs — normal: <code>]d</code> , <code><space>enj</code> , <code><space>enn</code> , <code><space>epj</code> , <code><space>epn</code> , select: <code><space>enj</code> , <code><space>enn</code> , <code><space>epj</code> , <code><space>epn</code> vim — normal: <code>]d</code> helix — normal: <code>]d</code> , select: <code>]d</code> emacs, cua — normal: <code><A-g>n</code> , insert: <code><A-g>n</code> , <code><A-g><A-n></code>
<code>goto_prev_diag</code>	Goto previous diagnostic	spacemacs — normal: <code>[d</code> , <code><space>enk</code> , <code><space>enp</code> , <code><space>epk</code> , <code><space>epp</code> , select: <code><space>enk</code> , <code><space>enp</code> , <code><space>epk</code> , <code><space>epp</code> vim — normal: <code>[d</code> helix — normal: <code>[d</code> , select: <code>[d</code> emacs, cua — normal: <code><A-g>p</code> , insert: <code><A-g>p</code> , <code><A-g><A-p></code>
<code>goto_next_change</code>	Goto next change	spacemacs, vim — normal: <code>]ghelix — normal: <code>]g</code>, select: <code>]g</code></code>
<code>goto_prev_change</code>	Goto previous change	spacemacs, vim — normal: <code>[ghelix — normal: <code>[g</code>, select: <code>[g</code></code>

Name	Description	Keybinds
<code>goto_next_conflict</code>	Goto next merge-conflict marker	spacemacs, vim — normal: <code>]n</code>
<code>goto_prev_conflict</code>	Goto previous merge-conflict marker	spacemacs, vim — normal: <code>[n</code>
<code>conflict_take_all_ours</code>	Resolve ALL conflicts: keep our side	spacemacs — normal: <code><space>gc0</code> , select: <code><space>gc0</code>
<code>conflict_take_all_theirs</code>	Resolve ALL conflicts: keep their side	spacemacs — normal: <code><space>gcT</code> , select: <code><space>gcT</code>
<code>git_diff</code>	Open side-by-side diff vs HEAD	spacemacs — normal: <code><[-x>v=</code> , <code><space>g=</code> , <code><space>Dfv</code> , <code><space>gfd</code> , select: <code><[-x>v=</code> , <code><space>g=</code> , <code><space>Dfv</code> , <code><space>gfd</code> , insert: <code><[-x>v=</code>
<code>resolve_conflicts</code>	Resolve merge conflicts (3-way)	spacemacs — normal: <code><space>gm</code> , <code><space>gcr</code> , select: <code><space>gm</code> , <code><space>gcr</code>
<code>git_status</code>	Magit status	spacemacs — normal: <code><[-x>v!</code> , <code><[-x>vd</code> , <code><space>gs</code> , <code><space>pv</code> , select: <code><[-x>v!</code> , <code><[-x>vd</code> , <code><space>gs</code> , <code><space>pv</code> , insert: <code><[-x>v!</code> , <code><[-x>vd</code>
<code>git_push</code>	Push the current branch to its remote (SPC g P)	spacemacs — normal: <code><[-x>vP</code> , <code><space>gP</code> , select: <code><[-x>vP</code> , <code><space>gP</code> , insert: <code><[-x>vP</code>
<code>git_pull</code>	Fast-forward pull from upstream (SPC g u)	spacemacs — normal: <code><[-x>v+</code> , <code><space>gu</code> , select: <code><[-x>v+</code> , <code><space>gu</code> , insert: <code><[-x>v+</code>
<code>git_fetch</code>	Fetch all remotes (SPC g F)	spacemacs — normal: <code><space>gF</code> , select: <code><space>gF</code>
<code>git_acp</code>	Stage all, commit, and push in one shot (C-x v c)	spacemacs — normal: <code><[-x>vc</code> , select: <code><[-x>vc</code> , insert: <code><[-x>vc</code>
<code>vc_print_log</code>	VC log for the current file (emacs vc-print-log)	spacemacs — normal: <code><[-x>vL</code> , select: <code><[-x>vL</code> , insert: <code><[-x>vL</code>
<code>vc_print_root_log</code>	VC log for the whole repository (emacs vc-print-root-log)	spacemacs — normal: <code><[-x>vL</code> , select: <code><[-x>vL</code> , insert: <code><[-x>vL</code>
<code>vc_print_branch_log</code>	VC log for a named branch (emacs vc-print-branch-log)	spacemacs — normal: <code><[-x>vbL</code> , select: <code><[-x>vbL</code> , insert: <code><[-x>vbL</code>
<code>vc_root_diff</code>	Diff the whole working tree vs HEAD (emacs vc-root-diff)	spacemacs — normal: <code><[-x>vD</code> , select: <code><[-x>vD</code> , insert: <code><[-x>vD</code>
<code>vc_region_history</code>	History of the selected line range (emacs vc-region-history)	spacemacs — normal: <code><[-x>vh</code> , select: <code><[-x>vh</code> , insert: <code><[-x>vh</code>
<code>vc_log_incoming</code>	Commits incoming from upstream (emacs vc-log-incoming)	spacemacs — normal: <code><[-x>vI</code> , select: <code><[-x>vI</code> , insert: <code><[-x>vI</code>
<code>vc_log_outgoing</code>	Commits outgoing to upstream (emacs vc-log-outgoing)	spacemacs — normal: <code><[-x>vD</code> , select: <code><[-x>vD</code> , insert: <code><[-x>vD</code>
<code>vc_log_search</code>	Search the commit log by pattern (emacs vc-log-search)	
<code>vc_create_branch</code>	Create and switch to a new branch (emacs vc-create-branch)	spacemacs — normal: <code><[-x>vbc</code> , select: <code><[-x>vbc</code> , insert: <code><[-x>vbc</code>
<code>vc_switch_branch</code>	Switch to an existing branch (emacs vc-switch-branch)	spacemacs — normal: <code><[-x>vbs</code> , select: <code><[-x>vbs</code> , insert: <code><[-x>vbs</code>
<code>vc_create_tag</code>	Create a git tag at HEAD (emacs vc-create-tag)	spacemacs — normal: <code><[-x>vs</code> , select: <code><[-x>vs</code> , insert: <code><[-x>vs</code>
<code>vc_retrieve_tag</code>	Check out a tag or branch (emacs vc-retrieve-tag)	spacemacs — normal: <code><[-x>vr</code> , select: <code><[-x>vr</code> , insert: <code><[-x>vr</code>
<code>vc_rename_file</code>	Rename the current file under VC (emacs vc-rename-file)	
<code>vc_delete_file</code>	Delete the current file under VC (emacs vc-delete-file)	

Name	Description	Keybinds
<code>vc_ignore</code>	Add the current file to .gitignore (emacs vc-ignore)	spacemacs — normal: <code><C-x>vG</code> , select: <code><C-x>vG</code> , insert: <code><C-x>vG</code> spacemacs — normal: <code><C-x>vu</code> , select: <code><C-x>vu</code> , insert: <code><C-x>vu</code>
<code>vc_revert</code>	Revert the current file to HEAD (emacs vc-revert)	
<code>vc_refresh_state</code>	Recompute the buffer's VC state (emacs vc-refresh-state)	
<code>vc_state_refresh</code>	Recompute the buffer's VC state (emacs vc-state-refresh)	
<code>vc_pull</code>	Pull from upstream (emacs vc-pull)	spacemacs — normal: <code><C-x>vv</code> , select: <code><C-x>vv</code> , insert: <code><C-x>vv</code>
<code>vc_push</code>	Push to upstream (emacs vc-push)	
<code>vc_next_action</code>	Do the next logical VC step: stage + commit (emacs vc-next-action)	
<code>vc_dir</code>	Open the VC directory / Magit status (emacs vc-dir)	
<code>project_vc_dir</code>	Open the project's VC directory / Magit status (emacs project-vc-dir)	emacs, cua — normal: <code><F10></code> , insert: <code><F10></code>
<code>project_any_command</code>	Pick a command and run it with the working directory set to the project root (emacs project-any-command)	
<code>imenu_add_menubar_index</code>	Add an Index menu of this buffer's definitions (emacs imenu-add-menubar-index)	
<code>menu_bar_open</code>	Open the menu bar (emacs menu-bar-open, F10)	
<code>menu_bar_mode</code>	Toggle whether the menu bar is displayed (emacs menu-bar-mode)	emacs, cua — normal: <code><A-`></code> , insert: <code><A-`></code>
<code>tool_bar_mode</code>	Toggle the tool bar; no visible effect on a tty (emacs tool-bar-mode)	
<code>modifier_bar_mode</code>	Toggle the modifier-key tool bar; no visible effect on a tty (emacs modifier-bar-mode)	
<code>android_request_directory_access</code>	Grant access to a directory (emacs android-request-directory-access)	
<code>android_relinquish_directory_access</code>	Revoke access to a directory (emacs android-relinquish-directory-access)	emacs, cua — normal: <code><A-`></code> , insert: <code><A-`></code>
<code>tooltip_mode</code>	Toggle GUI help tooltips; no visible effect on a tty (emacs tooltip-mode)	
<code>unicode_fonts_setup</code>	Configure per-block font fallback; no visible effect on a tty (spacemacs unicode-fonts-setup)	
<code>tmm_menubar</code>	Pick a menu-bar command from a text list (emacs tmm-menubar)	
<code>vc_dir_mark</code>	Mark the file at point in the VC directory (emacs vc-dir-mark)	
<code>vc_dir_mark_all_files</code>	Mark every file in the VC directory (emacs vc-dir-mark-all-files)	
<code>vc_dir_mark_by_regexp</code>	Mark VC-directory files matching a regexp (emacs vc-dir-mark-by-regexp)	
<code>vc_dir_mark_registered_files</code>	Mark every registered file in the VC directory (emacs vc-dir-mark-registered-files)	
<code>log_edit_done</code>	Finish the commit message and commit (emacs log-edit-done)	
<code>log_edit_show_diff</code>	Show the diff of what is being committed (emacs log-edit-show-diff)	
<code>log_edit_show_files</code>	List the files being committed (emacs log-edit-show-files)	

Name	Description	Keybinds
<code>log_edit_insert_changelog</code>	Seed the commit message from the ChangeLog (emacs log-edit-insert-changelog)	
<code>log_edit_generate_changelog_from_diff</code>	Write the commit message from the diff (emacs log-edit-generate-changelog-from-diff)	
<code>project_switch_project</code>	Switch to another project (emacs project-switch-project)	
<code>project_forget_project</code>	Remove a project from the known-project list (emacs project-forget-project)	
<code>project_search</code>	Grep-search the project (emacs project-search)	
<code>project_query_replace_regexp</code>	Project-wide regex replace (emacs project-query-replace-regexp)	
<code>project_list_buffers</code>	List open buffers (emacs project-list-buffers)	
<code>project_shell_command</code>	Run a shell command in the project root (emacs project-shell-command)	spacemacs — normal: <code><space>p!</code> , select: <code><space>p!</code>
<code>project_async_shell_command</code>	Run an async shell command in the project (emacs project-async-shell-command)	spacemacs — normal: <code><space>pġ</code> , select: <code><space>pġ</code>
<code>project_eshell</code>	Open a shell buffer for the project (emacs project-eshell)	
<code>xref_find_definitions_other_window</code>	Goto definition in another window (emacs xref-find-definitions-other-window)	spacemacs — normal: <code><C-w>d</code> , <code><C-x>4.</code> , <code><space>mġġ</code> , select: <code><C-x>4.</code> , <code><space>mġġ</code> , insert: <code><C-x>4.vim</code> — normal: <code><C-w>d</code> demacs — insert: <code><C-x>4.cua</code> — select: <code><C-X>4.</code> , insert: <code><C-x>4.</code>
<code>info_search_other_window</code>	Open the Info directory node in another window (emacs info-other-window)	spacemacs — normal: <code><C-h>4i</code> , select: <code><C-h>4i</code> , insert: <code><C-h>4i</code>
<code>xref_query_replace_in_results</code>	Query-replace across xref/project results (emacs xref-query-replace-in-results)	
<code>xref_find_references_and_replace</code>	Find references and replace them (emacs xref-find-references-and-replace)	
<code>cut_to_clipboard</code>	Cut the selection to the system clipboard	
<code>org_cycle</code>	Org: toggle subtree fold	spacemacs — normal: <code><space>mz</code> , <code><space>m<tab></code> , select: <code><space>mz</code> , <code><space>m<tab></code>
<code>org_todo</code>	Org: cycle TODO keyword	
<code>org_priority</code>	Org: cycle priority cookie	spacemacs — normal: <code><space>mp</code> , select: <code><space>mp</code>
<code>org_promote</code>	Org: promote heading	spacemacs — normal: <code><space>mH</code> , select: <code><space>mH</code>
<code>org_demote</code>	Org: demote heading	spacemacs — normal: <code><space>mL</code> , select: <code><space>mL</code>
<code>org_next_heading</code>	Org: next heading	spacemacs — normal: <code><space>mj</code> , select: <code><space>mj</code>
<code>org_prev_heading</code>	Org: previous heading	
<code>org_fold_all</code>	Org: fold all headings	spacemacs — normal: <code><space>ma</code> , select: <code><space>ma</code>
<code>outline_minor_mode</code>	Fold the buffer by outline headings; run again to unfold (emacs outline-minor-mode)	

Name	Description	Keybinds
<code>org_unfold_all</code>	Org: unfold all	spacemacs — normal: <code><space>mA</code> , select: <code><space>mA</code>
<code>org_agenda</code>	Org: open agenda	spacemacs — normal: <code><space>aoa</code> , select: <code><space>aoa</code>
<code>org_capture</code>	Org: capture note	spacemacs — normal: <code><space>oc</code> , <code><space>aoc</code> , select: <code><space>oc</code> , <code><space>aoc</code>
<code>goto_first_change</code>	Goto first change	helix — normal: <code>[G</code> , select: <code>[G</code>
<code>goto_last_change</code>	Goto last change	spacemacs — normal: <code><space>jc</code> , select: <code><space>jc</code> helix — normal: <code>]G</code> , select: <code>]G</code>
<code>goto_line_start</code>	Goto line start	spacemacs — normal: <code>0</code> , <code>g<home></code> , <code><space>j0</code> , select: <code><space>j0</code> , insert: <code><home>vim</code> — normal: <code>0</code> , <code>g<home></code> , insert: <code><home></code> helix — normal: <code>gh</code> , <code><home></code> , select: <code>gh</code> , insert: <code><home>emacs</code> , cua — normal: <code><[-a></code> , <code><home></code> , select: <code><[-a></code> , insert: <code><[-a></code> , <code><home></code>
<code>goto_line_end</code>	Goto line end	spacemacs — normal: <code>\$</code> , <code>gl</code> , <code>g<end></code> , <code><space>j\$</code> , select: <code><space>j\$</code> vim — normal: <code>\$</code> , <code>gl</code> , <code>g<end></code> helix — normal: <code>gl</code> , <code><end></code> , select: <code>glemacs</code> , cua — normal: <code><[-e></code> , <code><end></code> , select: <code><[-e></code> , insert: <code><[-e></code> , <code><end></code>
<code>goto_visual_line_start</code>	Goto visual line start (soft-wrap aware)	spacemacs , vim — normal: <code>g0</code>
<code>goto_visual_first_nonwhitespace</code>	Goto first non-blank of the screen line (vim g^)	spacemacs , vim — normal: <code>g^</code>
<code>goto_visual_line_end</code>	Goto visual line end (soft-wrap aware)	spacemacs , vim — normal: <code>g\$</code>
<code>extend_to_visual_line_start</code>	Extend to visual line start	
<code>extend_to_visual_line_end</code>	Extend to visual line end	
<code>goto_column</code>	Goto column	spacemacs — normal: <code>\ </code> , <code><A-g><tab></code> vim — normal: <code>\ </code> helix — normal: <code>g\ </code> , select: <code>g\ </code>
<code>extend_to_column</code>	Extend to column	
<code>goto_next_buffer</code>	Goto next buffer	spacemacs — normal: <code>]b</code> , <code><space>bn</code> , <code><space>b.n</code> , <code><[-x><right></code> , select: <code><space>bn</code> , <code><space>b.n</code> , <code><[-x><right></code> , insert: <code><[-x><right></code> vim — normal: <code>]b</code> helix — normal: <code>gn</code> , select: <code>gnemacs</code> — normal: <code><[-x><right></code> , insert: <code><[-x><right></code> , cua — normal: <code><[-x><right></code> , select: <code><[-x><right></code> , insert: <code><[-X><right></code>

Name	Description	Keybinds
<code>goto_previous_buffer</code>	Goto previous buffer	spacemacs — normal: <code>[b, <space>bp, <space>b.N, <space>b.p, <[-x]<left>, select: <space>bp, <space>b.N, <space>b.p, <[-x]<left>, insert: <[-x]<left></code> vim — normal: <code>[bhelix</code> — normal: <code>gp</code> , select: <code>gpemacs</code> — normal: <code><[-x]<left>, insert: <[-x]<left></code> cua — normal: <code><[-x]<left>, select: <[-X]<left>, insert: <[-x]<left></code>
<code>goto_line_end_newline</code>	Goto newline at line end	spacemacs, vim, helix — insert: <code><end></code>
<code>goto_first_nonwhitespace</code>	Goto first non-blank in line	spacemacs — normal: <code>^, <A-m></code> vim — normal: <code>^helix</code> — normal: <code>gs</code> , select: <code>gsemacs, cua</code> — normal: <code><A-m>, insert: <A-m></code>
<code>trim_selections</code>	Trim whitespace from selections	helix — normal: <code>_, select: _</code>
<code>extend_to_line_start</code>	Extend to line start	helix — select: <code><home></code> cua — select: <code><5-home></code>
<code>extend_to_first_nonwhitespace</code>	Extend to first non-blank in line	emacs, cua — select: <code><A-m></code>
<code>extend_to_line_end</code>	Extend to line end	helix — select: <code><end></code> cua — select: <code><5-end></code>
<code>extend_to_line_end_newline</code>	Extend to line end	
<code>signature_help</code>	Show signature help	spacemacs — normal: <code><space>ls</code> , select: <code><space>ls</code>
<code>smart_tab</code>	Insert tab if all cursors have all whitespace to their left; otherwise, run a separate command.	
<code>insert_tab</code>	Insert tab char	helix — insert: <code><5-tab></code> emacs, cua — insert: <code><A-i></code>
<code>insert_newline</code>	Insert newline char	spacemacs, vim, helix, emacs, cua — insert: <code><[-j>, <ret></code>
<code>default_indent_new_line</code>	Break line at point and continue the comment, indenting under it (emacs default-indent-new-line, M-j)	spacemacs — normal: <code><A-[-j>, insert: <A-j></code> vim — insert: <code><A-j></code>
<code>insert_char_interactive</code>	Insert an interactively-chosen char	spacemacs, vim — insert: <code><[-Q>, <[-V>, <[-q>, <[-v></code>
<code>append_char_interactive</code>	Append an interactively-chosen char	
<code>delete_char_backward</code>	Delete previous char	spacemacs — insert: <code><backspace></code> vim, emacs, cua — insert: <code><[-h>, <backspace></code> helix — insert: <code><[-h>, <backspace>, <5-backspace></code>
<code>delete_char_forward</code>	Delete next char	spacemacs, vim — insert: <code></code> helix — insert: <code><[-d>, </code> emacs, cua — normal: <code><[-d>, insert: <[-d>, </code> spacemacs, vim — normal: <code>x, </code>
<code>delete_chars_forward_vim</code>	Delete char(s) under cursor, line-bounded (vim x)	
<code>delete_chars_backward_vim</code>	Delete char(s) before cursor, no line join (vim X)	spacemacs, vim — normal: <code>X</code>
<code>replace_chars_vim</code>	Replace char(s) under cursor, line-bounded (vim r)	spacemacs, vim — normal: <code>r</code>
<code>delete_word_backward</code>	Delete previous word	spacemacs, vim, helix — insert: <code><[-w>, <A-backspace></code> emacs, cua — insert: <code><[-w>, <A-backspace>, <A-[-backspace></code>

Name	Description	Keybinds
<code>delete_word_forward</code>	Delete next word	spacemacs — normal: <code><A-d></code> , insert: <code><A-d>vim</code> , emacs , cua — insert: <code><A-d>helix</code> — insert: <code><A-d></code> , <code><A-del></code> spacemacs , vim — insert: <code><[-u></code>
<code>insert_kill_entered_vim</code>	Delete the text entered this insert session (vim i_CTRL-U)	helix — insert: <code><[-u></code>
<code>kill_to_line_start</code>	Delete till start of line	helix — insert: <code><[-k></code> emacs , cua — normal: <code><[-k></code> , insert: <code><[-k></code>
<code>kill_to_line_end</code>	Delete till end of line	spacemacs — normal: <code>u</code> , <code><[-/>></code> , <code><[-x>u</code> , <code><space>ku</code> , select: <code><[-x>u</code> , <code><space>ku</code> , insert: <code><[-/>></code> , <code><[-_></code> , <code><[-x>u</code> vim — normal: <code>u</code> helix — normal: <code>u</code> , select: <code>u</code> emacs — normal: <code><[-/>></code> , <code><[-_></code> , <code><[-x>u</code> , insert: <code><[-/>></code> , <code><[-_></code> , <code><[-x>u</code> cua — normal: <code><[-/>></code> , <code><[-_></code> , <code><[-z></code> , <code><[-x>u</code> , select: <code><[-X>u</code> , insert: <code><[-/>></code> , <code><[-_></code> , <code><[-z></code> , <code><[-x>u</code>
<code>undo</code>	Undo change	spacemacs , vim — normal: <code>U</code>
<code>undo_line</code>	Undo all latest changes on one line (vim U)	
<code>redo</code>	Redo change	spacemacs — normal: <code><[-r></code> , <code><space>k<[-r></code> , select: <code><space>k<[-r></code> vim — normal: <code><[-r></code> helix — normal: <code>U</code> , select: <code>U</code> spacemacs , vim — normal: <code>g<minus></code> helix — normal: <code><A-u></code> , select: <code><A-u></code>
<code>earlier</code>	Move backward in history	spacemacs , vim — normal: <code>g+helix</code> — normal: <code><A-U></code> , select: <code><A-U></code> spacemacs — normal: <code><space>aU</code> , select: <code><space>aU</code>
<code>later</code>	Move forward in history	
<code>undo_tree</code>	Browse the branching undo history (vim undotree)	
<code>edit_injected_fragment</code>	Edit the injected-language fragment at point in its own buffer	
<code>apply_injected_fragment</code>	Write the fragment buffer back into its host string	
<code>commit_undo_checkpoint</code>	Commit changes to new checkpoint	spacemacs , vim — insert: <code><[-g>U</code> , <code><[-g>u</code> helix — insert: <code><[-s></code>
<code>yank</code>	Yank selection	spacemacs — normal: <code><A-w></code> helix — normal: <code>y</code> , select: <code>y</code>
<code>yank_to_clipboard</code>	Yank selections to clipboard	helix — normal: <code><space>y</code> , select: <code><space>y</code>
<code>yank_to_primary_clipboard</code>	Yank selections to primary clipboard	
<code>yank_joined</code>	Join and yank selections	
<code>yank_joined_to_clipboard</code>	Join and yank selections to clipboard	
<code>yank_main_selection_to_clipboard</code>	Yank main selection to clipboard	helix — normal: <code><space>Y</code> , select: <code><space>Y</code>
<code>yank_joined_to_primary_clipboard</code>	Join and yank selections to primary clipboard	
<code>yank_main_selection_to_primary_clipboard</code>	Yank main selection to primary clipboard	
<code>replace_with_yanked</code>	Replace with yanked text	spacemacs , vim — select: <code>P</code> , phelix — normal: <code>R</code> , select: <code>R</code>
<code>replace_selections_with_clipboard</code>	Replace selections by clipboard content	helix — normal: <code><space>R</code> , select: <code><space>R</code>

Name	Description	Keybinds
<code>replace_selections_with_primary_clipboard</code>	Replace selections by primary clipboard	
<code>paste_after</code>	Paste after selection	spacemacs — normal: <code>p</code> , <code><space>kp</code> , select: <code><space>kp</code> vim — normal: phelix — normal: <code>p</code> , select: <code>p</code> spacemacs, vim — normal: <code>gp</code>
<code>paste_after_cursor_after</code>	Paste after selection, cursor after the pasted text (vim gp)	
<code>select_pasted_text</code>	Reselect the text the last put inserted (spacemacs g p)	
<code>paste_before_cursor_after</code>	Paste before selection, cursor after the pasted text (vim gP)	spacemacs, vim — normal: <code>gP</code>
<code>paste_before</code>	Paste before selection	spacemacs — normal: <code>P</code> , <code>[P,]P</code> , <code><space>kP</code> , select: <code><space>kP</code> vim — normal: <code>P</code> , <code>[P,]P</code> phelix — normal: <code>P</code> , select: <code>P</code>
<code>yank_from_kill_ring</code>	Yank the latest kill-ring entry (emacs C-y)	emacs — normal: <code><[-y></code> , insert: <code><[-y>cua</code> — normal: <code><[-v></code> , <code><[-y></code> , insert: <code><[-v></code> , <code><[-y></code>
<code>yank_pop</code>	Replace the just-yanked text with the next kill-ring entry (emacs M-y)	spacemacs — normal: <code><A-y></code> emacs, cua — normal: <code><A-y></code> , insert: <code><A-y></code>
<code>set_mark_command</code>	Set mark and activate region, pushing to the mark ring (emacs C-SPC)	spacemacs — normal: <code><[-@></code> emacs, cua — insert: <code><[-space></code>
<code>pop_to_mark</code>	Jump to the top of the mark ring, rotating it (emacs C-x C-SPC)	spacemacs — normal: <code><[-x><[-space></code> , select: <code><[-x><[-space></code> , insert: <code><[-x><[-space></code> emacs — insert: <code><[-x><[-space></code> cua — select: <code><[-X><[-space></code> , insert: <code><[-x><[-space></code>
<code>universal_argument</code>	Begin a prefix argument for the next command; repeat to multiply it by 4 (emacs C-u)	emacs, cua — normal: <code><[-u></code> , insert: <code><[-u></code>
<code>digit_argument</code>	Read a numeric prefix argument for the next command (emacs M-1 ... M-9)	
<code>negative_argument</code>	Make the prefix argument negative (emacs M-)	
<code>list_packages</code>	Open the package menu: browse, install and delete elisp packages (emacs M-x list-packages)	
<code>package_refresh_contents</code>	Re-read the package archives' listings (emacs package-refresh-contents)	
<code>package_install</code>	Install a package, with its dependencies, from an archive (emacs package-install)	
<code>package_install_file</code>	Install a package from a .el or .tar file on disk (emacs package-install-file)	
<code>package_delete</code>	Delete an installed package (emacs package-delete)	
<code>package_upgrade</code>	Install the newer version of a package (emacs package-upgrade)	
<code>package_upgrade_all</code>	Upgrade every package that has a newer version (emacs package-upgrade-all)	
<code>package_activate_all</code>	Load every installed package now (emacs package-activate-all)	
<code>package_quickstart_refresh</code>	Write package-quickstart.el, preloading every package's autoloader (emacs package-quickstart-refresh)	

Name	Description	Keybinds
<code>package_recompile</code>	Re-evaluate a package's elisp, reporting any file that fails (emacs package-recompile)	
<code>package_recompile_all</code>	Re-evaluate every installed package's elisp (emacs package-recompile-all)	
<code>describe_package</code>	Show a package's version, home page, keywords and dependencies (emacs C-h P)	
<code>finder_by_keyword</code>	List the packages that carry a keyword (emacs C-h p / finder-by-keyword)	
<code>package_browse_url</code>	Open the home page of the package under point (emacs package-browse-url)	
<code>package_menu_describe_package</code>	Describe the package under point in the package menu (emacs RET)	
<code>package_menu_mark_install</code>	Mark the package under point for installation (emacs package menu i)	
<code>package_menu_mark_delete</code>	Mark the package under point for deletion (emacs package menu d)	
<code>package_menu_mark_unmark</code>	Remove the mark from the package under point (emacs package menu u)	
<code>package_menu_mark_upgrades</code>	Mark every package that has a newer version (emacs package menu U)	
<code>package_menu_mark_obsolete_for_deletion</code>	Mark superseded installed versions for deletion (emacs package menu ~)	
<code>package_menu_execute</code>	Carry out the marked installations and deletions (emacs package menu x)	
<code>package_menu_quick_help</code>	Show the package menu's key bindings (emacs package menu ?)	
<code>package_menu_hide_package</code>	Hide the package under point from the listing (emacs package menu H)	
<code>package_menu_toggle_hiding</code>	Show or hide the packages hidden with H (emacs package menu ()	
<code>package_menu_filter_by_name</code>	Show only packages whose name matches (emacs package menu / n)	
<code>package_menu_filter_by_description</code>	Show only packages whose summary matches (emacs package menu / d)	
<code>package_menu_filter_by_name_or_description</code>	Show only packages whose name or summary matches (emacs package menu / N)	
<code>package_menu_filter_by_keyword</code>	Show only packages carrying a keyword (emacs package menu / k)	
<code>package_menu_filter_by_status</code>	Show only packages with a status: installed, available, obsolete, external (emacs package menu / s)	
<code>package_menu_filter_by_archive</code>	Show only packages from one archive (emacs package menu / a)	
<code>package_menu_filter_by_version</code>	Show only packages of a version (emacs package menu / v)	
<code>package_menu_filter_marked</code>	Show only the marked packages (emacs package menu / m)	
<code>package_menu_filter_upgradable</code>	Show only packages that can be upgraded (emacs package menu / u)	
<code>package_menu_filter_clear</code>	Clear every package-menu filter (emacs package menu / /)	
<code>package_vc_install</code>	Install a package straight from its source repository (emacs package-vc-install)	

Name	Description	Keybinds
<code>open_rectangle</code>	Insert blank space to shift the rectangle right (emacs C-x r o)	spacemacs — normal: <code><[-x>r o</code> , select: <code><[-x>r o</code> , insert: <code><[-x>r o</code>
<code>delete_whitespace_rectangle</code>	Delete whitespace after the rectangle's left column on each line (emacs delete-whitespace-rectangle)	
<code>bookmark_set</code>	Set a named persistent bookmark at point (emacs C-x r m)	spacemacs — normal: <code><[-x>r m</code> , select: <code><[-x>r m</code> , insert: <code><[-x>r m</code> <code><[-x>r m e m a c s</code> — insert: <code><[-x>r m c u a</code> — select: <code><[-X>r m</code> , insert: <code><[-x>r m</code>
<code>bookmark_set_no_overwrite</code>	Set a bookmark, refusing to overwrite an existing name (emacs C-x r M)	spacemacs — normal: <code><[-x>r M</code> , select: <code><[-x>r M</code> , insert: <code><[-x>r M</code>
<code>bookmark_jump</code>	Jump to a bookmark via a picker (emacs C-x r b)	spacemacs — normal: <code><[-x>r b</code> , <code><space>r j</code> , select: <code><[-x>r b</code> , <code><space>r j</code> , insert: <code><[-x>r b e m a c s</code> — insert: <code><[-x>r b</code> , <code><[-x>r l c u a</code> — select: <code><[-X>r b</code> , <code><[-X>r l</code> , insert: <code><[-x>r b</code> , <code><[-x>r l</code>
<code>list_bookmarks</code>	List bookmarks in a picker; select to jump (emacs C-x r l / list-bookmarks)	spacemacs — normal: <code><[-x>r l</code> , select: <code><[-x>r l</code> , insert: <code><[-x>r l</code>
<code>bookmark_insert_location</code>	Insert a bookmark's file path at point (emacs bookmark-insert-location, C-x r l)	
<code>bookmark_insert</code>	Insert the contents of a bookmark's file at point (emacs bookmark-insert)	
<code>bookmark_delete</code>	Delete a bookmark via a picker (emacs bookmark-delete)	
<code>bookmark_rename</code>	Rename a bookmark via a picker (emacs bookmark-rename)	
<code>define_abbrev</code>	Define a global abbrev: (emacs C-x a g)	spacemacs — normal: <code><[-x>a g</code> , select: <code><[-x>a g</code> , insert: <code><[-x>a g e m a c s</code> — insert: <code><[-x>a g c u a</code> — select: <code><[-X>a g</code> , insert: <code><[-x>a g</code>
<code>add_mode_abbrev</code>	Define the region or word before point as a mode-local abbrev, prompting for its name (emacs add-mode-abbrev, C-x a l)	spacemacs — normal: <code><[-x>a l</code> , select: <code><[-x>a l</code> , insert: <code><[-x>a l e m a c s</code> — insert: <code><[-x>a l c u a</code> — select: <code><[-X>a l</code> , insert: <code><[-x>a l</code>
<code>inverse_add_global_abbrev</code>	Define the word before point as an abbrev, prompting for its expansion (emacs inverse-add-global-abbrev, C-x a i g)	spacemacs — normal: <code><[-x>a i g</code> , select: <code><[-x>a i g</code> , insert: <code><[-x>a i g e m a c s</code> — insert: <code><[-x>a i g c u a</code> — select: <code><[-X>a i g</code> , insert: <code><[-x>a i g</code>
<code>inverse_add_mode_abbrev</code>	Define the word before point as a mode-local abbrev, prompting for its expansion (emacs inverse-add-mode-abbrev, C-x a i l)	spacemacs — normal: <code><[-x>a i l</code> , select: <code><[-x>a i l</code> , insert: <code><[-x>a i l e m a c s</code> — insert: <code><[-x>a i l c u a</code> — select: <code><[-X>a i l</code> , insert: <code><[-x>a i l</code>
<code>toggle_abbrev_mode</code>	Toggle abbrev-mode: auto-expand abbrevs when typing a word separator (emacs abbrev-mode)	
<code>wdired_finish_edit</code>	Apply the file renames edited in a wdired buffer (emacs wdired-finish-edit)	
<code>timeclock_in</code>	Clock in to a project, prompting for its name (emacs timeclock-in)	
<code>timeclock_out</code>	Clock out (emacs timeclock-out)	
<code>timeclock_change</code>	Clock out of the current project and into another (emacs timeclock-change)	
<code>timeclock_workday_remaining</code>	Report the time left in the workday (emacs timeclock-workday-remaining)	

Name	Description	Keybinds
<code>timeclock_when_to_leave</code>	Report how long until the workday is complete (emacs timeclock-when-to-leave)	
<code>timeclock_reread_log</code>	Reload the timelog and report the clock state (emacs timeclock-reread-log)	
<code>timeclock_mode_line_display</code>	Show today's worked/remaining time and clock state (emacs timeclock-mode-line-display)	
<code>expand_abbrev</code>	Expand the abbrev before point (emacs C-x ")	spacemacs — normal: <code><A-`></code> , <code><[-x>'</code> , select: <code><[-x>'</code> , insert: <code><[-]></code> , <code><[-x>'vim</code> — insert: <code><[-]></code> emacs — insert: <code><[-x>'cua</code> — select: <code><[-X>'</code> , insert: <code><[-x>'</code>
<code>abbrev_prefix_mark</code>	Mark point as an abbrev prefix boundary; insert a hyphen the next expand-abbrev removes (emacs abbrev-prefix-mark, M-')	spacemacs — normal: <code><space>xem</code> , select: <code><space>xem</code>
<code>unexpand_abbrev</code>	Undo the last abbrev expansion, restoring the original abbrev text (emacs unexpand-abbrev)	spacemacs — normal: <code><space>xeu</code> , select: <code><space>xeu</code>
<code>insert_abbrevs</code>	Insert a description of every defined abbrev at point (emacs insert-abbrevs)	
<code>define_abbrevs</code>	Define abbrevs from the buffer text after point (emacs define-abbrevs)	
<code>paste_clipboard_after</code>	Paste clipboard after selections	helix — normal: <code><space>p</code> , select: <code><space>p</code>
<code>paste_clipboard_before</code>	Paste clipboard before selections	helix — normal: <code><space>P</code> , select: <code><space>P</code>
<code>paste_primary_clipboard_after</code>	Paste primary clipboard after selections	
<code>paste_primary_clipboard_before</code>	Paste primary clipboard before selections	
<code>indent</code>	Indent selection	spacemacs, vim — normal: <code>=</code> , select: <code><gt></code> , insert: <code><[-t></code> helix — normal: <code><gt></code> , select: <code><gt></code>
<code>unindent</code>	Unindent selection	spacemacs, vim — select: <code><lt></code> helix — normal: <code><lt></code> , select: <code><lt></code>
<code>format_selections</code>	Format selection	spacemacs — normal: <code><space>j+</code> , <code><space>j=</code> , <code><space>lf</code> , select: <code><space>j+</code> , <code><space>j=</code> , <code><space>lf</code> helix — normal: <code>=</code> , select: <code>=</code>
<code>reflow_selections</code>	vim gg: reflow selection to text-width	spacemacs — normal: <code><A-q></code> emacs, cua — normal: <code><A-q></code> , insert: <code><A-q></code>
<code>reflow_selections_keep_cursor</code>	vim gw: reflow to text-width, keep cursor	
<code>reflow_mark_cursor</code>	Record the cursor for the vim gw that follows	
<code>join_selections</code>	Join lines inside selection	spacemacs — normal: <code><A-^></code> , <code><space>kJ</code> , select: <code><space>kJ</code> helix — normal: <code>J</code> , select: <code>J</code> emacs, cua — insert: <code><A-^></code>
<code>join_selections_space</code>	Join lines inside selection and select spaces	helix — normal: <code><A-J></code> , select: <code><A-J></code>
<code>join_lines_vim</code>	Join line(s) with a space, cursor at join (vim J)	spacemacs, vim — normal: <code>J</code>
<code>join_lines_vim_nospace</code>	Join line(s) without a space (vim gJ)	
<code>keep_selections</code>	Keep selections matching regex	helix — normal: <code>K</code> , select: <code>K</code>

Name	Description	Keybinds
<code>remove_selections</code>	Remove selections matching regex	helix — normal: <code><A-K></code> , select: <code><A-K></code>
<code>align_selections</code>	Align selections in column	spacemacs — normal: <code><space>xaa</code> , select: <code><space>xaa</code> helix — normal: <code>⌘</code> , select: <code>⌘</code>
<code>keep_primary_selection</code>	Keep primary selection	helix — normal: <code>,</code> , select: <code>,</code>
<code>remove_primary_selection</code>	Remove primary selection	helix — normal: <code><A-,></code> , select: <code><A-,></code>
<code>completion</code>	Invoke completion popup	spacemacs — normal: <code><A-[i></code> , <code><A-tab></code> , <code><[c></code> , <code>l</code> , <code><[c></code> , <code><space></code> , select: <code><[c></code> , <code>l</code> , <code><[c></code> , <code><space></code> , insert: <code><A-/></code> , <code><[n></code> , <code><[p></code> , <code><[c></code> , <code>l</code> , <code><[c></code> , <code><space></code> vim — insert: <code><[n></code> , <code><[p></code> , <code><[x><[i></code> , <code><[x><[n></code> , <code><[x><[p></code> helix — insert: <code><[x></code> emacs , cua — normal: <code><A-tab></code> , insert: <code><A-tab></code>
<code>hover</code>	Show docs for item under cursor	spacemacs — normal: <code><[h>.</code> , <code><space>lk</code> , <code><space>hda</code> , <code><space>hdf</code> , <code><space>hdv</code> , <code><space>mhh</code> , select: <code>K</code> , <code><[h>.</code> , <code><space>lk</code> , <code><space>hda</code> , <code><space>hdf</code> , <code><space>hdv</code> , <code><space>mhh</code> , insert: <code><[h>.</code> vim — select: <code>K</code> helix — normal: <code><space>k</code> , select: <code><space>k</code> spacemacs , vim — normal: <code>K</code>
<code>keyword_lookup</code>	vim K: run keywordprg on the word under cursor, else LSP hover	spacemacs , vim — normal: <code>K</code>
<code>goto_first_nonwhitespace_down</code>	vim <code>_</code> : first non-blank, count-1 lines down	spacemacs , vim — normal: <code>_</code>
<code>toggle_replace_mode</code>	vim <code>:</code> toggle insert/overtyp	spacemacs , vim — insert: <code><ins></code>
<code>insert_unindent</code>	vim <code>i_CTRL-D</code> : unindent, or <code>O/^ CTRL-D</code> delete all indent	spacemacs , vim — insert: <code><[d></code>
<code>toggle_comments</code>	Comment/uncomment selections	spacemacs — normal: <code><A-;></code> , <code><space>;</code> , <code><space>cP</code> , <code><space>cc</code> , <code><space>cp</code> , <code><[x><[-;></code> , select: <code><space>;</code> , <code><space>cP</code> , <code><space>cc</code> , <code><space>cp</code> , <code><[x><[-;></code> , insert: <code><[x><[-;></code> helix — normal: <code><[c></code> , <code><space>c</code> , select: <code><[c></code> , <code><space>c</code> emacs — normal: <code><A-;></code> , insert: <code><A-;></code> , <code><[x><[-;></code> cua — normal: <code><A-;></code> , select: <code><[X><[-;></code> , insert: <code><A-;></code> , <code><[x><[-;></code>
<code>toggle_line_comments</code>	Line comment/uncomment selections	spacemacs — normal: <code>gcc</code> , <code><space>cL</code> , <code><space>cL</code> , select: <code><space>cL</code> , <code><space>cL</code> vim — normal: <code>gcc</code> helix — normal: <code><space><A-c></code> , select: <code><space><A-c></code>
<code>comment_to_line</code>	Comment/uncomment from the cursor line to a prompted line (SPC c t)	spacemacs — normal: <code><space>ct</code> , select: <code><space>ct</code>
<code>invert_comment_to_line</code>	Invert comments per line from the cursor to a prompted line (SPC c T)	spacemacs — normal: <code><space>cT</code> , select: <code><space>cT</code>
<code>toggle_block_comments</code>	Block comment/uncomment selections	spacemacs — normal: <code><space>cb</code> , select: <code><space>cb</code> helix — normal: <code><space>[</code> , select: <code><space>[</code>
<code>comment_kill</code>	Kill the comment on the current line to the kill ring (emacs comment-kill; count kills that many lines)	spacemacs — normal: <code><space>cx</code> , select: <code><space>cx</code>

Name	Description	Keybinds
<code>rotate_selections_forward</code>	Rotate selections forward	helix — normal: <code>)</code> , select: <code>)</code>
<code>rotate_selections_backward</code>	Rotate selections backward	helix — normal: <code>(</code> , select: <code>(</code>
<code>rotate_selection_contents_forward</code>	Rotate selection contents forward	helix — normal: <code><A-)></code> , select: <code><A-)></code>
<code>rotate_selection_contents_backward</code>	Rotate selections contents backward	helix — normal: <code><A-(></code> , select: <code><A-(></code>
<code>reverse_selection_contents</code>	Reverse selections contents	
<code>expand_selection</code>	Expand selection to parent syntax node	spacemacs — normal: <code><space>v</code> , <code><space>kU</code> , select: <code><space>v</code> , <code><space>kU</code> helix — normal: <code><A-o></code> , <code><A-up></code> , select: <code><A-o></code> , <code><A-up></code>
<code>shrink_selection</code>	Shrink selection to previously expanded syntax node	helix — normal: <code><A-i></code> , <code><A-down></code> , select: <code><A-i></code> , <code><A-down></code>
<code>wildfire</code>	Wildfire: select/expand to the closest text object	spacemacs, vim — normal: <code><ret></code>
<code>wildfire_shrink</code>	Wildfire: shrink to the previously selected text object	spacemacs, vim — normal: <code><backspace></code>
<code>select_next_sibling</code>	Select next sibling in the syntax tree	spacemacs — normal: <code><space>kL</code> , <code><space>kL</code> , select: <code><space>kL</code> , <code><space>kL</code> helix — normal: <code><A-n></code> , <code><A-right></code> , select: <code><A-n></code> , <code><A-right></code>
<code>select_prev_sibling</code>	Select previous sibling the in syntax tree	spacemacs — normal: <code><space>kH</code> , <code><space>kH</code> , select: <code><space>kH</code> , <code><space>kH</code> helix — normal: <code><A-p></code> , <code><A-left></code> , select: <code><A-p></code> , <code><A-left></code>
<code>select_all_siblings</code>	Select all siblings of the current node	helix — normal: <code><A-a></code> , select: <code><A-a></code>
<code>select_all_children</code>	Select all children of the current node	helix — normal: <code><A-I></code> , <code><S-A-down></code> , select: <code><A-I></code> , <code><S-A-down></code>
<code>jump_forward</code>	Jump forward on jumplist	spacemacs, vim — normal: <code><[-i></code> , <code><tab></code> helix — normal: <code><[-i></code> , <code><tab></code> , select: <code><[-i></code> , <code><tab></code> emacs, cua — insert: <code><A[-,></code>
<code>jump_backward</code>	Jump backward on jumplist	spacemacs — normal: <code><[-o></code> , <code><space>jb</code> , <code><space>s`</code> , select: <code><space>jb</code> , <code><space>s`</code> vim — normal: <code><[-o></code> helix — normal: <code><[-o></code> , select: <code><[-o></code> emacs, cua — normal: <code><A-,></code> , insert: <code><A-,></code>
<code>save_selection</code>	Save current selection to jumplist	helix — normal: <code><[-s></code> , select: <code><[-s></code>

Name	Description	Keybinds
<code>jump_view_right</code>	Jump to right split	spacemacs — normal: <code><C-w>l</code> , <code><C-w>.l</code> , <code><C-w>[l</code> , <code><C-w>{l</code> , <code><space>wl</code> , <code><C-w><C-l></code> , <code><space>w.l</code> , <code><space>w[l</code> , <code><space>w{l</code> , <code><C-w><right></code> , <code><space>w<C-l></code> , <code><space>w<right></code> , select: <code><space>wl</code> , <code><space>w.l</code> , <code><space>w[l</code> , <code><space>w{l</code> , <code><space>w<C-l></code> , <code><space>w<right></code> vim — normal: <code><C-w>l</code> , <code><C-w>.l</code> , <code><C-w>[l</code> , <code><C-w>{l</code> , <code><C-w><C-l></code> , <code><C-w><right></code> helix — normal: <code><C-w>l</code> , <code><space>wl</code> , <code><C-w><C-l></code> , <code><C-w><right></code> , <code><space>w<C-l></code> , <code><space>w<right></code> , select: <code><C-w>l</code> , <code><space>wl</code> , <code><C-w><C-l></code> , <code><C-w><right></code> , <code><space>w<C-l></code> , <code><space>w<right></code>
<code>jump_view_left</code>	Jump to left split	spacemacs — normal: <code><C-w>h</code> , <code><C-w>.h</code> , <code><C-w>[h</code> , <code><C-w>{h</code> , <code><space>wh</code> , <code><C-w><C-h></code> , <code><space>w.h</code> , <code><space>w[h</code> , <code><space>w{h</code> , <code><C-w><left></code> , <code><space>w<C-h></code> , <code><space>w<left></code> , select: <code><space>wh</code> , <code><space>w.h</code> , <code><space>w[h</code> , <code><space>w{h</code> , <code><space>w<C-h></code> , vim — normal: <code><C-w>h</code> , <code><C-w>.h</code> , <code><C-w>[h</code> , <code><C-w>{h</code> , <code><C-w><C-h></code> , <code><C-w><left></code> helix — normal: <code><C-w>h</code> , <code><space>wh</code> , <code><C-w><C-h></code> , <code><C-w><left></code> , <code><space>w<C-h></code> , <code><space>w<left></code> , select: <code><C-w>h</code> , <code><space>wh</code> , <code><C-w><C-h></code> , <code><C-w><left></code> , <code><space>w<C-h></code> , <code><space>w<left></code>
<code>jump_view_up</code>	Jump to split above	spacemacs — normal: <code><C-w>k</code> , <code><C-w>.k</code> , <code><C-w>[k</code> , <code><C-w>{k</code> , <code><C-w><up></code> , <code><space>wk</code> , <code><C-w><C-k></code> , <code><C-w><C-t></code> , <code><space>w.k</code> , <code><space>w[k</code> , <code><space>w{k</code> , <code><space>w<up></code> , <code><space>w<C-k></code> , <code><space>w<C-t></code> , select: <code><space>wk</code> , <code><space>w.k</code> , <code><space>w[k</code> , <code><space>w{k</code> , <code><space>w<C-k></code> , <code><space>w<C-t></code> vim — normal: <code><C-w>k</code> , <code><C-w>.k</code> , <code><C-w>[k</code> , <code><C-w>{k</code> , <code><C-w><up></code> , <code><C-w><C-k></code> , <code><C-w><C-t></code> helix — normal: <code><C-w>k</code> , <code><C-w><up></code> , <code><space>wk</code> , <code><C-w><C-k></code> , <code><space>w<up></code> , <code><space>w<C-k></code> , select: <code><C-w>k</code> , <code><C-w><up></code> , <code><space>wk</code> , <code><C-w><C-k></code> , <code><space>w<up></code> , <code><space>w<C-k></code>

Name	Description	Keybinds
<code>jump_view_down</code>	Jump to split below	spacemacs — normal: <code><C-w>b,</code> <code><C-w>j, <C-w>.j, <C-w>[j, <C-w>{j,</code> <code><space>wb, <space>wj, <C-w><C-b>,</code> <code><C-w><C-j>, <space>w.j,</code> <code><space>w[j, <space>w{j,</code> <code><C-w><down>, <space>w<C-b>,</code> <code><space>w<C-j>, <space>w<down>,</code> select: <code><space>wb, <space>wj,</code> <code><space>w.j, <space>w[j,</code> <code><space>w{j, <space>w<C-b>,</code> <code><space>w<C-j>, <space>w<down></code> vim — normal: <code><C-w>b, <C-w>j, <C-w>.j, <C-w>[j,</code> <code><C-w>{j, <C-w><C-b>, <C-w><C-j>, <C-w><down></code> helix — normal: <code><C-w>j,</code> <code><space>wj, <C-w><C-j>, <C-w><down>, <space>w<C-j>, <space>w<down>,</code> select: <code><C-w>j, <space>wj, <C-w><C-j>, <C-w><down>, <space>w<C-j>, <space>w<down></code>
<code>swap_view_right</code>	Swap with right split	spacemacs — normal: <code><C-w>L,</code> <code><C-w>.L, <C-w>[L, <C-w>{L,</code> <code><space>wL, <space>w.L,</code> <code><space>w[L, <space>w{L, select:</code> <code><space>wL, <space>w.L,</code> <code><space>w[L, <space>w{L</code> vim — normal: <code><C-w>L, <C-w>.L, <C-w>[L,</code> <code><C-w>{L</code> helix — normal: <code><C-w>L,</code> <code><space>wL, select: <C-w>L, <space>wL</code>
<code>swap_view_left</code>	Swap with left split	spacemacs — normal: <code><C-w>H,</code> <code><C-w>.H, <C-w>[H, <C-w>{H,</code> <code><space>wH, <space>w.H,</code> <code><space>w[H, <space>w{H, select:</code> <code><space>wH, <space>w.H,</code> <code><space>w[H, <space>w{H</code> vim — normal: <code><C-w>H, <C-w>.H, <C-w>[H,</code> <code><C-w>{H</code> helix — normal: <code><C-w>H,</code> <code><space>wH, select: <C-w>H, <space>wH</code>
<code>swap_view_up</code>	Swap with split above	spacemacs — normal: <code><C-w>K,</code> <code><C-w>.K, <C-w>[K, <C-w>{K,</code> <code><space>wK, <space>w.K,</code> <code><space>w[K, <space>w{K, select:</code> <code><space>wK, <space>w.K,</code> <code><space>w[K, <space>w{K</code> vim — normal: <code><C-w>K, <C-w>.K, <C-w>[K,</code> <code><C-w>{K</code> helix — normal: <code><C-w>K,</code> <code><space>wK, select: <C-w>K, <space>wK</code>

Name	Description	Keybinds
<code>swap_view_down</code>	Swap with split below	spacemacs — normal: <code><[-w>]</code> , <code><[-w> .]</code> , <code><[-w>[]</code> , <code><[-w>{]</code> , <code><space>w]</code> , <code><space>w .]</code> , <code><space>w[]</code> , <code><space>w{]</code> , select: <code><space>w]</code> , <code><space>w .]</code> , <code><space>w[]</code> , <code><space>w{]</code> vim — normal: <code><[-w>]</code> , <code><[-w> .]</code> , <code><[-w>[]</code> , <code><[-w>{]</code> helix — normal: <code><[-w>]</code> , <code><space>w]</code> , select: <code><[-w>]</code> , <code><space>w]</code>
<code>tmux_navigate_left</code>	Move left, on out into tmux at the edge (spacemacs tmux layer)	
<code>tmux_navigate_down</code>	Move down, on out into tmux at the edge (spacemacs tmux layer)	
<code>tmux_navigate_up</code>	Move up, on out into tmux at the edge (spacemacs tmux layer)	
<code>tmux_navigate_right</code>	Move right, on out into tmux at the edge (spacemacs tmux layer)	
<code>windmove_delete_left</code>	Delete the window to the left (emacs windmove-delete-left)	
<code>windmove_delete_right</code>	Delete the window to the right (emacs windmove-delete-right)	
<code>windmove_delete_up</code>	Delete the window above (emacs windmove-delete-up)	
<code>windmove_delete_down</code>	Delete the window below (emacs windmove-delete-down)	
<code>windmove_default_keybindings</code>	Bind S- to move between windows (emacs windmove-default-keybindings)	
<code>windmove_swap_states_default_keybindings</code>	Bind C-S- to swap windows (emacs windmove-swap-states-default-keybindings)	
<code>windmove_delete_default_keybindings</code>	Bind C-x S- to delete a neighbouring window (emacs windmove-delete-default-keybindings)	
<code>windmove_display_left</code>	Display the next buffer in the window to the left (emacs windmove-display-left)	
<code>windmove_display_right</code>	Display the next buffer in the window to the right (emacs windmove-display-right)	
<code>windmove_display_up</code>	Display the next buffer in the window above (emacs windmove-display-up)	
<code>windmove_display_down</code>	Display the next buffer in the window below (emacs windmove-display-down)	
<code>windmove_display_same_window</code>	Display the next buffer in this window (emacs windmove-display-same-window)	
<code>windmove_display_new_frame</code>	Display the next buffer in a new frame (emacs windmove-display-new-frame)	
<code>windmove_display_new_tab</code>	Display the next buffer in a new tab (emacs windmove-display-new-tab)	
<code>windmove_display_default_keybindings</code>	Bind M-S- to display the next buffer in that direction (emacs windmove-display-default-keybindings)	

Name	Description	Keybinds
<code>transpose_view</code>	Transpose splits	spacemacs — normal: <code><[-w>M</code> , <code><[-w>x</code> , <code><space>wM</code> , <code><space>wx</code> , <code><[-w><[-x></code> , <code><space>w<[-x></code> , select: <code><space>wM</code> , <code><space>wx</code> , <code><space>w<[-x></code> vim — normal: <code><[-w>M</code> , <code><[-w>x</code> , <code><[-w><[-x></code> helix — normal: <code><[-w>t</code> , <code><space>wt</code> , <code><[-w><[-t></code> , <code><space>w<[-t></code> , select: <code><[-w>t</code> , <code><space>wt</code> , <code><[-w><[-t></code> , <code><space>w<[-t></code>
<code>quickfix_next</code>	Quickfix: jump to next entry (:cnext)	spacemacs , vim — normal: <code>]q</code>
<code>quickfix_prev</code>	Quickfix: jump to previous entry (:cprev)	spacemacs , vim — normal: <code>[q</code>
<code>quickfix_first</code>	Quickfix: jump to first entry (:cfirst)	
<code>quickfix_last</code>	Quickfix: jump to last entry (:clast)	
<code>quickfix_open</code>	Quickfix: open the quickfix list window (:copen)	
<code>loclist_next</code>	Location list: jump to next entry (:lnext)	spacemacs , vim — normal: <code>]l</code>
<code>loclist_prev</code>	Location list: jump to previous entry (:lprev)	spacemacs , vim — normal: <code>[l</code>
<code>loclist_first</code>	Location list: jump to first entry (:lfirst)	
<code>loclist_last</code>	Location list: jump to last entry (:llast)	
<code>loclist_open</code>	Location list: open the location list window (:lopen)	
<code>goto_next_tabpage</code>	Go to the next tabpage (gt / :tabnext)	spacemacs — normal: <code>gt</code> , <code><[-w>gt</code> , <code><[-x>to</code> , <code><space>wgt</code> , select: <code><[-x>to</code> , <code><space>wgt</code> , insert: <code><[-x>to</code> vim — normal: <code>gt</code> , <code><[-w>gt</code> spacemacs — normal: <code>gT</code> , <code><[-w>gT</code> , <code><S -[-tab></code> , <code><space>wgT</code> , select: <code><space>wgT</code> vim — normal: <code>gT</code> , <code><[-w>gT</code>
<code>goto_previous_tabpage</code>	Go to the previous tabpage (gT / :tabprevious)	spacemacs — normal: <code><[-x>t2</code> , select: <code><[-x>t2</code> , insert: <code><[-x>t2</code> spacemacs — normal: <code><[-x>t0</code> , select: <code><[-x>t0</code> , insert: <code><[-x>t0</code> spacemacs — normal: <code><[-x>t1</code> , select: <code><[-x>t1</code> , insert: <code><[-x>t1</code> spacemacs , vim — normal: <code><[-w>T</code>
<code>new_tab</code>	Open a new tabpage (:tabnew)	
<code>close_tab</code>	Close the current tabpage (:tabclose)	
<code>tab_only</code>	Close all other tabpages (:tabonly)	
<code>window_to_new_tab</code>	Move the current window to a new tabpage (vim CTRL-W T)	
<code>goto_first_tabpage</code>	Go to the first tabpage (:tabfirst)	
<code>goto_last_tabpage</code>	Go to the last tabpage (:tablast)	
<code>tab_select</code>	Go to the [count]-th tab (emacs tab-select)	
<code>tab_recent</code>	Switch to the most recently visited tab (emacs tab-recent)	
<code>tab_bar_mode</code>	Toggle the tab bar (emacs tab-bar-mode)	
<code>tab_rename</code>	Name the current tab (emacs tab-rename)	spacemacs — normal: <code><[-x>tr</code> , select: <code><[-x>tr</code> , insert: <code><[-x>tr</code>
<code>tab_switch</code>	Switch to a tab by name or number (emacs tab-switch)	spacemacs — normal: <code><[-x>t<ret></code> , select: <code><[-x>t<ret></code> , insert: <code><[-x>t<ret></code>

Name	Description	Keybinds
<code>tab_undo</code>	Reopen the most recently closed tab (emacs tab-undo)	
<code>tab_bar_history_mode</code>	Toggle tab-visit history recording (emacs tab-bar-history-mode)	
<code>tab_bar_history_back</code>	Return to the previously visited tab (emacs tab-bar-history-back)	
<code>tab_bar_history_forward</code>	Re-visit a tab left via history-back (emacs tab-bar-history-forward)	
<code>forward_list</code>	Move forward over a balanced () group (emacs forward-list, C-M-n)	spacemacs — normal: <code><A-[n></code>
<code>backward_list</code>	Move backward over a balanced () group (emacs backward-list, C-M-p)	spacemacs — normal: <code><A-[p></code>
<code>down_list</code>	Descend into the next list (emacs down-list, C-M-d)	spacemacs — normal: <code><A-[d></code>
<code>up_list</code>	Move forward out of the enclosing list (emacs up-list)	
<code>backward_up_list</code>	Move backward out of the enclosing list (emacs backward-up-list, C-M-u)	spacemacs — normal: <code><A-[u></code>
<code>kill_sexp</code>	Kill the s-expression after point (emacs kill-sexp, C-M-k)	spacemacs — normal: <code><A-[k></code>
<code>mark_sexp</code>	Set the region over the following s-expression (emacs mark-sexp, C-M-SPC)	spacemacs — normal: <code><A-[@></code> , <code><A-[space></code>
<code>forward_sexp</code>	Move forward over the next s-expression (emacs forward-sexp, C-M-f)	spacemacs — normal: <code><A-[f></code>
<code>backward_sexp</code>	Move backward over the previous s-expression (emacs backward-sexp, C-M-b)	spacemacs — normal: <code><A-[b></code>
<code>prog_indent_sexp</code>	Re-indent the s-expression after point, or the enclosing defun with a prefix (emacs prog-indent-sexp, C-M-q; here = s)	spacemacs — normal: <code>=s</code> , <code><A-[q></code> vim — normal: <code>=s</code>
<code>copy_region_as_kill</code>	Copy the region to the kill ring without deleting (emacs copy-region-as-kill, M-w)	
<code>copy_as_format</code>	Copy the region or line to the kill ring, wrapped for the default format (spacemacs copy-as-format)	
<code>copy_as_format_asciidoc</code>	Copy the region or line as an AsciiDoc source block (copy-as-format-asciidoc)	
<code>copy_as_format_bitbucket</code>	Copy the region or line as Bitbucket markup (copy-as-format-bitbucket)	
<code>copy_as_format_disqus</code>	Copy the region or line as Disqus markup (copy-as-format-disqus)	
<code>copy_as_format_github</code>	Copy the region or line as a GitHub fenced block (copy-as-format-github)	
<code>copy_as_format_gitlab</code>	Copy the region or line as GitLab markup (copy-as-format-gitlab)	
<code>copy_as_format_hipchat</code>	Copy the region or line as a HipChat /code message (copy-as-format-hipchat)	
<code>copy_as_format_html</code>	Copy the region or line as HTML pre/code (copy-as-format-html)	
<code>copy_as_format_jira</code>	Copy the region or line as a Jira {code} block (copy-as-format-jira)	
<code>copy_as_format_markdown</code>	Copy the region or line as indented Markdown code (copy-as-format-markdown)	

Name	Description	Keybinds
<code>copy_as_format_mediawiki</code>	Copy the region or line as MediaWiki syntaxhighlight (copy-as-format-mediawiki)	
<code>copy_as_format_org_mode</code>	Copy the region or line as an Org src block (copy-as-format-org-mode)	
<code>copy_as_format_pod</code>	Copy the region or line as POD (copy-as-format-pod)	
<code>copy_as_format_rst</code>	Copy the region or line as a reStructuredText code directive (copy-as-format-rst)	
<code>copy_as_format_telegram</code>	Copy the region or line as Telegram markup (copy-as-format-telegram)	
<code>copy_as_format_slack</code>	Copy the region or line as Slack markup (copy-as-format-slack)	
<code>copy_as_format_whatsapp</code>	Copy the region or line as WhatsApp markup (copy-as-format-whatsapp)	
<code>mark_word</code>	Set the region over the next word (emacs mark-word, M-@)	spacemacs — normal: <code><A-@></code>
<code>mark_paragraph</code>	Select the paragraph around point (emacs mark-paragraph, M-h)	spacemacs — normal: <code><A-h></code> emacs , cua — normal: <code><A-h></code> , insert: <code><A-h></code>
<code>mark_defun</code>	Select the function/defun around point (emacs mark-defun, C-M-h)	spacemacs — normal: <code><A-[h></code>
<code>kill_sentence</code>	Kill from point to end of sentence (emacs kill-sentence, M-k)	spacemacs — normal: <code><A-k></code>
<code>backward_kill_sentence</code>	Kill from start of sentence to point (emacs backward-kill-sentence, C-x DEL)	spacemacs — normal: <code><[-x><backspace></code> , select: <code><[-x><backspace></code> , insert: <code><[-x><backspace></code>
<code>append_next_kill</code>	Make the next kill append to the last kill-ring entry (emacs append-next-kill, C-M-w)	spacemacs — normal: <code><A-[w></code> , <code><space>ra</code> , select: <code><space>ra</code>
<code>forward_page</code>	Move to the next form-feed page (emacs forward-page, C-x J)	spacemacs — normal: <code><[-x>]</code> , select: <code><[-x>]</code> , insert: <code><[-x>]</code>
<code>backward_page</code>	Move to the previous form-feed page (emacs backward-page, C-x I)	spacemacs — normal: <code><[-x>[</code> , select: <code><[-x>[</code> , insert: <code><[-x>[</code>
<code>mark_page</code>	Select the current form-feed page (emacs mark-page, C-x C-p)	spacemacs — normal: <code><[-x><[-p></code> , select: <code><[-x><[-p></code> , insert: <code><[-x><[-p></code>
<code>move_to_opposite_group</code>	Move the current editor to the opposite split group (JetBrains)	

Name	Description	Keybinds
<code>rotate_view</code>	Goto next window	spacemacs — normal: <code>]w, <[-w>p, <[-w>r, <[-w>w, <[-x>o, <[-w>.o, <[-w>.r, <[-w>.w, <[-w>[o, <[-w>[r, <[-w>[w, <[-w>{o, <[-w>{r, <[-w>{w, <space>wp, <space>wr, <space>ww, <[-w><[-p>, <[-w><[-r>, <[-w><[-w>, <[-w><tab>, <space>b.o, <space>w.o, <space>w.r, <space>w.w, <space>w[o, <space>w[r, <space>w[w, <space>w{o, <space>w{r, <space>w{w, <space>w<[-p>, <space>w<[-r>, <space>w<[-w>, <space>w<tab>, select: <[-x>o, <space>wp, <space>wr, <space>ww, <space>b.o, <space>w.o, <space>w.r, <space>w.w, <space>w[o, <space>w[r, <space>w[w, <space>w{o, <space>w{r, <space>w{w, <space>w<[-p>, <space>w<[-r>, <space>w<[-w>, <space>w<tab>, insert: <[-x>o vim — normal: <code>]w, <[-w>p, <[-w>r, <[-w>w, <[-w>.o, <[-w>.r, <[-w>.w, <[-w>[o, <[-w>[r, <[-w>[w, <[-w>{o, <[-w>{r, <[-w>{w, <[-w><[-p>, <[-w><[-r>, <[-w><[-w>, <[-w><tab> helix — normal: <code><[-w>w, <space>ww, <[-w><[-w>, <space>w<[-w>, select: <[-w>w, <space>ww, <[-w><[-w>, <space>w<[-w> emacs — normal: <code><[-x>o, insert: <[-x>o ocua — normal: <code><[-x>o, select: <[-X>o, insert: <[-x>o </code></code></code></code></code>
<code>goto_preview_window</code>	Goto the preview window (vim CTRL-W P)	spacemacs — normal: <code><[-w>P, <space>wP, select: <space>wP vim — normal: <code><[-w>P </code></code>
<code>rotate_view_reverse</code>	Goto previous window	spacemacs — normal: <code>[t, [w, <[-w>R, <[-w>W, <[-w>.R, <[-w>[R, <[-w>{R, <space>wR, <space>wW, <space>w.R, <space>w[R, <space>w{R, select: <space>wR, <space>wW, <space>w.R, <space>w[R, <space>w{R vim — normal: <code>[t, [w, <[-w>R, <[-w>W, <[-w>.R, <[-w>[R, <[-w>{R spacemacs — normal: <code><A-[-v> </code></code></code>
<code>scroll_other_window</code>	Scroll the other window forward (emacs scroll-other-window, C-M-v)	spacemacs — normal: <code><A-[-V></code>
<code>scroll_other_window_down</code>	Scroll the other window backward (emacs scroll-other-window-down, C-M-S-v)	
<code>recenter_other_window</code>	Recenter point in the other window (emacs recenter-other-window, C-M-S-l)	spacemacs — normal: <code><[-w>z, <A-[-L>, <space>wz, <[-w><[-z>, <space>w<[-z>, select: <space>wz, <space>w<[-z> vim — normal: <code><[-w>z, <[-w><[-z> </code></code>

Name	Description	Keybinds
<code>hsplit</code>	Horizontal bottom split	spacemacs — normal: <code><C-w>S</code> , <code><C-w>s</code> , <code><C-x>2</code> , <code><C-w>.S</code> , <code><C-w>.s</code> , <code><C-w>[S</code> , <code><C-w>[s</code> , <code><C-w>{S</code> , <code><C-w>{s</code> , <code><space>wS</code> , <code><space>ws</code> , <code><C-w><C-s></code> , <code><space>w.S</code> , <code><space>w.s</code> , <code><space>w[S</code> , <code><space>w[s</code> , <code><space>w{S</code> , <code><space>w{s</code> , <code><C-w>.<minus></code> , <code><C-w>[<minus></code> , <code><C-w>{<minus></code> , <code><space>w<C-s></code> , <code><space>w.<minus></code> , <code><space>w[<minus></code> , <code><space>w{<minus></code> , select: <code><C-x>2</code> , <code><space>wS</code> , <code><space>ws</code> , <code><space>w.S</code> , <code><space>w.s</code> , <code><space>w[S</code> , <code><space>w[s</code> , <code><space>w{S</code> , <code><space>w{s</code> , <code><space>w<C-s></code> , <code><space>w.<minus></code> , <code><space>w[<minus></code> , <code><space>w{<minus></code> , insert: <code><C-x>2</code> vim — normal: <code><C-w>S</code> , <code><C-w>s</code> , <code><C-w>.S</code> , <code><C-w>.s</code> , <code><C-w>[S</code> , <code><C-w>[s</code> , <code><C-w>{S</code> , <code><C-w>{s</code> , <code><C-w><C-s></code> , <code><C-w>.<minus></code> , <code><C-w>[<minus></code> , <code><C-w>{<minus></code> helix — normal: <code><C-w>s</code> , <code><space>ws</code> , <code><C-w><C-s></code> , <code><space>w<C-s></code> , select: <code><C-w>s</code> , <code><space>ws</code> , <code><C-w><C-s></code> , <code><space>w<C-s></code> emacs — normal: <code><C-x>2</code> , insert: <code><C-x>2</code> cua — normal: <code><C-x>2</code> , select: <code><C-X>2</code> , insert: <code><C-x>2</code>
<code>hsplit_new</code>	Horizontal bottom split scratch buffer	spacemacs — normal: <code><C-w>n</code> , <code><space>wn</code> , <code><C-w><C-n></code> , <code><space>bNj</code> , <code><space>bNk</code> , <code><space>w<C-n></code> , select: <code><space>wn</code> , <code><space>bNj</code> , <code><space>bNk</code> , <code><space>w<C-n></code> vim — normal: <code><C-w>n</code> , <code><C-w><C-n></code> helix — normal: <code><C-w>ns</code> , <code><space>wns</code> , <code><C-w>n<C-s></code> , <code><space>wn<C-s></code> , select: <code><C-w>ns</code> , <code><space>wns</code> , <code><C-w>n<C-s></code> , <code><space>wn<C-s></code>

Name	Description	Keybinds
<code>vsplit</code>	Vertical right split	spacemacs — normal: <code><C-w>/</code>, <code><C-w>2</code>, <code><C-w>V</code>, <code><C-w>v</code>, <code><C-x>3</code>, <code><C-w>./</code>, <code><C-w> .V</code>, <code><C-w> .v</code>, <code><C-w>[/</code>, <code><C-w>[V</code>, <code><C-w>[v</code>, <code><C-w>{/</code>, <code><C-w>{V</code>, <code><C-w>{v</code>, <code><space>w/</code>, <code><space>w2</code>, <code><space>wV</code>, <code><space>wv</code>, <code><C-w><C-v></code>, <code><space>w . /</code>, <code><space>w . V</code>, <code><space>w . v</code>, <code><space>w[/</code>, <code><space>w[V</code>, <code><space>w[v</code>, <code><space>w{/</code>, <code><space>w{V</code>, <code><space>w{v</code>, <code><space>w<C-v></code>, <code><space>u<space>w2</code>, select: <code><C-x>3</code>, <code><space>w/</code>, <code><space>w2</code>, <code><space>wV</code>, <code><space>wv</code>, <code><space>w . /</code>, <code><space>w . V</code>, <code><space>w . v</code>, <code><space>w[/</code>, <code><space>w[V</code>, <code><space>w[v</code>, <code><space>w{/</code>, <code><space>w{V</code>, <code><space>w{v</code>, <code><space>w<C-v></code>, <code><space>u<space>w2</code>, insert: <code><C-x>3</code>vim — normal: <code><C-w>/</code>, <code><C-w>2</code>, <code><C-w>V</code>, <code><C-w>v</code>, <code><C-w>./</code>, <code><C-w> .V</code>, <code><C-w> .v</code>, <code><C-w>[/</code>, <code><C-w>[V</code>, <code><C-w>[v</code>, <code><C-w>{/</code>, <code><C-w>{V</code>, <code><C-w>{v</code>, <code><C-w><C-v></code>helix — normal: <code><C-w>v</code>, <code><space>wv</code>, <code><C-w><C-v></code>, <code><space>w<C-v></code>, select: <code><C-w>v</code>, <code><space>wv</code>, <code><C-w><C-v></code>, <code><space>w<C-v></code>emacs — normal: <code><C-x>3</code>, insert: <code><C-x>3</code>cua — normal: <code><C-x>3</code>, select: <code><C-X>3</code>, insert: <code><C-x>3</code>
<code>vsplit_new</code>	Vertical right split scratch buffer	spacemacs — normal: <code><space>bNh</code>, <code><space>bNl</code>, select: <code><space>bNh</code>, <code><space>bNl</code>helix — normal: <code><C-w>nv</code>, <code><space>wnv</code>, <code><C-w>n<C-v></code>, <code><space>wn<C-v></code>, select: <code><C-w>nv</code>, <code><space>wnv</code>, <code><C-w>n<C-v></code>, <code><space>wn<C-v></code>

Name	Description	Keybinds
<code>wclose</code>	Close window	spacemacs — normal: <code><C-w>D</code>, <code><C-w>c</code>, <code><C-w>q</code>, <code><C-x>0</code>, <code><C-w>.d</code>, <code><C-w>[d</code>, <code><C-w>{d</code>, <code><space>cd</code>, <code><space>wD</code>, <code><space>wd</code>, <code><space>wq</code>, <code><C-w><C-d></code>, <code><C-w><C-q></code>, <code><space>w.d</code>, <code><space>w[d</code>, <code><space>w{d</code>, <code><space>w<C-d></code>, <code><space>w<C-q></code>, <code><space>u<space>wd</code>, select: <code><C-x>0</code>, <code><space>cd</code>, <code><space>wD</code>, <code><space>wd</code>, <code><space>wq</code>, <code><space>w.d</code>, <code><space>w[d</code>, <code><space>w{d</code>, <code><space>w<C-d></code>, <code><space>w<C-q></code>, <code><space>u<space>wd</code>, insert: <code><C-x>0</code> vim — normal: <code><C-w>D</code>, <code><C-w>c</code>, <code><C-w>q</code>, <code><C-w>.d</code>, <code><C-w>[d</code>, <code><C-w>{d</code>, <code><C-w><C-d></code>, <code><C-w><C-q></code> helix — normal: <code><C-w>q</code>, <code><space>wq</code>, <code><C-w><C-q></code>, <code><space>w<C-q></code>, select: <code><C-w>q</code>, <code><space>wq</code>, <code><C-w><C-q></code>, <code><space>w<C-q></code> emacs — normal: <code><C-x>0</code>, insert: <code><C-x>0</code> cua — normal: <code><C-x>0</code>, select: <code><C-X>0</code>, insert: <code><C-x>0</code>

Name	Description	Keybinds
<code>wonly</code>	Close windows except current	spacemacs — normal: <code><C-w>1</code> , <code><C-w>_</code> , <code><C-w>m</code> , <code><C-w>o</code> , <code><C-x>1</code> , <code><C-w>.D</code> , <code><C-w>._</code> , <code><C-w>.m</code> , <code><C-w>[D</code> , <code><C-w>[_</code> , <code><C-w>[m</code> , <code><C-w>\ </code> , <code><C-w>{D</code> , <code><C-w>{ _</code> , <code><C-w>{m</code> , <code><C-w>._ </code> , <code><C-w>[_ </code> , <code><C-w>{\ </code> , <code><space>w1</code> , <code><space>w_</code> , <code><space>wm</code> , <code><space>wo</code> , <code><C-w><C-_></code> , <code><C-w><C-o></code> , <code><space>w.D</code> , <code><space>w._</code> , <code><space>w.m</code> , <code><space>w[D</code> , <code><space>w[_</code> , <code><space>w[m</code> , <code><space>w\ </code> , <code><space>w{D</code> , <code><space>w{ _</code> , <code><space>w{m</code> , <code><space>w._ </code> , <code><space>w{\ </code> , <code><space>w{ _ </code> , <code><space>w{ \ </code> , <code><space>w<C-o></code> , <code><space>u<space>w1</code> , select: <code><C-x>1</code> , <code><space>w1</code> , <code><space>w_</code> , <code><space>wm</code> , <code><space>wo</code> , <code><space>w.D</code> , <code><space>w._</code> , <code><space>w.m</code> , <code><space>w[D</code> , <code><space>w[_</code> , <code><space>w[m</code> , <code><space>w\ </code> , <code><space>w{D</code> , <code><space>w{ _</code> , <code><space>w{m</code> , <code><space>w._ </code> , <code><space>w{\ </code> , <code><space>w{ _ </code> , <code><space>w{ \ </code> , <code><space>w<C-o></code> , <code><space>u<space>w1</code> , insert: <code><C-x>1</code> vim — normal: <code><C-w>1</code> , <code><C-w>_</code> , <code><C-w>m</code> , <code><C-w>o</code> , <code><C-w>.D</code> , <code><C-w>._</code> , <code><C-w>.m</code> , <code><C-w>[D</code> , <code><C-w>[_</code> , <code><C-w>[m</code> , <code><C-w>\ </code> , <code><C-w>{D</code> , <code><C-w>{ _</code> , <code><C-w>{m</code> , <code><C-w>._ </code> , <code><C-w>[_ </code> , <code><C-w>\ </code> , <code><C-w>{\ </code> , <code><C-w><C-_></code> , <code><C-w><C-o></code> helix — normal: <code><C-w>o</code> , <code><space>wo</code> , <code><C-w><C-o></code> , <code><space>w<C-o></code> , select: <code><C-w>o</code> , <code><space>wo</code> , <code><C-w><C-o></code> , <code><space>w<C-o></code> emacs — normal: <code><C-x>1</code> , insert: <code><C-x>1</code> cua — normal: <code><C-x>1</code> , select: <code><C-X>1</code> , insert: <code><C-x>1</code>
<code>select_register</code>	Select register	spacemacs , vim , helix — normal: <code>"</code> , select: <code>"</code>
<code>insert_register</code>	Insert register	spacemacs , vim , helix — insert: <code><C-r></code>
<code>view_register</code>	Show a register's contents (emacs view-register, C-x r v)	
<code>not_modified</code>	Mark the buffer unmodified without saving (emacs not-modified, M-~)	spacemacs — normal: <code><A--></code>
<code>insert_char_by_code</code>	Insert a character by Unicode code point (emacs insert-char, C-x 8 RET)	
<code>backward_delete_char_untabify</code>	Delete backward, expanding a tab into spaces first (emacs backward-delete-char-untabify)	
<code>insert_last_inserted_text</code>	Insert the previously inserted text (vim i_CTRL-A)	spacemacs , vim — insert: <code><C-a></code>
<code>insert_command_normal</code>	Run one Normal-mode command, then return to Insert (vim i_CTRL-O)	spacemacs , vim — insert: <code><C-o></code>
<code>insert_last_inserted_and_stop</code>	Insert previously inserted text and stop insert (vim i_CTRL-@)	spacemacs , vim — insert: <code><C-@></code>

Name	Description	Keybinds
<code>copy_between_registers</code>	Copy between two registers	
<code>copy_to_register</code>	Copy the region into a register (emacs copy-to-register, C-x r s)	spacemacs — normal: <code><[<-x>r s</code> , select: <code><[<-x>r s</code> , insert: <code><[<-x>r s</code>
<code>append_to_register</code>	Append the region to a register (emacs append-to-register, C-x r a)	
<code>prepend_to_register</code>	Prepend the region to a register (emacs prepend-to-register, C-x r p)	
<code>align_view_middle</code>	Align view middle	helix — normal: <code>Zm</code> , <code>zm</code> , select: <code>Zm</code> , <code>zm</code>
<code>align_view_top</code>	Align view top	spacemacs , vim — normal: <code>zt</code> , select: <code>zt</code> helix — normal: <code>Zt</code> , <code>zt</code> , select: <code>Zt</code> , <code>zt</code>
<code>align_view_center</code>	Align view center	spacemacs — normal: <code>zz</code> , <code><[<-l></code> , <code><[<-w>.z</code> , <code><[<-w>[z</code> , <code><[<-w>{z</code> , <code><space>b.z</code> , <code><space>w.z</code> , <code><space>w[z</code> , <code><space>w{z</code> , <code><space>w{z</code> , select: <code>zz</code> , <code><space>b.z</code> , <code><space>w.z</code> , <code><space>w[z</code> , <code><space>w{z</code> , <code><space>w{z</code> , <code><space>w{z</code> — normal: <code>zz</code> , <code><[<-w>.z</code> , <code><[<-w>[z</code> , <code><[<-w>{z</code> , select: <code>zz</code> , <code>zc</code> , <code>Zz</code> , <code>zc</code> , <code>zz</code> , select: <code>Zc</code> , <code>Zz</code> , <code>zc</code> , <code>zz</code> emacs , cua — insert: <code><[<-l></code>
<code>align_view_bottom</code>	Align view bottom	spacemacs , vim — normal: <code>zb</code> , select: <code>zb</code> helix — normal: <code>Zb</code> , <code>zb</code> , select: <code>Zb</code> , <code>zb</code>
<code>scroll_up</code>	Scroll view up	spacemacs — normal: <code><[<-y></code> , <code>vim</code> — normal: <code><[<-y></code> , insert: <code><[<-x><[<-y></code> helix — normal: <code>Zk</code> , <code>zk</code> , <code>Z<up></code> , <code>z<up></code> , select: <code>Zk</code> , <code>zk</code> , <code>Z<up></code> , <code>z<up></code>
<code>scroll_down</code>	Scroll view down	spacemacs — normal: <code><[<-e></code> , <code>vim</code> — normal: <code><[<-e></code> , insert: <code><[<-x><[<-e></code> helix — normal: <code>Zj</code> , <code>zj</code> , <code>Z<down></code> , <code>z<down></code> , select: <code>Zj</code> , <code>zj</code> , <code>Z<down></code> , <code>z<down></code>
<code>scroll_column_left</code>	Scroll view left one column (zh)	spacemacs , vim — normal: <code>zh</code> , <code>z<left></code> , select: <code>zh</code> , <code>z<left></code>
<code>scroll_column_right</code>	Scroll view right one column (zl)	spacemacs , vim — normal: <code>zl</code> , <code>z<right></code> , select: <code>zl</code> , <code>z<right></code>
<code>scroll_half_column_left</code>	Scroll view left half a screen (zH)	spacemacs — normal: <code>zH</code> , <code><[<-x><gt;</code> , select: <code>zH</code> , <code><[<-x><gt;</code> , insert: <code><[<-x><gt;</code> vim — normal: <code>zH</code> , select: <code>zH</code>
<code>scroll_half_column_right</code>	Scroll view right half a screen (zL)	spacemacs — normal: <code>zL</code> , <code><[<-x><lt;</code> , select: <code>zL</code> , <code><[<-x><lt;</code> , insert: <code><[<-x><lt;</code> vim — normal: <code>zL</code> , select: <code>zL</code>
<code>scroll_cursor_to_left_edge</code>	Scroll so the cursor sits at the left edge (zs)	spacemacs , vim — normal: <code>zs</code> , select: <code>zs</code>
<code>scroll_cursor_to_right_edge</code>	Scroll so the cursor sits at the right edge (ze)	spacemacs , vim — normal: <code>ze</code> , select: <code>ze</code>

Name	Description	Keybinds
<code>resize_view_wider</code>	Make current window wider (CTRL-W >)	spacemacs — normal: <[<-x>], <[<-w>.<], <[<-w>[<], <[<-w>{<], <[<-w><gt;>, <[<-w>.<gt;>, <[<-w>[<gt;>, <[<-w>{<gt;>, <space>w.<], <space>w[<], <space>w{<], <space>w<gt;>, <space>w.<gt;>, <space>w[<gt;>, <space>w{<gt;>, select: <[<-x>], <space>w.<], <space>w[<], <space>w{<], <space>w<gt;>, <space>w.<gt;>, <space>w[<gt;>, <space>w{<gt;>, insert: <[<-x>] vim — normal: <[<-w>.<], <[<-w>[<], <[<-w>{<], <[<-w><gt;>, <[<-w>.<gt;>, <[<-w>[<gt;>, <[<-w>{<gt;> emacs — normal: <[<-x>], insert: <[<-x>] cua — normal: <[<-x>], select: <[<-X>], insert: <[<-x>]
<code>resize_view_narrower</code>	Make current window narrower (CTRL-W <)	spacemacs — normal: <[<-x>{<], <[<-w>.<], <[<-w>[<], <[<-w>{<], <[<-w><lt;>, <[<-w>.<lt;>, <[<-w>[<lt;>, <[<-w>{<lt;>, <space>w.<], <space>w[<], <space>w{<], <space>w<lt;>, <space>w.<lt;>, <space>w[<lt;>, <space>w{<lt;>, select: <[<-x>{<], <space>w.<], <space>w[<], <space>w{<], <space>w<lt;>, <space>w.<lt;>, <space>w[<lt;>, <space>w{<lt;>, insert: <[<-x>{ vim — normal: <[<-w>.<], <[<-w>[<], <[<-w>{<], <[<-w><lt;>, <[<-w>.<lt;>, <[<-w>[<lt;>, <[<-w>{<lt;> emacs — normal: <[<-x>{<], insert: <[<-x>{ cua — normal: <[<-x>{<], select: <[<-X>{<], insert: <[<-x>{<]
<code>resize_view_taller</code>	Make current window taller (CTRL-W +)	spacemacs — normal: <[<-w>+<], <[<-x>^<], <[<-w>.<], <[<-w>[<], <[<-w>{<], <space>w+<], <space>w.<], <space>w[<], <space>w{<], select: <[<-x>^<], <space>w+<], <space>w.<], <space>w[<], <space>w{<], insert: <[<-x>^<] vim — normal: <[<-w>+<], <[<-w>.<], <[<-w>[<], <[<-w>{<] emacs — normal: <[<-x>^<], insert: <[<-x>^< cua — normal: <[<-x>^<], select: <[<-X>^<], insert: <[<-x>^<]
<code>resize_view_shorter</code>	Make current window shorter (CTRL-W -)	spacemacs — normal: <[<-w>.<{<], <[<-w>[<{<], <[<-w>{<{<], <space>w.<{<], <space>w[<{<], <space>w{<{<], <[<-w><minus>>, <space>w<minus>>, select: <space>w.<{<], <space>w[<{<], <space>w{<{<], <space>w<minus>> vim — normal: <[<-w>.<{<], <[<-w>[<{<], <[<-w>{<{<]

Name	Description	Keybinds
<code>resize_view_equalize</code>	Make all windows equal size (CTRL-W =)	spacemacs — normal: <code><[-w>=</code> , <code><[-x>+</code> , <code><[-w>.=</code> , <code><[-w>[=</code> , <code><[-w>{=</code> , <code><space>w=</code> , <code><space>w.=</code> , <code><space>w[=</code> , <code><space>w{=</code> , select: <code><[-x>+</code> , <code><space>w=</code> , <code><space>w.=</code> , <code><space>w[=</code> , <code><space>w{=</code> , insert: <code><[-x>+vim</code> — normal: <code><[-w>=</code> , <code><[-w>.=</code> , <code><[-w>[=</code> , <code><[-w>{=</code> emacs — normal: <code><[-x>+</code> , insert: <code><[-x>+cua</code> — normal: <code><[-x>+</code> , select: <code><[-X>+</code> , insert: <code><[-x>+</code>
<code>golden_ratio_resize</code>	Resize the focused window to the golden ratio (SPC t g)	spacemacs — normal: <code><[-w> .g</code> , <code><[-w>[g</code> , <code><[-w>{g</code> , <code><space>t g</code> , <code><space>w .g</code> , <code><space>w[g</code> , <code><space>w{g</code> , select: <code><space>t g</code> , <code><space>w .g</code> , <code><space>w[g</code> , <code><space>w{g</code> vim — normal: <code><[-w> .g</code> , <code><[-w>[g</code> , <code><[-w>{g</code>
<code>rot13</code>	ROT13-encode the selection (g?)	
<code>check_parens</code>	Move to the first unbalanced bracket, or report all balanced (check-parens)	
<code>url_encode</code>	Percent-encode (URL-encode) the selection	
<code>url_decode</code>	Percent-decode (URL-decode) the selection	
<code>parse_query_selection</code>	Expand a URL query string into decoded key=value lines	
<code>build_query_selection</code>	Build a URL query string from key=value lines	
<code>url_info_selection</code>	Break the selected URL into scheme/host/port/path/query lines	
<code>encode_base64</code>	Base64-encode the selection	
<code>decode_base64</code>	Base64-decode the selection	
<code>encode_base64url</code>	URL-safe base64-encode the selection (no padding)	
<code>decode_base64url</code>	URL-safe base64-decode the selection (JWT-friendly)	
<code>jwt_decode_selection</code>	Decode the selected JWT into pretty header + payload JSON	
<code>encode_html</code>	HTML-escape the selection (& < > " ')	
<code>decode_html</code>	Decode HTML entities in the selection	
<code>html_to_text_selection</code>	Strip HTML tags and decode entities to plain text	
<code>title_case_selection</code>	Title-case the selection (capitalize each word)	
<code>sentence_case_selection</code>	Capitalize the first letter of each sentence in the selection	
<code>straighten_quotes_selection</code>	Convert smart quotes/dashes in the selection to plain ASCII	
<code>hex_to_rgb_selection</code>	Convert a #hex color in the selection to rgb(r, g, b)	
<code>rgb_to_hex_selection</code>	Convert an rgb(r, g, b) color in the selection to #hex	
<code>to_roman_selection</code>	Convert the selected integer to a Roman numeral	

Name	Description	Keybinds
<code>from_roman_selection</code>	Convert the selected Roman numeral to an integer	
<code>add_commas_selection</code>	Add thousands separators to numbers in the selection	
<code>strip_commas_selection</code>	Remove thousands separators from numbers in the selection	
<code>swap_quotes_selection</code>	Swap ' and " quote characters in the selection	
<code>strip_quotes_selection</code>	Remove surrounding quotes from the selection	
<code>reverse_words_selection</code>	Reverse the word order within each selected line	
<code>unwrap_tag_selection</code>	Strip the outermost ... wrapper from the selection	
<code>sort_paragraphs_selection</code>	Sort blank-line-separated paragraphs in the selection	
<code>lighten_selection</code>	Lighten the hex color in the selection by 10%	
<code>darken_selection</code>	Darken the hex color in the selection by 10%	
<code>contrast_text</code>	Recommend black/white text for the selected hex background color	
<code>toggle_value_selection</code>	Toggle the boolean/keyword in the selection (true<->false, ...)	
<code>normalize_whitespace_selection</code>	Collapse internal whitespace runs in the selection	
<code>insert_toc</code>	Insert a markdown table of contents from the buffer's headings	
<code>slugify_selection</code>	Slugify the selection (lowercase, hyphen-separated)	
<code>humanize_selection</code>	Humanize a slug/identifier into a Title-Cased label	
<code>transpose_csv_selection</code>	Transpose the selected CSV/TSV table (rows <-> columns)	
<code>csv_to_json_selection</code>	Convert the selected CSV/TSV to a JSON array of objects	
<code>regex_escape_selection</code>	Escape regex metacharacters in the selection	
<code>blockquote_selection</code>	Prefix each selected line with ">" (markdown blockquote)	
<code>unblockquote_selection</code>	Strip a leading ">" from each selected line	
<code>bullet_list_selection</code>	Make a markdown bullet list from the selected lines	
<code>unbullet_selection</code>	Strip a leading bullet (- * +) from each selected line	
<code>strip_ansi_selection</code>	Strip ANSI/VT escape codes from the selection	
<code>html_escape_selection</code>	HTML-escape the selection (& < > " ' to entities)	
<code>html_unescape_selection</code>	HTML-unescape entities in the selection back to characters	
<code>reverse_chars_selection</code>	Reverse the characters in the selection	
<code>json_escape_selection</code>	JSON-escape the selection (for a string literal)	
<code>to_json_string_selection</code>	Wrap the selection in quotes as a JSON string literal	
<code>json_unescape_selection</code>	JSON-unescape the selection	

Name	Description	Keybinds
<code>to_hex_selection</code>	Encode the selection as hex bytes	
<code>from_hex_selection</code>	Decode hex bytes in the selection back to text	
<code>format_table_selection</code>	Align the selected markdown table's columns	
<code>csv_to_table_selection</code>	Convert the selected CSV/TSV to a markdown table	
<code>table_to_csv_selection</code>	Convert the selected markdown table to CSV	
<code>json_pretty_selection</code>	Pretty-print the selected JSON (preserves key order)	
<code>json_minify_selection</code>	Minify the selected JSON	
<code>xml_pretty_selection</code>	Pretty-print the selected XML/HTML	
<code>insert_digraph</code>	Insert a digraph by two-character mnemonic (CTRL-K)	spacemacs, vim — insert: <code><[-k></code>
<code>insert_uuid_v4</code>	Insert a random UUIDv4 (SPC i U 4)	spacemacs — normal: <code><space>iU4,</code> <code><space>iUU</code> , select: <code><space>iU4,</code> <code><space>iUU</code>
<code>insert_uuid_v1</code>	Insert a time-based UUIDv1 (SPC i U 1)	spacemacs — normal: <code><space>iU1,</code> select: <code><space>iU1</code>
<code>insert_lorem_sentence</code>	Insert a lorem-ipsuM sentence (SPC i l s)	spacemacs — normal: <code><space>ils,</code> select: <code><space>ils</code>
<code>insert_lorem_paragraph</code>	Insert a lorem-ipsuM paragraph (SPC i l p)	spacemacs — normal: <code><space>ilp,</code> select: <code><space>ilp</code>
<code>insert_lorem_list</code>	Insert a lorem-ipsuM list (SPC i l l)	spacemacs — normal: <code><space>ill,</code> select: <code><space>ill</code>
<code>insert_password_simple</code>	Insert a simple alphanumeric password (SPC i p 1)	spacemacs — normal: <code><space>ip1,</code> select: <code><space>ip1</code>
<code>insert_password_strong</code>	Insert a stronger password with symbols (SPC i p 2)	spacemacs — normal: <code><space>ip2,</code> select: <code><space>ip2</code>
<code>insert_password_paranoio</code>	Insert a long password for paranoids (SPC i p 3)	spacemacs — normal: <code><space>ip3,</code> select: <code><space>ip3</code>
<code>insert_password_numerical</code>	Insert a numeric password (SPC i p n)	spacemacs — normal: <code><space>ipn,</code> select: <code><space>ipn</code>
<code>insert_password_phonetic</code>	Insert a phonetically easy password (SPC i p p)	spacemacs — normal: <code><space>ipp,</code> select: <code><space>ipp</code>
<code>symbol_upper_camel</code>	Change symbol style to UpperCamelCase (SPC x i C)	spacemacs — normal: <code><space>xiC,</code> select: <code><space>xiC</code>
<code>symbol_up_case</code>	Change symbol style to UP_CASE (SPC x i U)	spacemacs — normal: <code><space>xiU,</code> select: <code><space>xiU</code>
<code>symbol_under_score</code>	Change symbol style to under_score (SPC x i _)	spacemacs — normal: <code><space>xi_,</code> select: <code><space>xi_</code>
<code>symbol_lower_camel</code>	Change symbol style to camelCase (vim-abolish crC)	
<code>symbol_kebab</code>	Change symbol style to kebab-case (vim-abolish cr-)	
<code>symbol_dot</code>	Change symbol style to dot.case (vim-abolish cr.)	spacemacs — normal: <code><space>xi.,</code> select: <code><space>xi.</code>
<code>randomize_lines_in_region</code>	Randomize lines in the selection (SPC x l r)	spacemacs — normal: <code><space>xlr,</code> select: <code><space>xlr</code>
<code>randomize_words_in_region</code>	Randomize words in the selection (SPC x w r)	spacemacs — normal: <code><space>xwr,</code> select: <code><space>xwr</code>
<code>copy_char_below</code>	Insert the character below the cursor (i_CTRL-E)	spacemacs, vim — insert: <code><[-e></code>
<code>copy_char_above</code>	Insert the character above the cursor (i_CTRL-Y)	spacemacs, vim — insert: <code><[-y></code>

Name	Description	Keybinds
<code>toggle_revins</code>	Toggle 'revins', inserting text right-to-left (vim i_CTRL-_)	vim — insert: <code><[-_></code>
<code>page_up_key</code>	Page up, ending Select mode with 'keymodel' stopsel (vim)	spacemacs, vim — normal: <code><pageup></code>
<code>page_down_key</code>	Page down, ending Select mode with 'keymodel' stopsel (vim)	spacemacs, vim — normal: <code><pagedown></code>
<code>home_key</code>	Go to line start, ending Select mode with 'keymodel' stopsel (vim)	spacemacs, vim — normal: <code><home></code>
<code>end_key</code>	Go to line end, ending Select mode with 'keymodel' stopsel (vim)	spacemacs, vim — normal: <code><end></code>
<code>shift_home_key</code>	Select to line start with 'keymodel' startsel, else go there (vim)	spacemacs, vim — normal: <code><S-home></code>
<code>shift_end_key</code>	Select to line end with 'keymodel' startsel, else go there (vim)	spacemacs, vim — normal: <code><S-end></code>
<code>left_key</code>	Move left, ending Select mode with 'keymodel' stopsel (vim)	spacemacs, vim — normal: <code><left></code>
<code>right_key</code>	Move right, ending Select mode with 'keymodel' stopsel (vim)	spacemacs, vim — normal: <code><right></code>
<code>up_key</code>	Move up, ending Select mode with 'keymodel' stopsel (vim)	spacemacs, vim — normal: <code><up></code>
<code>down_key</code>	Move down, ending Select mode with 'keymodel' stopsel (vim)	spacemacs, vim — normal: <code><down></code>
<code>shift_right_key</code>	Select a char right with 'keymodel' startsel, else move a word right (vim)	spacemacs, vim — normal: <code><S-right></code>
<code>shift_left_key</code>	Select a char left with 'keymodel' startsel, else move a word left (vim)	spacemacs, vim — normal: <code><S-left></code>
<code>shift_down_key</code>	Select a line down with 'keymodel' startsel, else page down (vim)	spacemacs, vim — normal: <code><S-down></code>
<code>shift_up_key</code>	Select a line up with 'keymodel' startsel, else page up (vim)	spacemacs, vim — normal: <code><S-up></code>
<code>shift_page_up_key</code>	Extend a page up with 'keymodel' startsel, else page up (vim)	spacemacs, vim — normal: <code><S-pageup></code>
<code>shift_page_down_key</code>	Extend a page down with 'keymodel' startsel, else page down (vim)	spacemacs, vim — normal: <code><S-pagedown></code>
<code>select_left_key</code>	Select-mode : stop Select with 'keymodel' stopsel, else extend left	
<code>select_right_key</code>	Select-mode : stop Select with 'keymodel' stopsel, else extend right	
<code>select_up_key</code>	Select-mode : stop Select with 'keymodel' stopsel, else extend up	
<code>select_down_key</code>	Select-mode : stop Select with 'keymodel' stopsel, else extend down	
<code>select_home_key</code>	Select-mode : stop Select with 'keymodel' stopsel, else extend to line start	
<code>select_end_key</code>	Select-mode : stop Select with 'keymodel' stopsel, else extend to line end	spacemacs, vim — select: <code><end></code>
<code>select_page_up_key</code>	Select-mode : stop Select with 'keymodel' stopsel, else extend a page up	
<code>select_page_down_key</code>	Select-mode : stop Select with 'keymodel' stopsel, else extend a page down	
<code>file_info</code>	Show file name and cursor position (CTRL-G)	spacemacs, vim — normal: <code><[-g></code>
<code>document_stats</code>	Show document line/word/char counts (g CTRL-G)	spacemacs, vim — normal: <code>g<[-g></code> , select: <code>g<[-g></code>

Name	Description	Keybinds
<code>git_blame_line</code>	Show git blame for the current line (g b)	spacemacs — normal: <code><[-x>vg</code> , <code><space>gM</code> , <code><space>gb</code> , select: <code><[-x>vg</code> , <code><space>gM</code> , <code><space>gb</code> , insert: <code><[-x>vg</code>
<code>toggle_inline_blame</code>	Toggle GitLens-style inline blame on the current line	spacemacs — normal: <code><space>gI</code> , select: <code><space>gI</code>
<code>toggle_blame_annotate</code>	Toggle the git-blame annotate gutter column (SPC g B)	spacemacs — normal: <code><space>gB</code> , select: <code><space>gB</code>
<code>git_branch_picker</code>	Pick a git branch and check it out	
<code>preferences</code>	Open the unified Preferences window	spacemacs — normal: <code><space></code> , , select: <code><space></code> , ,
<code>set_selective_display</code>	Hide lines indented past the prefix-arg column; no arg turns it off (emacs set-selective-display, C-x \$)	spacemacs — normal: <code><[-x>\$</code> , select: <code><[-x>\$</code> , insert: <code><[-x>\$</code>
<code>global_whitespace_toggle_options</code>	Toggle rendering of whitespace characters (emacs global-whitespace-toggle-options)	
<code>global_tab_line_mode</code>	Toggle the buffer tab line (emacs global-tab-line-mode)	
<code>global_visual_wrap_prefix_mode</code>	Toggle soft-wrap with indentation carry-over (emacs global-visual-wrap-prefix-mode)	
<code>help</code>	Open the inline Help browser	spacemacs — normal: <code><F1></code> , <code><[-h>?</code> , <code><[-h>F</code> , <code><[-h>I</code> , <code><[-h>K</code> , <code><[-h>I</code> , <code><[-h>4s</code> , <code><space>bH</code> , <code><space>h?</code> , <code><space>hc</code> , <code><space>hh</code> , <code><space>hk</code> , <code><space>hr</code> , <code><[-h><[-h></code> , <code><space>hdb</code> , <code><space>hdk</code> , <code><space>hdx</code> , <code><space>h<space></code> , select: <code><[-h>?</code> , <code><[-h>F</code> , <code><[-h>I</code> , <code><[-h>K</code> , <code><[-h>I</code> , <code><[-h>4s</code> , <code><space>bH</code> , <code><space>h?</code> , <code><space>hc</code> , <code><space>hh</code> , <code><space>hk</code> , <code><space>hr</code> , <code><[-h><[-h></code> , <code><space>hdb</code> , <code><space>hdk</code> , <code><space>hdx</code> , <code><space>h<space></code> , insert: <code><[-h>?</code> , <code><[-h>F</code> , <code><[-h>I</code> , <code><[-h>K</code> , <code><[-h>I</code> , <code><[-h>4s</code> , <code><[-h><[-h></code> vim — normal: <code><F1></code>
<code>dashboard</code>	Open the system-stats Dashboard (Preferences)	spacemacs — normal: <code><space>bh</code> , select: <code><space>bh</code>
<code>search_in_files</code>	Open the project-wide Find in Files panel	
<code>terminal</code>	Open an integrated terminal (PTY shell)	spacemacs — normal: <code><space>p'</code> , select: <code><space>p'</code>
<code>comint_shell</code>	Open a comint line-oriented shell buffer (emacs M-x shell)	
<code>gud_gdb</code>	Run gdb in a comint buffer (emacs gud-gdb)	
<code>gud_up</code>	Select the stack frame one level up (emacs gud-up)	spacemacs — normal: <code><[-c><lt></code> , <code><[-x><[-a><lt></code> , select: <code><[-c><lt></code> , <code><[-x><[-a><lt></code> , insert: <code><[-c><lt></code> , <code><[-x><[-a><lt></code>
<code>gud_down</code>	Select the stack frame one level down (emacs gud-down)	spacemacs — normal: <code><[-c><gt></code> , <code><[-x><[-a><gt></code> , select: <code><[-c><gt></code> , <code><[-x><[-a><gt></code> , insert: <code><[-c><gt></code> , <code><[-x><[-a><gt></code>

Name	Description	Keybinds
<code>gud_stepi</code>	Step one machine instruction (emacs gud-stepi)	spacemacs — normal: <code><[-c]><[-i></code> , <code><[-x]><[-a]><[-i></code> , select: <code><[-c]><[-i></code> , <code><[-x]><[-a]><[-i></code> , insert: <code><[-c]><[-i></code> , <code><[-x]><[-a]><[-i></code>
<code>gud_tbreak</code>	Set a temporary breakpoint at the current line (emacs gud-tbreak)	spacemacs — normal: <code><[-x]><[-a]><[-t></code> , select: <code><[-x]><[-a]><[-t></code> , insert: <code><[-x]><[-a]><[-t></code>
<code>gud_print</code>	Print the expression at point in the debugger (emacs gud-print)	spacemacs — normal: <code><[-c]><[-p></code> , <code><[-x]><[-a]><[-p></code> , select: <code><[-c]><[-p></code> , <code><[-x]><[-a]><[-p></code> , insert: <code><[-c]><[-p></code> , <code><[-x]><[-a]><[-p></code>
<code>gud_watch</code>	Watch the expression at point in the debugger (emacs gud-watch)	spacemacs — normal: <code><[-x]><[-a]><[-w></code> , select: <code><[-x]><[-a]><[-w></code> , insert: <code><[-x]><[-a]><[-w></code>
<code>gud_jump</code>	Set the debugger execution point to the current line (emacs gud-jump)	spacemacs — normal: <code><[-x]><[-a]><[-j></code> , select: <code><[-x]><[-a]><[-j></code> , insert: <code><[-x]><[-a]><[-j></code>
<code>gud_refresh</code>	Redisplay the debugger buffer (emacs gud-refresh)	spacemacs — normal: <code><[-c]><[-l></code> , <code><[-x]><[-a]><[-l></code> , select: <code><[-c]><[-l></code> , <code><[-x]><[-a]><[-l></code> , insert: <code><[-c]><[-l></code> , <code><[-x]><[-a]><[-l></code>
<code>gud_def</code>	Define a named debugger command from a template (emacs gud-def)	
<code>gud_call</code>	Run a gud-def'd command by name in the debugger (emacs gud-call)	
<code>comint_kill_input</code>	Discard the pending comint input line (emacs comint-kill-input)	
<code>comint_bol_or_process_mark</code>	Move to the process mark or beginning of line (emacs comint-bol-or-process-mark)	
<code>comint_delchar_or_maybe_eof</code>	Delete char or send EOF on empty input (emacs comint-delchar-or-maybe-eof)	
<code>comint_magic_space</code>	Expand history designators then insert a space (emacs comint-magic-space)	
<code>comint_insert_previous_argument</code>	Insert the last argument of the previous command (emacs comint-insert-previous-argument)	
<code>comint_get_next_from_history</code>	Yank the next history entry onto the input (emacs comint-get-next-from-history)	
<code>comint_next_prompt</code>	Move to the next command prompt (emacs comint-next-prompt)	
<code>comint_previous_prompt</code>	Move to the previous command prompt (emacs comint-previous-prompt)	
<code>comint_show_output</code>	Scroll the last command's output to the top (emacs comint-show-output)	
<code>comint_show_maximum_output</code>	Scroll the newest output to the bottom (emacs comint-show-maximum-output)	
<code>comint_delete_output</code>	Delete the last command's output (emacs comint-delete-output)	
<code>comint_write_output</code>	Write the last command's output to a file (emacs comint-write-output)	

Name	Description	Keybinds
<code>comint_copy_old_input</code>	Copy the most recent input onto the input line (emacs <code>comint-copy-old-input</code>)	
<code>comint_truncate_buffer</code>	Trim the comint scrollbar to the maximum size (emacs <code>comint-truncate-buffer</code>)	
<code>comint_strip_ctrl_m</code>	Strip carriage returns from the scrollbar (emacs <code>comint-strip-ctrl-m</code>)	
<code>comint_interrupt_subjob</code>	Send SIGINT to the comint child (emacs <code>comint-interrupt-subjob</code>)	
<code>comint_stop_subjob</code>	Suspend the comint child with SIGTSTP (emacs <code>comint-stop-subjob</code>)	
<code>comint_continue_subjob</code>	Resume the comint child with SIGCONT (emacs <code>comint-continue-subjob</code>)	
<code>comint_quit_subjob</code>	Send SIGQUIT to the comint child (emacs <code>comint-quit-subjob</code>)	
<code>comint_kill_subjob</code>	Send SIGKILL to the comint child (emacs <code>comint-kill-subjob</code>)	
<code>comint_dynamic_list_input_ring</code>	List the comint input history (emacs <code>comint-dynamic-list-input-ring</code>)	
<code>comint_dynamic_list_filename</code>	List the file names completing the one before point (emacs <code>comint-dynamic-list-filename-completions</code>)	
<code>comint_send_invisible</code>	Send a non-echoed line (a password) to the process (emacs <code>comint-send-invisible</code>)	
<code>comint_history_isearch_backward_regex</code>	Search the comint input history backward (emacs <code>comint-history-isearch-backward-regex</code>)	
<code>comint_run</code>	Run a program in a new comint buffer (emacs <code>comint-run</code>)	
<code>shell_forward_command</code>	Move forward over a shell command on the input line (emacs <code>shell-forward-command</code>)	
<code>shell_backward_command</code>	Move backward over a shell command on the input line (emacs <code>shell-backward-command</code>)	
<code>run_config_manager</code>	Manage run/debug configurations	spacemacs — normal: <code><space>Rc</code> , <code><space>Re</code> , <code><space>cm</code> , <code><space>pi</code> , select: <code><space>Rc</code> , <code><space>Re</code> , <code><space>cm</code> , <code><space>pi</code>
<code>run_active_config</code>	Run the active run configuration	spacemacs — normal: <code><F5></code> , <code><space>Rr</code> , <code><space>cC</code> , <code><space>pc</code> , <code><space>pu</code> , <code><C-c><C-c></code> , select: <code><space>Rr</code> , <code><space>cC</code> , <code><space>pc</code> , <code><space>pu</code> , <code><C-c><C-c></code> , insert: <code><C-c><C-c></code> vim — normal: <code><F5></code>
<code>clear_run_output</code>	Clear the Run tool window output	spacemacs — normal: <code><space>Rl</code> , <code><space>Rx</code> , <code><space>ck</code> , select: <code><space>Rl</code> , <code><space>Rx</code> , <code><space>ck</code>
<code>rerun_last_run</code>	Re-run the last command in the Run console	spacemacs — normal: <code><space>RR</code> , <code><space>cr</code> , <code><C-c><C-r></code> , select: <code><space>RR</code> , <code><space>cr</code> , <code><C-c><C-r></code> , insert: <code><C-c><C-r></code>
<code>run_next_error</code>	Jump to the next file:line in the run output	spacemacs — normal: <code><A-g>n</code> , <code><C-x>`</code> , <code><space>Rn</code> , <code><A-g><A-n></code> , select: <code><C-x>`</code> , <code><space>Rn</code> , insert: <code><C-x>`</code>

Name	Description	Keybinds
<code>run_prev_error</code>	Jump to the previous file:line in the run output	spacemacs — normal: <code><A-g>p</code> , <code><space>Rp</code> , <code><A-g><A-p></code> , select: <code><space>Rp</code>
<code>reveal_in_tree</code>	Reveal the current file in the project tree	
<code>toggle_auto_reveal</code>	Toggle always-select-opened-file (autoscroll from source)	spacemacs — normal: <code><space>pV</code> , select: <code><space>pV</code>
<code>focus_file_tree</code>	Focus the project file tree panel	spacemacs — normal: <code><space>Wp</code> , <code><space>Wt</code> , select: <code><space>Wp</code> , <code><space>Wt</code>
<code>focus_structure</code>	Focus the structure/symbol outline panel	spacemacs — normal: <code><space>Wo</code> , <code><space>Ws</code> , select: <code><space>Wo</code> , <code><space>Ws</code>
<code>hide_active_tool_window</code>	Return focus to the editor, hiding the active tool window (JetBrains Shift-Esc)	spacemacs — normal: <code><space>Wq</code> , select: <code><space>Wq</code>
<code>jump_to_last_tool_window</code>	Toggle focus between the editor and the last tool window (JetBrains F12)	spacemacs — normal: <code><space>W<tab></code> , select: <code><space>W<tab></code>
<code>focus_bookmarks</code>	Focus the Bookmarks tool window (pinned files; JetBrains Bookmarks)	spacemacs — normal: <code><space>Wb</code> , select: <code><space>Wb</code>
<code>focus_marks_panel</code>	Focus the Marks tool window	spacemacs — normal: <code><space>Wk</code> , select: <code><space>Wk</code>
<code>focus_registers_panel</code>	Focus the Registers tool window	spacemacs — normal: <code><space>WR</code> , select: <code><space>WR</code>
<code>focus_jumplist_panel</code>	Focus the Jumplist tool window	spacemacs — normal: <code><space>Wj</code> , select: <code><space>Wj</code>
<code>focus_recent_panel</code>	Focus the Recent Files tool window	spacemacs — normal: <code><space>Wu</code> , select: <code><space>Wu</code>
<code>focus_todo_panel</code>	Focus the TODO tool window	spacemacs — normal: <code><space>Wd</code> , select: <code><space>Wd</code>
<code>focus_problems</code>	Focus the problems/diagnostics panel	spacemacs — normal: <code><space>We</code> , select: <code><space>We</code>
<code>focus_run_console</code>	Focus the Run console (scroll output with j/k/PgUp/PgDn)	spacemacs — normal: <code><space>Wr</code> , select: <code><space>Wr</code>
<code>focus_git_panel</code>	Focus the Git changes panel (j/k select, Enter opens)	spacemacs — normal: <code><space>Wg</code> , <code><space>gG</code> , select: <code><space>Wg</code> , <code><space>gG</code>
<code>focus_ci_panel</code>	Focus the CI status panel (GitHub Actions runs; Enter opens in browser)	spacemacs — normal: <code><space>Wc</code> , select: <code><space>Wc</code>
<code>toggle_bottom_zoom</code>	Maximize / restore the bottom panel	spacemacs — normal: <code><space>Wm</code> , select: <code><space>Wm</code>
<code>toggle_drawer_mid</code>	Fold / unfold the middle column of the bottom drawer	spacemacs — normal: <code><space>Wf</code> , select: <code><space>Wf</code>
<code>toggle_ide</code>	Toggle the IDE workbench (Zen / focus mode)	spacemacs — normal: <code><space>Wz</code> , <code><space>zz</code> , select: <code><space>Wz</code> , <code><space>zz</code>
<code>settings_page</code>	Open the settings page (config.toml editor)	spacemacs — normal: <code><space>S</code> , select: <code><space>S</code>
<code>goto_next_spell_error</code>	Move to the next misspelled word (Js)	spacemacs, vim — normal: <code>]s</code>
<code>goto_prev_spell_error</code>	Move to the previous misspelled word (Is)	spacemacs, vim — normal: <code>[s</code>
<code>spell_add_good</code>	Mark word under cursor as correctly spelled (zg)	spacemacs, vim — normal: <code>zg</code>
<code>spell_add_bad</code>	Mark word under cursor as misspelled (zw)	spacemacs, vim — normal: <code>zw</code>
<code>spell_undo</code>	Undo a zg/zw for the word under cursor (zug)	spacemacs, vim — normal: <code>zug</code> , <code>zuw</code>
<code>spell_add_good_internal</code>	Mark word under cursor good for this session only (zG)	spacemacs, vim — normal: <code>zG</code>

Name	Description	Keybinds
<code>spell_add_bad_internal</code>	Mark word under cursor misspelled for this session only (zW)	spacemacs, vim — normal: zW
<code>spell_undo_internal</code>	Undo a zG/zW for the word under cursor (zuG)	spacemacs, vim — normal: zuG , zuW
<code>spell_suggest</code>	Show spelling suggestions for the word under cursor (z=)	spacemacs, vim — normal: z=
<code>ispell_word</code>	Spell-check the word at point with aspell/hunspell (emacs ispell-word, M-\$)	spacemacs — normal: <A-\$>
<code>flyspell_auto_correct_word</code>	Correct the word at point with the top suggestion (emacs flyspell-auto-correct-word)	
<code>view_file</code>	Open a file read-only for viewing (emacs view-file, C-x C-r)	
<code>view_buffer</code>	Make the current buffer read-only for viewing (emacs view-buffer)	
<code>view_buffer_other_window</code>	Show the current buffer read-only in a new split (emacs view-buffer-other-window)	
<code>ispell_region</code>	Spell-check the selection with an external speller (emacs ispell-region)	
<code>ispell_buffer</code>	Spell-check the whole buffer with an external speller (emacs ispell-buffer)	
<code>ispell_continue</code>	Continue the halted spell-check from the word at point (emacs ispell-continue)	
<code>ispell_message</code>	Spell-check a mail message body, skipping headers/citations/signature (emacs ispell-message)	
<code>ispell</code>	Spell-check the region or buffer with an external speller (emacs ispell)	
<code>ispell_change_dictionary</code>	Set the ispell dictionary/language (emacs ispell-change-dictionary)	
<code>flyspell_buffer</code>	Check the whole buffer with the wordlist speller and move to the first misspelling (emacs flyspell-buffer)	
<code>flyspell_region</code>	Check the selection with the wordlist speller (emacs flyspell-region)	
<code>flyspell_word</code>	Check the word at point with the wordlist speller (emacs flyspell-word)	
<code>flyspell_check_previous_highlighted_word</code>	Move to the previous misspelled word before point (emacs flyspell-check-previous-highlighted-word)	
<code>flyspell_goto_next_error</code>	Move to the next misspelled word (emacs flyspell-goto-next-error)	
<code>flyspell_mode</code>	Toggle on-the-fly spell checking (emacs flyspell-mode)	spacemacs — normal: <space>t5 , select: <space>t5
<code>flyspell_prog_mode</code>	Toggle on-the-fly spell checking of comments and strings (emacs flyspell-prog-mode)	
<code>outline_next_visible_heading</code>	Move to the next outline heading (emacs outline-next-visible-heading)	spacemacs — normal: <[- c>@<[- n> , select: <[- c>@<[- n> , insert: <[- c>@<[- n>
<code>outline_previous_visible_heading</code>	Move to the previous outline heading (emacs outline-previous-visible-heading)	spacemacs — normal: <[- c>@<[- p> , select: <[- c>@<[- p> , insert: <[- c>@<[- p>
<code>outline_up_heading</code>	Move to the parent outline heading (emacs outline-up-heading)	spacemacs — normal: <[- c>@<[- u> , select: <[- c>@<[- u> , insert: <[- c>@<[- u>

Name	Description	Keybinds
<code>outline_forward_same_level</code>	Move to the next same-level heading (emacs outline-forward-same-level)	spacemacs — normal: <code><[- c>@<[- f></code> , select: <code><[- c>@<[- f></code> , insert: <code><[- c>@<[- f></code>
<code>outline_backward_same_level</code>	Move to the previous same-level heading (emacs outline-backward-same-level)	spacemacs — normal: <code><[- c>@<[- b></code> , select: <code><[- c>@<[- b></code> , insert: <code><[- c>@<[- b></code>
<code>outline_hide_subtree</code>	Fold the subtree of the heading at point (emacs outline-hide-subtree)	spacemacs — normal: <code><[- c>@<[- d></code> , select: <code><[- c>@<[- d></code> , insert: <code><[- c>@<[- d></code>
<code>outline_show_subtree</code>	Reveal the subtree of the heading at point (emacs outline-show-subtree)	
<code>outline_hide_entry</code>	Fold this heading's body (emacs outline-hide-entry)	
<code>outline_show_entry</code>	Reveal this heading's body (emacs outline-show-entry)	spacemacs — normal: <code><[- c>@<[- e></code> , select: <code><[- c>@<[- e></code> , insert: <code><[- c>@<[- e></code>
<code>outline_hide_body</code>	Fold all bodies, showing only headings (emacs outline-hide-body)	spacemacs — normal: <code><[- c>@<[- t></code> , select: <code><[- c>@<[- t></code> , insert: <code><[- c>@<[- t></code>
<code>outline_show_all</code>	Reveal all outline body text (emacs outline-show-all)	spacemacs — normal: <code><[- c>@<[- a></code> , select: <code><[- c>@<[- a></code> , insert: <code><[- c>@<[- a></code>
<code>outline_hide_sublevels</code>	Show only the top N levels of headings (emacs outline-hide-sublevels)	spacemacs — normal: <code><[- c>@<[- q></code> , select: <code><[- c>@<[- q></code> , insert: <code><[- c>@<[- q></code>
<code>outline_hide_leaves</code>	Fold bodies in the current subtree, keeping subheadings (emacs outline-hide-leaves)	
<code>outline_show_children</code>	Reveal the immediate subheadings of the heading at point (emacs outline-show-children)	spacemacs — normal: <code><[- c>@<[- i></code> , select: <code><[- c>@<[- i></code> , insert: <code><[- c>@<[- i></code>
<code>outline_show_branches</code>	Reveal every subheading in the subtree at point (emacs outline-show-branches)	spacemacs — normal: <code><[- c>@<[- k></code> , select: <code><[- c>@<[- k></code> , insert: <code><[- c>@<[- k></code>
<code>outline_cycle</code>	Cycle the heading at point folded -> children -> subtree (emacs outline-cycle)	
<code>outline_cycle_buffer</code>	Cycle the whole buffer show-all -> overview -> contents (emacs outline-cycle-buffer)	
<code>fold_create</code>	Create a fold over the selection (zf)	
<code>fold_toggle</code>	Toggle fold under cursor (za)	spacemacs — normal: <code>zA, za, zi</code> , <code><[- c>@<[- c></code> , select: <code><[- c>@<[- c></code> , insert: <code><[- c>@<[- c></code> vim — normal: <code>zA, za, zi</code>
<code>fold_open</code>	Open fold under cursor (zo)	spacemacs — normal: <code>zo, zv, zx</code> , <code><[- c>@<[- r></code> , <code><[- c>@<[- s></code> , select: <code><[- c>@<[- r></code> , <code><[- c>@<[- s></code> , insert: <code><[- c>@<[- r></code> , <code><[- c>@<[- s></code> vim — normal: <code>zo, zv, zx</code>
<code>fold_close</code>	Close fold under cursor (zc)	spacemacs — normal: <code>zC, zc</code> , <code><[- c>@<[- h></code> , select: <code><[- c>@<[- h></code> , insert: <code><[- c>@<[- h></code> vim — normal: <code>zC, zc</code>
<code>fold_open_recursive</code>	Open fold under cursor and all nested folds (IntelliJ Expand Recursively)	spacemacs, vim — normal: <code>zO</code>
<code>fold_close_recursive</code>	Close fold under cursor and all nested folds (IntelliJ Collapse Recursively)	

Name	Description	Keybinds
<code>fold_open_all</code>	Open all folds (zR)	spacemacs — normal: zR , zn , <C-c>@<A-C-s> , select: <C-c>@<A-C-s> , insert: <C-c>@<A-C-s>vim — normal: zR , zn
<code>fold_close_all</code>	Close all folds (zM)	spacemacs — normal: zM , zN , zX , <C-c>@<A-C-h> , select: <C-c>@<A-C-h> , insert: <C-c>@<A-C-h>vim — normal: zM , zN , zX
<code>fold_more</code>	Fold more: close one more level of nested folds (zm)	spacemacs, vim — normal: zm
<code>fold_less</code>	Fold less: open one more level of nested folds (zr)	spacemacs, vim — normal: zr
<code>fold_delete</code>	Delete fold under cursor (zd)	spacemacs, vim — normal: zD , zd
<code>fold_delete_all</code>	Delete all folds (zE)	spacemacs, vim — normal: zE
<code>narrow_to_region</code>	Narrow the buffer to the selected region (SPC n r)	spacemacs — normal: <C-x>nn , <space>nr , select: <C-x>nn , <space>nr , insert: <C-x>nn
<code>widen</code>	Widen: remove narrowing and reveal the whole buffer (SPC n w)	spacemacs — normal: <C-x>nw , <space>nw , select: <C-x>nw , <space>nw , insert: <C-x>nw
<code>narrow_to_function_indirect</code>	Narrow to the function in an indirect (split) view (SPC n F)	spacemacs — normal: <space>nF , select: <space>nF
<code>narrow_region_indirect</code>	Narrow to the selected region in an indirect (split) view (SPC n R)	spacemacs — normal: <space>nR , select: <space>nR
<code>layout_add_buffers</code>	Add another layout's buffers into the current windows (SPC l A)	spacemacs — normal: <space>lA , select: <space>lA
<code>winner_undo</code>	Undo the last window-layout change (winner-undo, SPC w u)	spacemacs — normal: <C-w>u , <C-w>.u , <C-w>[u , <C-w>{u , <space>wu , <space>w.u , <space>w[u , <space>w{u , select: <space>wu , <space>w.u , <space>w[u , <space>w{u vim — normal: <C-w>u , <C-w>.u , <C-w>[u , <C-w>{u
<code>winner_redo</code>	Redo a window-layout change (winner-redo, SPC w . U)	spacemacs — normal: <C-w>.U , <C-w>[U , <C-w>{U , <space>w.U , <space>w[U , <space>w{U , select: <space>w.U , <space>w[U , <space>w{U vim — normal: <C-w>.U , <C-w>[U , <C-w>{U

Name	Description	Keybinds
<code>exit_transient_state</code>	Leave the current transient state (q)	spacemacs — normal: <code><[w>.q</code> , <code><[w>[q</code> , <code><[w>{q</code> , <code><space>b.q</code> , <code><space>enq</code> , <code><space>epq</code> , <code><space>lwq</code> , <code><space>n+q</code> , <code><space>n=q</code> , <code><space>n_q</code> , <code><space>w.q</code> , <code><space>w[q</code> , <code><space>w{q</code> , <code><space>x.q</code> , <code><space>zfq</code> , <code><space>z xq</code> , <code><space>l<ret></code> , <code><space>n<minus>q</code> , select: <code><space>b.q</code> , <code><space>enq</code> , <code><space>epq</code> , <code><space>lwq</code> , <code><space>n+q</code> , <code><space>n=q</code> , <code><space>n_q</code> , <code><space>w.q</code> , <code><space>w[q</code> , <code><space>w{q</code> , <code><space>x.q</code> , <code><space>zfq</code> , <code><space>z xq</code> , <code><space>l<ret></code> , <code><space>n<minus>q</code> qvim — normal: <code><[w>.q</code> , <code><[w>[q</code> , <code><[w>{q</code>
<code>text_scale_increase</code>	Increase the text scale one step (SPC z x +)	spacemacs — normal: <code><[x><[+></code> , <code><[x><[-=></code> , <code><space>z x+</code> , <code><space>z x=</code> , <code><space>z xk</code> , <code><[x><A-[+></code> , <code><[x><A-[-=></code> , select: <code><[x><[+></code> , <code><[x><[-=></code> , <code><space>z x+</code> , <code><space>z x=</code> , <code><space>z xk</code> , <code><[x><A-[+></code> , <code><[x><A-[-=></code> , insert: <code><[x><[+></code> , <code><[x><[-=></code> , <code><[x><A-[+></code> , <code><[x><A-[-=></code>
<code>text_scale_decrease</code>	Decrease the text scale one step (SPC z x -)	spacemacs — normal: <code><space>z x_</code> , <code><space>z xj</code> , <code><[x><[-minus></code> , <code><[x><A-[minus></code> , <code><space>z x<minus></code> , select: <code><space>z x_</code> , <code><space>z xj</code> , <code><[x><[-minus></code> , <code><[x><A-[minus></code> , <code><space>z x<minus></code> , insert: <code><[x><[-minus></code> , <code><[x><A-[minus></code>
<code>text_scale_reset</code>	Reset the text scale to the default size (SPC z x 0)	spacemacs — normal: <code><[x><[-0></code> , <code><space>z x0</code> , <code><[x><A-[0></code> , select: <code><[x><[-0></code> , <code><space>z x0</code> , <code><[x><A-[0></code> , insert: <code><[x><[-0></code> , <code><[x><A-[0></code>
<code>frame_zoom_in</code>	Zoom the frame in one step (SPC z f +)	spacemacs — normal: <code><space>z f+</code> , <code><space>z f=</code> , <code><space>z fk</code> , select: <code><space>z f+</code> , <code><space>z f=</code> , <code><space>z fk</code>
<code>frame_zoom_out</code>	Zoom the frame out one step (SPC z f -)	spacemacs — normal: <code><space>z f_</code> , <code><space>z fj</code> , <code><space>z f<minus></code> , select: <code><space>z f_</code> , <code><space>z fj</code> , <code><space>z f<minus></code>
<code>frame_zoom_reset</code>	Reset the frame zoom to the default size (SPC z f 0)	spacemacs — normal: <code><space>z f0</code> , select: <code><space>z f0</code>
<code>copy_version</code>	Display and copy the zmax version to the clipboard (SPC f e v)	spacemacs — normal: <code><space>fev</code> , select: <code><space>fev</code>
<code>narrow_to_page_indirect</code>	Narrow to the page in an indirect (split) view (SPC n P)	spacemacs — normal: <code><space>nP</code> , select: <code><space>nP</code>

Name	Description	Keybinds
<code>kmacro_ring_next</code>	Cycle to the next macro in the ring (SPC K r n)	spacemacs — normal: <code><space>Krn</code> , <code><[-x]<[-k]<[-n></code> , select: <code><space>Krn</code> , <code><[-x]<[-k]<[-n></code> , insert: <code><[-x]<[-k]<[-n></code>
<code>kmacro_ring_prev</code>	Cycle to the previous macro in the ring (SPC K r p)	spacemacs — normal: <code><space>Krn</code> , <code><space>Krp</code> , <code><[-x]<[-k]<[-p></code> , select: <code><space>Krn</code> , <code><space>Krp</code> , <code><[-x]<[-k]<[-p></code> , insert: <code><[-x]<[-k]<[-p></code>
<code>kmacro_ring_delete</code>	Delete the head macro in the ring (SPC K r d)	spacemacs — normal: <code><space>Krd</code> , <code><[-x]<[-k]<d</code> , select: <code><space>Krd</code> , <code><[-x]<[-k]<d</code> , insert: <code><[-x]<[-k]<d</code>
<code>kmacro_ring_swap</code>	Swap the first two macros in the ring (SPC K r s)	spacemacs — normal: <code><space>Krs</code> , <code><[-x]<[-k]<[-t></code> , select: <code><space>Krs</code> , <code><[-x]<[-k]<[-t></code> , insert: <code><[-x]<[-k]<[-t></code>
<code>kmacro_ring_view</code>	View the head macro in the ring (SPC K r L)	spacemacs — normal: <code><space>Kv</code> , <code><space>Krl</code> , select: <code><space>Kv</code> , <code><space>Krl</code>
<code>kmacro_call_ring_2nd</code>	Run the second macro in the ring without rotating it (emacs <code>kmacro-call-ring-2nd</code> , SPC K r l)	
<code>kmacro_to_register</code>	Write the last macro to a register (SPC K e r)	spacemacs — normal: <code><space>Ken</code> , <code><space>Ker</code> , <code><[-x]<[-k]<x</code> , select: <code><space>Ken</code> , <code><space>Ker</code> , <code><[-x]<[-k]<x</code> , insert: <code><[-x]<[-k]<x</code>
<code>kmacro_add_counter</code>	Add [count] to the keyboard-macro counter (SPC K c a)	spacemacs — normal: <code><space>Kca</code> , <code><[-x]<[-k]<[-a></code> , select: <code><space>Kca</code> , <code><[-x]<[-k]<[-a></code> , insert: <code><[-x]<[-k]<[-a></code>
<code>kmacro_insert_counter</code>	Insert the macro counter value, then increment (SPC K c c)	spacemacs — normal: <code><space>Kcc</code> , <code><[-x]<[-k]<[-i></code> , select: <code><space>Kcc</code> , <code><[-x]<[-k]<[-i></code> , insert: <code><[-x]<[-k]<[-i></code>
<code>kmacro_set_counter</code>	Set the keyboard-macro counter to [count] (emacs <code>kmacro-set-counter</code>)	spacemacs — normal: <code><space>KcC</code> , <code><[-x]<[-k]<[-c></code> , select: <code><space>KcC</code> , <code><[-x]<[-k]<[-c></code> , insert: <code><[-x]<[-k]<[-c></code>
<code>kmacro_set_format</code>	Set the macro-counter insert format, e.g. %03d (emacs <code>kmacro-set-format</code>)	spacemacs — normal: <code><space>Kcf</code> , <code><[-x]<[-k]<[-f></code> , select: <code><space>Kcf</code> , <code><[-x]<[-k]<[-f></code> , insert: <code><[-x]<[-k]<[-f></code>
<code>kmacro_name_last_macro</code>	Name the last kbd macro and register it as an invokable command (emacs <code>kmacro-name-last-macro</code>)	spacemacs — normal: <code><[-x]<[-k]<n</code> , select: <code><[-x]<[-k]<n</code> , insert: <code><[-x]<[-k]<n</code>
<code>insert_kbd_macro</code>	Insert a textual definition of the last kbd macro (emacs <code>insert-kbd-macro</code>)	
<code>apply_macro_to_region_lines</code>	Run the last kbd macro at the start of each line in the region (emacs <code>apply-macro-to-region-lines</code>)	spacemacs — normal: <code><[-x]<[-k]<r</code> , select: <code><[-x]<[-k]<r</code> , insert: <code><[-x]<[-k]<r</code>
<code>kmacro_end_or_call_macro</code>	End recording, or call the last kbd macro (emacs <code>kmacro-end-or-call-macro</code> , F4)	spacemacs — normal: <code><F4></code> , <code><[-x]<e</code> , <code><space>KK</code> , select: <code><[-x]<e</code> , <code><space>KK</code> , insert: <code><[-x]<evim</code> — normal: <code><F4></code>
<code>kmacro_end_or_call_macro_repeat</code>	Repeat-variant of end-or-call macro (emacs <code>kmacro-end-or-call-macro-repeat</code>)	spacemacs — normal: <code><[-x]<[-k]<[-k></code> , select: <code><[-x]<[-k]<[-k></code> , insert: <code><[-x]<[-k]<[-k></code>

Name	Description	Keybinds
<code>kmacro_edit_macro</code>	Edit the last kbd macro's keys as text (emacs edit-kbd-macro / kmacro-edit-macro)	spacemacs — normal: <code><space>Kee</code> , <code><[-x]<[-k>e</code> , <code><[-x]<[-k><[-e></code> , <code><[-x]<[-k><ret></code> , select: <code><space>Kee</code> , <code><[-x]<[-k>e</code> , <code><[-x]<[-k><[-e></code> , <code><[-x]<[-k><ret></code> , insert: <code><[-x]<[-k>e</code> , <code><[-x]<[-k><[-e></code> , <code><[-x]<[-k><ret></code>
<code>edmacro_insert_key</code>	Read one literal key and insert its written name at point (emacs edmacro-insert-key)	
<code>edmacro_set_macro_to_region_lines</code>	Set the kbd macro to the lines the region covers (emacs edmacro-set-macro-to-region-lines)	
<code>kmacro_step_edit_macro</code>	Step through the last kbd macro key by key, editing as you go (emacs kmacro-step-edit-macro)	spacemacs — normal: <code><space>Kes</code> , <code><[-x]<[-k><space></code> , select: <code><space>Kes</code> , <code><[-x]<[-k><space></code> , insert: <code><[-x]<[-k><space></code>
<code>kmacro_edit_lossage</code>	Edit the recently pressed keys as a macro (emacs kmacro-edit-lossage)	spacemacs — normal: <code><space>Ke l</code> , <code><[-x]<[-k>l</code> , select: <code><space>Ke l</code> , <code><[-x]<[-k>l</code> , insert: <code><[-x]<[-k>l</code>
<code>kmacro_bind_to_key</code>	Report the config binding for the last kbd macro (emacs kmacro-bind-to-key)	spacemacs — normal: <code><space>Keb</code> , <code><[-x]<[-k>b</code> , select: <code><space>Keb</code> , <code><[-x]<[-k>b</code> , insert: <code><[-x]<[-k>b</code>
<code>kmacro_redisplay</code>	Force a redisplay while a kbd macro runs (emacs kmacro-redisplay, C-x C-k d)	
<code>toggle_readonly</code>	Toggle the buffer's read-only (writable) state (SPC b w)	spacemacs — normal: <code><space>bw</code> , <code><[-x]<[-q></code> , select: <code><space>bw</code> , <code><[-x]<[-q></code> , insert: <code><[-x]<[-q></code>
<code>toggle_window_dedication</code>	Toggle window dedication (spacemacs SPC w t)	spacemacs — normal: <code><[-w>t</code> , <code><[-x>wd</code> , <code><space>wt</code> , select: <code><[-x>wd</code> , <code><space>wt</code> , insert: <code><[-x>wdvim</code> — normal: <code><[-w>t</code>
<code>toggle_subword</code>	Toggle sub-word w/b/e motions (spacemacs SPC t c)	spacemacs — normal: <code><space>tc</code> , select: <code><space>tc</code>
<code>toggle_superword</code>	Toggle super-word w/b/e motions: symbols join words (emacs superword-mode, SPC t C)	spacemacs — normal: <code><space>tC</code> , select: <code><space>tC</code>
<code>toggle_auto_fill</code>	Toggle auto-fill: wrap at text-width while typing (spacemacs SPC t F)	spacemacs — normal: <code><space>tF</code> , select: <code><space>tF</code>
<code>toggle_follow_mode</code>	Toggle follow mode: windows on the same doc scroll together (spacemacs SPC w f)	spacemacs — normal: <code><[-w>f</code> , <code><space>wf</code> , select: <code><space>wfvim</code> — normal: <code><[-w>f</code>
<code>subword_w</code>	Next word start, sub-word aware (w)	spacemacs, vim — normal: <code>w</code> , <code><[-right></code> , insert: <code><[-right></code> , <code><S-right></code>
<code>subword_b</code>	Previous word start, sub-word aware (b)	spacemacs, vim — normal: <code>b</code>
<code>subword_e</code>	Next word end, sub-word aware (e)	spacemacs, vim — normal: <code>e</code>
<code>subword_extend_w</code>	Extend to next word start, sub-word aware	
<code>subword_extend_b</code>	Extend to previous word start, sub-word aware	
<code>subword_extend_e</code>	Extend to next word end, sub-word aware	
<code>subword_extend_ge</code>	Extend to previous word end, sub-word aware (ge)	

Name	Description	Keybinds
<code>paredit_slurp_forward</code>	Paredit: slurp the next s-expression forward (SPC k s)	spacemacs — normal: <code><space>ks</code> , <code><space>k`s</code> , select: <code><space>ks</code> , <code><space>k`s</code>
<code>paredit_barf_forward</code>	Paredit: barf the last s-expression forward (SPC k b)	spacemacs — normal: <code><space>kb</code> , select: <code><space>kb</code>
<code>paredit_slurp_backward</code>	Paredit: slurp the previous s-expression backward (SPC k S)	spacemacs — normal: <code><space>kS</code> , select: <code><space>kS</code>
<code>paredit_barf_backward</code>	Paredit: barf the first s-expression backward (SPC k B)	spacemacs — normal: <code><space>kB</code> , select: <code><space>kB</code>
<code>paredit_splice</code>	Paredit: splice/unwrap the enclosing s-expression (SPC k W)	spacemacs — normal: <code><space>kW</code> , select: <code><space>kW</code>
<code>paredit_raise</code>	Paredit: raise the current s-expression (SPC k r)	spacemacs — normal: <code><space>kr</code> , select: <code><space>kr</code>
<code>paredit_transpose</code>	Paredit: transpose the s-expressions around point (SPC k t)	spacemacs — normal: <code><space>kt</code> , <code><space>k`t</code> , <code><space>k`p</code> , select: <code><space>kt</code> , <code><space>k`t</code> , <code><space>k`p</code> , <code><space>k`t</code>
<code>paredit_split</code>	Paredit: split the enclosing list at point (SPC j s)	spacemacs — normal: <code><space>jS</code> , <code><space>jS</code> , select: <code><space>jS</code> , <code><space>jS</code>
<code>paredit_absorb</code>	Paredit: absorb the previous sexp into the current form (SPC k a)	spacemacs — normal: <code><space>ka</code> , select: <code><space>ka</code>
<code>paredit_convolute</code>	Paredit: convolute — swap enclosing/inner prefixes (SPC k c)	spacemacs — normal: <code><space>kc</code> , select: <code><space>kc</code>
<code>buffer_swap_window_1</code>	Swap current buffer with window 1 (SPC b . M-1)	spacemacs — normal: <code><space>b.<A-1></code> , select: <code><space>b.<A-1></code>
<code>buffer_swap_window_2</code>	Swap current buffer with window 2 (SPC b . M-2)	spacemacs — normal: <code><space>b.<A-2></code> , select: <code><space>b.<A-2></code>
<code>buffer_swap_window_3</code>	Swap current buffer with window 3 (SPC b . M-3)	spacemacs — normal: <code><space>b.<A-3></code> , select: <code><space>b.<A-3></code>
<code>buffer_swap_window_4</code>	Swap current buffer with window 4 (SPC b . M-4)	spacemacs — normal: <code><space>b.<A-4></code> , select: <code><space>b.<A-4></code>
<code>buffer_swap_window_5</code>	Swap current buffer with window 5 (SPC b . M-5)	spacemacs — normal: <code><space>b.<A-5></code> , select: <code><space>b.<A-5></code>
<code>buffer_swap_window_6</code>	Swap current buffer with window 6 (SPC b . M-6)	spacemacs — normal: <code><space>b.<A-6></code> , select: <code><space>b.<A-6></code>
<code>buffer_swap_window_7</code>	Swap current buffer with window 7 (SPC b . M-7)	spacemacs — normal: <code><space>b.<A-7></code> , select: <code><space>b.<A-7></code>
<code>buffer_swap_window_8</code>	Swap current buffer with window 8 (SPC b . M-8)	spacemacs — normal: <code><space>b.<A-8></code> , select: <code><space>b.<A-8></code>
<code>buffer_swap_window_9</code>	Swap current buffer with window 9 (SPC b . M-9)	spacemacs — normal: <code><space>b.<A-9></code> , select: <code><space>b.<A-9></code>
<code>paredit_splice_kill_forward</code>	Paredit: splice, killing forward (SPC k e)	spacemacs — normal: <code><space>ke</code> , select: <code><space>ke</code>
<code>paredit_splice_kill_backward</code>	Paredit: splice, killing backward (SPC k E)	spacemacs — normal: <code><space>kE</code> , select: <code><space>kE</code>
<code>paredit_insert_sexp_after</code>	Paredit: insert a new () sexp after the current one (SPC k))	spacemacs — normal: <code><space>k)</code> , select: <code><space>k)</code>
<code>paredit_insert_sexp_before</code>	Paredit: insert a new () sexp before the current one (SPC k ()	spacemacs — normal: <code><space>k(</code> , select: <code><space>k(</code>

Name	Description	Keybinds
<code>fold_next</code>	Move to next fold (zj)	spacemacs, vim — normal: <code>]z, zj</code>
<code>fold_prev</code>	Move to previous fold (zk)	spacemacs, vim — normal: <code>[z, zk</code>
<code>goto_line_last_nonblank</code>	Goto last non-blank on line (g_)	spacemacs, vim — normal: <code>g_</code>
<code>goto_line_middle</code>	Goto middle of text line (gM)	spacemacs, vim — normal: <code>gM, gm</code>
<code>goto_byte</code>	Goto byte {count} in buffer (go)	spacemacs, vim — normal: <code>go</code>
<code>goto_prev_unmatched_paren</code>	Goto previous unmatched (([()	spacemacs — normal: <code>[(, <space>j(,</code> select: <code><space>j(</code> (vim — normal: <code>[(</code>
<code>goto_prev_unmatched_brace</code>	Goto previous unmatched { ([{)	spacemacs, vim — normal: <code>[{</code>
<code>goto_next_unmatched_paren</code>	Goto next unmatched) (])	spacemacs, vim — normal: <code>])</code>
<code>goto_next_unmatched_brace</code>	Goto next unmatched } (])	spacemacs, vim — normal: <code>] }</code>
<code>goto_prev_preproc</code>	Goto previous unmatched #if/#else ([#)	spacemacs, vim — normal: <code>[#</code>
<code>goto_next_preproc</code>	Goto next unmatched #endif/#else (])	spacemacs, vim — normal: <code>] #</code>
<code>vim_sleep</code>	Sleep for {count} seconds (vim gs)	spacemacs, vim — normal: <code>gs</code>
<code>goto_prev_mark</code>	Goto previous lowercase mark ([')	spacemacs, vim — normal: <code>[`</code>
<code>goto_next_mark</code>	Goto next lowercase mark ([')	spacemacs, vim — normal: <code>] `</code>
<code>goto_prev_mark_line</code>	Goto previous lowercase mark, line start ([')])	spacemacs, vim — normal: <code>['</code>
<code>goto_next_mark_line</code>	Goto next lowercase mark, line start ([')])	spacemacs, vim — normal: <code>] '</code>
<code>yank_file_path</code>	Yank current file path to clipboard	spacemacs — normal: <code><space>fyC,</code> <code><space>fyY, <space>fyy,</code> select: <code><space>fyC, <space>fyY,</code> <code><space>fyy</code>
<code>yank_file_name</code>	Yank current file name to clipboard	spacemacs — normal: <code><space>fyN,</code> <code><space>fyn,</code> select: <code><space>fyN,</code> <code><space>fyn</code>
<code>yank_file_path_with_line</code>	Yank current file path:line to clipboard	spacemacs — normal: <code><space>fyl,</code> <code><space>fyl,</code> select: <code><space>fyl,</code> <code><space>fyl</code>
<code>yank_file_path_with_line_col</code>	Yank current file path:line:col to clipboard	spacemacs — normal: <code><space>fyc,</code> select: <code><space>fyc</code>
<code>yank_file_dir</code>	Yank current file's directory to clipboard	spacemacs — normal: <code><space>fyD,</code> <code><space>fyd,</code> select: <code><space>fyD,</code> <code><space>fyd</code>
<code>copy_remote_url</code>	Copy web permalink (host/blob//path#Ln) for current line	spacemacs — normal: <code><space>glC,</code> <code><space>gll, <space>glP,</code> select: <code><space>glC, <space>gll,</code> <code><space>glP</code>
<code>open_remote_url</code>	Open current line's web permalink in the browser	spacemacs — normal: <code><space>glc,</code> <code><space>gll, <space>glp,</code> select: <code><space>glc, <space>gll,</code> <code><space>glp</code>
<code>open_url_under_cursor</code>	Open the URL under the cursor in the browser	spacemacs — normal: <code><C-c><ret>,</code> select: <code><C-c><ret>,</code> insert: <code><C-c><ret></code>
<code>open_all_buffer_links</code>	Open every URL in the current buffer in the browser (link-hint-open-all-links, SPC x A)	spacemacs — normal: <code><space>xA,</code> select: <code><space>xA</code>
<code>copy_all_buffer_links</code>	Copy every URL in the current buffer to the clipboard (link-hint-copy-all-links, SPC x Y)	spacemacs — normal: <code><space>xY,</code> select: <code><space>xY</code>

Name	Description	Keybinds
<code>link_hint_open_link</code>	Pick a URL from the current buffer and open it in the browser (link-hint-open-link, SPC x O)	spacemacs — normal: <code><space>xO</code> , select: <code><space>x0</code>
<code>helm_google_suggest</code>	Google suggestions, opening the results in the browser (helm-google-suggest, SPC s w g)	
<code>link_hint_copy_link</code>	Pick a URL from the current buffer and copy it to the clipboard (link-hint-copy-link, SPC x y)	spacemacs — normal: <code><space>xy</code> , select: <code><space>xy</code>
<code>profiler_start</code>	Start (and reset) the command profiler (SPC h P s)	spacemacs — normal: <code><space>hPs</code> , select: <code><space>hPs</code>
<code>profiler_stop</code>	Stop the command profiler, keeping its samples (SPC h P k)	spacemacs — normal: <code><space>hPk</code> , select: <code><space>hPk</code>
<code>profiler_report</code>	Show the command profiler report, slowest first (SPC h P r)	spacemacs — normal: <code><space>hPr</code> , select: <code><space>hPr</code>
<code>profiler_write_report</code>	Write the command profiler report to a prompted file (SPC h P w)	spacemacs — normal: <code><space>hPw</code> , select: <code><space>hPw</code>
<code>regexp_generate_strings</code>	Generate every string matched by a finite regexp (SPC x r ')	spacemacs — normal: <code><space>xr'</code> , <code><space>xrp'</code> , select: <code><space>xr'</code> , <code><space>xrp'</code>
<code>regexp_generate_strings_emacs</code>	Generate every string matched by a finite Emacs regexp (SPC x r e ')	spacemacs — normal: <code><space>xre'</code> , select: <code><space>xre'</code>
<code>toggle_fringe</code>	Hide or show the whole fringe (gutter column strip) (fringe-mode, SPC T f)	spacemacs — normal: <code><space>Tf</code> , select: <code><space>Tf</code>
<code>restart_editor</code>	Close every view and relaunch zmax with the same arguments (restart-emacs, SPC q r)	spacemacs — normal: <code><space>qr</code> , select: <code><space>qr</code>
<code>duplicate_selection_down</code>	Duplicate current line(s) downward	
<code>duplicate_selection_up</code>	Duplicate current line(s) upward	
<code>move_text_line_down</code>	Move current line(s) down past the next line	
<code>move_text_line_up</code>	Move current line(s) up past the previous line	
<code>count_selection</code>	Count chars/words/lines in selection	spacemacs — normal: <code><A-=></code> , <code><space>xc</code> , select: <code><space>xc</code>
<code>match_brackets</code>	Goto matching bracket	spacemacs — normal: <code><space>k%</code> , select: <code><space>k%</code> helix — normal: <code>mm</code> , select: <code>mm</code>
<code>match_brackets_extend</code>	Extend to matching bracket, inclusive (vim d%/c%/y%)	
<code>match_brackets_or_goto_percent</code>	Goto matching bracket, or {count} percent through the file	spacemacs, vim — normal: <code>%</code> , select: <code>%</code>
<code>surround_add</code>	Surround add	helix — normal: <code>ms</code> , select: <code>ms</code>
<code>surround_replace</code>	Surround replace	helix — normal: <code>mr</code> , select: <code>mr</code>
<code>surround_delete</code>	Surround delete	helix — normal: <code>md</code> , select: <code>md</code>
<code>select_textobject_around</code>	Select around object	spacemacs, vim — select: <code>ahelix</code> — normal: <code>ma</code> , select: <code>ma</code>
<code>select_textobject_inner</code>	Select inside object	spacemacs, vim — select: <code>ihelix</code> — normal: <code>mi</code> , select: <code>mi</code>
<code>change_textobject_inner</code>	Change inside object (ci)	spacemacs, vim — normal: <code>ci</code>
<code>change_textobject_around</code>	Change around object (ca)	spacemacs, vim — normal: <code>ca</code>
<code>delete_textobject_inner</code>	Delete inside object (di)	spacemacs, vim — normal: <code>di</code>
<code>delete_textobject_around</code>	Delete around object (da)	spacemacs, vim — normal: <code>da</code>

Name	Description	Keybinds
<code>yank_textobject</code>	Yank the selection and put the cursor at its start (vim visual y)	
<code>yank_textobject_inner</code>	Yank inside object (yi)	spacemacs, vim — normal: <code>yi</code>
<code>yank_textobject_around</code>	Yank around object (ya)	spacemacs, vim — normal: <code>ya</code>
<code>extend_backward_exclusive_vim</code>	Make a backward selection exclusive of the cursor (vim backward motions)	
<code>extend_forward_exclusive_vim</code>	Make a forward selection exclusive of the cursor (vim n, /pat under an operator)	
<code>delete_to_mark</code>	Delete to a mark (vim d')	spacemacs, vim — normal: <code>d`</code>
<code>delete_to_mark_line</code>	Delete whole lines to a mark (vim d')	spacemacs, vim — normal: <code>d'</code>
<code>change_to_mark</code>	Change to a mark (vim c')	spacemacs, vim — normal: <code>c`</code>
<code>change_to_mark_line</code>	Change whole lines to a mark (vim c')	spacemacs, vim — normal: <code>c'</code>
<code>yank_to_mark</code>	Yank to a mark (vim y')	spacemacs, vim — normal: <code>y`</code>
<code>yank_to_mark_line</code>	Yank whole lines to a mark (vim y')	spacemacs, vim — normal: <code>y'</code>
<code>reflow_textobject_inner</code>	Reflow inside object (vim gqi)	spacemacs, vim — normal: <code>gqi</code>
<code>reflow_textobject_around</code>	Reflow around object (vim gqa)	spacemacs, vim — normal: <code>gqa</code>
<code>reflow_keep_textobject_inner</code>	Reflow inside object, keep cursor (vim gwi)	spacemacs, vim — normal: <code>gwi</code>
<code>reflow_keep_textobject_around</code>	Reflow around object, keep cursor (vim gwa)	spacemacs, vim — normal: <code>gwa</code>
<code>indent_textobject_inner</code>	Indent inside object (vim >i)	spacemacs, vim — normal: <code><gt>i</code>
<code>indent_textobject_around</code>	Indent around object (vim >a)	spacemacs, vim — normal: <code><gt>a</code>
<code>unindent_textobject_inner</code>	Unindent inside object (vim <i)	spacemacs, vim — normal: <code><lt>i</code>
<code>unindent_textobject_around</code>	Unindent around object (vim <a)	spacemacs, vim — normal: <code><lt>a</code>
<code>uppercase_textobject_inner</code>	Uppercase inside object (vim gUi)	spacemacs, vim — normal: <code>gUi</code>
<code>uppercase_textobject_around</code>	Uppercase around object (vim gUa)	spacemacs, vim — normal: <code>gUa</code>
<code>lowercase_textobject_inner</code>	Lowercase inside object (vim gui)	spacemacs, vim — normal: <code>gui</code>
<code>lowercase_textobject_around</code>	Lowercase around object (vim gua)	spacemacs, vim — normal: <code>gua</code>
<code>togglecase_textobject_inner</code>	Toggle case inside object (vim g~i)	spacemacs, vim — normal: <code>g~i</code>
<code>togglecase_textobject_around</code>	Toggle case around object (vim g~a)	spacemacs, vim — normal: <code>g~a</code>
<code>delete_find_char_forward</code>	Delete to next char (df)	spacemacs, vim — normal: <code>df</code>
<code>delete_till_char_forward</code>	Delete till next char (dt)	spacemacs, vim — normal: <code>dt</code>
<code>zap_to_char</code>	Kill through the next char, inclusive (emacs zap-to-char, M-z)	spacemacs — normal: <code><A-z>emacs</code> , cua — normal: <code><A-z></code> , insert: <code><A-z></code>
<code>zap_up_to_char</code>	Kill up to the next char, exclusive (emacs zap-up-to-char)	
<code>five_by_five</code>	Play 5x5, the light-flipping puzzle (emacs 5x5)	
<code>solitaire</code>	Play English peg solitaire (emacs solitaire)	
<code>hanoi</code>	Watch the Towers of Hanoi solution (emacs hanoi)	

Name	Description	Keybinds
<code>life</code>	Run Conway's Game of Life (emacs life)	
<code>doctor</code>	Talk to the ELIZA psychotherapist (emacs doctor)	
<code>gomoku</code>	Play five-in-a-row against the computer (emacs gomoku)	
<code>butterfly</code>	Flip the desired bit with a butterfly (emacs butterfly, xkcd 378)	
<code>mpuz</code>	Play the multiplication puzzle (emacs mpuz)	
<code>bubbles</code>	Play the bubbles same-game (emacs bubbles)	
<code>blackbox</code>	Play blackbox, the ray-tracing puzzle (emacs blackbox)	
<code>snake</code>	Play snake (emacs snake)	
<code>tetris</code>	Play tetris (emacs tetris)	
<code>pong</code>	Play pong against the computer (emacs pong)	
<code>space_invaders</code>	Play Space Invaders	
<code>breakout</code>	Play Breakout, the brick-breaker	
<code>asteroids</code>	Play Asteroids	
<code>frogger</code>	Play Frogger, cross the traffic	
<code>twenty_forty_eight</code>	Play 2048, the sliding-tile puzzle	
<code>minesweeper</code>	Play Minesweeper	
<code>tic_tac_toe</code>	Play Tic-Tac-Toe against the computer	
<code>connect_four</code>	Play Connect Four against the computer	
<code>reversi</code>	Play Reversi / Othello against the computer	
<code>sokoban</code>	Play Sokoban, push the boxes onto the goals	
<code>sudoku</code>	Play Sudoku	
<code>fifteen</code>	Play the 15-puzzle sliding tiles	
<code>hangman</code>	Play Hangman, guess the word	
<code>wordle</code>	Play Wordle, guess the five-letter word	
<code>mastermind</code>	Play Mastermind, crack the code	
<code>pacman</code>	Play Pac-Man	
<code>landmark</code>	Play landmark, find the hidden tree (emacs landmark)	
<code>centipede</code>	Play Centipede	
<code>missile_command</code>	Play Missile Command, defend the cities	
<code>tron</code>	Play Tron light-cycles against the computer	
<code>flappy</code>	Play Flappy, flap through the pipes	
<code>checkers</code>	Play Checkers / draughts against the computer	

Name	Description	Keybinds
<code>battleship</code>	Play Battleship against the computer	
<code>blackjack</code>	Play Blackjack against the dealer	
<code>yahtzee</code>	Play Yahtzee	
<code>simon</code>	Play Simon, the memory game	
<code>galaga</code>	Play Galaga	
<code>dig_dug</code>	Play Dig Dug	
<code>donkey_kong</code>	Play Donkey Kong	
<code>bomberman</code>	Play Bomberman	
<code>lunar_lander</code>	Play Lunar Lander	
<code>chess</code>	Play chess against the computer	
<code>mancala</code>	Play Mancala / Kalah against the computer	
<code>video_poker</code>	Play Jacks-or-Better video poker	
<code>klondike</code>	Play Klondike solitaire	
<code>nonogram</code>	Play nonogram / picross	
<code>xref_find_references</code>	Find references to a symbol across the workspace (emacs xref-find-references)	spacemacs — normal: <code><A-?></code> , <code><space>pE</code> , select: <code><space>pE</code>
<code>project_find_file</code>	Find a file under the project root (emacs project-find-file)	
<code>diffmode</code>	Open the unified-diff viewer (emacs diff-mode)	
<code>diff_hunk_kill</code>	Kill the diff hunk at point (emacs diff-hunk-kill)	
<code>diff_file_kill</code>	Kill the whole file section at point (emacs diff-file-kill)	
<code>diff_split_hunk</code>	Split the diff hunk at point into two (emacs diff-split-hunk)	
<code>diff_reverse_direction</code>	Reverse the direction of the diff (emacs diff-reverse-direction)	
<code>diff_context_to_unified</code>	Convert a context diff to a unified diff (emacs diff-context->unified)	
<code>diff_unified_to_context</code>	Convert a unified diff to a context diff (emacs diff-unified->context)	
<code>diff_delete_trailing_whitespace</code>	Strip trailing whitespace from added lines (emacs diff-delete-trailing-whitespace)	
<code>diff_restrict_view</code>	Narrow the diff buffer to the hunk at point (emacs diff-restrict-view)	
<code>diff_apply_hunk</code>	Apply the diff hunk at point to its target file (emacs diff-apply-hunk)	
<code>diff_apply_buffer</code>	Apply every hunk in the diff buffer to its target files (emacs diff-apply-buffer)	
<code>picture</code>	Draw ASCII pictures on a canvas (emacs picture-mode)	
<code>picture_mode</code>	Toggle picture-mode overwrite drawing on the buffer (emacs picture-mode)	
<code>edit_picture</code>	Enter picture-mode on the current buffer (emacs edit-picture)	
<code>picture_movement_right</code>	Picture-mode: draw toward the right (emacs picture-movement-right)	

Name	Description	Keybinds
<code>picture_movement_left</code>	Picture-mode: draw toward the left (emacs picture-movement-left)	
<code>picture_movement_up</code>	Picture-mode: draw upward (emacs picture-movement-up)	
<code>picture_movement_down</code>	Picture-mode: draw downward (emacs picture-movement-down)	
<code>picture_movement_nw</code>	Picture-mode: draw up-and-left (emacs picture-movement-nw)	
<code>picture_movement_ne</code>	Picture-mode: draw up-and-right (emacs picture-movement-ne)	
<code>picture_movement_sw</code>	Picture-mode: draw down-and-left (emacs picture-movement-sw)	
<code>picture_movement_se</code>	Picture-mode: draw down-and-right (emacs picture-movement-se)	
<code>picture_motion</code>	Picture-mode: move point in the drawing direction (emacs picture-motion)	
<code>picture_motion_reverse</code>	Picture-mode: move point opposite the drawing direction (emacs picture-motion-reverse)	
<code>picture_set_tab_stops</code>	Picture-mode: set tab stops from this line (emacs picture-set-tab-stops)	
<code>picture_tab</code>	Picture-mode: move to the next tab stop (emacs picture-tab)	
<code>picture_tab_search</code>	Picture-mode: move under next word-start above (emacs picture-tab-search)	
<code>picture_open_line</code>	Picture-mode: split the line at point (emacs picture-open-line)	emacs, cua — insert: <code><C-o></code>
<code>picture_clear_line</code>	Picture-mode: clear to end of line (emacs picture-clear-line)	
<code>picture_clear_column</code>	Picture-mode: blank columns after point (emacs picture-clear-column)	
<code>picture_backward_clear_column</code>	Picture-mode: blank columns before point (emacs picture-backward-clear-column)	
<code>picture_clear_rectangle_to_register</code>	Picture-mode: clear rectangle into a register (emacs picture-clear-rectangle-to-register)	
<code>picture_yank_rectangle</code>	Picture-mode: overlay the killed rectangle (emacs picture-yank-rectangle)	
<code>picture_yank_rectangle_from_register</code>	Picture-mode: overlay a register's rectangle (emacs picture-yank-rectangle-from-register)	
<code>twocol_two_columns</code>	Two-column: create a side-by-side partner buffer (emacs 2C-two-columns)	spacemacs — normal: <code><F2>2</code> , <code><C-x>62</code> , select: <code><C-x>62</code> , insert: <code><C-x>62</code>
<code>twocol_associate_buffer</code>	Two-column: associate the other window's buffer (emacs 2C-associate-buffer)	spacemacs — normal: <code><F2>b</code> , <code><C-x>6b</code> , select: <code><C-x>6b</code> , insert: <code><C-x>6b</code>
<code>twocol_split</code>	Two-column: split the buffer at point into two columns (emacs 2C-split)	spacemacs — normal: <code><F2>s</code> , <code><C-x>6s</code> , select: <code><C-x>6s</code> , insert: <code><C-x>6s</code>
<code>twocol_merge</code>	Two-column: merge the partner column back in (emacs 2C-merge)	spacemacs — normal: <code><F2>1</code> , <code><C-x>61</code> , select: <code><C-x>61</code> , insert: <code><C-x>61</code>
<code>twocol_dissociate</code>	Two-column: break the association (emacs 2C-dissociate)	spacemacs — normal: <code><F2>d</code> , <code><C-x>6d</code> , select: <code><C-x>6d</code> , insert: <code><C-x>6d</code>

Name	Description	Keybinds
<code>twocol_newline</code>	Two-column: newline in both columns (emacs 2C-newline)	spacemacs — normal: <code><F2><ret></code> , <code><[-x>6<ret></code> , select: <code><[-x>6<ret></code> , insert: <code><[-x>6<ret></code>
<code>table</code>	Edit a text table (emacs table.el)	
<code>table_recognize</code>	Recognize the ASCII table at point and report its dimensions (emacs table-recognize)	
<code>table_recognize_region</code>	Recognize the table in the selection and report its dimensions (emacs table-recognize-region)	
<code>table_recognize_table</code>	Recognize the whole table at point (emacs table-recognize-table)	
<code>table_recognize_cell</code>	Report the table cell at point (emacs table-recognize-cell)	
<code>table_query_dimension</code>	Report the size of the table cell and table at point (emacs table-query-dimension)	
<code>table_justify</code>	Cycle the justification of the current table column (emacs table-justify)	
<code>table_widen_cell</code>	Widen the current table column by one column (emacs table-widen-cell)	
<code>table_narrow_cell</code>	Narrow the current table column by one column (emacs table-narrow-cell)	
<code>table_heighten_cell</code>	Heighten the current table row by one line (emacs table-heighten-cell)	
<code>table_shorten_cell</code>	Shorten the current table row by one line (emacs table-shorten-cell)	
<code>table_span_cell</code>	Merge the current table cell with the one to its right (emacs table-span-cell)	
<code>table_split_cell</code>	Split the current table cell vertically at its middle (emacs table-split-cell)	
<code>table_split_cell_horizontally</code>	Split the current table cell into two columns (emacs table-split-cell-horizontally)	
<code>table_split_cell_vertically</code>	Split the current table cell into two rows (emacs table-split-cell-vertically)	
<code>table_insert_sequence</code>	Fill table cells from point with an incrementing sequence (emacs table-insert-sequence)	
<code>table_generate_source</code>	Emit HTML source for the table at point (emacs table-generate-source)	
<code>table_capture</code>	Capture the selected plain text into a table (emacs table-capture)	
<code>table_release</code>	Release the table at point back to plain text (emacs table-release)	
<code>table_fixed_width_mode</code>	Toggle table fixed-width mode: cells wrap instead of widening (emacs table-fixed-width-mode)	
<code>table_unrecognize</code>	Deactivate every table in the buffer (emacs table-unrecognize)	
<code>table_unrecognize_table</code>	Deactivate the table at point (emacs table-unrecognize-table)	
<code>table_unrecognize_region</code>	Deactivate every table in the region (emacs table-unrecognize-region)	
<code>table_unrecognize_cell</code>	Deactivate the table cell at point (emacs table-unrecognize-cell)	
<code>word_search_forward</code>	Search forward for words, ignoring punctuation (emacs word-search-forward)	

Name	Description	Keybinds
<code>word_search_backward</code>	Search backward for words, ignoring punctuation (emacs word-search-backward)	
<code>fill_region_as_paragraph</code>	Fill the region as a single paragraph (emacs fill-region-as-paragraph)	
<code>indent_rigidly</code>	Shift the region by N columns (emacs indent-rigidly)	spacemacs — normal: <code><[-x]>tab></code> , select: <code><[-x]>tab></code> , insert: <code><[-x]>tab></code> emacs, cua — insert: <code><[-q]></code>
<code>quoted_insert</code>	Insert the next key literally, control chars included (emacs quoted-insert)	
<code>foldout_zoom_subtree</code>	Narrow to the outline subtree at point (emacs foldout-zoom-subtree)	spacemacs — normal: <code><[-c]><[-z]></code> , select: <code><[-c]><[-z]></code> , insert: <code><[-c]><[-z]></code>
<code>foldout_exit_fold</code>	Leave the zoomed subtree and widen (emacs foldout-exit-fold)	spacemacs — normal: <code><[-c]><[-x]></code> , select: <code><[-c]><[-x]></code> , insert: <code><[-c]><[-x]></code>
<code>ff_find_related_file</code>	Visit the related header/source file (emacs ff-find-related-file)	
<code>revert_buffer_quick</code>	Revert the buffer from its file without confirmation (emacs revert-buffer-quick)	spacemacs — normal: <code><[-x]>xg</code> , select: <code><[-x]>xg</code> , insert: <code><[-x]>xg</code>
<code>delete_file</code>	Delete a file from disk (emacs delete-file)	
<code>quit_window</code>	Quit this window, burying the buffer (emacs quit-window)	
<code>switch_to_buffer_other_window</code>	Show a buffer in another window (emacs switch-to-buffer-other-window)	spacemacs — normal: <code><[-x]>4b</code> , select: <code><[-x]>4b</code> , insert: <code><[-x]>4b</code> emacs — insert: <code><[-x]>4bcua</code> — select: <code><[-X]>4b</code> , insert: <code><[-x]>4b</code>
<code>find_file_other_window</code>	Open a file in another window (emacs find-file-other-window)	spacemacs — normal: <code><[-x]>4f</code> , <code><[-x]>4<[-f]></code> , select: <code><[-x]>4f</code> , <code><[-x]>4<[-f]></code> , insert: <code><[-x]>4f</code> , <code><[-x]>4<[-f]></code> emacs — insert: <code><[-x]>4f</code> , <code><[-x]>4<[-f]></code> cua — select: <code><[-X]>4f</code> , <code><[-X]>4<[-f]></code> , insert: <code><[-x]>4f</code> , <code><[-x]>4<[-f]></code>
<code>shrink_window_if_larger_than_buffer</code>	Shrink the window to fit its buffer (emacs shrink-window-if-larger-than-buffer)	spacemacs — normal: <code><[-x]><minus></code> , select: <code><[-x]><minus></code> , insert: <code><[-x]><minus></code>
<code>edit_abbrevs</code>	Show every abbrev definition for editing (emacs edit-abbrevs)	
<code>quietly_read_abbrev_file</code>	Read abbrev definitions from a file, silently (emacs quietly-read-abbrev-file)	
<code>whitespace_toggle_options</code>	Toggle one whitespace visualization by key (emacs whitespace-toggle-options)	
<code>xref_find_apropos</code>	List project identifiers matching a pattern (emacs xref-find-apropos)	
<code>keyboard_escape_quit</code>	Drop extra cursors and deactivate the region (emacs keyboard-escape-quit)	emacs, cua — insert: <code><[-]></code> , <code><esc><esc><esc></code>
<code>kill_ring_deindent_mode</code>	Toggle deindenting text saved to the kill ring (emacs kill-ring-deindent-mode)	
<code>kill_some_buffers</code>	Offer to kill each buffer in turn (emacs kill-some-buffers)	
<code>async_shell_command</code>	Run a shell command without blocking, output to a buffer (emacs async-shell-command)	emacs, cua — insert: <code><A-f></code>
<code>xref_quit_and_pop_marker_stack</code>	Close the xref results and jump back (emacs xref-quit-and-pop-marker-stack)	
<code>display_fill_column_indicator_mode</code>	Toggle a rule at the fill column (emacs display-fill-column-indicator-mode)	

Name	Description	Keybinds
<code>fortran_next_statement</code>	Move to the next fixed-form Fortran statement (emacs <code>fortran-next-statement</code>)	
<code>fortran_previous_statement</code>	Move to the previous fixed-form Fortran statement (emacs <code>fortran-previous-statement</code>)	
<code>fortran_beginning_of_block</code>	Move to the opening of the Fortran block at point (emacs <code>fortran-beginning-of-block</code>)	
<code>fortran_end_of_block</code>	Move to the END of the Fortran block at point (emacs <code>fortran-end-of-block</code>)	
<code>f90_next_statement</code>	Move to the next free-form F90 statement (emacs <code>f90-next-statement</code>)	
<code>f90_previous_statement</code>	Move to the previous free-form F90 statement (emacs <code>f90-previous-statement</code>)	
<code>f90_next_block</code>	Move to the next F90 block opening (emacs <code>f90-next-block</code>)	
<code>f90_previous_block</code>	Move to the previous F90 block opening (emacs <code>f90-previous-block</code>)	
<code>f90_beginning_of_block</code>	Move to the opening of the F90 block at point (emacs <code>f90-beginning-of-block</code>)	
<code>f90_end_of_block</code>	Move to the end of the F90 block at point (emacs <code>f90-end-of-block</code>)	
<code>fortran_split_line</code>	Break the line at point onto a Fortran continuation line (emacs <code>fortran-split-line</code>)	
<code>fortran_join_line</code>	Join the current line with the following Fortran continuation line (emacs <code>fortran-join-line</code>)	
<code>fortran_comment_region</code>	Comment (or uncomment) the selected lines as Fortran comments (emacs <code>fortran-comment-region</code>)	
<code>fortran_indent_subprogram</code>	Re-indent the buffer by fixed-form Fortran block nesting (emacs <code>fortran-indent-subprogram</code>)	
<code>fortran_strip_sequence_nos</code>	Delete sequence numbers in columns 73+ on every line (emacs <code>fortran-strip-sequence-nos</code>)	
<code>fortran_column_ruler</code>	Show the fixed-form Fortran column ruler in the echo area (emacs <code>fortran-column-ruler</code>)	
<code>fortran_mode</code>	Enter fixed-form Fortran mode (emacs <code>fortran-mode</code>)	
<code>f90_mode</code>	Enter free-form Fortran/F90 mode (emacs <code>f90-mode</code>)	
<code>facemenu</code>	Browse faces and colors (emacs <code>list-faces-display</code> / <code>facemenu</code>)	
<code>facemenu_set_face</code>	Put a named face on the region (emacs <code>facemenu-set-face</code>)	
<code>facemenu_set_foreground</code>	Put a foreground color on the region (emacs <code>facemenu-set-foreground</code>)	
<code>facemenu_set_background</code>	Put a background color on the region (emacs <code>facemenu-set-background</code>)	
<code>facemenu_set_bold</code>	Make the region bold (emacs <code>facemenu-set-bold</code>)	
<code>facemenu_set_italic</code>	Make the region italic (emacs <code>facemenu-set-italic</code>)	
<code>facemenu_set_bold_italic</code>	Make the region bold italic (emacs <code>facemenu-set-bold-italic</code>)	

Name	Description	Keybinds
<code>facemenu_set_underline</code>	Underline the region (emacs facemenu-set-underline)	
<code>facemenu_set_default</code>	Put the default face on the region (emacs facemenu-set-default)	
<code>facemenu_remove_face_props</code>	Remove the face property from the region (emacs facemenu-remove-face-props)	
<code>facemenu_remove_all</code>	Remove every text property from the region (emacs facemenu-remove-all)	
<code>format_decode_buffer</code>	Decode this buffer's format annotations into text properties (emacs format-decode-buffer)	
<code>hide_ifdef_mode</code>	Hide the code the C preprocessor skips (emacs hide-ifdef-mode)	
<code>cpp_highlight_buffer</code>	Shade the preprocessor branches that are compiled out (emacs cpp-highlight-buffer)	
<code>cwarn_mode</code>	Highlight suspicious C constructs in this buffer (emacs cwarn-mode)	
<code>global_cwarn_mode</code>	Highlight suspicious C constructs everywhere (emacs global-cwarn-mode)	
<code>sgml_tags_invisible</code>	Hide the markup tags, show only the text (emacs sgml-tags-invisible)	
<code>goto_address_mode</code>	Highlight the URLs and e-mails in this buffer (emacs goto-address-mode)	
<code>bug_reference_mode</code>	Turn bug references like Bug#1234 into tracker links (emacs bug-reference-mode)	
<code>bug_reference_prog_mode</code>	Bug references, but only inside comments and strings (emacs bug-reference-prog-mode)	
<code>transient_mark_mode</code>	Toggle highlighting of the region (emacs transient-mark-mode)	
<code>prettify_symbols_mode</code>	Draw -> as an arrow, lambda as a lambda (emacs prettify-symbols-mode)	
<code>glyphless_display_mode</code>	Reveal control and zero-width characters (emacs glyphless-display-mode)	
<code>bookmark_bmenu_list</code>	List bookmarks in an overlay (emacs bookmark-bmenu-list)	
<code>proced</code>	Open the process viewer/manager (emacs proced)	
<code>zone</code>	Run the zone screen-saver (emacs zone)	
<code>decipher</code>	Solve a cryptogram (emacs decipher)	
<code>dunnet</code>	Play the dunnet text adventure (emacs dunnet)	
<code>animate_birthday_present</code>	Animate a birthday-present message (emacs animate-birthday-present)	
<code>dissociated_press</code>	Scramble the buffer with the travesty generator (emacs dissociated-press)	
<code>spook</code>	Insert random NSA-bait phrases (emacs spook)	
<code>studlify_region</code>	StudlyCaps the selected region (emacs studlify-region)	
<code>studlify_buffer</code>	StudlyCaps the whole buffer (emacs studlify-buffer)	

Name	Description	Keybinds
<code>studlify_word</code>	StudlyCaps the word after point (emacs <code>studlify-word</code>)	
<code>indent_relative</code>	Indent to under the next indent point in the previous line (emacs <code>indent-relative</code>)	
<code>indent_code_rigidly</code>	Shift region lines by [count] columns, skipping lines that start in a string (emacs <code>indent-code-rigidly</code> , = r)	spacemacs, vim — normal: =r
<code>indent_sexp_rigidly</code>	Reindent this line and shift the grouping starting on it by the same amount (emacs <code>C-u TAB</code>)	
<code>c_hungry_delete_forward</code>	Delete all whitespace after point, else one char (emacs <code>c-hungry-delete-forward</code> , SPC x d f)	spacemacs — normal: <space>xdf , select: <space>xdf
<code>c_hungry_delete_backwards</code>	Delete all whitespace before point, else one char (emacs <code>c-hungry-delete-backwards</code> , SPC x d b)	spacemacs — normal: <space>xdb , select: <space>xdb
<code>c_beginning_of_defun</code>	Move to the start of the function at point (emacs <code>c-beginning-of-defun</code>)	spacemacs — normal: <A-[a>
<code>c_end_of_defun</code>	Move to the end of the function at point (emacs <code>c-end-of-defun</code>)	spacemacs — normal: <A-[e>
<code>c_mark_function</code>	Select the whole function around point (emacs <code>c-mark-function</code>)	
<code>c_beginning_of_statement</code>	Move to the start of the C statement at point (emacs <code>c-beginning-of-statement</code>)	
<code>c_end_of_statement</code>	Move to the end of the C statement at point (emacs <code>c-end-of-statement</code>)	
<code>c_forward_conditional</code>	Move forward across a preprocessor conditional (emacs <code>c-forward-conditional</code>)	
<code>c_backward_conditional</code>	Move backward across a preprocessor conditional (emacs <code>c-backward-conditional</code>)	
<code>c_up_conditional</code>	Move up out of the containing preprocessor conditional (emacs <code>c-up-conditional</code>)	
<code>c_indent_line_or_region</code>	Re-indent the current line or the selected lines (emacs <code>c-indent-line-or-region</code>)	
<code>c_indent_defun</code>	Re-indent the whole function at point (emacs <code>c-indent-defun</code>)	
<code>c_ts_mode_indent_defun</code>	Re-indent the whole function at point via tree-sitter (emacs <code>c-ts-mode-indent-defun</code>)	
<code>c_indent_exp</code>	Re-indent the balanced expression after point (emacs <code>c-indent-exp</code>)	
<code>c_fill_paragraph</code>	Fill the C comment block around point (emacs <code>c-fill-paragraph</code> , M-q)	
<code>comment_set_column</code>	Set the comment column from point, aligning this line's comment with a prefix (emacs <code>comment-set-column</code> , C-x ;)	
<code>fortran_fill_paragraph</code>	Fill the Fortran comment block or statement at point (emacs <code>fortran-fill-paragraph</code> , M-q)	
<code>c_backslash_region</code>	Align trailing backslash continuations in the region (emacs <code>c-backslash-region</code>)	
<code>c_context_line_break</code>	Break the line, continuing a comment or macro (emacs <code>c-context-line-break</code>)	
<code>c_toggle_auto_newline</code>	Toggle auto-newline insertion in C mode (emacs <code>c-toggle-auto-newline</code>)	

Name	Description	Keybinds
<code>c_toggle_hungry_state</code>	Toggle hungry-delete state in C mode (emacs c-toggle-hungry-state)	
<code>c_toggle_electric_state</code>	Toggle electric behavior in C mode (emacs c-toggle-electric-state)	
<code>c_show_syntactic_information</code>	Report the tree-sitter node kind at point (emacs c-show-syntactic-information)	
<code>c_macro_expand</code>	Expand C preprocessor macros in the region via cpp (emacs c-macro-expand)	
<code>c_set_style</code>	Report the C indentation style (emacs c-set-style)	
<code>ps_print_buffer</code>	Print the buffer as PostScript via lpr (emacs ps-print-buffer)	
<code>ps_print_region</code>	Print the region as PostScript via lpr (emacs ps-print-region)	
<code>ps_print_buffer_with_faces</code>	Print the buffer as PostScript (plain, no faces) (emacs ps-print-buffer-with-faces)	
<code>ps_print_region_with_faces</code>	Print the region as PostScript (plain, no faces) (emacs ps-print-region-with-faces)	
<code>ps_spool_buffer</code>	Spool the buffer as PostScript for later printing (emacs ps-spool-buffer)	
<code>ps_spool_region</code>	Spool the region as PostScript for later printing (emacs ps-spool-region)	
<code>ps_spool_buffer_with_faces</code>	Spool the buffer as PostScript (plain, no faces) (emacs ps-spool-buffer-with-faces)	
<code>ps_spool_region_with_faces</code>	Spool the region as PostScript (plain, no faces) (emacs ps-spool-region-with-faces)	
<code>ps_despool</code>	Print the accumulated PostScript spool via lpr (emacs ps-despool)	
<code>handwrite</code>	Print the buffer as PostScript in a handwriting style via lpr (emacs handwrite)	
<code>pr_interface</code>	Open the printing package's front-end for the print commands (emacs pr-interface)	
<code>delete_find_char_backward</code>	Delete to prev char (dF)	spacemacs, vim — normal: dF
<code>delete_till_char_backward</code>	Delete till prev char (dT)	spacemacs, vim — normal: dT
<code>change_find_char_forward</code>	Change to next char (cf)	spacemacs, vim — normal: cf
<code>change_till_char_forward</code>	Change till next char (ct)	spacemacs, vim — normal: ct
<code>change_find_char_backward</code>	Change to prev char (cF)	spacemacs, vim — normal: cF
<code>change_till_char_backward</code>	Change till prev char (cT)	spacemacs, vim — normal: cT
<code>yank_find_char_forward</code>	Yank to next char (yf)	spacemacs, vim — normal: yf
<code>yank_till_char_forward</code>	Yank till next char (yt)	spacemacs, vim — normal: yt
<code>yank_find_char_backward</code>	Yank to prev char (yF)	spacemacs, vim — normal: yF
<code>yank_till_char_backward</code>	Yank till prev char (yT)	spacemacs, vim — normal: yT
<code>set_mark</code>	Set mark (m{a-z} buffer, m{A-Z} global)	spacemacs, vim — normal: m
<code>goto_mark</code>	Goto mark exact ('{a-z/A-Z/0-9}, " for last jump)	spacemacs, vim — normal: `

Name	Description	Keybinds
<code>goto_mark_line</code>	Goto mark line ('{a-z/A-Z/0-9},' for last jump)	spacemacs, vim — normal: <code>'</code>
<code>goto_mark_nojump</code>	Goto mark exact without changing jumplist (g')	spacemacs, vim — normal: <code>g`</code>
<code>goto_mark_line_nojump</code>	Goto mark line without changing jumplist (g')	spacemacs, vim — normal: <code>g'</code>
<code>repeat_substitute</code>	Repeat last :substitute (&)	spacemacs, vim — normal: <code>ft</code>
<code>repeat_substitute_global</code>	Repeat last :substitute on whole file (g&)	spacemacs, vim — normal: <code>gft</code>
<code>vim_record_macro</code>	Record macro into register (q{reg})	spacemacs, vim — normal: <code>q</code>
<code>vim_replay_macro</code>	Replay macro from register (@{reg})	spacemacs, vim — normal: <code>@</code>
<code>save_visual_selection</code>	Save the visual selection (for gv)	
<code>reselct_visual</code>	Reselect the last visual area (gv)	spacemacs, vim — normal: <code>gv</code> , select: <code>gv</code>
<code>mark_insert_exit</code>	Record the insert-exit position (for gi)	
<code>insert_at_last_insert</code>	Insert at the last insert position (gi)	spacemacs, vim — normal: <code>gi</code>
<code>goto_next_function</code>	Goto next function	spacemacs, vim — normal: <code>][</code> , mhelix — normal: <code>]f</code> , select: <code>]f</code> , femacs, cua — normal: <code><A-[-e></code> , insert: <code><A-[-e></code>
<code>goto_prev_function</code>	Goto previous function	spacemacs, vim — normal: <code>[]</code> , mhelix — normal: <code>[f</code> , select: <code>[f</code> , femacs, cua — normal: <code><A-[-a></code> , insert: <code><A-[-a></code>
<code>goto_next_class</code>	Goto next type definition	helix — normal: <code>]t</code> , select: <code>]t</code>
<code>goto_prev_class</code>	Goto previous type definition	helix — normal: <code>[t</code> , select: <code>[t</code>
<code>goto_next_parameter</code>	Goto next parameter	helix — normal: <code>]a</code> , select: <code>]a</code>
<code>goto_prev_parameter</code>	Goto previous parameter	helix — normal: <code>[a</code> , select: <code>[a</code>
<code>goto_next_comment</code>	Goto next comment	spacemacs, vim — normal: <code>]*</code> , <code>] /</code> , helix — normal: <code>]c</code> , select: <code>]c</code>
<code>goto_prev_comment</code>	Goto previous comment	spacemacs, vim — normal: <code>[*</code> , <code>[/</code> , helix — normal: <code>[c</code> , select: <code>[c</code>
<code>goto_next_test</code>	Goto next test	helix — normal: <code>]T</code> , select: <code>]T</code>
<code>goto_prev_test</code>	Goto previous test	helix — normal: <code>[T</code> , select: <code>[T</code>
<code>goto_next_xml_element</code>	Goto next (X)HTML element	helix — normal: <code>]x</code> , select: <code>]x</code>
<code>goto_prev_xml_element</code>	Goto previous (X)HTML element	helix — normal: <code>[x</code> , select: <code>[x</code>
<code>goto_next_entry</code>	Goto next pairing	helix — normal: <code>]e</code> , select: <code>]e</code>
<code>goto_prev_entry</code>	Goto previous pairing	helix — normal: <code>[e</code> , select: <code>[e</code>
<code>goto_next_paragraph</code>	Goto next paragraph	spacemacs — normal: <code>}</code> , <code><A-></code> , select: <code>}</code> , vim — normal: <code>}</code> , select: <code>}</code> , mhelix — normal: <code>]p</code> , select: <code>]p</code> , femacs, cua — normal: <code><A-></code> , insert: <code><A-></code>
<code>goto_prev_paragraph</code>	Goto previous paragraph	spacemacs — normal: <code>{</code> , <code><A-{></code> , select: <code>{</code> , vim — normal: <code>{</code> , select: <code>{</code> , mhelix — normal: <code>[p</code> , select: <code>[p</code> , femacs, cua — normal: <code><A-{></code> , insert: <code><A-{></code>
<code>goto_next_section</code>	Goto next section (vim <code>]])</code>	spacemacs, vim — normal: <code>]])</code>

Name	Description	Keybinds
<code>goto_prev_section</code>	Goto previous section (vim [])	spacemacs , vim — normal: []
<code>move_sentence_forward</code>	Move to next sentence	spacemacs — normal:), <A-e>, select:) vim — normal:), select:)
<code>move_sentence_backward</code>	Move to previous sentence	spacemacs — normal: (, <A-a>, select: (vim — normal: (, select: (
<code>dap_launch</code>	Launch debug target	spacemacs — normal: <S-F5>, <space>dd, <[-c><[-d>, select: <space>dd, <[-c><[-d>, insert: <[-c><[-d> vim — normal: <S-F5> helix — normal: <space>G _l , select: <space>G _l
<code>dap_restart</code>	Restart debugging session	spacemacs — normal: <space>dr, select: <space>dr helix — normal: <space>G _r , select: <space>G _r
<code>dap_toggle_breakpoint</code>	Toggle breakpoint	spacemacs — normal: <F9>, <space>db, <[-x><[-a><[-b>, select: <space>db, <[-x><[-a><[-b>, insert: <[-x><[-a><[-b> vim — normal: <F9> helix — normal: <space>G _b , select: <space>G _b
<code>dap_remove_breakpoint</code>	Remove breakpoint on current line (Emacs gud-remove)	spacemacs — normal: <[-x><[-a><[-d>, select: <[-x><[-a><[-d>, insert: <[-x><[-a><[-d>
<code>dap_continue</code>	Continue program execution	spacemacs — normal: <space>dc, <[-x><[-a><[-r>, select: <space>dc, <[-x><[-a><[-r>, insert: <[-x><[-a><[-r> helix — normal: <space>G _c , select: <space>G _c
<code>dap_run_to_cursor</code>	Run the debugger up to the cursor line (JetBrains Run To Cursor)	spacemacs — normal: <space>dC, <[-c><[-u>, <[-x><[-a><[-u>, select: <space>dC, <[-c><[-u>, <[-x><[-a><[-u>, insert: <[-c><[-u>, <[-x><[-a><[-u>
<code>dap_pause</code>	Pause program execution	spacemacs — normal: <space>dp, select: <space>dph helix — normal: <space>G _h , select: <space>G _h
<code>dap_step_in</code>	Step in	spacemacs — normal: <F11>, <space>di, <[-c><[-s>, <[-x><[-a><[-s>, select: <space>di, <[-c><[-s>, <[-x><[-a><[-s>, insert: <[-c><[-s>, <[-x><[-a><[-s> vim — normal: <F11> helix — normal: <space>G _i , select: <space>G _i
<code>dap_step_out</code>	Step out	spacemacs — normal: <S-F11>, <space>do, <[-c><[-f>, <[-x><[-a><[-f>, select: <space>do, <[-c><[-f>, <[-x><[-a><[-f>, insert: <[-c><[-f>, <[-x><[-a><[-f> vim — normal: <S-F11> helix — normal: <space>G _o , select: <space>G _o

Name	Description	Keybinds
<code>dap_next</code>	Step to next	spacemacs — normal: <code><F10></code> , <code><space>dn</code> , <code><[-c]><[-n></code> , <code><[-x]><[-a]><[-n></code> , select: <code><space>dn</code> , <code><[-c]><[-n></code> , <code><[-x]><[-a]><[-n></code> , insert: <code><[-c]><[-n></code> , <code><[-x]><[-a]><[-n></code> vim — normal: <code><F10></code> helix — normal: <code><space>Gn</code> , select: <code><space>Gn</code>
<code>dap_variables</code>	List variables	spacemacs — normal: <code><space>dv</code> , select: <code><space>dv</code> helix — normal: <code><space>Gv</code> , select: <code><space>Gv</code>
<code>dap_terminate</code>	End debug session	spacemacs — normal: <code><space>dq</code> , select: <code><space>dq</code> helix — normal: <code><space>Gt</code> , select: <code><space>Gt</code>
<code>dap_edit_condition</code>	Edit breakpoint condition on current line	helix — normal: <code><space>G<[-c></code> , select: <code><space>G<[-c></code>
<code>dap_breakpoints_picker</code>	View all breakpoints in a picker (JetBrains View Breakpoints)	spacemacs — normal: <code><space>dB</code> , select: <code><space>dB</code>
<code>dap_edit_log</code>	Edit breakpoint log message on current line	helix — normal: <code><space>G<[-l></code> , select: <code><space>G<[-l></code>
<code>dap_switch_thread</code>	Switch current thread	spacemacs — normal: <code><space>dgt</code> , select: <code><space>dgt</code> helix — normal: <code><space>Gst</code> , select: <code><space>Gst</code>
<code>dap_switch_stack_frame</code>	Switch stack frame	spacemacs — normal: <code><space>dgf</code> , select: <code><space>dgf</code> helix — normal: <code><space>Gsf</code> , select: <code><space>Gsf</code>
<code>dap_enable_exceptions</code>	Enable exception breakpoints	helix — normal: <code><space>Ge</code> , select: <code><space>Ge</code>
<code>dap_disable_exceptions</code>	Disable exception breakpoints	helix — normal: <code><space>GE</code> , select: <code><space>GE</code>
<code>gdb_display_locals_buffer</code>	Show local variables of the current frame (emacs gdb-display-locals-buffer)	spacemacs — normal: <code><space>dgl</code> , select: <code><space>dgl</code>
<code>gdb_display_registers_buffer</code>	Show CPU registers of the current frame (emacs gdb-display-registers-buffer)	spacemacs — normal: <code><space>dgr</code> , select: <code><space>dgr</code>
<code>gdb_display_stack_for_thread</code>	Show the call stack of the current thread (emacs gdb-display-stack-for-thread)	spacemacs — normal: <code><space>dgs</code> , select: <code><space>dgs</code>
<code>gdb_display_locals_for_thread</code>	Show locals of the current thread's innermost frame (emacs gdb-display-locals-for-thread)	
<code>gdb_display_registers_for_thread</code>	Show registers of the current thread's innermost frame (emacs gdb-display-registers-for-thread)	
<code>gdb_display_disassembly_buffer</code>	Disassemble around the current frame PC (emacs gdb-display-disassembly-buffer)	spacemacs — normal: <code><space>dgd</code> , select: <code><space>dgd</code>
<code>gdb_display_disassembly_for_thread</code>	Disassemble around the current thread PC (emacs gdb-display-disassembly-for-thread)	
<code>gdb_display_io_buffer</code>	Show the inferior IO / Run console (emacs gdb-display-io-buffer)	spacemacs — normal: <code><space>dgi</code> , select: <code><space>dgi</code>
<code>gdb_display_memory_buffer</code>	Read and hexdump target memory (emacs gdb-display-memory-buffer)	spacemacs — normal: <code><space>dgm</code> , select: <code><space>dgm</code>
<code>gdb_delete_breakpoint</code>	Delete the breakpoint on the current line (emacs gdb-delete-breakpoint)	spacemacs — normal: <code><space>dgk</code> , select: <code><space>dgk</code>
<code>gdb_edit_value</code>	Set a variable/expression value in the debugger (emacs gdb-edit-value)	spacemacs — normal: <code><space>dge</code> , select: <code><space>dge</code>
<code>gdb_many_windows</code>	Open the multi-pane debugger layout (emacs gdb-many-windows)	spacemacs — normal: <code><space>dgw</code> , select: <code><space>dgw</code>

Name	Description	Keybinds
<code>gdb_restore_windows</code>	Restore the debugger window layout (emacs gdb-restore-windows)	
<code>gud_gdb_complete_command</code>	Complete the gdb command at point (emacs gud-gdb-complete-command)	
<code>pandoc_menu</code>	Convert the buffer through pandoc, picking the output format (SPC P /)	spacemacs — normal: <code><space>P/</code> , select: <code><space>P/</code>
<code>shell_pipe</code>	Pipe selections through shell command	spacemacs — normal: <code><A-\\ ></code> helix — normal: <code>\\ </code> , select: <code>\\ </code>
<code>shell_pipe_to</code>	Pipe selections into shell command ignoring output	helix — normal: <code><A-\\ ></code> , select: <code><A-\\ ></code>
<code>shell_insert_output</code>	Insert shell command output before selections	spacemacs — normal: <code><A-!></code> helix — normal: <code>!</code> , select: <code>!</code>
<code>shell_append_output</code>	Append shell command output after selections	helix — normal: <code><A-!></code> , select: <code><A-!></code>
<code>shell_keep_pipe</code>	Filter selections with shell predicate	helix — normal: <code>\$</code> , select: <code>\$</code>
<code>suspend</code>	Suspend and return to shell	spacemacs — normal: <code><[-z></code> , <code><[-x><[-z></code> , select: <code><[-x><[-z></code> , insert: <code><[-x><[-z></code> vim — normal: <code><[-z></code> helix — normal: <code><[-z></code> , select: <code><[-z></code>
<code>rename_symbol</code>	Rename symbol	spacemacs — normal: <code><space>lr</code> , select: <code><space>lr</code> helix — normal: <code><space>r</code> , select: <code><space>r</code>
<code>increment</code>	Increment item under cursor	spacemacs — normal: <code><[-a></code> , <code>g<[-a></code> , <code><space>n++</code> , <code><space>n+=</code> , <code><space>n=+</code> , <code><space>n=</code> , <code><space>n_+</code> , <code><space>n_</code> , <code><space>n<minus>+</code> , <code><space>n<minus>=</code> , select: <code><[-a></code> , <code><space>n++</code> , <code><space>n+=</code> , <code><space>n=+</code> , <code><space>n=</code> , <code><space>n_+</code> , <code><space>n_</code> , <code><space>n<minus>+</code> , <code><space>n<minus>=</code> vim — normal: <code><[-a></code> , <code>g<[-a></code> , select: <code><[-a></code> helix — normal: <code><[-a></code> , select: <code><[-a></code>
<code>decrement</code>	Decrement item under cursor	spacemacs — normal: <code>g<[-x></code> , <code><space>n+_₁</code> , <code><space>n=_₁</code> , <code><space>n<minus>_₁</code> , <code><space>n=<minus>_₁</code> , <code><space>n<minus><minus>_₁</code> , select: <code><space>n+_₁</code> , <code><space>n=_₁</code> , <code><space>n<minus>_₁</code> , <code><space>n<minus><minus>_₁</code> , <code><space>n+<minus>_₁</code> , <code><space>n=<minus>_₁</code> , <code><space>n<minus><minus>_₁</code> vim — normal: <code><[-x></code> , <code>g<[-x></code> , select: <code><[-x></code> helix — normal: <code><[-x></code> , select: <code><[-x></code>
<code>increment_sequential</code>	Increment each line in the selection by a growing amount (vim g CTRL-A)	spacemacs, vim — select: <code>g<[-a></code>
<code>decrement_sequential</code>	Decrement each line in the selection by a growing amount (vim g CTRL-X)	spacemacs, vim — select: <code>g<[-x></code>
<code>record_macro</code>	Record macro	spacemacs — normal: <code><[-x>(</code> , select: <code><[-x>(</code> , insert: <code><[-x>(</code> helix — normal: <code>Q</code> , select: <code>Q</code>

Name	Description	Keybinds
<code>replay_macro</code>	Replay macro	spacemacs, vim — normal: <code>Qhelix</code> — normal: <code>q</code> , select: <code>q</code>
<code>command_palette</code>	Open command palette	spacemacs — normal: <code><A-X></code> , <code><A-x></code> , <code><C-x>*</code> , <code><C-x>;</code> , <code><C-x>q</code> , <code><space>?</code> , <code><space><space></code> , select: <code><C-x>*</code> , <code><C-x>;</code> , <code><C-x>q</code> , <code><space>?</code> , <code><space><space></code> , insert: <code><C-x>*</code> , <code><C-x>;</code> , <code><C-x>qhelix</code> — normal: <code><space>?</code> , select: <code><space>?emacs</code> , cua — normal: <code><A-x></code> , insert: <code><A-X></code> , <code><A-x></code>
<code>search_everywhere</code>	Search Everywhere: choose Files/Symbols/Text/Actions/Buffers (JetBrains)	spacemacs — normal: <code><space>sE</code> , select: <code><space>sE</code>
<code>recent_files_switcher</code>	Recent Files switcher: tool windows + recent files (SPC b r)	spacemacs — normal: <code><space>br</code> , select: <code><space>br</code>
<code>recentf_mode</code>	Toggle recording of opened files in the recent-files list (emacs recentf-mode)	
<code>recentf_save_list</code>	Write the recent-files list to its store file (emacs recentf-save-list)	
<code>recentf_edit_list</code>	Edit the recent-files list: mark entries and delete them (emacs recentf-edit-list)	
<code>repl</code>	Open the embedded-language REPL (elisp/viml/stryke/awk/zsh)	
<code>goto_word</code>	Jump to a two-character label	spacemacs — normal: <code><space>jl</code> , <code><space>jw</code> , select: <code><space>jl</code> , <code><space>jwhelix</code> — normal: <code>gw</code>
<code>extend_to_word</code>	Extend to a two-character label	helix — select: <code>gw</code>
<code>goto_char</code>	Label every visible occurrence of a char and jump (vim-easymotion s)	spacemacs — normal: <code><A-g>c</code> , <code><space>ja</code> , <code><space>je</code> , select: <code><space>ja</code> , <code><space>je</code>
<code>extend_to_char</code>	Label every visible occurrence of a char and extend to it	
<code>find_char_forward_label</code>	easymotion f: label forward occurrences of a char and jump	
<code>find_char_backward_label</code>	easymotion F: label backward occurrences of a char and jump	
<code>till_char_forward_label</code>	easymotion t: label forward, jump till before a char	
<code>till_char_backward_label</code>	easymotion T: label backward, jump till after a char	
<code>goto_next_tabstop</code>	Goto next snippet placeholder	
<code>goto_prev_tabstop</code>	Goto next snippet placeholder	
<code>emmet_expand</code>	Expand emmet/zen HTML abbreviation (or Tab)	helix, emacs, cua — insert: <code><tab></code>
<code>snippet_expand</code>	Expand the user snippet whose trigger precedes the cursor	
<code>rotate_selections_first</code>	Make the first selection your primary one	
<code>rotate_selections_last</code>	Make the last selection your primary one	
<code>show_keyword_line_from_start</code>	Show the first line containing the keyword (Ii)	spacemacs, vim — normal: <code>[i</code>
<code>show_keyword_line_from_cursor</code>	Show the next line containing the keyword (Ji)	spacemacs, vim — normal: <code>]i</code>

Name	Description	Keybinds
<code>list_keyword_lines_from_start</code>	List every line containing the keyword ([I])	spacemacs, vim — normal: <code>[I</code>
<code>list_keyword_lines_from_cursor</code>	List the lines below containing the keyword ([I])	spacemacs, vim — normal: <code>]I</code>
<code>list_defines_from_start</code>	List every #define of the keyword ([D])	spacemacs, vim — normal: <code>[D</code>
<code>list_defines_from_cursor</code>	List the #defines of the keyword below the cursor ([D])	spacemacs, vim — normal: <code>]D</code>
<code>goto_keyword_line_from_start</code>	Jump to the first line containing the keyword ([CTRL-I])	spacemacs, vim — normal: <code>[<[-i></code>
<code>goto_keyword_line_from_cursor</code>	Jump to the next line containing the keyword ([CTRL-I])	spacemacs, vim — normal: <code>]<[-i></code>
<code>goto_define_from_start</code>	Jump to the first #define of the keyword ([CTRL-D])	spacemacs, vim — normal: <code>[<[-d></code>
<code>goto_define_from_cursor</code>	Jump to the next #define of the keyword ([CTRL-D])	spacemacs, vim — normal: <code>]<[-d></code>
<code>scroll_line_below_window</code>	Put the line below the window at the top of it (z+)	
<code>scroll_line_above_window</code>	Put the line above the window at the bottom of it (z^)	
<code>yank_no_trailing_whitespace</code>	Yank without the trailing whitespace of each line (zy)	spacemacs, vim — normal: <code>zy</code>
<code>paste_after_no_trailing_whitespace</code>	Paste after without trailing whitespace (zp)	spacemacs, vim — normal: <code>zp</code>
<code>paste_before_no_trailing_whitespace</code>	Paste before without trailing whitespace (zP)	spacemacs, vim — normal: <code>zP</code>
<code>command_mode_count</code>	Open the Ex line, with the count's line range ({count}:)	spacemacs, vim — normal: <code>:</code>
<code>filter_equalprg</code>	Filter the selection through 'equalprg', else reindent (v_=)	
<code>complete_line</code>	Complete a whole line (i_CTRL-X CTRL-L)	vim — insert: <code><[-x><[-l></code>
<code>complete_filename</code>	Complete a file name (i_CTRL-X CTRL-F)	vim — insert: <code><[-x><[-f></code>
<code>complete_dictionary</code>	Complete from 'dictionary' (i_CTRL-X CTRL-K)	vim — insert: <code><[-x><[-k></code>
<code>complete_tag</code>	Complete a tag from the tags file (i_CTRL-X CTRL-J)	vim — insert: <code><[-x><[-j></code>
<code>complete_cmdline</code>	Complete like on the : command line (i_CTRL-X CTRL-V)	vim — insert: <code><[-x><[-v></code>
<code>complete_thesaurus</code>	Complete from 'thesaurus' (i_CTRL-X CTRL-T)	vim — insert: <code><[-x><[-t></code>
<code>complete_register_word</code>	Complete a word from the registers (i_CTRL-X CTRL-R)	vim — insert: <code><[-x><[-r></code>
<code>complete_define</code>	Complete a defined identifier (i_CTRL-X CTRL-D)	vim — insert: <code><[-x><[-d></code>
<code>complete_user_func</code>	Complete with 'completefunc' (i_CTRL-X CTRL-U)	vim — insert: <code><[-x><[-u></code>
<code>complete_omni_func</code>	Complete with 'omnifunc' (i_CTRL-X CTRL-O)	vim — insert: <code><[-x><[-o></code>
<code>operator_func</code>	Call 'operatorfunc' on the selection (g@)	spacemacs, vim — normal: <code>g@</code>
<code>toggle_lang_keymap</code>	Toggle the :Imap language keymap (i_CTRL-^, 'iminsert')	spacemacs, vim — insert: <code><[-^></code>
<code>insert_spell_suggest</code>	Spelling suggestions for the word being typed (i_CTRL-X s)	vim — insert: <code><[-x>s, <[-x><[-s></code>

Name	Description	Keybinds
<code>kmacro_menu</code>	List the keyboard-macro ring — mark, delete, copy, edit (emacs <code>kmacro-menu</code>)	
<code>kmacro_menu_mark</code>	Mark the macro at point in the <code>kmacro</code> list (emacs <code>kmacro-menu-mark</code>)	
<code>kmacro_menu_unmark</code>	Unmark the macro at point in the <code>kmacro</code> list (emacs <code>kmacro-menu-unmark</code>)	
<code>kmacro_menu_unmark_backward</code>	Move back a line and unmark (emacs <code>kmacro-menu-unmark-backward</code>)	
<code>kmacro_menu_unmark_all</code>	Remove every mark in the <code>kmacro</code> list (emacs <code>kmacro-menu-unmark-all</code>)	
<code>kmacro_menu_flag_for_deletion</code>	Flag the macro at point for deletion (emacs <code>kmacro-menu-flag-for-deletion</code>)	
<code>kmacro_menu_do_flagged_delete</code>	Delete the flagged macros (emacs <code>kmacro-menu-do-flagged-delete</code>)	
<code>kmacro_menu_do_delete</code>	Delete the marked macros (emacs <code>kmacro-menu-do-delete</code>)	
<code>kmacro_menu_do_copy</code>	Duplicate the marked macros in the ring (emacs <code>kmacro-menu-do-copy</code>)	
<code>kmacro_menu_transpose</code>	Transpose the macro at point with the previous one (emacs <code>kmacro-menu-transpose</code>)	
<code>kmacro_menu_edit_column</code>	Edit the column at point in the <code>kmacro</code> list (emacs <code>kmacro-menu-edit-column</code>)	
<code>kmacro_menu_edit_keys</code>	Edit the keys of the macro at point (emacs <code>kmacro-menu-edit-keys</code>)	
<code>kmacro_menu_edit_counter</code>	Edit the counter of the macro at point (emacs <code>kmacro-menu-edit-counter</code>)	
<code>kmacro_menu_edit_format</code>	Edit the counter format of the macro at point (emacs <code>kmacro-menu-edit-format</code>)	
<code>kmacro_menu_edit_position</code>	Move the macro at point to another ring position (emacs <code>kmacro-menu-edit-position</code>)	
<code>kmacro_end_macro</code>	End the keyboard macro being defined (emacs <code>kmacro-end-macro</code> , C-x))	spacemacs — normal: <code><[-x ></code>), select: <code><[-x ></code>), insert: <code><[-x ></code>)
<code>kmacro_start_macro_or_insert_counter</code>	Start a kbd macro, or insert the counter while defining one (emacs <code>kmacro-start-macro-or-insert-counter</code> , F3)	spacemacs — normal: <code><F3></code> , <code><space>Kk</code> , select: <code><space>Kk</code>
<code>keymap_global_set</code>	Bind a key sequence to a command, live (emacs <code>keymap-global-set</code>)	
<code>keymap_global_unset</code>	Remove a key sequence's global binding (emacs <code>keymap-global-unset</code>)	
<code>keymap_set</code>	Bind a key in one keymap — normal/select/insert (emacs <code>keymap-set</code>)	
<code>keymap_unset</code>	Remove a key's binding from one keymap (emacs <code>keymap-unset</code>)	
<code>keymap_substitute</code>	Rebind every key that runs OLD so it runs NEW (emacs <code>keymap-substitute</code>)	
<code>describe_function</code>	Describe a command — its doc and key bindings (emacs <code>describe-function</code> , C-h f)	spacemacs — normal: <code><[-h >f</code> , select: <code><[-h >f</code> , insert: <code><[-h >f</code>
<code>describe_key_briefly</code>	Echo, in one line, the command a key runs (emacs <code>describe-key-briefly</code> , C-h c)	spacemacs — normal: <code><[-h >c</code> , select: <code><[-h >c</code> , insert: <code><[-h >c</code>
<code>describe_variable</code>	Describe an editor/vim variable — value and default (emacs <code>describe-variable</code> , C-h v)	spacemacs — normal: <code><[-h >v</code> , select: <code><[-h >v</code> , insert: <code><[-h >v</code>

Name	Description	Keybinds
<code>describe_symbol</code>	Describe a name, command or variable (emacs describe-symbol, C-h o)	spacemacs — normal: <code><[-h>o</code> , select: <code><[-h>o</code> , insert: <code><[-h>o</code>
<code>button_describe</code>	Describe the push-button (URL/command/file) under point (emacs button-describe)	
<code>widget_describe</code>	Describe the Customize widget (option) under point (emacs widget-describe)	
<code>debug_print</code>	Dump the internal state at point to the log and a <i>Debug</i> buffer (emacs debug-print)	
<code>describe_repeat_maps</code>	List the sticky (transient-state) keymaps — zmax's repeat maps (emacs describe-repeat-maps)	
<code>describe_character_set</code>	Describe a character set / Unicode block (emacs describe-character-set)	
<code>help_quick</code>	Show the quick-help sheet with the live keys (emacs help-quick, C-h C-q)	
<code>help_quick_toggle</code>	Show or close the quick-help sheet (emacs help-quick-toggle)	spacemacs — normal: <code><[-h><[-q></code> , select: <code><[-h><[-q></code> , insert: <code><[-h><[-q></code>
<code>help_go_back</code>	Go back to the previously visited Help entry (emacs help-go-back)	spacemacs, vim — normal: <code>gb</code>
<code>help_go_forward</code>	Go forward again in the Help history (emacs help-go-forward)	
<code>help_goto_next_page</code>	Scroll the Help text down one screenful (emacs help-goto-next-page)	
<code>help_goto_previous_page</code>	Scroll the Help text up one screenful (emacs help-goto-previous-page)	
<code>calendar_mark_today</code>	Mark today's date in the calendar (emacs calendar-mark-today)	
<code>calendar_unmark</code>	Remove the marks from the calendar (emacs calendar-unmark)	
<code>calendar_star_date</code>	Replace the date under the cursor with asterisks (emacs calendar-star-date)	
<code>calendar_redraw</code>	Redraw the calendar, re-reading the diary (emacs calendar-redraw)	
<code>calendar_scroll_left</code>	Show the next month in the calendar (emacs calendar-scroll-left)	
<code>calendar_scroll_right</code>	Show the previous month in the calendar (emacs calendar-scroll-right)	
<code>diary_hebrew_list_entries</code>	List today's Hebrew-dated diary entries (emacs diary-hebrew-list-entries)	
<code>diary_hebrew_mark_entries</code>	Mark the calendar days with Hebrew-dated diary entries (emacs diary-hebrew-mark-entries)	
<code>diary_islamic_list_entries</code>	List today's Islamic-dated diary entries (emacs diary-islamic-list-entries)	
<code>diary_islamic_mark_entries</code>	Mark the calendar days with Islamic-dated diary entries (emacs diary-islamic-mark-entries)	
<code>diary_bahai_list_entries</code>	List today's Baha'i-dated diary entries (emacs diary-bahai-list-entries)	
<code>diary_bahai_mark_entries</code>	Mark the calendar days with Baha'i-dated diary entries (emacs diary-bahai-mark-entries)	
<code>diary_chinese_list_entries</code>	List today's Chinese-dated diary entries (emacs diary-chinese-list-entries)	
<code>diary_chinese_mark_entries</code>	Mark the calendar days with Chinese-dated diary entries (emacs diary-chinese-mark-entries)	

Name	Description	Keybinds
<code>diary_lunar_phases</code>	Report today's lunar phase, if any (emacs diary-lunar-phases)	
<code>diary_sunrise_sunset</code>	Report today's sunrise and sunset (emacs diary-sunrise-sunset)	
<code>diary_hebrew_sabbath_candles</code>	Report Friday's candle-lighting time (emacs diary-hebrew-sabbath-candles)	
<code>dired_undo</code>	Undo the last change to the Dired listing (emacs dired-undo)	
<code>minibuffer_complete_word</code>	Complete the minibuffer input one word further (emacs minibuffer-complete-word)	
<code>minibuffer_complete_and_exit</code>	Complete uniquely and accept the minibuffer (emacs minibuffer-complete-and-exit)	
<code>minibuffer_choose_completion</code>	Accept the minibuffer with the selected completion (emacs minibuffer-choose-completion)	
<code>minibuffer_complete_history</code>	Complete the minibuffer input against its history (emacs minibuffer-complete-history)	
<code>dabbrev_expand</code>	Expand the word before point from the buffer's other words (emacs dabbrev-expand, JetBrains Cyclic Expand Word)	emacs, cua — normal: <code><A- /></code> , insert: <code><A- /></code>
<code>dabbrev_completion</code>	List the buffer words that could expand the word before point (emacs dabbrev-completion)	spacemacs — normal: <code><A- [- /></code>
<code>copy_reference</code>	Copy a project-relative file:line reference to the clipboard (JetBrains Copy Reference)	
<code>error_description</code>	Show the full text of the diagnostic under the cursor (JetBrains Error Description)	
<code>find_sibling_file</code>	Open a sibling of this file: header/source, test/impl, style/template (emacs find-sibling-file, JetBrains Related File)	
<code>navigation_bar</code>	Jump to an enclosing function/class from the breadcrumb trail (JetBrains Navigation Bar)	
<code>select_in</code>	Select this file in another view: project tree, dired, terminal (JetBrains Select In)	
<code>unwrap_remove</code>	Unwrap the block around the cursor, keeping its body (JetBrains Unwrap/Remove)	
<code>move_file_refactor</code>	Move the current file to another directory (JetBrains Move, F6)	
<code>run_anything</code>	Run any shell command in the Run tool window (JetBrains Run Anything)	
<code>show_debug_window</code>	Open the Debug tool window (JetBrains Cmd-5)	
<code>auto_indent_lines</code>	Reindent the selected lines (JetBrains Auto-Indent Lines)	
<code>find_file_at_point</code>	Open the file or URL under the cursor (emacs find-file-at-point / ffap)	
<code>find_file_other_tab</code>	Open a file in a new tab (emacs find-file-other-tab)	spacemacs — normal: <code><C-x>tf</code> , select: <code><C-x>tf</code> , insert: <code><C-x>tf</code>
<code>find_grep</code>	Grep the project for a regex and list the hits (emacs find-grep)	

Name	Description	Keybinds
<code>list_keyboard_macros</code>	List the keyboard-macro ring (emacs <code>list-keyboard-macos</code>)	
<code>vc_register</code>	Put the current file under version control: <code>git add</code> (emacs <code>vc-register</code>)	spacemacs — normal: <code><[-x>vi</code> , select: <code><[-x>vi</code> , insert: <code><[-x>vi</code>
<code>vc_annotate</code>	Annotate each line of the file with its last commit: <code>git blame</code> (emacs <code>vc-annotate</code>)	
<code>vc_prepare_patch</code>	Prepare a patch of the last commit for mailing (emacs <code>vc-prepare-patch</code>)	
<code>vc_insert_headers</code>	Insert a version header comment at point (emacs <code>vc-insert-headers</code>)	
<code>vc_update_change_log</code>	Insert ChangeLog entries from the recent commits (emacs <code>vc-update-change-log</code>)	spacemacs — normal: <code><[-x>va</code> , select: <code><[-x>va</code> , insert: <code><[-x>va</code>
<code>vc_ediff</code>	Diff the current file against a revision (emacs <code>vc-ediff</code>)	
<code>set_buffer_file_coding_system</code>	Set the encoding this buffer is saved in (emacs <code>set-buffer-file-coding-system</code>)	spacemacs — normal: <code><[-x><ret>f</code> , select: <code><[-x><ret>f</code> , insert: <code><[-x><ret>f</code>
<code>set_terminal_coding_system</code>	Set the coding system the hosted terminal's output is decoded with (emacs <code>C-x RET t</code>)	spacemacs — normal: <code><[-x><ret>t</code> , select: <code><[-x><ret>t</code> , insert: <code><[-x><ret>t</code>
<code>set_keyboard_coding_system</code>	Set the coding system keys are encoded into for the hosted terminal (emacs <code>C-x RET k</code>)	spacemacs — normal: <code><[-x><ret>k</code> , select: <code><[-x><ret>k</code> , insert: <code><[-x><ret>k</code>
<code>set_selection_coding_system</code>	Set the coding system used for the system clipboard (emacs <code>C-x RET x</code>)	spacemacs — normal: <code><[-x><ret>x</code> , select: <code><[-x><ret>x</code> , insert: <code><[-x><ret>x</code>
<code>set_next_selection_coding_system</code>	Set the coding system for the next clipboard transfer only (emacs <code>C-x RET X</code>)	spacemacs — normal: <code><[-x><ret>X</code> , select: <code><[-x><ret>X</code> , insert: <code><[-x><ret>X</code>
<code>set_buffer_process_coding_system</code>	Set the coding systems a subprocess's pipes are decoded/encoded with (emacs <code>C-x RET p</code>)	spacemacs — normal: <code><[-x><ret>p</code> , select: <code><[-x><ret>p</code> , insert: <code><[-x><ret>p</code>
<code>set_file_name_coding_system</code>	Set the coding system file names are encoded into (emacs <code>C-x RET F</code>)	spacemacs — normal: <code><[-x><ret>F</code> , select: <code><[-x><ret>F</code> , insert: <code><[-x><ret>F</code>
<code>prefer_coding_system</code>	Make a coding system the default for files that are opened (emacs <code>C-x RET c</code> / <code>prefer-coding-system</code>)	
<code>universal_coding_system_argument</code>	Run the next file read or write with a coding system you name (emacs <code>C-x RET c</code>)	spacemacs — normal: <code><[-x><ret>c</code> , select: <code><[-x><ret>c</code> , insert: <code><[-x><ret>c</code>
<code>recode_file_name</code>	Rename a file whose name was decoded with the wrong coding system (emacs <code>recode-file-name</code>)	
<code>set_language_environment</code>	Choose a language environment, setting its default coding systems (emacs <code>set-language-environment</code>)	
<code>set_locale_environment</code>	Take the default coding systems from the locale (<code>\$LC_ALL/\$LC_CTYPE/\$LANG</code>) (emacs <code>set-locale-environment</code>)	
<code>set_input_method</code>	Choose the input method this buffer composes characters with (emacs <code>set-input-method</code> , <code>C-x RET C-</code>)	
<code>quail_set_keyboard_layout</code>	Tell the input-method system your physical keyboard layout (emacs <code>quail-set-keyboard-layout</code>)	

Name	Description	Keybinds
<code>activate_transient_input_method</code>	Turn an input method on for one character only (emacs activate-transient-input-method, C-x)	
<code>custom_prompt_customize_unsaved_options</code>	Ask whether to examine options customized this session but not saved (emacs custom-prompt-customize-unsaved-options)	
<code>describe_input_method</code>	Show an input method's documentation and key table (emacs describe-input-method, C-h C-)	
<code>global_display_line_numbers_mode</code>	Toggle line numbers in every window (emacs global-display-line-numbers-mode)	
<code>global_font_lock_mode</code>	Toggle syntax highlighting in every window (emacs global-font-lock-mode)	
<code>global_visual_line_mode</code>	Toggle soft line wrapping in every window (emacs global-visual-line-mode)	
<code>global_display_fill_column_indicator_mode</code>	Toggle the fill-column rule in every window (emacs global-display-fill-column-indicator-mode)	
<code>global_text_scale_adjust</code>	Adjust the terminal font size for every window (emacs global-text-scale-adjust)	
<code>diff_buffers</code>	Diff two open buffers (emacs diff-buffers)	
<code>diff_backup</code>	Diff this file against its backup file (emacs diff-backup)	spacemacs — normal: <code><space>DB</code> , select: <code><space>DB</code>
<code>c_guess</code>	Guess this buffer's indent style and report it (emacs c-guess)	
<code>c_guess_install</code>	Guess this buffer's indent style and apply it (emacs c-guess-install)	
<code>plain_tex_mode</code>	Edit this buffer as plain TeX (emacs plain-tex-mode)	
<code>slitex_mode</code>	Edit this buffer as SliTeX (emacs slitex-mode)	
<code>doctex_mode</code>	Edit this buffer as DocTeX (emacs doctex-mode)	
<code>bibtex_mode</code>	Edit this buffer as a BibTeX database (emacs bibtex-mode)	
<code>nxml_mode</code>	Edit this buffer as XML (emacs nxml-mode)	
<code>org_mode</code>	Edit this buffer as Org (emacs org-mode)	
<code>outline_mode</code>	Outline major mode: C-c C-n/C-p headings, C-c C-d/C-s fold (emacs outline-mode)	
<code>text_mode</code>	Text major mode (emacs text-mode)	
<code>enriched_mode</code>	Enriched major mode: margins and justification (emacs enriched-mode)	
<code>nroff_mode</code>	Nroff major mode: M-n/M-p text lines (emacs nroff-mode)	
<code>nroff_electric_mode</code>	Toggle RET closing the nroff request the line opens (emacs nroff-electric-mode)	
<code>change_log_mode</code>	ChangeLog major mode: entries for files and the functions in them (emacs change-log-mode)	

Name	Description	Keybinds
<code>view_mode</code>	View major mode: SPC/DEL page the buffer; run again to leave (emacs view-mode)	
<code>view_exit</code>	Leave view-mode, staying at the current position (emacs View-exit)	
<code>view_quit</code>	Leave view-mode (emacs View-quit)	
<code>fundamental_mode</code>	Leave the major mode; back to the base keymap (emacs fundamental-mode)	
<code>scheme_mode</code>	Edit this buffer as Scheme (emacs scheme-mode)	
<code>emacs_lisp_mode</code>	Edit this buffer as Emacs Lisp (emacs emacs-lisp-mode)	
<code>lisp_interaction_mode</code>	Open a scratch buffer for evaluating elisp forms (emacs lisp-interaction-mode)	
<code>iso_tex2iso</code>	Convert TeX escape sequences in the region to accented characters (emacs iso-tex2iso)	
<code>iso_iso2tex</code>	Convert accented characters in the region to TeX escapes (emacs iso-iso2tex)	
<code>iso_gtex2iso</code>	Convert German-TeX shorthands in the region to accented characters (emacs iso-gtex2iso)	
<code>iso_iso2gtex</code>	Convert accented characters in the region to German-TeX shorthands (emacs iso-iso2gtex)	
<code>time_stamp</code>	Update the Time-stamp template near the top of the buffer (emacs time-stamp)	
<code>visit_tags_table</code>	Read an etags TAGS file and use it for tag lookups (emacs visit-tags-table)	
<code>etags_regen_mode</code>	Generate the project's tags table automatically instead of running etags by hand (emacs etags-regen-mode)	
<code>xref_etags_mode</code>	Resolve xref definitions through the tags table instead of the language server (emacs xref-etags-mode)	
<code>list_tags</code>	List the tags one file of the tags table defines (emacs list-tags)	
<code>find_tag_other_window</code>	Open a tag's definition in a split (emacs find-tag-other-window)	
<code>tags_search</code>	Search every file in the tags table for a regexp (emacs tags-search)	
<code>tags_next_file</code>	Visit the next file in the tags table (emacs tags-next-file)	
<code>fileloop_continue</code>	Jump to the next match of the running multi-file search (emacs fileloop-continue)	
<code>multi_isearch_buffers</code>	Search every open buffer for a string (emacs multi-isearch-buffers)	
<code>multi_isearch_buffers_regexp</code>	Search every open buffer for a regexp (emacs multi-isearch-buffers-regexp)	
<code>multi_isearch_files</code>	Search a list of files for a string (emacs multi-isearch-files)	
<code>multi_isearch_files_regexp</code>	Search a list of files for a regexp (emacs multi-isearch-files-regexp)	
<code>ispell_comments_and_strings</code>	Spell-check every comment and string in the buffer (emacs ispell-comments-and-strings)	

Name	Description	Keybinds
<code>ispell_comment_or_string_at_point</code>	Spell-check the comment or string at point (emacs ispell-comment-or-string-at-point)	
<code>ispell_hunspell_add_multi_dic</code>	Spell-check against several hunspell dictionaries at once (emacs ispell-hunspell-add-multi-dic)	
<code>disable_command</code>	Put a command on the disabled list so it refuses to run (emacs disable-command)	
<code>enable_command</code>	Take a command off the disabled list (emacs enable-command)	
<code>emerge_buffers</code>	Merge two buffers into a merge buffer (emacs emerge-buffers)	spacemacs — normal: <code><space>Dmbb</code> , select: <code><space>Dmbb</code>
<code>emerge_buffers_with_ancestor</code>	Three-way merge two buffers against their ancestor (emacs emerge-buffers-with-ancestor)	spacemacs — normal: <code><space>Dmb3</code> , select: <code><space>Dmb3</code>
<code>emerge_files_with_ancestor</code>	Three-way merge two files against their ancestor (emacs emerge-files-with-ancestor)	spacemacs — normal: <code><space>Dmf3</code> , select: <code><space>Dmf3</code>
<code>text_scale_set</code>	Set the terminal font size to an absolute level (emacs text-scale-set)	
<code>text_scale_adjust</code>	Keep adjusting the font size with +/-0 (emacs text-scale-adjust)	
<code>text_scale_mode</code>	Turn text scaling off and back on (emacs text-scale-mode)	
<code>sunrise_sunset</code>	Report today's sunrise and sunset (emacs sunrise-sunset)	
<code>lunar_phases</code>	Report this month's moon phases (emacs lunar-phases)	
<code>add_change_log_entry_other_window</code>	Start a ChangeLog entry for this file and function in a split (emacs add-change-log-entry-other-window)	spacemacs — normal: <code><C-x>4a</code> , select: <code><C-x>4a</code> , insert: <code><C-x>4a</code>
<code>diff_add_change_log_entries_other_window</code>	Start ChangeLog entries for every file in this patch (emacs diff-add-change-log-entries-other-window)	
<code>change_log_goto_source</code>	Open the file and function the ChangeLog entry at point names (emacs change-log-goto-source)	
<code>change_log_merge</code>	Merge another ChangeLog into this one, newest entries first (emacs change-log-merge)	
<code>move_file_to_trash</code>	Move this file to the system trash and close the buffer (emacs move-file-to-trash)	
<code>revert_buffer_with_fine_grain</code>	Re-read the file as an edit, keeping point and undo (emacs revert-buffer-with-fine-grain)	
<code>revert_buffer_with_coding_system</code>	Re-read the file with a coding system you name (emacs revert-buffer-with-coding-system)	spacemacs — normal: <code><C-x><ret>r</code> , select: <code><C-x><ret>r</code> , insert: <code><C-x><ret>r</code>
<code>clean_buffer_list</code>	Kill the buffers left untouched for three days (emacs clean-buffer-list)	
<code>switch_to_buffer_other_tab</code>	Show a buffer in a new tab (emacs switch-to-buffer-other-tab)	spacemacs — normal: <code><C-x>tb</code> , select: <code><C-x>tb</code> , insert: <code><C-x>tb</code>
<code>dired_other_tab</code>	Open Dired in a new tab (emacs dired-other-tab)	spacemacs — normal: <code><C-x>td</code> , select: <code><C-x>td</code> , insert: <code><C-x>td</code>
<code>rot13_other_window</code>	Show this buffer ROT13'd in a split (emacs rot13-other-window)	

Name	Description	Keybinds
<code>outline_hide_other</code>	Hide everything but the current entry, its parents and the top-level headings (emacs outline-hide-other)	spacemacs — normal: <code><[- c > @ <[- o ></code> , select: <code><[- c > @ <[- o ></code> , insert: <code><[- c > @ <[- o ></code> emacs, cua — insert: <code><A - [- l ></code>
<code>reposition_window</code>	Scroll so the whole function at point is on screen (emacs reposition-window)	
<code>load</code>	Evaluate an elisp file in the embedded interpreter (emacs load)	
<code>load_library</code>	Load an elisp library by name from the load path (emacs load-library)	
<code>complete_keyword</code>	Complete the keyword before the cursor from the 'complete' sources (vim i_CTRL-N)	
<code>show_paren_mode</code>	Toggle highlighting of the paren matching the one at point (emacs show-paren-mode)	
<code>show_paren_local_mode</code>	Toggle paren-match highlighting in this buffer only (emacs show-paren-local-mode)	
<code>hi_lock_mode</code>	Toggle display of the interactive highlight-regexp patterns (emacs hi-lock-mode)	
<code>delete_selection_mode</code>	Toggle: typing with an active region replaces it (emacs delete-selection-mode)	
<code>electric_indent_mode</code>	Toggle re-indenting the line a newline opens (emacs electric-indent-mode)	
<code>electric_quote_mode</code>	Toggle self-inserting curved quotes (emacs electric-quote-mode)	
<code>electric_layout_mode</code>	Toggle opening a new line after ';' '{' '}' (emacs electric-layout-mode)	
<code>aggressive_indent_mode</code>	Toggle re-indenting the enclosing form after every edit (aggressive-indent-mode, SPC t l)	
<code>which_function_mode</code>	Toggle showing the enclosing function on the status line (emacs which-function-mode)	
<code>compilation_next_file</code>	Jump to the first error of the next file in the run output (emacs compilation-next-file)	
<code>compilation_previous_file</code>	Jump to the first error of the previous file in the run output (emacs compilation-previous-file)	
<code>compile_goto_error</code>	Visit the error the run output is parked on (emacs compile-goto-error)	
<code>kill_compilation</code>	Kill the running compilation (emacs kill-compilation)	
<code>rgrep</code>	Recursive grep under a directory, filtered by a file glob (emacs rgrep)	
<code>lgrep</code>	Grep the files of one directory, not its subdirectories (emacs lgrep)	
<code>grep_find</code>	Recursive grep from the project root (emacs grep-find)	
<code>hs_hide_level</code>	Fold every block at a nesting depth you name (emacs hs-hide-level)	spacemacs — normal: <code><[- c > @ <[- l ></code> , select: <code><[- c > @ <[- l ></code> , insert: <code><[- c > @ <[- l ></code>
<code>hs_show_region</code>	Unfold every fold overlapping the selection (emacs hs-show-region)	
<code>string_rectangle</code>	Replace each line's rectangle segment with a string you type (emacs string-rectangle)	

Name	Description	Keybinds
<code>rectangle_mark_mode</code>	Toggle rectangular region selection (emacs rectangle-mark-mode)	spacemacs — normal: <code><[-x><space></code> , select: <code><[-x><space></code> , insert: <code><[-x><space></code> emacs — insert: <code><[-x><space></code> cua — normal: <code><[-ret></code> , select: <code><[-ret></code> , <code><[-X><space></code> , insert: <code><[-ret></code> , <code><[-x><space></code>
<code>rectangle_exchange_point_and_mark</code>	Move point to the opposite corner of the rectangle (emacs rectangle-exchange-point-and-mark)	
<code>customize_group</code>	Edit the settings of one configuration group (emacs customize-group)	
<code>customize_apropos</code>	Search the settings whose name or value matches a pattern (emacs customize-apropos)	
<code>customize_changed</code>	List the settings changed from their built-in defaults (emacs customize-changed)	
<code>customize_saved</code>	List the settings saved in the config file (emacs customize-saved)	
<code>customize_unsaved</code>	List the settings set for this session but not saved (emacs customize-unsaved)	
<code>customize_dirlocals</code>	Edit this project's directory-local settings (emacs customize-dirlocals)	
<code>org_metaleft</code>	Org: promote the heading at point (emacs org-metaleft)	
<code>org_metaright</code>	Org: demote the heading at point (emacs org-metaright)	
<code>org_metaup</code>	Org: move the subtree at point above its previous sibling (emacs org-metaup)	
<code>org_metadown</code>	Org: move the subtree at point below its next sibling (emacs org-metadown)	
<code>org_shifttab</code>	Org: cycle the whole buffer show-all -> overview -> contents (emacs org-shifttab)	
<code>c_ts_mode_set_style</code>	Report the C indentation style (emacs c-ts-mode-set-style)	
<code>diff_ignore_whitespace_hunk</code>	Re-diff the hunk at point ignoring whitespace changes (emacs diff-ignore-whitespace-hunk)	
<code>diff_refresh_hunk</code>	Re-diff the hunk at point (emacs diff-refresh-hunk)	
<code>diff_ediff_patch</code>	Apply this patch and review it side by side (emacs diff-ediff-patch)	spacemacs — normal: <code><space>Dfp</code> , select: <code><space>Dfp</code>
<code>ediff_patch_buffer</code>	Patch a buffer with a patch buffer/file and review it (emacs ediff-patch-buffer, SPC D b p)	
<code>diff_add_change_log_entry_other_window</code>	Start a ChangeLog entry for the hunk at point (emacs add-change-log-entry-other-window, in Diff mode)	
<code>next_completion</code>	Move to the next completion candidate (emacs next-completion)	
<code>previous_completion</code>	Move to the previous completion candidate (emacs previous-completion, M-)	
<code>term_line_mode</code>	Terminal: edit input locally, Enter sends the line (emacs term-line-mode)	spacemacs — normal: <code><[-c><[-j></code> , select: <code><[-c><[-j></code> , insert: <code><[-c><[-j></code>

Name	Description	Keybinds
<code>comint_completion_at_point</code>	Shell buffer: complete the command or file name before point (emacs completion-at-point in Shell mode)	
<code>window_configuration_to_register</code>	Save the window configuration in a register (emacs window-configuration-to-register)	spacemacs — normal: <code><[-x>rw</code> , select: <code><[-x>rw</code> , insert: <code><[-x>rw</code>
<code>do_auto_save</code>	Write every modified buffer to its auto-save file now (emacs do-auto-save)	
<code>ffap_next</code>	Visit the next file named in this buffer (emacs ffap-next)	
<code>lossage_size</code>	Set how many recent keystrokes view-lossage remembers (emacs lossage-size)	
<code>tabulated_list_sort</code>	Sort the tabulated list by the column at point (emacs tabulated-list-sort)	
<code>emerge_auto_advance</code>	Merge: choosing a side moves to the next difference (emacs emerge-auto-advance)	
<code>emerge_skip_prefers</code>	Merge: step over the differences already decided (emacs emerge-skip-prefers)	
<code>tabulated_list_narrow_current_column</code>	Narrow the tabulated-list column at point (emacs tabulated-list-narrow-current-column)	
<code>tabulated_list_widen_current_column</code>	Widen the tabulated-list column at point (emacs tabulated-list-widen-current-column)	
<code>ffap_menu</code>	Pick from the files named in this buffer and visit one (emacs ffap-menu)	
<code>auto_save_mode</code>	Toggle auto-saving of the buffer (emacs auto-save-mode)	
<code>recover_session</code>	Recover a file from the auto-save data of an interrupted session (emacs recover-session)	
<code>frameset_to_register</code>	Save the frame's window configuration in a register (emacs frameset-to-register)	spacemacs — normal: <code><[-x>rf</code> , select: <code><[-x>rf</code> , insert: <code><[-x>rf</code>
<code>shell_dynamic_complete_command</code>	Shell buffer: complete the command name before point from PATH (emacs shell-dynamic-complete-command)	
<code>term_char_mode</code>	Terminal: send every key straight to the process (emacs term-char-mode)	spacemacs — normal: <code><[-c><[-k></code> , select: <code><[-c><[-k></code> , insert: <code><[-c><[-k></code>
<code>term_pager_toggle</code>	Terminal: stop output after each screenful (emacs term-pager-toggle)	spacemacs — normal: <code><[-c><[-q></code> , select: <code><[-c><[-q></code> , insert: <code><[-c><[-q></code>
<code>switch_to_completions</code>	Move into the list of completions (emacs switch-to-completions)	
<code>previous_matching_history_element</code>	Recall the newest older history entry matching the regexp on the line (emacs previous-matching-history-element)	
<code>next_matching_history_element</code>	Recall the oldest newer history entry matching the regexp on the line (emacs next-matching-history-element)	
<code>line_number_mode</code>	Toggle the cursor position display in the mode line (emacs line-number-mode)	
<code>column_number_mode</code>	Toggle the column number display in the mode line (emacs column-number-mode)	

Name	Description	Keybinds
<code>size_indication_mode</code>	Toggle the buffer size display in the mode line (emacs size-indication-mode)	spacemacs — normal: <code><space>tmt</code> , select: <code><space>tmt</code> spacemacs — normal: <code><space>tmb</code> , select: <code><space>tmb</code>
<code>display_time</code>	Toggle the clock in the mode line (emacs display-time)	
<code>display_battery_mode</code>	Toggle the battery charge in the mode line (emacs display-battery-mode)	
<code>make_local_variable</code>	Give a variable a buffer-local binding in this document (emacs make-local-variable)	
<code>make_variable_buffer_local</code>	Make a variable become buffer-local whenever it is set (emacs make-variable-buffer-local)	
<code>kill_local_variable</code>	Drop this document's local binding of a variable (emacs kill-local-variable)	
<code>setq_default</code>	Set a variable's global value, seen by every document without a local binding (emacs setq-default)	
<code>default_value</code>	Show a variable's global value, ignoring this document's local binding (emacs default-value)	
<code>add_hook</code>	Add a function to a hook variable (emacs add-hook)	
<code>remove_hook</code>	Take a function off a hook variable (emacs remove-hook)	
<code>org_schedule</code>	Prompt for a date and add/update SCHEDULED: on the heading at point (emacs org-schedule)	
<code>org_deadline</code>	Prompt for a date and add/update DEADLINE: on the heading at point (emacs org-deadline)	
<code>org_clock_toggle</code>	Start or stop the org task clock on the heading at point (emacs org-clock-in/org-clock-out)	
<code>outline_hide_by_heading_regexp</code>	Prompt for a regexp and fold the subtree of every heading matching it (emacs outline-hide-by-heading-regexp)	
<code>outline_show_by_heading_regexp</code>	Prompt for a regexp and reveal the subtree of every heading matching it (emacs outline-show-by-heading-regexp)	
<code>dbx</code>	Run the dbx debugger in a comint buffer (emacs M-x dbx)	
<code>xdb</code>	Run the xdb debugger in a comint buffer (emacs M-x xdb)	
<code>sdb</code>	Run the sdb debugger in a comint buffer (emacs M-x sdb)	
<code>lldb</code>	Run the LLDB debugger in a comint buffer (emacs M-x lldb)	
<code>jdb</code>	Run the Java debugger in a comint buffer (emacs M-x jdb)	
<code>pdb</code>	Run the Python debugger on this file in a comint buffer (emacs M-x pdb)	
<code>perlldb</code>	Run the Perl debugger on this file in a comint buffer (emacs M-x perlldb)	
<code>guiler</code>	Run the Guile debugger in a comint buffer (emacs M-x guiler)	
<code>run_lisp</code>	Run an inferior Common Lisp in a comint buffer (emacs run-lisp)	

Name	Description	Keybinds
<code>run_scheme</code>	Run an inferior Scheme in a comint buffer (emacs run-scheme)	
<code>lisp_eval_defun</code>	Send the top-level form at point to the inferior Lisp (emacs lisp-eval-defun)	
<code>rebuild_mail_abbrevs</code>	Re-read the mail aliases in ~/.mailrc (emacs rebuild-mail-abbrevs)	
<code>merge_mail_abbrevs</code>	Merge the mail aliases of another file into the table (emacs merge-mail-abbrevs)	
<code>define_mail_abbrev</code>	Define a mail alias and save it to ~/.mailrc (emacs define-mail-abbrev)	
<code>mail_abbrev_insert_alias</code>	Pick a mail alias and insert its addresses at point (emacs mail-abbrev-insert-alias)	
<code>mail_abbrev_complete_alias</code>	Expand the mail alias before point into its addresses (emacs mail-abbrev-complete-alias)	
<code>message_tab</code>	In a draft's headers complete the mail alias before point, else indent (emacs message-tab)	
<code>message_goto_fcc</code>	Go to the draft's Fcc: header, creating it if absent (emacs message-goto-fcc)	
<code>goto_reply_to</code>	Go to the draft's Reply-To: header, creating it if absent (emacs goto-reply-to)	
<code>goto_followup_to</code>	Go to the draft's Followup-To: header, creating it if absent (emacs goto-followup-to)	
<code>message_yank_original</code>	Insert the message being replied to, cited (emacs message-yank-original)	
<code>message_yank_prefix</code>	Set the prefix message-yank-original puts on cited lines (emacs message-yank-prefix)	
<code>mail_fill_yanked_message</code>	Refill the cited paragraphs of the yanked message (emacs mail-fill-yanked-message)	
<code>define_mail_user_agent</code>	Register a mail user agent's compose/send/abort/hook functions (emacs define-mail-user-agent)	
<code>add_dir_local_variable</code>	Set a variable for this whole tree in .dir-locals.el (emacs add-dir-local-variable)	spacemacs — normal: <code><space>fvd</code> , select: <code><space>fvd</code>
<code>delete_dir_local_variable</code>	Remove a variable from the tree's .dir-locals.el (emacs delete-dir-local-variable)	
<code>copy_file_locals_to_dir_locals</code>	Copy this file's file-local variables into .dir-locals.el (emacs copy-file-locals-to-dir-locals)	
<code>copy_dir_locals_to_file_locals</code>	Write the tree's directory-local variables into this file's Local Variables block (emacs copy-dir-locals-to-file-locals)	
<code>copy_dir_locals_to_file_locals_prop_line</code>	Write the tree's directory-local variables into this file's -*- prop line (emacs copy-dir-locals-to-file-locals-prop-line)	
<code>dir_locals_set_class_variables</code>	Register a named set of directory-local variables (emacs dir-locals-set-class-variables)	
<code>dir_locals_set_directory_class</code>	Apply a registered directory-local class to a directory (emacs dir-locals-set-directory-class)	

Name	Description	Keybinds
<code>top_level</code>	Close every open overlay and go back to the editor (emacs top-level)	
<code>report_emacs_bug</code>	Compose a bug report with the version and system information (emacs report-emacs-bug)	
<code>report_spacemacs_issue</code>	Open the Spacemacs issue tracker pre-filled; C-u adds your last keys (Spacemacs SPC h I)	
<code>align_highlight_rule</code>	Highlight what an alignment rule matches (emacs align-highlight-rule)	
<code>align_unhighlight_rule</code>	Remove the alignment rule highlighting (emacs align-unhighlight-rule)	
<code>font_lock_add_keywords</code>	Highlight an extra regexp on top of the syntax highlighting (emacs font-lock-add-keywords)	
<code>font_lock_remove_keywords</code>	Stop highlighting an extra regexp (emacs font-lock-remove-keywords)	
<code>edit_tab_stops</code>	Edit the list of tab-stop columns (emacs edit-tab-stops)	
<code>fortran_window_create</code>	Mark column 72, the fixed-form Fortran line limit (emacs fortran-window-create)	
<code>fortran_window_create_momentarily</code>	Mark column 72 until the next key (emacs fortran-window-create-momentarily)	
<code>recode_region</code>	Re-decode the region with the coding system it was really in (emacs recode-region)	
<code>make_frame_command</code>	Create a new frame showing this buffer (emacs make-frame-command)	spacemacs — normal: <code><C-x>52</code> , <code><space>Fn</code> , select: <code><C-x>52</code> , <code><space>Fn</code> , insert: <code><C-x>52</code>
<code>make_frame_on_display</code>	Create a frame on a named display; one display on a tty (emacs make-frame-on-display)	
<code>make_frame_on_monitor</code>	Create a frame on a named monitor; one monitor on a tty (emacs make-frame-on-monitor)	
<code>toggle_frame_fullscreen</code>	Toggle the frame's fullscreen parameter; no visible effect on a tty (emacs toggle-frame-fullscreen, F11)	
<code>toggle_frame_maximized</code>	Toggle the frame's maximized parameter; no visible effect on a tty (emacs toggle-frame-maximized, M-F10)	spacemacs — normal: <code><space>TM</code> , select: <code><space>TM</code>
<code>iconify_or_deiconify_frame</code>	Iconify or deiconify the frame; no visible effect on a tty (emacs iconify-or-deiconify-frame, C-z under X)	
<code>dos_mode25</code>	Resize the terminal to 80x25 (emacs dos-mode25)	
<code>dos_mode4350</code>	Resize the terminal to 80x50 (emacs dos-mode4350)	
<code>w32_set_console_codepage</code>	Set the codepage the hosted terminal's output is decoded with (emacs w32-set-console-codepage)	
<code>w32_add_untranslated_filesystem</code>	Add a filesystem that never gets CR/LF translation; bookkeeping-only on Unix (emacs w32-add-untranslated-filesystem)	
<code>w32_remove_untranslated_filesystem</code>	Remove a filesystem from the untranslated list (emacs w32-remove-untranslated-filesystem)	

Name	Description	Keybinds
<code>create_fontset_from_fontset_spec</code>	Parse a fontset spec and record the named fontset; no visible effect on a tty (emacs <code>create-fontset-from-fontset-spec</code>)	
<code>set_fontset_font</code>	Assign a font to a character range in a fontset; no visible effect on a tty (emacs <code>set-fontset-font</code>)	
<code>minibuffer_inactive_mode</code>	Major mode for the minibuffer when no minibuffer is active (emacs <code>minibuffer-inactive-mode</code>)	
<code>mouse_avoidance_mode</code>	Warp the mouse pointer away from the cursor; no visible effect on a tty (emacs <code>mouse-avoidance-mode</code>)	
<code>switch_to_minibuffer</code>	Select the active minibuffer window (spacemacs <code>SPC w b</code> , <code>switch-to-minibuffer-window</code>)	
<code>fortune_to_signature</code>	Insert the output of the fortune program as the message signature (emacs <code>fortune-to-signature</code>)	
<code>category_set_mnemonics</code>	Report the category mnemonics of the character at point (emacs <code>category-set-mnemonics</code>)	
<code>char_category_set</code>	Report the category set of the character at point (emacs <code>char-category-set</code>)	
<code>clone_frame</code>	Create a new frame with a copy of this frame's layout (emacs <code>clone-frame</code>)	spacemacs — normal: <code><C-x>5c</code> , select: <code><C-x>5c</code> , insert: <code><C-x>5c</code>
<code>delete_frame</code>	Delete the displayed frame (emacs <code>delete-frame</code>)	spacemacs — normal: <code><C-x>50</code> , select: <code><C-x>50</code> , insert: <code><C-x>50</code>
<code>delete_other_frames</code>	Delete every frame but this one (emacs <code>delete-other-frames</code>)	spacemacs — normal: <code><C-x>51</code> , <code><space>FD</code> , select: <code><C-x>51</code> , <code><space>FD</code> , insert: <code><C-x>51</code>
<code>other_frame</code>	Display the next frame (emacs <code>other-frame</code>)	spacemacs — normal: <code>]t</code> , <code><C-x>5o</code> , select: <code><C-x>5o</code> , insert: <code><C-x>5o</code> vim — normal: <code>]t</code>
<code>undeleate_frame</code>	Bring back the most recently deleted frame (emacs <code>undeleate-frame</code>)	spacemacs — normal: <code><C-x>5u</code> , select: <code><C-x>5u</code> , insert: <code><C-x>5u</code>
<code>undeleate_frame_mode</code>	Record deleted frames so <code>undeleate-frame</code> can bring them back (emacs <code>undeleate-frame-mode</code>)	
<code>select_frame_by_name</code>	Pick a frame by name and display it (emacs <code>select-frame-by-name</code>)	
<code>set_frame_name</code>	Rename the displayed frame (emacs <code>set-frame-name</code>)	
<code>find_file_other_frame</code>	Open a file in a new frame (emacs <code>find-file-other-frame</code>)	spacemacs — normal: <code><C-x>5f</code> , select: <code><C-x>5f</code> , insert: <code><C-x>5f</code>
<code>find_file_read_only_other_frame</code>	Open a file read-only in a new frame (emacs <code>find-file-read-only-other-frame</code>)	spacemacs — normal: <code><C-x>5r</code> , select: <code><C-x>5r</code> , insert: <code><C-x>5r</code>
<code>switch_to_buffer_other_frame</code>	Display a buffer in a new frame (emacs <code>switch-to-buffer-other-frame</code>)	spacemacs — normal: <code><C-x>5b</code> , select: <code><C-x>5b</code> , insert: <code><C-x>5b</code>
<code>display_buffer_other_frame</code>	Put a buffer in another frame without selecting it (emacs <code>display-buffer-other-frame</code> , <code>SPC F B</code>)	
<code>dired_other_frame</code>	Open Dired in a new frame (emacs <code>dired-other-frame</code>)	spacemacs — normal: <code><C-x>5d</code> , <code><space>F0</code> , select: <code><C-x>5d</code> , <code><space>F0</code> , insert: <code><C-x>5d</code>
<code>other_frame_prefix</code>	Display the next command's buffer in a new frame (emacs <code>other-frame-prefix</code>)	spacemacs — normal: <code><C-x>55</code> , select: <code><C-x>55</code> , insert: <code><C-x>55</code>

Name	Description	Keybinds
<code>other_window_prefix</code>	Display the next command's buffer in another window (emacs other-window-prefix)	spacemacs — normal: <code><[-x>44</code> , select: <code><[-x>44</code> , insert: <code><[-x>44</code>
<code>other_tab_prefix</code>	Display the next command's buffer in a new tab (emacs other-tab-prefix)	spacemacs — normal: <code><[-x>tt</code> , select: <code><[-x>tt</code> , insert: <code><[-x>tt</code>
<code>display_buffer</code>	Show a buffer in another window without selecting it (emacs display-buffer)	spacemacs — normal: <code><[-x>4<[-o></code> , select: <code><[-x>4<[-o></code> , insert: <code><[-x>4<[-o></code>
<code>server_start</code>	Start the zmax server so clients can hand it files (emacs server-start)	
<code>server_edit</code>	Finish with this buffer and let the waiting client return (emacs server-edit)	spacemacs — normal: <code><[-x>#</code> , select: <code><[-x>#</code> , insert: <code><[-x>#</code>
<code>server_edit_abort</code>	Abandon this buffer's edit and tell the waiting client (emacs server-edit-abort)	
<code>server_generate_key</code>	Mint the shared key clients must authenticate with (emacs server-generate-key)	
<code>server_eval_at</code>	Evaluate an expression in another zmax server (emacs server-eval-at)	
<code>mouse_set_point</code>	Move point to the last click (emacs mouse-set-point)	
<code>mouse_goto_tag</code>	Go to the definition of the symbol at the click (vim)	
<code>mouse_pop_tag</code>	Pop the tag/jump stack (vim , CTRL-T)	
<code>mouse_search_word_forward</code>	Search forward for the word at the click (vim , *)	
<code>mouse_search_word_backward</code>	Search backward for the word at the click (vim , #)	
<code>paste_after_reindent</code>	Paste after, indent shifted to the current line (vim]p)	spacemacs, vim — normal: <code>]p</code>
<code>paste_before_reindent</code>	Paste before, indent shifted to the current line (vim [p)	spacemacs, vim — normal: <code>[p</code>
<code>mouse_paste_before</code>	Paste before the click, adjusting indent (vim [, [p)	
<code>mouse_paste_after</code>	Paste after the click, adjusting indent (vim],]p)	
<code>mouse_scroll_left</code>	Move the window <code>mousetscroll=hor:N</code> columns left (vim)	
<code>mouse_scroll_right</code>	Move the window <code>mousetscroll=hor:N</code> columns right (vim)	
<code>mouse_scroll_page_left</code>	Move the window one page left (vim)	
<code>mouse_scroll_page_right</code>	Move the window one page right (vim)	
<code>mouse_scroll_page_up</code>	Move the window one page up (vim)	
<code>mouse_scroll_page_down</code>	Move the window one page down (vim)	
<code>mouse_set_region</code>	Set the region from the drag's start to the click (emacs mouse-set-region)	
<code>mouse_save_then_kill</code>	Extend the region to the click and copy it; again to kill it (emacs mouse-save-then-kill)	
<code>mouse_yank_at_click</code>	Move point to the click and yank there (emacs mouse-yank-at-click)	
<code>mouse_yank_primary</code>	Yank the primary selection at point (emacs mouse-yank-primary)	
<code>mouse_wheel_mode</code>	Turn wheel scrolling on or off (emacs mouse-wheel-mode)	

Name	Description	Keybinds
<code>mouse_buffer_menu</code>	The buffer menu, from the mouse (emacs mouse-buffer-menu)	
<code>browse_url_at_mouse</code>	Open the URL under the mouse in a browser (emacs browse-url-at-mouse)	
<code>mouse_start_secondary</code>	Anchor the secondary selection at the click (emacs mouse-start-secondary)	
<code>mouse_set_secondary</code>	Set the secondary selection from the anchor to the click (emacs mouse-set-secondary)	
<code>mouse_yank_secondary</code>	Insert the secondary selection at the click (emacs mouse-yank-secondary)	
<code>mouse_secondary_save_then_kill</code>	Copy the secondary selection; again to kill it (emacs mouse-secondary-save-then-kill)	
<code>overwrite_mode</code>	Typing replaces the character under point (emacs overwrite-mode)	
<code>binary_overwrite_mode</code>	Overwrite mode that replaces newlines too (emacs binary-overwrite-mode)	
<code>compose_mail_other_frame</code>	Open a mail draft in a new frame (emacs compose-mail-other-frame)	spacemacs — normal: <code><C-x>5m</code> , select: <code><C-x>5m</code> , insert: <code><C-x>5m</code>
<code>context_menu_mode</code>	Turn the right-button popup menu on or off (emacs context-menu-mode)	
<code>reveal_mode</code>	Reveal hidden text while point is inside it (emacs reveal-mode)	
<code>desktop_save_mode</code>	Save the desktop on exit (emacs desktop-save-mode)	
<code>visual_wrap_prefix_mode</code>	Indent soft-wrapped continuation lines under their line (emacs visual-wrap-prefix-mode)	
<code>open_dribble_file</code>	Record every key into a file (emacs open-dribble-file)	
<code>winner_mode</code>	Record window-layout changes so winner-undo can take them back (emacs winner-mode)	

Language Support

Language	Syntax Highlighting	Treesitter Textobjects	Auto Indent	Code Navigation Tags	Rainbow Brackets	Default language servers
ada	✓	✓				ada_language_server
adl	✓	✓	✓			
agda	✓					als
alda						
alloy	✓					
amber	✓	✓	✓	✓	✓	amber-lsp
apache asciidoc	✓					ltex-ls-plus
astro	✓					astro-ls
autohotkey	✓					autohotkey_lsp
awk	✓	✓				awk-language-server
bash	✓	✓	✓	✓	✓	zshrs
basic	✓	✓	✓	✓		
bass	✓					bass
batch						
beancount	✓					beancount-language-server
bibtex	✓					texlab, ltex-ls-plus
bicep	✓					bicep-langserver
bitbake	✓					bitbake-language-server
blade	✓	✓			✓	
blueprint	✓					blueprint-compiler
bovex						
c	✓	✓	✓	✓	✓	clangd

Language	Syntax Highlighting	Treesitter Textobjects	Auto Indent	Code Navigation Tags	Rainbow Brackets	Default language servers
c-sharp	✓	✓	✓	✓	✓	roslyn-language-server, OmniSharp, csharp-ls
c3	✓					c3-lsp
cabal						haskell-language-server-wrapper
caddyfile	✓	✓	✓			
cairo	✓	✓	✓			cairo-language-server
capnp	✓		✓			
cel	✓					
cfengine						
chuck	✓					
circom	✓					circom-lsp
clarity	✓					clarinet
clojure	✓	✓	✓	✓	✓	clojure-lsp
cmake	✓	✓	✓		✓	neomakelisp, cmake-language-server
codeql	✓	✓				codeql
coffeescript	✓					coffeesense-language-server
comment	✓					
common-lisp	✓	✓		✓	✓	cl-lsp
concerto	✓	✓	✓			
coq	✓					coq-lsp
cpon	✓		✓			
cpp	✓	✓	✓	✓	✓	clangd
cross-config	✓	✓			✓	taplo, tombi
crystal	✓	✓	✓	✓	✓	crystalline, ameba-ls
css	✓	✓	✓		✓	vscode-css-language-server
csv	✓					
cue	✓					cue
cylc	✓	✓	✓			
cython	✓		✓	✓		

Language	Syntax Highlighting	Treesitter Textobjects	Auto Indent	Code Navigation Tags	Rainbow Brackets	Default language servers
d	✓	✓	✓			<code>serve-d</code>
dart	✓	✓	✓	✓	✓	<code>dart</code>
dbml	✓					
debian	✓					<code>debian-lsp</code>
devicetree	✓					<code>dts-lsp</code>
dhall	✓	✓				<code>dhall-lsp-server</code>
diff	✓					
djot	✓					
docker-bake	✓	✓	✓	✓	✓	<code>docker-language-server</code>
docker-compose	✓	✓	✓			<code>docker-compose-langserver</code> , <code>yaml-language-server</code> , <code>docker-language-server</code>
dockerfile	✓	✓				<code>docker-langserver</code> , <code>docker-language-server</code>
dot	✓					<code>dot-language-server</code>
doxyfile	✓	✓	✓	✓		
drools						<code>drools-lsp</code>
dtd	✓					
dune	✓					
dunstrc	✓					
earthfile	✓	✓	✓			<code>earthlyls</code>
ebnf	✓					<code>ebnfer</code>
edoc	✓					
eex	✓					
eiffel	✓	✓	✓			<code>eiffel-language-server</code>
ejs	✓					
elisp	✓			✓		
elixir	✓	✓	✓	✓	✓	<code>elixir-ls</code> , <code>expert</code>

Language	Syntax Highlighting	Treesitter Textobjects	Auto Indent	Code Navigation Tags	Rainbow Brackets	Default language servers
elm	✓	✓		✓	✓	<code>elm-language-server</code>
elvish	✓					<code>elvish</code>
embedded-perl	✓					
env	✓	✓				
erb	✓					
erlang	✓	✓	✓	✓	✓	<code>erlang_ls</code> , <code>elp</code>
esdl	✓					
factor						
faust	✓					<code>faustlsp</code>
fennel	✓				✓	<code>fennel-ls</code>
fga	✓	✓	✓			
fidl	✓					
fish	✓	✓	✓			<code>fish-lsp</code>
flatbuffers	✓					
forth	✓					<code>forth-lsp</code>
fortran	✓		✓			<code>fortls</code>
fountain	✓					<code>fountain-lsp-server</code>
freebasic	✓	✓	✓	✓		
fsharp	✓				✓	<code>fsautocomplete</code>
gas	✓	✓				<code>asm-lsp</code>
gdscript	✓	✓	✓	✓	✓	
gemin	✓					
gherkin	✓					
ghostty	✓					
git-attributes	✓					
git-cliff-config	✓	✓			✓	<code>taplo</code> , <code>tombi</code>
git-commit	✓	✓				<code>commit-lsp</code>
git-config	✓	✓		✓		
git-ignore	✓					
git-notes	✓					
git-rebase	✓					
github-action	✓	✓	✓		✓	<code>actions-languageserver</code> , <code>yaml-language-server</code> , <code>zizmor</code>
gitlab-ci	✓	✓	✓	✓	✓	<code>yaml-language-server</code> , <code>gitlab-ci-ls</code>

Language	Syntax Highlighting	Treesitter Textobjects	Auto Indent	Code Navigation Tags	Rainbow Brackets	Default language servers
gjs	✓	✓	✓	✓		typescript-language-server, vscode-eslint-language-server, ember-language-server
gleam	✓	✓			✓	gleam
glimmer	✓					ember-language-server
gls	✓	✓	✓	✓	✓	gls_analyzer, glsld
gn	✓					
gnuplot	✓					
go	✓	✓	✓	✓	✓	gopls, golangci-lint-langserver
go-format-string	✓				✓	
godot-resource	✓	✓		✓		
gomod	✓					gopls
gotmpl	✓					gopls
gowork	✓					gopls
gpr	✓					ada_language_server
graphql	✓	✓			✓	graphql-lsp
gren	✓	✓				
groovy	✓				✓	groovy-language-server
gts	✓	✓	✓	✓		typescript-language-server, vscode-eslint-language-server, ember-language-server
hare	✓	✓				hare-lsp

Language	Syntax Highlighting	Treesitter Textobjects	Auto Indent	Code Navigation Tags	Rainbow Brackets	Default language servers
haskell	✓	✓		✓	✓	haskell-language-server-wrapper
haskell-literate	✓					haskell-language-server-wrapper
haskell-persistent	✓					
haxe	✓			✓		
hcl	✓	✓	✓		✓	terraform-ls
hdl	✓					hdl
heex	✓	✓				elixir-ls , expert
helm	✓					helm_ls
hocon	✓	✓	✓			
hoon	✓					
hosts	✓					
html	✓	✓	✓		✓	vscode-html-language-server , superhtml
htmldjango	✓					djls , vscode-html-language-server , superhtml
http	✓					kulala-ls
hurl	✓	✓	✓			
hy	✓					hyuga
hyprrlang	✓		✓	✓		hyprrls
idris	✓					idris2-lsp
iex	✓					
ini	✓			✓		
ink	✓					
inko	✓	✓	✓	✓		
janet	✓		✓		✓	
java	✓	✓	✓	✓	✓	
javascript	✓	✓	✓	✓	✓	jdtls
						typescript-language-server
jinja	✓					
jjconfig	✓	✓	✓			taplo , tombi

Language	Syntax Highlighting	Treesitter Textobjects	Auto Indent	Code Navigation Tags	Rainbow Brackets	Default language servers
jjdescription	✓					
jjrevset	✓					
jjtemplate	✓					
jq	✓	✓				jq-lsp
jsdoc	✓					
json	✓	✓	✓		✓	vscode-json-language-server
json-ld	✓	✓	✓		✓	vscode-json-language-server
json5	✓	✓	✓		✓	
jsonc	✓	✓	✓		✓	vscode-json-language-server
jsonnet	✓					jsonnet-language-server
jsx	✓	✓	✓	✓	✓	typescript-language-server
julia	✓	✓	✓	✓	✓	julia
just	✓	✓	✓	✓		just-lsp
kcl	✓					kcl-language-server
kconfig	✓		✓			
kdl	✓	✓	✓	✓	✓	
kivy						
klog	✓					
koka	✓		✓			koka
kotlin	✓	✓	✓	✓	✓	kotlin-language-server
koto	✓	✓	✓		✓	koto-ls
latex	✓	✓	✓			texlab, ltex-ls-plus
ld	✓		✓			
ldif	✓					
lean	✓					lake
ledger	✓					

Language	Syntax Highlighting	Treesitter Textobjects	Auto Indent	Code Navigation Tags	Rainbow Brackets	Default language servers
less	✓		✓		✓	<code>vscode-css-language-server</code>
llvm	✓	✓	✓			
llvm-mir	✓	✓	✓			
llvm-mir-yaml	✓		✓			
log						
lpf	✓					
lua	✓	✓	✓	✓	✓	<code>lua-language-server</code>
lua-format-string	✓					
luap	✓					
luau	✓	✓	✓			<code>luau-lsp</code>
mail	✓	✓			✓	
make	✓		✓			
markdoc	✓					<code>markdoc-ls</code>
markdown	✓			✓		<code>marksman</code> , <code>markdown-oxide</code> , <code>rumdl</code> , <code>ltex-ls-plus</code>
markdown-rustdoc	✓					
markdown.inline	✓					
matlab	✓	✓	✓			
mercury						
mermaid	✓					
meson	✓		✓			<code>merman-lsp</code> <code>mesonlsp</code>
mint						<code>mint</code>
miseconfig	✓	✓	✓			<code>taplo</code> , <code>tombi</code>
mojo	✓	✓	✓			<code>pixi</code>
move	✓		✓			
msbuild	✓		✓			
nasm	✓	✓				<code>asm-lsp</code>
nearley	✓				✓	
nestedtext	✓	✓	✓			
nginx	✓					<code>nginx-language-server</code>
nickel	✓		✓			<code>nls</code>
nim	✓	✓	✓		✓	<code>nimlangserver</code>
nix	✓	✓	✓	✓	✓	<code>nil</code> , <code>nixd</code>
nu	✓	✓	✓			<code>nu</code> , <code>nu-lint</code>

Language	Syntax Highlighting	Treesitter Textobjects	Auto Indent	Code Navigation Tags	Rainbow Brackets	Default language servers
nunjucks	✓					
ocaml	✓	✓	✓		✓	ocaml lsp
ocaml- interface	✓					ocaml lsp
odin	✓	✓	✓			ols
ohm	✓	✓	✓			
opencl	✓	✓	✓			clangd
openscad	✓					openscad- lsp
org	✓					ltex-ls- plus
pact	✓					pact-lsp
pascal	✓	✓				pasls
passwd	✓					
pem	✓					
penrose	✓	✓				
perl	✓	✓	✓	✓	✓	perlnavigator
pest	✓	✓	✓			pest- language- server
php	✓	✓	✓	✓	✓	intelephense
php-only	✓			✓		
picat	✓	✓	✓	✓	✓	
pip- requirements	✓					
pkgbuild	✓	✓	✓			termux- language- server, bash- language- server
pkd	✓		✓			pkd-lsp
plantuml	✓					plantuml- lsp
po	✓	✓				
pod	✓					
ponylang	✓	✓	✓			pony-lsp
powershell	✓				✓	
prisma	✓	✓				prisma- language- server
prolog	✓		✓			swipl
properties	✓	✓				
protobuf	✓	✓	✓	✓		buf, pb, protols
proverif	✓					

Language	Syntax Highlighting	Treesitter Textobjects	Auto Indent	Code Navigation Tags	Rainbow Brackets	Default language servers
prql	✓					
ptx	✓		✓			
pug	✓					
puppet	✓					puppet-language-server
purescript	✓	✓				purescript-language-server
python	✓	✓	✓	✓	✓	ty, ruff, jedi-language-server, pylsp, zuban
qml	✓	✓	✓			qmls
qmv						
quarto	✓		✓			
quint	✓					quint-language-server
r	✓	✓	✓	✓	✓	R
racket	✓		✓		✓	racket
reason	✓					reason-language-server
regex	✓				✓	
rego	✓					regols
rescript	✓	✓				rescript-language-server
ripple	✓			✓	✓	ripple-language-server
rmarkdown	✓		✓			R
robot	✓		✓			robotcode, robotframework_ls
robots.txt	✓	✓		✓		
ron	✓		✓	✓	✓	ron-lsp
rpmspec	✓					
rshtml	✓	✓				rshtml-analyzer, vscode-html-language-server, superhtml

Language	Syntax Highlighting	Treesitter Textobjects	Auto Indent	Code Navigation Tags	Rainbow Brackets	Default language servers
rst	✓			✓		python3, ltex-ls-plus
ruby	✓	✓	✓	✓	✓	ruby-lsp, solargraph
rust	✓	✓	✓	✓	✓	rust-analyzer
rust-format-args	✓					
rust-format-args-macro	✓	✓	✓		✓	
sage	✓	✓				
scala	✓	✓	✓	✓	✓	metals
scfg	✓					
scheme	✓	✓	✓	✓	✓	guile-lsp-server, chicken-lsp-server
scss	✓		✓		✓	vscode-css-language-server
shellcheckrc	✓	✓				
slang	✓	✓	✓			slangd
slint	✓	✓	✓			slint-lsp
slisp	✓			✓		
sls	✓					salt_lsp_server
smali	✓		✓			smalisp
smithy	✓					cs
sml	✓					millet-ls
snakemake	✓		✓			pylsp
solidity	✓	✓			✓	solc
sourcepawn	✓	✓				sourcepawn-studio
spade	✓		✓		✓	swim
sparql	✓					sparql-language-server
spicedb	✓			✓		
sql	✓	✓			✓	sqls, sql-language-server
ssh_client_config	✓					

Language	Syntax Highlighting	Treesitter Textobjects	Auto Indent	Code Navigation Tags	Rainbow Brackets	Default language servers
starlark	✓	✓	✓		✓	<code>starpls</code> , <code>buck2</code>
strace	✓			✓		
strictdoc	✓					
stryke	✓					<code>stryke</code>
styx	✓	✓	✓		✓	<code>styx</code>
supercollider	✓					
svelte	✓	✓	✓	✓	✓	<code>svelteserver</code>
sway	✓	✓	✓			<code>forc</code>
swift	✓	✓	✓	✓	✓	<code>sourcekit-lsp</code>
systemd	✓			✓		<code>systemd-lsp</code>
systemverilog	✓					<code>svlangserver</code> , <code>verible-verilog-ls</code>
t32	✓					
tablegen	✓	✓	✓			
tact	✓	✓	✓			
task	✓					
tcl	✓		✓			
teal	✓					<code>teal-language-server</code>
templ	✓					<code>templ</code>
tera	✓					
text						<code>ltex-ls-plus</code>
textproto	✓	✓	✓			
tfvars	✓		✓			<code>terraform-ls</code>
thrift	✓					
tilt	✓	✓	✓		✓	<code>tilt</code>
tlaplus	✓					
todotxt	✓					
tolk	✓		✓			
toml	✓	✓	✓	✓	✓	<code>taplo</code> , <code>tombi</code>
tql	✓		✓			
tsq	✓				✓	<code>ts_query_ls</code>
tsx	✓	✓	✓	✓	✓	<code>typescript-language-server</code>
turtle	✓					<code>turtle-language-server</code>
twig	✓					

Language	Syntax Highlighting	Treesitter Textobjects	Auto Indent	Code Navigation Tags	Rainbow Brackets	Default language servers
typescript	✓	✓	✓	✓	✓	typescript-language-server
typespec	✓	✓	✓			tsp-server
typst	✓	✓		✓		tinymist, ltex-ls-plus
ungrammar	✓					
unison	✓	✓	✓	✓	✓	
uxntal	✓					
v	✓	✓	✓			v-analyzer
vala	✓	✓				vala-language-server
vento	✓					
verilog	✓	✓				verible-verilog-ls
vhdl	✓					vhdl_ls
vhs	✓					
vim	✓				✓	
vue	✓	✓	✓			vue-language-server
wast	✓					
wat	✓					wat_server
webc	✓					
werk	✓					
wesl	✓	✓				
wgsl	✓	✓	✓	✓	✓	wgsl-analyzer
wikitext	✓					wikitext-lsp
wit	✓		✓			
woodpecker-ci	✓	✓	✓	✓	✓	yaml-language-server
wren	✓	✓	✓			
xit	✓					
xml	✓	✓	✓		✓	
xtc	✓					
yaml	✓	✓	✓		✓	yaml-language-server, ansible-language-server

Language	Syntax Highlighting	Treesitter Textobjects	Auto Indent	Code Navigation Tags	Rainbow Brackets	Default language servers
yang	✓					<code>yang-language-server</code>
yara	✓					<code>yls</code>
yuck	✓		✓		✓	
zig	✓	✓	✓	✓	✓	<code>zls</code>